

Modeling Epidemics with Julia

Table of contents

Modeling Epidemics with Julia	12
I Prerequisites	13
1 Julia essentials for modelers	14
1.1 Learning goals	14
1.2 Model assumptions and classification	14
1.3 Equations	15
1.4 Julia implementation	15
1.4.1 Functions and in-place mutation	15
1.4.2 Broadcasting and comprehensions	16
1.4.3 Structs, named tuples, and multiple dispatch	17
1.4.4 Reproducible randomness	17
1.4.5 A figure from code	18
1.5 Validation	19
1.6 Interpretation and limitations	19
1.7 Exercises	19
1.7.1 Exercise 1	19
1.7.2 Exercise 2	20
1.8 Companion materials	20
2 Probability and stochastic processes	21
2.1 Learning goals	21
2.2 Model assumptions and classification	21
2.3 Equations	22
2.4 Julia implementation	22
2.5 Validation	25
2.6 Interpretation and limitations	25
2.7 Exercises	26
2.7.1 Exercise 1	26
2.7.2 Exercise 2	26
2.8 Companion materials	26
3 Calculus and numerical methods	27
3.1 Learning goals	27

3.2	Model assumptions and classification	27
3.3	Equations	28
3.4	Julia implementation	28
3.5	Validation	31
3.6	Interpretation and limitations	31
3.7	Exercises	32
3.7.1	Exercise 1	32
3.7.2	Exercise 2	32
3.8	Companion materials	32
II	Core course	33
4	Foundations: SIR theory and the Julia workflow	34
4.1	Learning goals	34
4.2	Model assumptions and classification	34
4.3	Equations	35
4.4	Julia implementation	36
4.5	Validation	38
4.6	Interpretation and limitations	39
4.7	Exercises	40
4.7.1	Exercise 1	40
4.7.2	Exercise 2	40
4.8	Companion materials	40
5	Discrete-time models	41
5.1	Learning goals	41
5.2	Model assumptions and classification	41
5.3	Equations	42
5.4	Julia implementation	42
5.4.1	Convergence of the map to the ODE	45
5.5	Validation	46
5.6	Interpretation and limitations	47
5.7	Exercises	47
5.7.1	Exercise 1	47
5.7.2	Exercise 2	48
5.8	Companion materials	48
6	Ordinary differential equations	49
6.1	Learning goals	49
6.2	Model assumptions and classification	49
6.3	Equations	50

6.4	Julia implementation	50
6.4.1	Problem, solver, and interpolation	50
6.4.2	A lockdown via <code>PresetTimeCallback</code>	51
6.4.3	Peak detection via <code>ContinuousCallback</code>	52
6.5	Validation	53
6.6	Interpretation and limitations	54
6.7	Exercises	54
6.7.1	Exercise 1	54
6.7.2	Exercise 2	55
6.8	Companion materials	55
7	Continuous-time Markov chains and jump processes	56
7.1	Learning goals	56
7.2	Model assumptions and classification	56
7.3	Equations	57
7.4	Julia implementation	58
7.5	Validation	60
7.6	Interpretation and limitations	61
7.7	Exercises	62
7.7.1	Exercise 1	62
7.7.2	Exercise 2	62
7.8	Companion materials	62
8	Stochastic differential equations	63
8.1	Learning goals	63
8.2	Model assumptions and classification	63
8.3	Equations	63
8.4	Julia implementation	64
8.5	Validation	67
8.6	Interpretation and limitations	68
8.7	Exercises	68
8.7.1	Exercise 1	68
8.7.2	Exercise 2	68
8.8	Companion materials	68
9	Agent-based models	69
9.1	Learning goals	69
9.2	Model assumptions and classification	69
9.3	Equations	70
9.4	Julia implementation	70
9.5	Validation	75
9.6	Interpretation and limitations	76

9.7	Exercises	76
9.7.1	Exercise 1	76
9.7.2	Exercise 2	76
9.8	Companion materials	76
10	Observation models and simulated data	77
10.1	Learning goals	77
10.2	Model assumptions and classification	77
10.3	Equations	78
10.4	Julia implementation	79
10.4.1	The latent process	79
10.4.2	Poisson and negative-binomial observations	80
10.4.3	Alternative ways to obtain daily incidence	82
10.5	Validation	83
10.6	Interpretation and limitations	84
10.7	Exercises	84
10.7.1	Exercise 1	84
10.7.2	Exercise 2	85
10.8	Companion materials	85
11	Frequentist parameter estimation	86
11.1	Learning goals	86
11.2	Model assumptions and classification	86
11.3	Equations	87
11.4	Julia implementation	87
11.4.1	Process kernel and simulated data	88
11.4.2	The log-likelihood via the integrator interface	89
11.4.3	Maximum-likelihood estimate	89
11.4.4	Profile-likelihood intervals	90
11.5	Validation	92
11.6	Interpretation and limitations	94
11.7	Exercises	95
11.7.1	Exercise 1	95
11.7.2	Exercise 2	95
11.8	Companion materials	95
12	Bayesian inference	96
12.1	Learning goals	96
12.2	Model assumptions and classification	96
12.3	Equations	97
12.4	Julia implementation	97
12.4.1	Process kernel and simulated data	97
12.4.2	The probabilistic model	99

12.4.3	Sampling with NUTS	100
12.5	Validation	100
12.5.1	Posterior predictive check	102
12.6	Interpretation and limitations	104
12.7	Exercises	105
12.7.1	Exercise 1	105
12.7.2	Exercise 2	105
12.8	Companion materials	105
13	Sensitivity and identifiability	106
13.1	Learning goals	106
13.2	Model assumptions and classification	106
13.3	Equations	107
13.4	Julia implementation	107
13.4.1	Local sensitivities	108
13.4.2	Global sensitivity	110
13.4.3	Structural identifiability	112
13.5	Validation	113
13.6	Interpretation and limitations	114
13.7	Exercises	115
13.7.1	Exercise 1	115
13.7.2	Exercise 2	115
13.8	Companion materials	115
14	Interventions and optimization	116
14.1	Learning goals	116
14.2	Model assumptions and classification	116
14.3	Equations	117
14.4	Julia implementation	117
14.4.1	Scenario simulation with a callback	118
14.4.2	A bounded Optimization.jl problem	122
14.5	Validation	124
14.6	Interpretation and limitations	125
14.7	Exercises	125
14.7.1	Exercise 1	125
14.7.2	Exercise 2	126
14.8	Companion materials	126
15	Data-to-decision capstone	127
15.1	Learning goals	127
15.2	Model assumptions and classification	127
15.3	Equations	128

15.4	Julia implementation	128
15.4.1	Fitting the effective SIR model	130
15.4.2	Diagnostics	132
15.4.3	Deterministic uncertainty envelope	133
15.4.4	Sensitivity and intervention comparison	135
15.5	Validation	138
15.6	Interpretation and limitations	139
15.7	Exercises	139
15.7.1	Exercise 1	139
15.7.2	Exercise 2	139
15.8	Companion materials	140
Course resources		141
	Julia essentials for modelers	141
	Probability and stochastic processes	141
	Calculus and numerical methods	141
	Foundations: SIR theory and the Julia workflow	142
	Discrete-time models	142
	Ordinary differential equations	142
	Continuous-time Markov chains and jump processes	142
	Stochastic differential equations	142
	Agent-based models	143
	Observation models and simulated data	143
	Frequentist parameter estimation	143
	Bayesian inference	143
	Sensitivity and identifiability	143
	Interventions and optimization	144
	Data-to-decision capstone	144
	Delay differential equations	144
	Stochastic delay differential equations	144
	Discrete-event simulation	144
	Universal differential equations	145
	Finite-state projection	145
	Probabilistic numerical integration	145
	Surrogate and emulator models	145
	Advanced optimal control	145
	Model composition and structured populations	146
Appendices		147
A	Delay differential equations	147
A.1	Learning goals	147

A.2	Model assumptions and classification	147
A.3	Equations	148
A.4	Julia implementation	149
A.4.1	An independent two-stage construction	150
A.5	Validation	152
A.6	Interpretation and limitations	153
A.7	Exercises	153
A.7.1	Exercise 1	153
A.7.2	Exercise 2	153
A.8	Companion materials	154
B	Stochastic delay differential equations	155
B.1	Learning goals	155
B.2	Model assumptions and classification	155
B.3	Equations	156
B.4	Julia implementation	156
B.4.1	Adaptive stepping breaks down at the history discontinuity	158
B.5	Validation	160
B.6	Interpretation and limitations	161
B.7	Exercises	162
B.7.1	Exercise 1	162
B.7.2	Exercise 2	162
B.8	Companion materials	162
C	Discrete-event simulation	163
C.1	Learning goals	163
C.2	Model assumptions and classification	163
C.3	Equations	164
C.4	Julia implementation	165
C.4.1	Primary: <code>Agents.jl EventQueueABM</code>	165
C.4.2	Alternative: <code>ConcurrentSim.jl</code> general-purpose processes	166
C.5	Validation	169
C.6	Interpretation and limitations	171
C.7	Exercises	171
C.7.1	Exercise 1	171
C.7.2	Exercise 2	172
C.8	Companion materials	172
D	Universal differential equations	173
D.1	Learning goals	173
D.2	Model assumptions and classification	173
D.3	Equations	174

D.4	Julia implementation	175
D.4.1	Setup and synthetic data	175
D.4.2	A neural closure for the force of infection	176
D.4.3	Assembling and training the UDE	176
D.5	Validation	179
D.6	Interpretation and limitations	181
D.7	Exercises	181
D.7.1	Exercise 1	181
D.7.2	Exercise 2	182
D.8	Companion materials	182
E	Finite-state projection	183
E.1	Learning goals	183
E.2	Model assumptions and classification	183
E.3	Equations	184
E.4	Julia implementation	185
E.4.1	A note on <code>FiniteStateProjection.jl</code>	185
E.4.2	Building the sparse master-equation generator	185
E.4.3	Solving the projected master equation	187
E.5	Validation	188
E.6	Interpretation and limitations	190
E.7	Exercises	191
E.7.1	Exercise 1	191
E.7.2	Exercise 2	191
E.8	Companion materials	192
F	Probabilistic numerical integration	193
F.1	Learning goals	193
F.2	Model assumptions and classification	194
F.3	Equations	194
F.4	Julia implementation	195
F.4.1	A reference solution and a working solver	195
F.4.2	Quantifying the numerical error	195
F.4.3	The probabilistic solve (current <code>ProbNumDiffEq.jl</code> API)	196
F.5	Validation	197
F.6	Interpretation and limitations	198
F.7	Exercises	199
F.7.1	Exercise 1	199
F.7.2	Exercise 2	199
F.8	Companion materials	199
G	Surrogate and emulator models	200
G.1	Learning goals	200

G.2	Model assumptions and classification	200
G.3	Equations	201
G.4	Julia implementation	201
G.4.1	Simulator and training design	201
G.4.2	Current <code>Surrogates.jl</code> API	202
G.4.3	Visualizing the approximation and its error	203
G.5	Validation	205
G.6	Interpretation and limitations	206
G.7	Exercises	206
G.7.1	Exercise 1	206
G.7.2	Exercise 2	206
G.8	Companion materials	207
H	Advanced optimal control	208
H.1	Learning goals	208
H.2	Model assumptions and classification	208
H.3	Equations	209
H.4	Julia implementation	209
H.4.1	InfiniteOpt formulation	209
H.4.2	Plotting the policy and trajectory	211
H.5	Validation	212
H.6	Interpretation and limitations	213
H.7	Exercises	213
H.7.1	Exercise 1	213
H.7.2	Exercise 2	213
H.8	Companion materials	214
I	Model composition and structured populations	215
I.1	Learning goals	215
I.2	Model assumptions and classification	215
I.3	Equations	216
I.4	Julia implementation	216
I.4.1	Composing symbolic SIR components	216
I.4.2	One structured reaction network, three problem types	219
I.4.3	Comparing aggregate infectious prevalence	222
I.5	Validation	223
I.6	Interpretation and limitations	224
I.7	Exercises	224
I.7.1	Exercise 1	224
I.7.2	Exercise 2	224
I.8	Companion materials	225

J	Model and API cheat sheet	226
J.1	Common SIR baseline	229
K	Exercise solutions	230
K.1	Prerequisite and foundational solutions	230
K.1.1	Julia essentials for modelers	230
K.1.2	Probability and stochastic processes	231
K.1.3	Calculus and numerical methods	233
K.1.4	Foundations: SIR theory and the Julia workflow	235
K.1.5	Discrete-time models	237
K.1.6	Ordinary differential equations	239
K.2	Stochastic modeling solutions	240
K.2.1	Continuous-time Markov chains and jump processes	241
K.2.2	Stochastic differential equations	244
K.2.3	Agent-based models	246
K.3	Observation and inference solutions	250
K.3.1	Observation models and simulated data	251
K.3.2	Frequentist parameter estimation	253
K.3.3	Bayesian inference	258
K.4	Analysis, decisions, and capstone solutions	263
K.4.1	Sensitivity and identifiability	263
K.4.2	Interventions and optimization	265
K.4.3	Data-to-decision capstone	267
K.5	Advanced dynamics solutions	270
K.5.1	Delay differential equations	270
K.5.2	Stochastic delay differential equations	273
K.5.3	Discrete-event simulation	277
K.6	Advanced scientific machine learning solutions	282
K.6.1	Universal differential equations	282
K.6.2	Finite-state projection	288
K.6.3	Probabilistic numerical integration	292
K.7	Advanced decision and composition solutions	295
K.7.1	Surrogate and emulator models	295
K.7.2	Advanced optimal control	297
K.7.3	Model composition and structured populations	300
	References	304

Modeling Epidemics with Julia

This executable course develops mathematical models from their assumptions and equations through simulation, inference, and decision-making in Julia.

The course supports a mixed audience:

- Optional prerequisites introduce Julia, probability, calculus, and numerical methods.
- Twelve core chapters progress from the SIR model to a complete data-to-decision case study.
- Nine advanced appendices extend the same modeling ideas to delays, scientific machine learning, probabilistic numerics, control, and model composition.

Every teaching unit has a detailed book chapter, a Reveal.js slide deck, an interactive Pluto notebook, and two exercises with solutions.

See the [course resources](#) for all companion materials.

Part I

Prerequisites

1 Julia essentials for modelers

This prerequisite equips you with the specific slice of Julia that the rest of the course depends on: numeric types, functions with in-place mutation, broadcasting, **structs** and named tuples, multiple dispatch, and reproducible random numbers. Rather than surveying the language, we introduce each feature through the object we return to in every chapter — the susceptible–infectious–recovered (SIR) model of Kermack and McKendrick (1927).

1.1 Learning goals

By the end of this unit you should be able to:

- read and write Julia functions that mutate a preallocated array in place, the pattern every SciML solver expects;
- use broadcasting `(.)` and comprehensions to vectorise scalar operations without writing loops;
- group model quantities with **structs** and **NamedTuples** and access them by field;
- exploit multiple dispatch to write one function that accepts several parameter representations;
- produce bit-for-bit reproducible stochastic output with **StableRNGs**;
- render a figure from code with **CairoMakie**.

1.2 Model assumptions and classification

The running example is the deterministic SIR model. It assumes a closed, well-mixed population of fixed size N , split into susceptible S , infectious I , and recovered R compartments. Individuals mix homogeneously, each contacting others at rate c ; a contact between a susceptible and an infectious individual transmits with probability β ; infectious individuals recover at constant rate γ and are then permanently immune.

Property	Classification
Determinism	Deterministic
Time	Continuous
State	Continuous

Property	Classification
Representation	Population (compartmental)

We do not solve the model here — that is Chapter 4. In this unit the model is a vehicle for the Julia constructs you will reuse throughout.

1.3 Equations

The force of infection is $\lambda(t) = \beta c I(t)/N$, and the compartmental rates of change are

$$\frac{dS}{dt} = -\lambda S, \quad \frac{dI}{dt} = \lambda S - \gamma I, \quad \frac{dR}{dt} = \gamma I. \quad (1.1)$$

Because the three right-hand sides sum to zero, the total population is a conserved quantity:

$$\frac{d}{dt} (S + I + R) = 0 \implies S(t) + I(t) + R(t) = N. \quad (1.2)$$

When we later discretise time we will need to turn a continuous rate r into the probability that at least one event occurs in a window of length δt , assuming events follow a Poisson process:

$$p(r, \delta t) = 1 - e^{-r \delta t}. \quad (1.3)$$

1.4 Julia implementation

We use the course baseline scenario $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $N = 1000$, $c = 10$, $\beta = 0.05$, $\gamma = 0.25$.

1.4.1 Functions and in-place mutation

Every SciML right-hand side has the signature `f(du, u, p, t)`: it *writes* the derivatives into a preallocated vector `du` and returns nothing. This avoids allocating a fresh array on each of the thousands of evaluations a solver makes. Here `u = [S, I, R]` maps directly onto the states in Equation 1.1, and `p` carries (β, c, γ, N) .

```

function sir_ode!(du, u, p, t)
    S, I, R = u
    β, c, γ, N = p
    fi = β * c * I / N      # force of infection, fi = cI/N
    du[1] = -β * c * I / N * S      # dS/dt
    du[2] = β * c * I / N * S - γ * I      # dI/dt
    du[3] = γ * I      # dR/dt
    return nothing
end

u0 = [990.0, 10.0, 0.0]      # S, I, R
p = (β = 0.05, c = 10.0, γ = 0.25, N = 1000.0)
du = zeros(3)
sir_ode!(du, u0, values(p), 0.0)
du

```

```

3-element Vector{Float64}:
-4.95
 2.45
 2.5

```

The trailing `!` is a Julia convention marking a function that mutates its arguments. The `NamedTuple` `p` lets us write `p.β` instead of remembering that the transmission probability lives at index 1.

1.4.2 Broadcasting and comprehensions

The dot syntax applies a scalar function elementwise, fusing all operations into a single loop with no temporaries. We use it constantly to evaluate a model over a grid. Below we compute Equation 1.3 for a range of window widths and two rates.

```

rate_to_proportion(r, t) = -expm1(-r * t)      # numerically stable 1 - exp(-r * t)

ts = 0.0:0.05:2.0
p_fast = rate_to_proportion.(1.0, ts)      # broadcast over ts
p_slow = rate_to_proportion.(0.25, ts)
(first(p_fast), last(p_fast), first(p_slow), last(p_slow))

```

```

(0.0, 0.8646647167633873, 0.0, 0.3934693402873666)

```


`expm1(x)` computes $e^x - 1$ accurately for small x , avoiding the cancellation error of `1 - exp(...)`.

1.4.3 Structs, named tuples, and multiple dispatch

A `struct` bundles related fields into a new type. Multiple dispatch then lets one function name resolve to different methods depending on argument types — the mechanism SciML uses everywhere. Here `force_of_infection` accepts either a plain `NamedTuple` or a typed `SIRParameters`.

```
struct SIRParameters
    ::Float64
    c::Float64
    ::Float64
    N::Float64
end

force_of_infection(I, p::NamedTuple)    = p. * p.c * I / p.N
force_of_infection(I, p::SIRParameters) = p. * p.c * I / p.N

params = SIRParameters(0.05, 10.0, 0.25, 1000.0)
(force_of_infection(10.0, p), force_of_infection(10.0, params))
```

(0.005, 0.005)

1.4.4 Reproducible randomness

Later chapters simulate stochastic epidemics. To make figures reproducible we seed a *local* random stream with `StableRNGs`, whose output is fixed across Julia versions — unlike the global `Random.default_rng()`.

```
using StableRNGs
rng = StableRNG(2024)
draws = rand(rng, 5)
```

```
5-element Vector{Float64}:
 0.8177021060277301
 0.3804323567786363
 0.5425813266814807
 0.7085670484724618
 0.7911959541742282
```

1.4.5 A figure from code

We now visualise Equation 1.3 with CairoMakie. The last expression in the cell (the `Figure`) is what Quarto renders.

```
using CairoMakie

fig = Figure(size = (640, 400))
ax = Axis(fig[1, 1];
    xlabel = "time step  $t$ ",
    ylabel = "event probability  $1 - \exp(-r t)$ ",
    title = "Rate-to-proportion conversion")
lines!(ax, ts, p_fast; linewidth = 3, label = "r = 1.0")
lines!(ax, ts, p_slow; linewidth = 3, label = "r = 0.25")
axislegend(ax; position = :rb)
fig
```

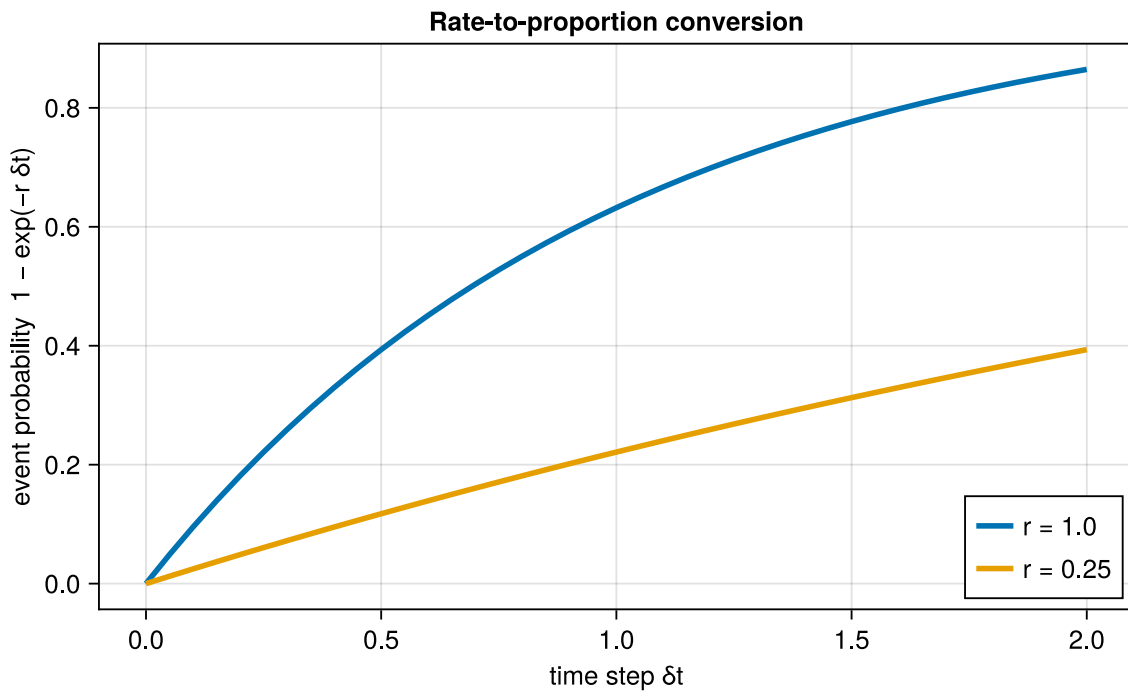


Figure 1.1: Probability of at least one event in a window of width δt for a Poisson process, for a fast ($r = 1$) and slow ($r = 0.25$) rate. Both curves rise from 0 and saturate at 1.

1.5 Validation

Two structural checks confirm the constructs behave as the mathematics requires.

First, the derivatives in Equation 1.1 must sum to zero, reproducing the conservation law Equation 1.2:

```
sir_ode!(du, u0, values(p), 0.0)
isapprox(sum(du), 0.0; atol = 1e-12)
```

true

Second, `rate_to_proportion` must reduce to its first-order Taylor expansion $p \approx r \delta t$ as $\delta t \rightarrow 0$, and must always return a valid probability in $[0, 1]$:

```
small = rate_to_proportion(0.25, 1e-6)
(isapprox(small, 0.25e-6; rtol = 1e-3), all(0 .<= p_fast .<= 1))
```

(true, true)

Both cells return `true`, so the in-place kernel conserves population and the rate conversion is well behaved.

1.6 Interpretation and limitations

You now have the exact Julia vocabulary the course relies on: in-place kernels, broadcasting, typed containers, dispatch, and seeded randomness. This unit is deliberately narrow — it is not a full Julia tutorial. It does not cover package management beyond `using`, performance profiling, metaprogramming, or the type system’s parametric machinery. For those, consult the official [Julia documentation](#) and the [performance tips](#). The [CairoMakie documentation](#) covers the plotting API in depth.

1.7 Exercises

1.7.1 Exercise 1

Write an in-place kernel `sir_incidence!(du, u, p, t)` that extends Equation 1.1 with a fourth state C tracking *cumulative* infections, so that $dC/dt = \lambda S$. Evaluate the derivative at the baseline initial condition and confirm that `du[4]` equals `-du[1]` (new infections leave S at exactly the rate they enter C).

1.7.2 Exercise 2

Using `StableRNG(2024)`, draw 10 000 samples from `rand`, and use broadcasting to count how many fall below `rate_to_proportion(0.25, 1.0)`. Compare the observed fraction with the target probability and explain why a *local* seeded RNG makes this figure reproducible across machines.

1.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

2 Probability and stochastic processes

Epidemics in finite populations are inherently random: whether the next contact transmits, and whether an infectious person recovers today or tomorrow, are chance events. This unit builds the probability foundations the course needs — random variables, the Bernoulli/Binomial family, expectation and variance, and the idea of a discrete-time Markov chain — and applies them immediately to a stochastic SIR model driven by binomial draws.

2.1 Learning goals

By the end of this unit you should be able to:

- describe a Bernoulli trial and derive the Binomial distribution as a sum of independent trials;
- compute the mean and variance of a Binomial random variable;
- define the Markov property and explain why the binomial SIR chain has it;
- convert continuous transition rates into per-step probabilities;
- simulate an ensemble of stochastic epidemics reproducibly with `Distributions` and `StableRNGs`;
- compare the ensemble mean against a deterministic expectation.

2.2 Model assumptions and classification

We model each time step of width δt as a batch of independent Bernoulli trials. Every susceptible individual either becomes infected or not; every infectious individual either recovers or not. The number of new infections is therefore Binomial in the number of susceptibles, and likewise for recoveries. The population is closed and well mixed, and — crucially — the transition probabilities depend only on the *current* counts (S_t, I_t, R_t) , not on the history. This is the Markov property.

Property	Classification
Determinism	Stochastic
Time	Discrete (step δt)
State	Discrete (integer counts)

Property	Classification
Representation	Population (compartmental)

2.3 Equations

A Bernoulli(p) variable takes value 1 with probability p . The sum of n independent Bernoulli(p) trials is Binomial(n, p), with

$$\Pr(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad \mathbb{E}[X] = np, \quad \text{Var}[X] = np(1-p). \quad (2.1)$$

Over a step of width δt , a susceptible individual is infected with probability p_I and an infectious individual recovers with probability p_R , where the per-capita hazards are the SIR rates converted through Equation 1.3:

$$p_I = 1 - \exp\left(-\frac{\beta c I_t}{N} \delta t\right), \quad p_R = 1 - \exp(-\gamma \delta t). \quad (2.2)$$

The chain then updates by drawing the two event counts and moving individuals between compartments:

$$\begin{aligned} Y_t &\sim \text{Binomial}(S_t, p_I), & Z_t &\sim \text{Binomial}(I_t, p_R), \\ S_{t+1} &= S_t - Y_t, & I_{t+1} &= I_t + Y_t - Z_t, & R_{t+1} &= R_t + Z_t. \end{aligned} \quad (2.3)$$

Because every individual moves *forward* through S→I→R and none are created or destroyed, $S_{t+1} + I_{t+1} + R_{t+1} = S_t + I_t + R_t = N$ for all t .

2.4 Julia implementation

The kernel below maps Equation 2.2 and Equation 2.3 directly onto code. The random number generator `rng` is passed explicitly so that the stochastic stream is reproducible and local. `Binomial(n, p)` from `Distributions` is exactly the distribution in Equation 2.1.

```

using Distributions, StableRNGs

rate_to_proportion(r, t) = -expm1(-r * t)

function sir_markov_step(state, p, t, rng)
    S, I, R = state
    , c, , N = p
    p_I = rate_to_proportion( * c * I / N, t) # eq-p02-probs
    p_R = rate_to_proportion( , t)
    Y = rand(rng, Binomial(S, p_I)) # new infections
    Z = rand(rng, Binomial(I, p_R)) # new recoveries
    return (S - Y, I + Y - Z, R + Z) # eq-p02-chain
end

function simulate_chain(u0, p, t, nsteps, rng)
    traj = Matrix{Int}(undef, 3, nsteps + 1)
    traj[:, 1] .= u0
    state = Tuple(u0)
    for k in 1:nsteps
        state = sir_markov_step(state, p, t, rng)
        traj[:, k + 1] .= state
    end
    return traj
end
end

```

`simulate_chain` (generic function with 1 method)

We run an ensemble of 200 realisations from the baseline initial condition and also record the ensemble mean of the infectious compartment.

```

u0 = [990, 10, 0] # integer S, I, R
p = (0.05, 10.0, 0.25, 1000) # , c, , N
t = 1.0
nsteps = 40
nsims = 200

ensemble = [simulate_chain(u0, p, t, nsteps, StableRNG(seed))
            for seed in 1:nsims]
I_paths = reduce(hcat, (traj[2, :] for traj in ensemble)) # (nsteps+1) × nsims
mean_I = vec(sum(I_paths; dims = 2)) ./ nsims
(size(I_paths), round(maximum(mean_I); digits = 1))

```

((41, 200), 190.8)

```
using CairoMakie

ts = 0:nsteps
fig = Figure(size = (680, 420))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "infectious I(t)",
          title = "Binomial SIR chain: 200 realisations")
for j in 1:nsims
    lines!(ax, ts, I_paths[:, j]; color = (:grey, 0.15), linewidth = 1)
end
lines!(ax, ts, mean_I; color = :firebrick, linewidth = 3, label = "ensemble mean")
axislegend(ax; position = :rt)
fig
```

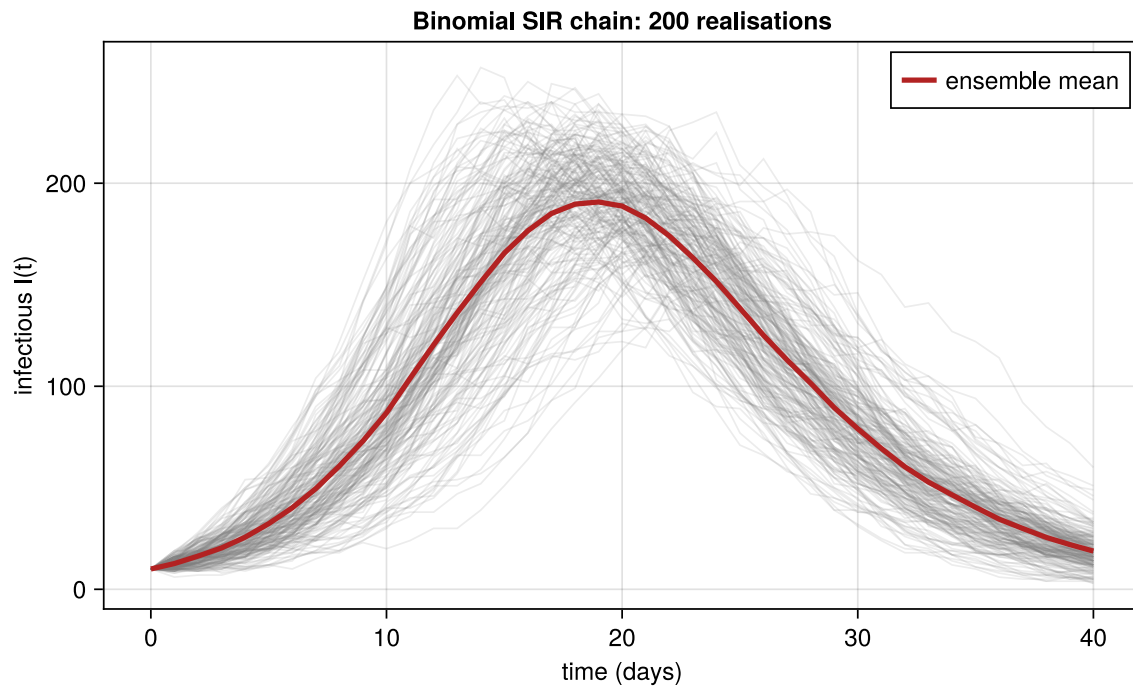


Figure 2.1: Two hundred stochastic SIR realisations of the infectious compartment (thin grey) with the ensemble mean overlaid (red). Individual epidemics fluctuate and some fade early, but the mean traces a smooth, reproducible curve.

2.5 Validation

For a single Binomial draw the sample mean over many replicates must approach the theoretical mean np from Equation 2.1. We check this directly:

```
rng = StableRNG(7)
n, prob = 990, 0.02
samples = [rand(rng, Binomial(n, prob)) for _ in 1:100_000]
theoretical = n * prob
empirical = sum(samples) / length(samples)
(theoretical, round(empirical; digits = 3), isapprox(empirical, theoretical; rtol = 0.02))
```

```
(19.8, 19.773, true)
```

As a structural check on the chain itself, every realisation must conserve the population exactly (Equation 2.3 moves integers between compartments):

```
totals = [sum(traj[:, k]) for traj in ensemble for k in 1:nsteps+1]
all(==(1000), totals)
```

```
true
```

Both checks pass: the empirical Binomial mean matches np to within 2%, and all 200×41 population totals equal $N = 1000$.

2.6 Interpretation and limitations

The binomial chain captures *demographic stochasticity* — the randomness of a finite population — that the deterministic ODE ignores. Individual epidemics can peak at different times or fade out entirely, yet the ensemble mean tracks a smooth trajectory close to the deterministic solution we study in Chapter 4. The binomial construction, however, allows more than one event per individual per step to be double-counted only if $p_I + p_R$ per compartment were mishandled; here susceptibles and infectious individuals are treated in separate binomials, which is exact only in the limit of small δt . It also assumes infections and recoveries within a step are independent. The exact, event-by-event alternative — the Gillespie algorithm of Gillespie (1977) — is developed in the continuous-time Markov chapter. For the distributions API see the [Distributions.jl documentation](#).

2.7 Exercises

2.7.1 Exercise 1

The variance of a $\text{Binomial}(n, p)$ is $np(1 - p)$ (Equation 2.1). Draw 100 000 samples from $\text{Binomial}(990, 0.02)$ using `StableRNG(7)` and compare the sample variance with the theoretical value. Then repeat with $p = 0.5$ and explain, using the formula, why the variance is largest near $p = 0.5$.

2.7.2 Exercise 2

Reduce the step width to $\delta t = 0.1$ (and increase `nsteps` to 400 so the horizon is unchanged) and rerun the ensemble. Report how the fraction of realisations that reach $I = 0$ before day 40 changes, and explain why finer steps change the discrete-time dynamics.

2.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

3 Calculus and numerical methods

Differential equations describe how a system changes instant to instant, but a computer can only take finite steps. This unit bridges that gap. We review the derivative as a rate of change, introduce the forward Euler method as the simplest numerical integrator, and analyse its error — distinguishing the local error of one step from the global error accumulated over a whole simulation. These ideas justify the adaptive solvers we adopt from Chapter 6 onward.

3.1 Learning goals

By the end of this unit you should be able to:

- interpret a derivative as an instantaneous rate and a first-order Taylor expansion;
- derive and implement the forward Euler update for a system of ODEs;
- distinguish local truncation error $O(\delta t^2)$ from global error $O(\delta t)$;
- estimate a method's empirical order of convergence from a refinement study;
- produce a log-log error plot with `CairoMakie` and read its slope.

3.2 Model assumptions and classification

We integrate the deterministic SIR system of Equation 1.1. Forward Euler assumes the derivative is approximately constant across each step of width δt — exact only in the limit $\delta t \rightarrow 0$. It is an *explicit, fixed-step, first-order* method.

Property	Classification
Determinism	Deterministic
Time	Continuous problem, discretised in fixed steps
State	Continuous
Method order	1 (global error $\propto \delta t$)

3.3 Equations

A first-order Taylor expansion of the state u about time t gives

$$u(t + \delta t) = u(t) + \delta t u'(t) + \frac{1}{2} \delta t^2 u''(\xi), \quad \xi \in (t, t + \delta t). \quad (3.1)$$

Dropping the quadratic remainder yields the forward Euler update. For the SIR system with right-hand side f we obtain the recursion

$$u_{n+1} = u_n + \delta t f(u_n), \quad u_0 = (S(0), I(0), R(0)). \quad (3.2)$$

The discarded term makes the **local truncation error** per step $O(\delta t^2)$. Over a fixed horizon $[0, T]$ the number of steps is $T/\delta t$, so the errors accumulate to a **global error** that is one order lower:

$$\max_n \|u_n - u(t_n)\| = O(\delta t). \quad (3.3)$$

Halving δt should therefore roughly halve the global error — a slope of 1 on a log-log plot of error against step size.

3.4 Julia implementation

The integrator reuses the in-place kernel pattern from Chapter 1. The `euler` function marches Equation 3.2 forward and stores every state.

```
function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, , N = p
    = * c * I / N
    du[1] = - * S
    du[2] = * S - * I
    du[3] = * I
    return nothing
end

function euler(f, u0, p, t, tmax)
    ts = 0.0:t:tmax
    us = Matrix{Float64}(undef, length(u0), length(ts))
    us[:, 1] .= u0
```

```

du = similar(u0)
for n in 1:length(ts) - 1
    u = @view us[:, n]
    f(du, u, p, ts[n])
    us[:, n + 1] .= u .+ t .* du      # u_{n+1} = u_n + t f(u_n)
end
return collect(ts), us
end

u0 = [990.0, 10.0, 0.0]
p = (0.05, 10.0, 0.25, 1000.0)
tmax = 40.0

ts, us = euler(sir_ode!, u0, p, 0.1, tmax)
(size(us), round.(us[:, end]; digits = 2))

```

```
((3, 401), [208.19, 14.33, 777.48])
```

To measure error we need a trustworthy reference. Forward Euler *converges to its own exact solution* as $\delta t \rightarrow 0$, so a very fine Euler run (step $2^{-14} \approx 6 \times 10^{-5}$) serves as ground truth for comparing coarser runs at the final time.

```

t_ref = tmax / 2^19
_, us_ref = euler(sir_ode!, u0, p, t_ref, tmax)
u_ref = us_ref[:, end]

ts = [tmax / 2^k for k in 6:12]
errors = map(ts) do t
    _, us = euler(sir_ode!, u0, p, t, tmax)
    maximum(abs.(us[:, end] .- u_ref))      # global error at t = tmax
end
[(round(t; sigdigits = 3), round(err; sigdigits = 3)) for (t, err) in zip(ts, errors)]

```

```
7-element Vector{Tuple{Float64, Float64}}:
```

```

(0.625, 11.5)
(0.312, 5.66)
(0.156, 2.8)
(0.0781, 1.4)
(0.0391, 0.696)
(0.0195, 0.347)
(0.00977, 0.173)

```

```

using CairoMakie

fig = Figure(size = (640, 440))
ax = Axis(fig[1, 1]; xscale = log10, yscale = log10,
          xlabel = "step size  $\delta t$ ", ylabel = "global error at  $t = 40$ ",
          title = "Forward Euler convergence")
scatter!(ax, ts, errors; markersize = 12, color = :firebrick, label = "measured")
ref_line = errors[1] .* (ts ./ ts[1]).^1 # slope-1 guide
lines!(ax, ts, ref_line; linestyle = :dash, color = :black, label = "slope 1 (order 1)")
axislegend(ax; position = :rb)
fig

```

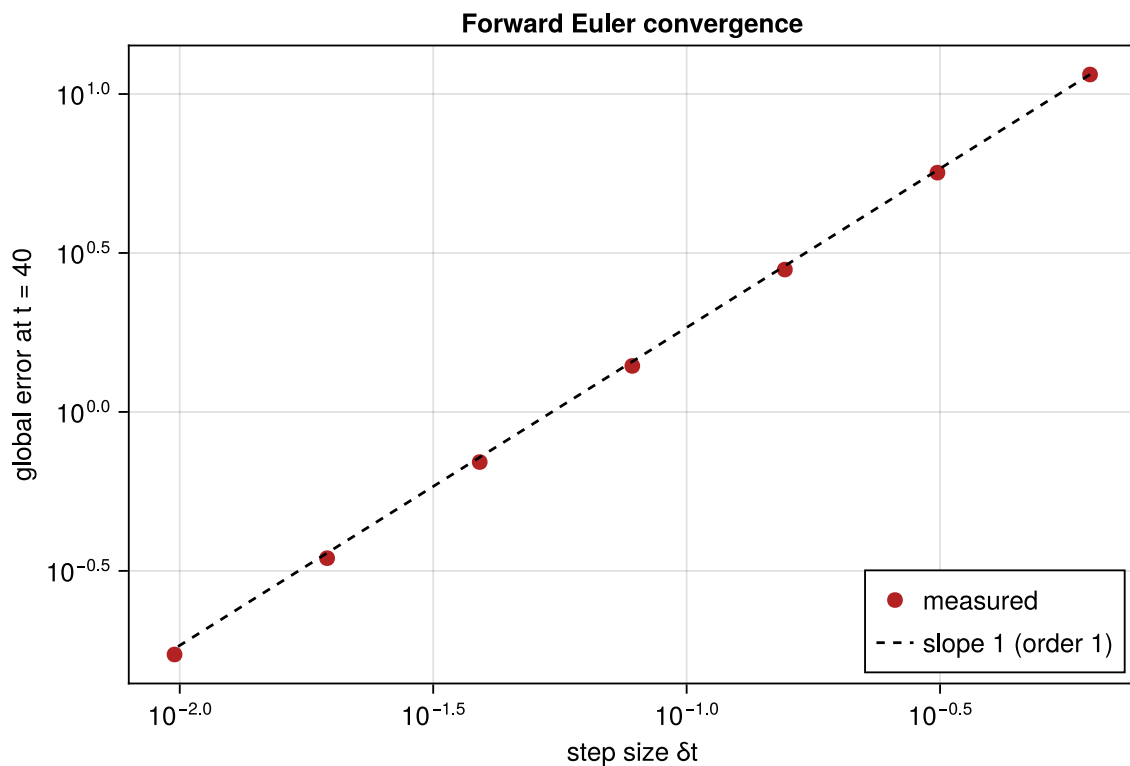


Figure 3.1: Global error of forward Euler at $t = 40$ versus step size, on log–log axes. The measured errors (points) lie parallel to the reference first-order line (dashed), confirming $O(\delta t)$ convergence.

3.5 Validation

The empirical order of convergence is the slope of $\log(\text{error})$ against $\log(\delta t)$. For a first-order method it should be close to 1. We estimate it from successive halvings, $\log_2(e_k/e_{k+1})$:

```
orders = [log2(errors[k] / errors[k + 1]) for k in 1:length(errors) - 1]
mean_order = sum(orders) / length(orders)
(round.(orders; digits = 3), round(mean_order; digits = 3), isapprox(mean_order, 1.0; atol =
```

```
([1.026, 1.013, 1.007, 1.004, 1.004, 1.006], 1.01, true)
```

The estimated orders cluster around 1 and the mean is within 0.1 of the theoretical value, confirming Equation 3.3. As a second check, Euler must preserve the conservation law of Equation 1.2 regardless of step size, because its update adds the three derivatives that sum to zero:

```
_, us_coarse = euler(sir_ode!, u0, p, 0.5, tmax)
totals = vec(sum(us_coarse; dims = 1))
all(t -> isapprox(t, 1000.0; atol = 1e-9), totals)
```

```
true
```

3.6 Interpretation and limitations

The refinement study shows the central trade-off of numerical integration: accuracy improves only linearly as we shrink the step, so halving the error doubles the cost. Forward Euler is also only *conditionally stable* — for stiff problems a too-large step produces oscillating or divergent output even though the true solution is smooth. Higher-order and adaptive methods obtain far more accuracy per unit work by estimating the local error and adjusting δt automatically. That is exactly what `Tsit5` and the rest of the `OrdinaryDiffEq` suite do, and why the course uses them from Chapter 6 onward. See the [OrdinaryDiffEq solver documentation](#) for the available methods and their orders.

3.7 Exercises

3.7.1 Exercise 1

Implement the *midpoint* (second-order Runge–Kutta) update $u_{n+1} = u_n + \delta t f(u_n + \frac{\delta t}{2} f(u_n))$ and repeat the refinement study. Estimate its empirical order of convergence and confirm that it is close to 2, i.e. one order higher than Euler.

3.7.2 Exercise 2

At $\delta t = 0.1$, compare the final-time state from `euler` against the reference solution and report the relative error in the infectious compartment. Then determine, by experiment, the largest step size for which forward Euler keeps every compartment nonnegative over $[0, 40]$.

3.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Part II

Core course

4 Foundations: SIR theory and the Julia workflow

This chapter is the conceptual anchor of the course. We derive the classical susceptible–infectious–recovered (SIR) model of Kermack and McKendrick (1927) from first principles, state its assumptions precisely, and read off the two quantities that govern every epidemic it describes: the basic reproduction number R_0 and the epidemic threshold. We solve the model with the modern SciML stack, verify conservation of the population and the analytic final-size relation, and close with a roadmap of how the rest of the book relaxes each assumption in turn.

4.1 Learning goals

By the end of this chapter you should be able to:

- derive the SIR equations from mass-action transmission and constant-rate recovery;
- define R_0 and explain the threshold condition for an epidemic to grow;
- solve the model in Julia with `OrdinaryDiffEq` and the `Tsit5` solver;
- verify population conservation and compute the final epidemic size with `NonlinearSolve`;
- situate every later chapter as a controlled relaxation of one SIR assumption.

4.2 Model assumptions and classification

The SIR model partitions a closed population of size N into susceptible S , infectious I , and recovered R individuals. Its assumptions are:

1. **Closed population.** No births, deaths, or migration, so $S + I + R = N$ is fixed.
2. **Homogeneous mixing.** Every individual contacts others at the same rate c , and contacts are drawn uniformly at random from the population.
3. **Mass-action transmission.** A susceptible individual’s hazard of infection is proportional to the prevalence I/N and the per-contact transmission probability β .
4. **Constant recovery.** Infectious individuals recover at a constant per-capita rate γ , so infectious periods are exponentially distributed with mean $1/\gamma$.
5. **Permanent immunity.** Recovered individuals never return to S .

Property	Classification
Determinism	Deterministic
Time	Continuous
State	Continuous (population fractions)
Representation	Population (compartmental)

These choices place the model in the top-left of the modelling landscape: deterministic and continuous. Subsequent chapters move along each axis — stochastic in Chapter 5, discrete-event and individual-based later — so it is worth understanding this baseline thoroughly.

4.3 Equations

Consider a small interval dt . Each of the S susceptibles makes $c dt$ contacts; a fraction I/N of those are with infectious individuals, and each such contact transmits with probability β . The expected number of new infections is therefore $\beta c(I/N)S dt$. Meanwhile $\gamma I dt$ individuals recover. Taking $dt \rightarrow 0$ gives the SIR system, with force of infection $\lambda = \beta c I/N$:

$$\frac{dS}{dt} = -\beta c \frac{I}{N} S, \quad \frac{dI}{dt} = \beta c \frac{I}{N} S - \gamma I, \quad \frac{dR}{dt} = \gamma I. \quad (4.1)$$

Summing the three equations gives $d(S + I + R)/dt = 0$, so

$$S(t) + I(t) + R(t) = N \quad \text{for all } t. \quad (4.2)$$

The infectious compartment grows when $dI/dt > 0$, i.e. when $\beta c S/N > \gamma$. Defining the **basic reproduction number** — the expected number of secondary infections produced by one infectious individual in a wholly susceptible population —

$$R_0 = \frac{\beta c}{\gamma}, \quad (4.3)$$

the epidemic grows if and only if the susceptible fraction exceeds the reciprocal threshold:

$$\frac{dI}{dt} > 0 \quad \Longleftrightarrow \quad \frac{S}{N} > \frac{1}{R_0}. \quad (4.4)$$

At the epidemic peak $S/N = 1/R_0$ exactly. After the epidemic burns out, the surviving susceptible fraction $s_\infty = S(\infty)/N$ satisfies the transcendental **final-size relation**, obtained by integrating $dS/dR = -R_0 S/N$ and using $R(\infty) = N - S(\infty)$,

$$s_{\infty} = s_0 \exp(-R_0(1 - s_{\infty})), \quad s_0 = \frac{S(0)}{N}, \quad (4.5)$$

whose root gives the total attack rate $1 - s_{\infty}$. (With $s_0 \rightarrow 1$ this reduces to the familiar textbook form.)

4.4 Julia implementation

The right-hand side maps line by line onto Equation 4.1. We use the course baseline scenario and solve with `Tsit5`, a robust non-stiff Runge–Kutta method.

```
using OrdinaryDiffEq

function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, , N = p
    = * c * I / N          # force of infection, eq-ch01-sir
    du[1] = - * S
    du[2] = * S - * I
    du[3] = * I
    return nothing
end

u0 = [990.0, 10.0, 0.0]      # S(0), I(0), R(0)
p = (0.05, 10.0, 0.25, 1000.0) # , c, , N
tspan = (0.0, 80.0)

prob = ODEProblem(sir_ode!, u0, tspan, p)
sol = solve(prob, Tsit5(); saveat = 0.5)
sol.u[end]
```

```
3-element Vector{Float64}:
 199.8228587945497
   0.03697185303398417
 800.1401693524167
```

The reproduction number and epidemic threshold follow directly from Equation 4.3 and Equation 4.4:

```

, c, , N = p
R = * c /
threshold_fraction = 1 / R
(R, threshold_fraction)

```

(2.0, 0.5)

With $R_0 = 2$, the epidemic grows while more than half the population is susceptible. We solve the final-size relation Equation 4.5 with a scalar Newton iteration from `NonlinearSolve`:

```

using NonlinearSolve

s = u0[1] / N                                     # initial susceptible fraction
finalsize(s, R0) = s - s * exp(-R0 * (1 - s))      # eq-ch01-finalsize
fs_prob = NonlinearProblem(finalsize, 0.2, R)
s_inf = solve(fs_prob, NewtonRaphson()).u
attack_rate = 1 - s_inf
(s_inf, attack_rate)

```

(0.19979603232320023, 0.8002039676767998)

```

using CairoMakie

t = sol.t
S = [u[1] for u in sol.u]
I = [u[2] for u in sol.u]
R = [u[3] for u in sol.u]

fig = Figure(size = (720, 440))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "number of individuals",
          title = "SIR baseline: S(0)=990, I(0)=10, R=2")
lines!(ax, t, S; color = :steelblue, linewidth = 3, label = "S")
lines!(ax, t, I; color = :firebrick, linewidth = 3, label = "I")
lines!(ax, t, R; color = :seagreen, linewidth = 3, label = "R")
hlines!(ax, [threshold_fraction * N]; color = :black, linestyle = :dash,
        label = "S = N/R ")
axislegend(ax; position = :rc)
fig

```

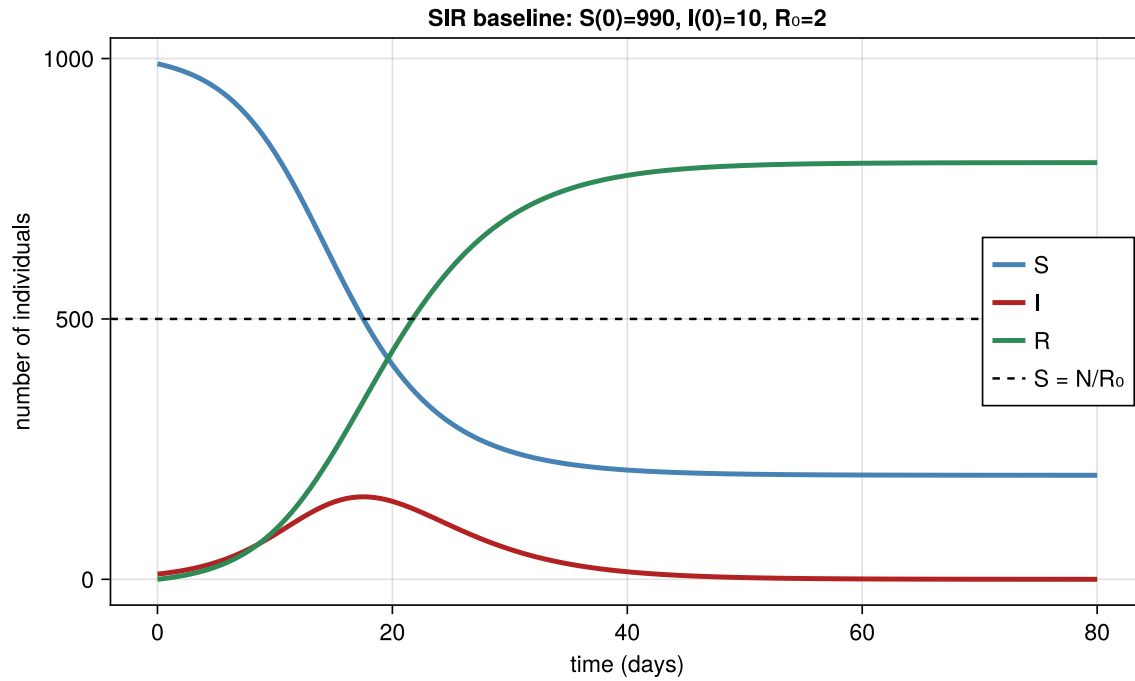


Figure 4.1: Baseline SIR dynamics ($R_0 = 2$). The dashed line marks the threshold susceptible fraction $S/N = 1/R_0$; the infectious curve peaks exactly where S crosses it, as predicted by Equation 4.4.

4.5 Validation

Three independent checks confirm the numerical solution respects the theory.

Conservation (Equation 4.2). The total across compartments must equal N at every saved time:

```
totals = [sum(u) for u in sol.u]
all(x -> isapprox(x, N; atol = 1e-6), totals)
```

true

Threshold (Equation 4.4). The infectious peak must coincide with the susceptible fraction crossing $1/R_0$. We locate the peak and read off S/N there:

```

peak_index = argmax(I)
S_over_N_at_peak = S[peak_index] / N
isapprox(S_over_N_at_peak, threshold_fraction; atol = 0.01)

```

true

Final size (Equation 4.5). Extending the horizon, the simulated surviving susceptible fraction must match the analytic root s_∞ :

```

prob_long = remake(prob; tspan = (0.0, 400.0))
sol_long = solve(prob_long, Tsit5(); reltol = 1e-10, abstol = 1e-10, save_everystep = false)
S_end_fraction = sol_long.u[end][1] / N
isapprox(S_end_fraction, s_inf; atol = 1e-3)

```

true

All three return `true`: population is conserved to machine tolerance, the peak sits on the threshold, and the long-run susceptible fraction matches the final-size prediction $s_\infty \approx 0.200$ (attack rate $\approx 80\%$).

4.6 Interpretation and limitations

The SIR model compresses an epidemic into three numbers — R_0 , the peak timing, and the final size — that follow from just two parameters, R_0 and the recovery rate. This parsimony is its strength and its limitation. The deterministic ODE describes the *mean* behaviour of a large population and cannot represent extinction, super-spreading, or the sampling noise of small outbreaks; those require the stochastic models of Chapter 5 and the jump processes that follow. Homogeneous mixing ignores age, space, and contact structure, addressed later by agent-based models. Exponential infectious periods are a modelling convenience relaxed by the delay models in the appendices.

The rest of the book is organised as a guided relaxation of these assumptions:

- **Chapter 5** — discrete-time maps and the stochastic Markov chain; convergence to the ODE as the step shrinks.
- **Chapter 6** — the full `OrdinaryDiffEq` workflow: dense interpolation, callbacks, and event handling.

- Continuous-time Markov chains and jump processes, stochastic differential equations, agent-based models, observation models, and finally inference, sensitivity, and control — each adding realism the baseline omits.

For the solver API see the [OrdinaryDiffEq documentation](#) and for root-finding the [NonlinearSolve documentation](#).

4.7 Exercises

4.7.1 Exercise 1

Using the baseline parameters, compute R_0 for contact rates $c \in \{4, 6, 8, 10\}$ (holding β and γ fixed). For each, solve the final-size relation Equation 4.5 and tabulate the attack rate. Identify the contact rate below which no epidemic occurs and relate it to the condition $R_0 = 1$.

4.7.2 Exercise 2

Verify the threshold prediction numerically: solve the baseline model, extract the time of the infectious peak, and confirm that S/N at that time equals $1/R_0$ to within 1%. Then raise γ to 0.5 (so $R_0 = 1$) and describe what happens to the infectious trajectory.

4.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

5 Discrete-time models

Many epidemic data arrive in discrete intervals — daily case counts, weekly reports — and many simulations advance in fixed steps. This chapter develops the discrete-time SIR model in two flavours: a **deterministic function map** with continuous state, and a **stochastic Markov chain** with integer counts. We then show that as the step width δt shrinks, the deterministic map converges to the continuous ODE of Chapter 4, tying the discrete and continuous worlds together.

5.1 Learning goals

By the end of this chapter you should be able to:

- write a discrete-time SIR update using the rate-to-proportion conversion;
- contrast the deterministic map with the binomial Markov chain of Chapter 2;
- run a stochastic ensemble and compare its mean with the deterministic map;
- demonstrate numerically that the map converges to the ODE as $\delta t \rightarrow 0$;
- verify population conservation in both formulations.

5.2 Model assumptions and classification

All three models share the SIR assumptions of Chapter 4 — closed, well-mixed population, mass-action transmission, constant recovery. They differ only in how they represent time and state:

Model	Determinism	Time	State
Function map	Deterministic	Discrete	Continuous
Markov chain	Stochastic	Discrete	Discrete (integer)
ODE (Chapter 4)	Deterministic	Continuous	Continuous

The function map is the deterministic *expectation* of the Markov chain: it moves the mean number of individuals each step, whereas the chain draws a random integer around that mean.

5.3 Equations

Over a step of width δt , the per-capita probabilities of infection and recovery are obtained from the continuous rates via the Poisson conversion introduced in Equation 1.3:

$$p_I = 1 - \exp\left(-\frac{\beta c I_t}{N} \delta t\right), \quad p_R = 1 - \exp(-\gamma \delta t). \quad (5.1)$$

The **deterministic map** moves the expected number of individuals:

$$\begin{aligned} S_{t+1} &= S_t - p_I S_t, \\ I_{t+1} &= I_t + p_I S_t - p_R I_t, \\ R_{t+1} &= R_t + p_R I_t. \end{aligned} \quad (5.2)$$

The **stochastic chain** replaces those expectations with binomial draws (cf. Equation 2.3):

$$\begin{aligned} S_{t+1} &= S_t - Y_t, \\ Y_t &\sim \text{Binomial}(S_t, p_I), \quad Z_t \sim \text{Binomial}(I_t, p_R), \quad I_{t+1} = I_t + Y_t - Z_t, \\ R_{t+1} &= R_t + Z_t. \end{aligned} \quad (5.3)$$

As $\delta t \rightarrow 0$, $p_I \rightarrow \frac{\beta c I}{N} \delta t$ and $p_R \rightarrow \gamma \delta t$, so the map Equation 5.2 reduces to a forward-Euler step of the ODE Equation 4.1 and converges to it.

5.4 Julia implementation

The two kernels sit side by side so the only difference — deterministic movement versus a binomial draw — is explicit.

```
using Distributions, StableRNGs

rate_to_proportion(r, t) = -expm1(-r * t)

function sir_map_step(state, p, t)          # deterministic, eq-ch02-map
    S, I, R = state
    , c, , N = p
    infection = rate_to_proportion( * c * I / N, t) * S
    recovery  = rate_to_proportion( , t) * I
    return (S - infection, I + infection - recovery, R + recovery)
end
```

```

function sir_chain_step(state, p, t, rng)    # stochastic, eq-ch02-chain
    S, I, R = state
    , c, , N = p
    Y = rand(rng, Binomial(round(Int, S), rate_to_proportion( * c * I / N, t)))
    Z = rand(rng, Binomial(round(Int, I), rate_to_proportion( , t)))
    return (S - Y, I + Y - Z, R + Z)
end

function iterate_model(step, u0, p, t, nsteps; extra = ())
    traj = Matrix{Float64}(undef, 3, nsteps + 1)
    traj[:, 1] .= u0
    state = Tuple(float.(u0))
    for k in 1:nsteps
        state = step(state, p, t, extra...)
        traj[:, k + 1] .= state
    end
    return traj
end

```

iterate_model (generic function with 1 method)

We run the deterministic map and a 200-member stochastic ensemble on the baseline scenario with $\delta t = 1$ day.

```

u0 = [990, 10, 0]
p = (0.05, 10.0, 0.25, 1000.0)
t = 1.0
nsteps = 40
ts = 0:nsteps

map_traj = iterate_model((s, p, dt) -> sir_map_step(s, p, dt), u0, p, t, nsteps)

nsims = 200
chain_ens = [iterate_model(sir_chain_step, u0, p, t, nsteps; extra = (StableRNG(seed),))
             for seed in 1:nsims]
chain_I = reduce(hcat, (traj[2, :] for traj in chain_ens))
chain_mean_I = vec(sum(chain_I; dims = 2)) ./ nsims
(round(maximum(map_traj[2, :]); digits = 1), round(maximum(chain_mean_I); digits = 1))

```

(200.9, 190.8)

```

using CairoMakie

fig = Figure(size = (720, 440))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "infectious I(t)",
          title = "Discrete-time SIR: map vs stochastic chain (t = 1)")
for j in 1:nsims
    lines!(ax, ts, chain_I[:, j]; color = (:grey, 0.12), linewidth = 1)
end
lines!(ax, ts, chain_mean_I; color = :firebrick, linewidth = 3, label = "chain mean")
lines!(ax, ts, map_traj[2, :]; color = :black, linestyle = :dash, linewidth = 3,
      label = "function map")
axislegend(ax; position = :rt)
fig

```

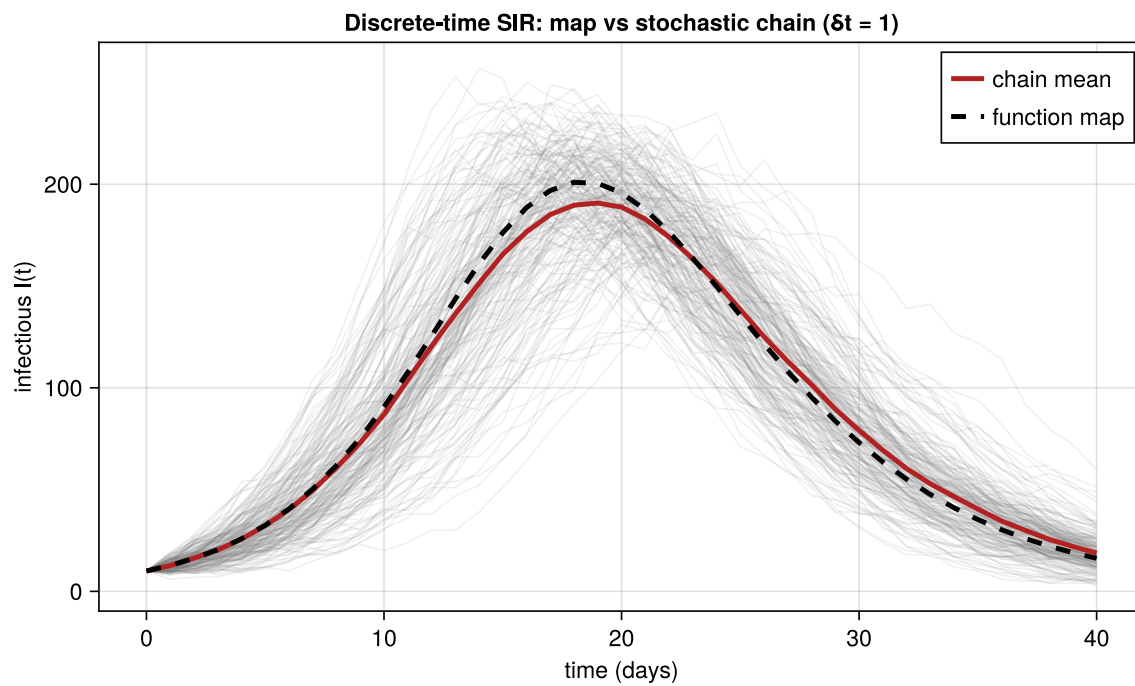


Figure 5.1: Infectious compartment: stochastic Markov-chain realisations (thin grey), their ensemble mean (red), and the deterministic function map (dashed black). The map closely tracks the stochastic mean.

5.4.1 Convergence of the map to the ODE

As $\delta t \rightarrow 0$ the deterministic map should approach the continuous-time ODE solution. We compare the map at the fixed horizon $t = 40$ against a high-accuracy `Tsit5` reference across a sequence of shrinking steps.

```
using OrdinaryDiffEq

function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, , N = p
    = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I
    return nothing
end

tmax = 40.0
ode_ref = solve(ODEProblem(sir_ode!, float.(u0), (0.0, tmax), p), Tsit5();
    reltol = 1e-12, abstol = 1e-12, save_everystep = false).u[end]

ts = [tmax / 2^k for k in 5:13]
map_errors = map(ts) do dt
    n = round{Int, tmax / dt}
    traj = iterate_model((s, p, d) -> sir_map_step(s, p, d), u0, p, dt, n)
    maximum(abs.(traj[:, end] .- ode_ref))
end
[(round(dt; sigdigits = 3), round(err; sigdigits = 3)) for (dt, err) in zip(ts, map_errors)]
```

```
9-element Vector{Tuple{Float64, Float64}}:
 (1.25, 71.3)
 (0.625, 39.2)
 (0.312, 20.6)
 (0.156, 10.6)
 (0.0781, 5.34)
 (0.0391, 2.69)
 (0.0195, 1.35)
 (0.00977, 0.675)
 (0.00488, 0.338)
```

```
fig2 = Figure(size = (640, 420))
ax2 = Axis(fig2[1, 1]; xscale = log10, yscale = log10,
```

```

        xlabel = "step size t", ylabel = "||map - ODE|| at t = 40",
        title = "Map → ODE convergence")
scatter!(ax2, ts, map_errors; markersize = 12, color = :steelblue, label = "measured")
guide = map_errors[1] .* (ts ./ ts[1])
lines!(ax2, ts, guide; linestyle = :dash, color = :black, label = "slope 1")
axislegend(ax2; position = :rb)
fig2

```

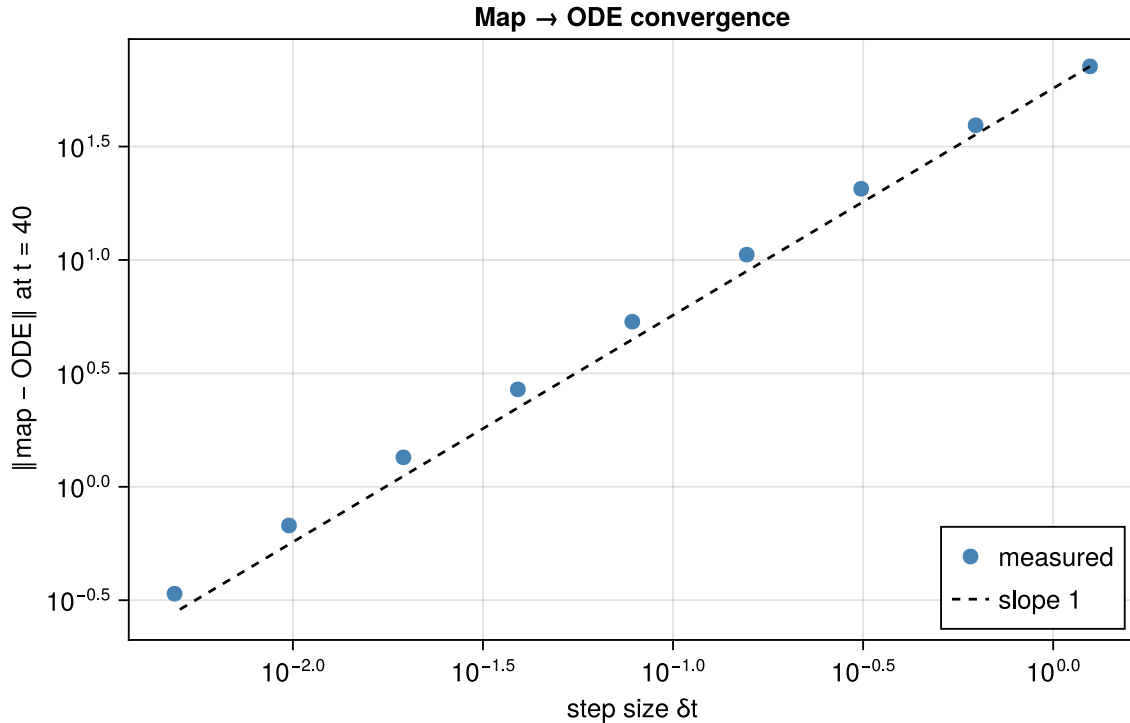


Figure 5.2: Error of the deterministic map at $t = 40$ relative to the ODE solution, versus step size (log-log). The points parallel the slope-1 guide, confirming first-order convergence of the map to the ODE.

5.5 Validation

Conservation. Both the deterministic map and every stochastic realisation must keep the population at $N = 1000$:

```
map_ok = all(k -> isapprox(sum(map_traj[:, k]), 1000.0; atol = 1e-8), 1:nsteps+1)
chain_ok = all(traj -> all(k -> sum(traj[:, k]) == 1000, 1:nsteps+1), chain_ens)
(map_ok, chain_ok)
```

```
(true, true)
```

Convergence. The empirical order of the map-to-ODE error should be close to 1, matching the forward-Euler analysis of Chapter 3:

```
orders = [log2(map_errors[k] / map_errors[k+1]) for k in 1:length(map_errors)-1]
mean_order = sum(orders) / length(orders)
(round(mean_order; digits = 3), isapprox(mean_order, 1.0; atol = 0.1))
```

```
(0.965, true)
```

Both cells confirm the expected behaviour: population is conserved in every trajectory, and the map converges to the ODE at first order.

5.6 Interpretation and limitations

The three models form a coherent family. The Markov chain is the most faithful: it produces integer counts, allows an epidemic to fade out by chance, and shows run-to-run variability that shrinks as the population grows. The deterministic map is its mean-field shadow — cheap and smooth, but blind to extinction and fluctuations. Both are only *approximations* to the continuous process, exact only as $\delta t \rightarrow 0$; at $\delta t = 1$ day the discretisation error is visible but modest. A subtle limitation of the binomial construction is that it processes all infections and recoveries within a step simultaneously, so an individual cannot become infected *and* recover in the same step — an artefact that vanishes as the step shrinks. The exact event-driven treatment is the subject of the continuous-time Markov chain chapter. See the [Distributions.jl documentation](#) and the [SciML FunctionMap solver](#) for the corresponding library tools.

5.7 Exercises

5.7.1 Exercise 1

Run the stochastic chain ensemble at $\delta t = 1$ and again at $\delta t = 0.25$ (with the horizon fixed at 40 days). Compare the ensemble-mean infectious peak against the deterministic map at each step size, and explain which discretisation sits closer to the ODE reference and why.

5.7.2 Exercise 2

Compute, from the 200-member ensemble, the probability that the epidemic goes extinct (reaches $I = 0$) before day 40 for the baseline $R_0 = 2$. Then lower the contact rate to $c = 6$ and recompute. Explain the change in terms of the reproduction number and demographic stochasticity — a phenomenon the deterministic map cannot capture.

5.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

6 Ordinary differential equations

Chapter Chapter 4 solved the SIR model to establish the theory; this chapter is about the *tools*. We work through the `OrdinaryDiffEq` workflow in depth: defining an `ODEProblem`, choosing the `Tsit5` solver, evaluating the dense continuous solution with interpolation, and — most importantly — using **callbacks** to inject events such as a time-limited lockdown and to detect the epidemic peak automatically. These techniques recur in every dynamical chapter that follows.

6.1 Learning goals

By the end of this chapter you should be able to:

- assemble an `ODEProblem` from a kernel, initial condition, timespan, and parameters;
- solve it with `Tsit5` and control accuracy through `reltol/abstol`;
- evaluate the dense interpolant `sol(t)` at arbitrary times;
- apply a `PresetTimeCallback` to change a parameter mid-simulation (a lockdown);
- use a `ContinuousCallback` to detect an event (the infectious peak) exactly.

6.2 Model assumptions and classification

The base model is the deterministic SIR system of Equation 4.1, unchanged. For the intervention example we make the transmission parameter time-dependent: during a lockdown window $[t_1, t_2]$ the per-contact transmission probability β drops to a reduced value β_{lock} , then returns. The model remains deterministic and continuous; the only new ingredient is a piecewise-constant parameter.

Property	Classification
Determinism	Deterministic
Time	Continuous
State	Continuous
Parameters	Piecewise-constant (lockdown example)

6.3 Equations

The governing equations are those of Chapter 4, written here with an explicit time-dependent force of infection:

$$\frac{dS}{dt} = -\lambda(t)S, \quad \frac{dI}{dt} = \lambda(t)S - \gamma I, \quad \frac{dR}{dt} = \gamma I, \quad \lambda(t) = \frac{\beta(t) c I}{N}. \quad (6.1)$$

The lockdown makes β a step function of time:

$$\beta(t) = \begin{cases} \beta_{\text{lock}}, & t_1 \leq t < t_2, \\ \beta, & \text{otherwise.} \end{cases} \quad (6.2)$$

The infectious peak is the instant at which growth turns to decline, characterised by

$$\frac{dI}{dt} = \lambda(t)S - \gamma I = 0, \quad (6.3)$$

which a `ContinuousCallback` can locate to solver tolerance by root-finding on this expression.

6.4 Julia implementation

6.4.1 Problem, solver, and interpolation

An `ODEProblem` bundles the kernel, initial state `u0`, timespan `tspan`, and parameters `p`. `Tsit5` is a 5th-order adaptive Runge–Kutta method suitable for non-stiff problems like SIR.

```
using OrdinaryDiffEq

function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, , N = p
    = * c * I / N          # eq-ch03-sir
    du[1] = - * S
    du[2] = * S - * I
    du[3] = * I
    return nothing
end

u0 = [990.0, 10.0, 0.0]
```

```

p = (0.05, 10.0, 0.25, 1000.0) # , c, , N
tspan = (0.0, 80.0)

prob = ODEProblem(sir_ode!, u0, tspan, p)
sol = solve(prob, Tsit5(); reltol = 1e-8, abstol = 1e-8)
length(sol.t) # number of adaptive steps chosen by the solver

```

133

Because `Tsit5` produces a *dense* output, `sol(t)` returns a high-order interpolant at any time inside the span — no need to have saved that point. This is how we later resample a solution onto a regular observation grid.

```

grid = 0.0:1.0:80.0
I_on_grid = [sol(t)[2] for t in grid] # interpolated infectious counts
(sol(12.5)[2], I_on_grid[13])

```

(120.78978226167975, 114.01974681005916)

6.4.2 A lockdown via `PresetTimeCallback`

A `PresetTimeCallback` fires the `affect!` function at prescribed times. Here it overwrites the mutable parameter vector to enact Equation 6.2: β drops to 0.025 over days 10–30, then recovers. We use a `Vector` for `p` so the callback can mutate it.

```

using DiffEqCallbacks

t1, t2 = 10.0, 30.0
_base, _lock = 0.05, 0.025

function lockdown!(integrator)
    integrator.p[1] = integrator.t < t2 ? _lock : _base
end
cb = PresetTimeCallback([t1, t2], lockdown!)

p_vec = [0.05, 10.0, 0.25, 1000.0]
prob_lock = ODEProblem(sir_ode!, u0, tspan, p_vec)
sol_lock = solve(prob_lock, Tsit5(); callback = cb, reltol = 1e-8, abstol = 1e-8)
(maximum(sol[2, :]), maximum(sol_lock[2, :])) # peak I: baseline vs lockdown

```

(158.34324529582247, 86.0394161043159)

6.4.3 Peak detection via ContinuousCallback

ContinuousCallback root-finds a scalar condition and fires when it crosses zero. Setting the condition to dI/dt (Equation 6.3) locates the infectious peak exactly, without scanning saved points.

```
peak_times = Float64[]
peak_values = Float64[]
dIdt(u, p) = p[1] * p[2] * u[2] / p[4] * u[1] - p[3] * u[2] # eq-ch03-peak
peak_cb = ContinuousCallback(
    (u, t, integrator) -> dIdt(u, integrator.p),
    integrator -> (push!(peak_times, integrator.t);
                   push!(peak_values, integrator.u[2])))

solve(ODEProblem(sir_ode!, u0, tspan, collect(p)), Tsit5();
      callback = peak_cb, reltol = 1e-8, abstol = 1e-8)
(peak_times, peak_values)
```

```
([17.51507384016969], [158.45157770119272])
```

```
using CairoMakie

tt = 0.0:0.25:80.0
I_base = [sol(t)[2] for t in tt]
I_lock = [sol_lock(t)[2] for t in tt]

fig = Figure(size = (720, 440))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "infectious I(t)",
          title = "SIR with a lockdown intervention")
vspan!(ax, t1, t2; color = (:grey, 0.15))
lines!(ax, tt, I_base; color = :firebrick, linewidth = 3, label = "no intervention")
lines!(ax, tt, I_lock; color = :steelblue, linewidth = 3, linestyle = :dash,
       label = "lockdown (days 10-30)")
vlines!(ax, peak_times; color = :black, linestyle = :dot, label = "detected peak")
axislegend(ax; position = :rt)
fig
```

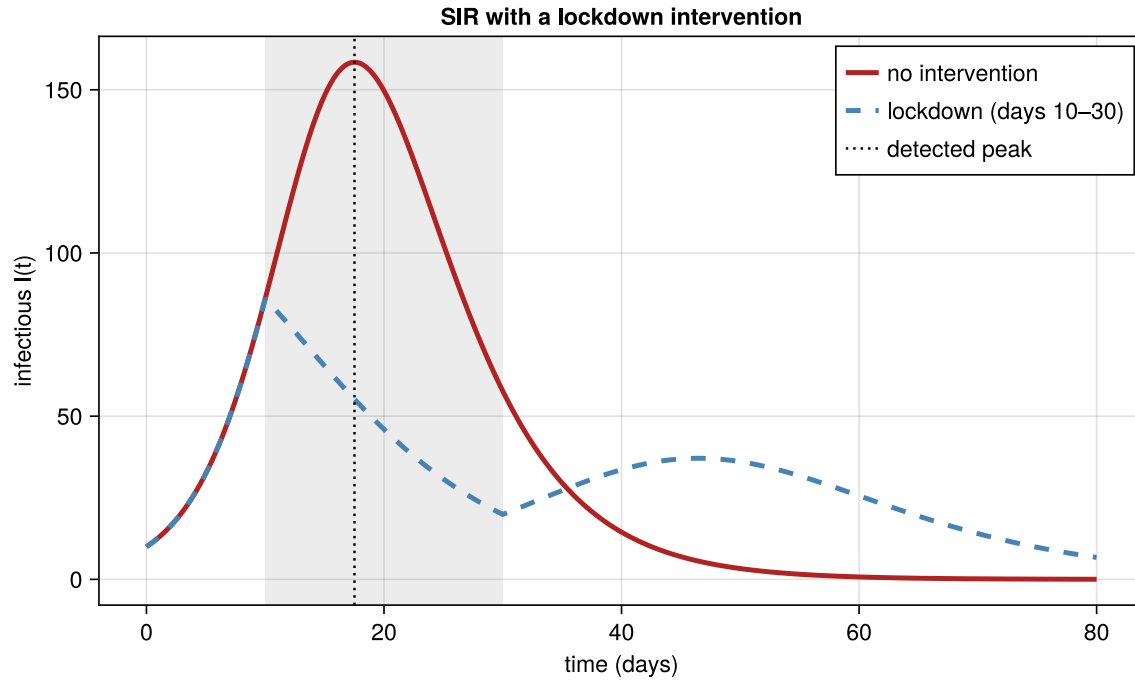


Figure 6.1: Infectious curve without intervention (solid red) and with a lockdown reducing β by half over days 10–30 (dashed blue, shaded window). The vertical line marks the callback-detected baseline peak. The lockdown flattens and delays the peak.

6.5 Validation

Conservation. The adaptive solution must conserve the population at every step (Equation 4.2):

```
all(u -> isapprox(sum(u), 1000.0; atol = 1e-6), sol.u)
```

true

Interpolation consistency. The dense interpolant evaluated at a saved time must reproduce the stored state, confirming $\text{sol}(t)$ is faithful:

```
t_check = sol.t[10]
isapprox(sol(t_check), sol.u[10]; atol = 1e-8)
```

true

Peak detection. The callback-detected peak must coincide with the maximum of a densely sampled solution and satisfy $dI/dt \approx 0$:

```
dense_t = 0.0:0.01:80.0
dense_I = [sol(t)[2] for t in dense_t]
brute_peak_time = dense_t[argmax(dense_I)]
residual = dIdt(sol(peak_times[1]), collect(p))
(isapprox(peak_times[1], brute_peak_time; atol = 0.05), isapprox(residual, 0.0; atol = 1e-4))
```

(true, true)

All three checks return **true**: the solver conserves population, its interpolant is self-consistent, and the event-detected peak matches the brute-force maximum while zeroing the growth rate.

6.6 Interpretation and limitations

The `OrdinaryDiffEq` workflow separates *what* the model is (the kernel and problem) from *how* it is solved (the algorithm and tolerances) and *what happens during* the solve (callbacks). This separation scales: the same `ODEProblem` can be handed to a stiff solver, differentiated for sensitivity analysis, or fitted to data with no change to the kernel. Callbacks turn a plain integrator into an event-driven simulator — lockdowns, vaccination pulses, and threshold-triggered interventions all follow the same pattern. The limitations are those of the deterministic SIR model itself (Chapter 4): no stochasticity, homogeneous mixing, exponential infectious periods. The lockdown here is a deterministic, perfectly-timed switch; realistic interventions involve uncertainty and feedback, taken up in the interventions and control chapter. For the full API see the [OrdinaryDiffEq documentation](#) and the [event-handling and callback guide](#).

6.7 Exercises

6.7.1 Exercise 1

Extend the lockdown study by sweeping the intervention strength: for $\beta_{\text{lock}} \in \{0.045, 0.035, 0.025, 0.015\}$ over the same window 10–30, solve the model and record the peak infectious count and its timing using the `ContinuousCallback`. Describe how intervention strength trades off peak height against peak timing.

6.7.2 Exercise 2

Add a fourth state C for cumulative incidence with $dC/dt = \lambda(t)S$, solve the baseline model, and use the dense interpolant `sol(t)` to compute daily incidence as the difference $C(t) - C(t-1)$ on the integer-day grid. Verify that summing the daily increments recovers the final cumulative incidence $C(80)$.

6.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

7 Continuous-time Markov chains and jump processes

Jump models are useful when the event itself is the object of interest. Instead of moving the SIR state a small distance at every numerical time step, we wait for the next infection or recovery and move the integer state by one. This is the continuous-time Markov-chain (CTMC) view of the epidemic.

7.1 Learning goals

By the end of this chapter, you should be able to:

- translate SIR transitions into propensities and stoichiometric change vectors;
- explain the Gillespie direct stochastic simulation algorithm (SSA);
- construct a current Catalyst reaction network and simulate it with JumpProcesses; and
- use population conservation and an ODE ensemble comparison as checks on a stochastic implementation.

7.2 Model assumptions and classification

We use the same closed, homogeneously mixing population as in the ODE chapter. There are $N = 1000$ people, with $S(0) = 990$, $I(0) = 10$, and $R(0) = 0$. Each infectious person makes $c = 10$ potentially infectious contacts per unit time, a contact transmits with probability $\beta = 0.05$, and an infectious person recovers at rate $\gamma = 0.25$. Recovered people do not return to the susceptible class; births, deaths, age structure, and imported infections are omitted.

This is a **stochastic, continuous-time, discrete-state, population-level** model. It is population-level because the state records counts, not identities. It is a CTMC because, conditional on the current counts, the future does not depend on the earlier path. The two event types are

event	state change ν_j	propensity $a_j(S, I, R)$
infection $S + I \rightarrow 2I$	$(-1, +1, 0)$	$a_1 = \kappa SI$

event	state change ν_j	propensity $a_j(S, I, R)$
recovery $I \rightarrow R$	$(0, -1, +1)$	$a_2 = \gamma I$

Here $\kappa = \beta c/N$. The division by N makes the total infection rate comparable across population sizes: every susceptible has force of infection $\beta c I/N$.

7.3 Equations

Let $X(t) = (S(t), I(t), R(t))^T$ and let Y_1, Y_2 be independent unit-rate Poisson processes. The random time-change representation is

$$X(t) = X(0) + \nu_1 Y_1 \left(\int_0^t a_1(X(s^-)) ds \right) + \nu_2 Y_2 \left(\int_0^t a_2(X(s^-)) ds \right).$$

The left limit $X(s^-)$ matters: the propensity is evaluated immediately before an event. Equivalently, if the current state is x , the total event rate is $a_0(x) = a_1(x) + a_2(x)$. The next waiting time is

$$\tau \sim \text{Exponential}(a_0(x)), \quad \Pr(J = j \mid x) = \frac{a_j(x)}{a_0(x)}.$$

This is the direct Gillespie algorithm (Gillespie 1977). Applying $x \leftarrow x + \nu_J$ and repeating gives an exact simulation of this CTMC (up to floating-point arithmetic).

Taking expected changes per unit time gives the familiar mean-field equations,

$$\frac{d}{dt} \begin{pmatrix} S \\ I \\ R \end{pmatrix} = \begin{pmatrix} -\kappa SI \\ \kappa SI - \gamma I \\ \gamma I \end{pmatrix}. \quad (7.1)$$

Both the CTMC and Equation Equation 7.1 conserve $S + I + R = N$. The equality of their *means* is not exact at finite population size because nonlinear terms such as SI require a moment closure: $\mathbb{E}[SI] \neq \mathbb{E}[S]\mathbb{E}[I]$.

7.4 Julia implementation

The course environment contains Catalyst 16.2.1 and JumpProcesses 9.29.1. The reaction network below is deliberately visible: its species are the coordinates of X , and its reaction rates are the propensities above. Catalyst's current `JumpProblem` constructor creates the compatible `JumpProcesses` representation; `Direct()` selects Gillespie's direct aggregator and `SSAS stepper()` advances from jump to jump. No fixed numerical step is used to define the event process.

```
using CairoMakie
using Catalyst
using JumpProcesses
using OrdinaryDiffEq
using StableRNGs
using Statistics

const N = 1000
const β = 0.05
const c = 10.0
const γ = 0.25
const μ = β * c / N
const u = [:S => 990, :I => 10, :R => 0]
const tspan = (0.0, 40.0)
const times = collect(0.0:0.2:40.0)

sir_network = @reaction_network begin
    , S + I --> 2I
    , I --> R
end
nothing
```

The symbols `β` and `μ` in the network are parameters. The initial state is integer-valued because a half-person is not a valid CTMC state. We pass parameter values by name, which is safer than relying on the order of a positional vector.

```
jump_parameters = [:β => β, :μ => μ]
jump_problem = JumpProblem(
    sir_network,
    u,
    tspan,
    jump_parameters;
    aggregator = Direct(),
```

```

    rng = StableRNG(20250711),
)
jump_solution = solve(jump_problem, SSAS stepper(); saveat = times)
jump_states = Array(jump_solution(times))

```

3×201 Matrix{Int64}:

990	990	990	989	989	989	988	988	...	277	275	274	273	273	273	272
10	9	7	8	8	8	8	6		30	29	26	24	23	23	20
0	1	3	3	3	3	4	6		693	696	700	703	704	704	708

The `JumpProblem` stores the CTMC and the random-number generator. Calling `solve` returns a piecewise-constant solution: the state changes at event times and remains unchanged between events. `saveat` samples that path for plotting; it does not turn the model into a time-stepped approximation.

For comparison, the deterministic right-hand side uses exactly the drift in Equation Equation 7.1:

```

function sir_drift!(du, u, p, t)
    , = p
    infection = * u[1] * u[2]
    recovery = * u[2]
    du[1] = -infection
    du[2] = infection - recovery
    du[3] = recovery
    return nothing
end

ode_problem = ODEProblem(sir_drift!, [990.0, 10.0, 0.0], tspan, (, ))
ode_solution = solve(ode_problem, Tsit5(); saveat = times)
ode_states = Array(ode_solution)

```

3×201 Matrix{Float64}:

990.0	988.986	987.922	986.806	...	210.46	210.142	209.834
10.0	10.5017	11.0274	11.5781		15.3356	14.8978	14.472
0.0	0.512444	1.05057	1.6156		774.204	774.96	775.694

The `ODEProblem` uses floating-point counts because it represents expected population flow, whereas the `Catalyst` problem uses integer species counts. Keeping the two constructors adjacent makes the code-to-equation mapping explicit.

```

fig = Figure(size = (760, 430))
ax = Axis(fig[1, 1], xlabel = "time", ylabel = "people",
          title = "Gillespie SIR path")
labels = ["S (jump)", "I (jump)", "R (jump)"]
colors = [:steelblue, :firebrick, :seagreen]
for j in 1:3
    stairs!(ax, times, jump_states[j, :]; color = colors[j],
            linewidth = 2, label = labels[j])
end
lines!(ax, times, ode_states[2, :]; color = :black, linestyle = :dash,
        linewidth = 2, label = "I (ODE)")
axislegend(ax, position = :rt)
fig

```

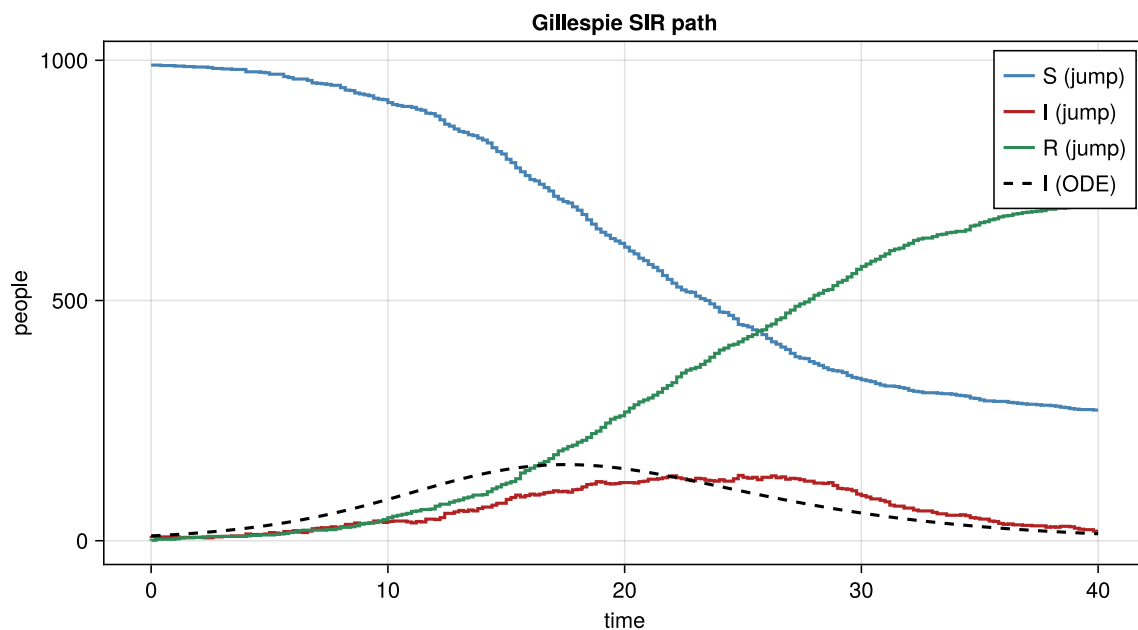


Figure 7.1: One exact CTMC SIR trajectory and its deterministic mean-field comparison.

7.5 Validation

The first check is structural rather than statistical: every reaction changes one compartment down and another up. Thus conservation should hold at every saved point.

```

mass_error = maximum(abs.(vec(sum(jump_states; dims = 1)) .- N))
minimum_count = minimum(jump_states)
@assert mass_error == 0
@assert minimum_count >= 0
(; mass_error, minimum_count, number_of_events = length(jump_solution.t) - 1)

```

```
(mass_error = 0, minimum_count = 0, number_of_events = 1626)
```

An individual path is not expected to lie on the ODE curve. A more informative check averages independent paths. Each repetition receives a fresh, deterministically chosen `StableRNG`, so the calculation is reproducible without changing Julia’s global random state.

```

function simulate_jump(seed)
    problem = JumpProblem(
        sir_network, u, tspan, jump_parameters;
        aggregator = Direct(), rng = StableRNG(seed),
    )
    solution = solve(problem, SSAS stepper(); saveat = times)
    return Array(solution(times))
end

ensemble = cat((simulate_jump(3000 + k) for k in 1:16)...; dims = 3)
ensemble_mean = dropdims(mean(ensemble; dims = 3), dims = 3)
infectious_rmse = sqrt(mean((ensemble_mean[2, :] .- ode_states[2, :]).^2))
@assert maximum(abs.(sum(ensemble; dims = 1) .- N)) == 0
infectious_rmse

```

```
6.240473193802095
```

With only 16 paths, the RMSE is a diagnostic rather than a precision estimate. Increasing the number of paths narrows Monte Carlo error approximately as $1/\sqrt{M}$, but costs proportionally more simulations.

7.6 Interpretation and limitations

The CTMC retains integer counts and event timing, so it can represent extinction, early fade-out, and variability in the epidemic peak. Its propensities still assume homogeneous mixing and exponential recovery times. Real contact networks, latent periods, reporting delays,

births, and time-varying interventions require additional states or a different model. For very large populations, exact SSA can spend most of its time processing events; tau-leaping, hybrid jump-ODE models, or an ODE may then be more appropriate, with an explicit approximation trade-off.

The [JumpProcesses continuous-time tutorial](#) and [Catalyst documentation](#) document the APIs used here. The event formulation follows Gillespie’s exact SSA (Gillespie 1977) and the epidemic interpretation follows the original SIR model (Kermack and McKendrick 1927).

7.7 Exercises

7.7.1 Exercise 1

Replace the recovery reaction by two sequential stages, $I_1 \rightarrow I_2 \rightarrow R$, with rates 2γ for both transitions. Add the fourth state to the Julia model, simulate 40 time units with a fixed seed, and verify both $S + I_1 + I_2 + R = N$ and nonnegative integer states. Explain why two exponential stages have a less variable infectious duration than one exponential stage with mean $1/\gamma$.

7.7.2 Exercise 2

Use the low-level `ConstantRateJump(rate, affect!)` interface from `JumpProcesses` to implement a one-person birth-death toy process, then use the same direct-SSA ideas to add a vaccination jump $S \rightarrow R$ to SIR. State the new propensity, update the ODE comparison, and discuss how a vaccination rate that depends on calendar time would change the jump type or simulation setup.

7.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

8 Stochastic differential equations

A jump process resolves every infection and recovery. A stochastic differential equation (SDE) instead describes continuous-valued fluctuations around a drift. This is useful when the population is large enough for a diffusion approximation to be reasonable, and when we want to connect stochastic simulation with the calculus and solvers used for ODEs.

8.1 Learning goals

You will:

- derive the chemical Langevin approximation from SIR reaction propensities;
- distinguish drift, diffusion columns, and independent Wiener processes;
- solve a non-diagonal-noise SDE with the current StochasticDiffEq API; and
- diagnose conservation and the possibility of negative populations.

8.2 Model assumptions and classification

The baseline is the closed, homogeneously mixing SIR population $(S, I, R) = (990, 10, 0)$ with $N = 1000$, $c = 10.0$, $\beta = 0.05$, and $\gamma = 0.25$. We retain the two transitions $S + I \rightarrow 2I$ and $I \rightarrow R$, but replace integer event counts by a diffusion.

This is a **stochastic, continuous-time, continuous-state, population-level** model. “Continuous state” means the numerical approximation can produce real values, not that people are fractional. It is an approximation to the discrete-state CTMC from Chapter 4, and is most defensible when propensities are large and the process is away from absorbing boundaries.

8.3 Equations

Write the stoichiometric columns and propensities as

$$V = \begin{pmatrix} -1 & 0 \\ 1 & -1 \\ 0 & 1 \end{pmatrix}, \quad a(X) = \begin{pmatrix} \kappa SI \\ \gamma I \end{pmatrix}, \quad \kappa = \frac{\beta c}{N}.$$

For a short interval dt , a Poisson event count with mean $a_j dt$ can be approximated by

$$\text{Poisson}(a_j dt) \approx a_j dt + \sqrt{a_j} dW_j,$$

where dW_j are independent Brownian increments with variance dt . Substituting both reactions into the time-change equation gives the chemical Langevin equation

$$dX = Va(X) dt + V \text{diag}(\sqrt{a(X)}) dW. \quad (8.1)$$

Thus the drift $f(X) = Va(X)$ is

$$f(X) = (-a_1, a_1 - a_2, a_2)^T,$$

and the 3×2 diffusion matrix $G(X)$ has columns

$$G_{.1} = (-\sqrt{a_1}, \sqrt{a_1}, 0)^T, \quad G_{.2} = (0, -\sqrt{a_2}, \sqrt{a_2})^T.$$

Because each column of V sums to zero, both the drift and every Brownian increment conserve $S + I + R$ algebraically. Numerical integration can still accumulate roundoff error.

8.4 Julia implementation

`SDEProblem` accepts an in-place drift `f!` and diffusion `g!`. A vector-valued state with a 3×2 diffusion requires `noise_rate_prototype=zeros(3, 2)`: two columns correspond to the two independent Wiener processes. `LambaEM()` is the `StochasticDiffEq` method for Euler–Maruyama with general (non-diagonal) noise in the installed 7.1.1 environment. The unusual spelling is part of the package API.

```
using CairoMakie
using OrdinaryDiffEq
using StochasticDiffEq
using StableRNGs

const N = 1000.0
const dt = 0.05
const c = 10.0
const beta = 0.25
const gamma = * c / N
const u = [990.0, 10.0, 0.0]
const tspan = (0.0, 30.0)
const times = collect(0.0:0.1:30.0)
```



```

function propensities(u, p)
    , = p
    S, I = max(u[1], 0.0), max(u[2], 0.0)
    return ( * S * I, * I)
end

```

propensities (generic function with 1 method)

The max operations make the code well-defined if an Euler step crosses a boundary. They do not repair the interpretation of a negative state; the boundary issue is discussed below.

```

function sir_drift!(du, u, p, t)
    infection, recovery = propensities(u, p)
    du[1] = -infection
    du[2] = infection - recovery
    du[3] = recovery
    return nothing
end

function sir_diffusion!(G, u, p, t)
    infection, recovery = propensities(u, p)
    infection_noise = sqrt(infection)
    recovery_noise = sqrt(recovery)
    G[1, 1] = -infection_noise
    G[2, 1] = infection_noise
    G[3, 1] = 0.0
    G[1, 2] = 0.0
    G[2, 2] = -recovery_noise
    G[3, 2] = recovery_noise
    return nothing
end

noise_shape = zeros(3, 2)
sde_problem = SDEProblem(
    sir_drift!, sir_diffusion!, u, tspan, (, );
    noise_rate_prototype = noise_shape,
)

sde_solution = solve(
    sde_problem, LambdaEM();
    dt = 0.02, adaptive = false, saveat = times,

```

```

    rng = StableRNG(20250711),
)
sde_states = Array(sde_solution)

```

3×301 Matrix{Float64}:

```

990.0  989.411    989.623    988.439    ...  182.568    183.329    183.136
 10.0    9.86467    10.446    11.4065    ...    23.2206    21.7437    20.9243
  0.0    0.723904   -0.0688276    0.15447    ...   794.212    794.927    795.939

```

The function `sir_diffusion!` is a direct transcription of the two columns of G in Equation Equation 8.1. It is intentionally not replaced by independent noise on S , I , and R : doing that would break the reaction-level correlation and population conservation.

For a baseline comparison, solve the drift alone:

```

ode_problem = ODEProblem(sir_drift!, u, tspan, (, ))
ode_solution = solve(ode_problem, Tsit5(); saveat = times)
ode_states = Array(ode_solution)

mass_error = maximum(abs.(vec(sum(sde_states; dims = 1)) .- N))
minimum_state = minimum(sde_states)
@assert mass_error < 1e-8
(; mass_error, minimum_state)

```

```
(mass_error = 2.7284841053187847e-12, minimum_state = -0.06882763332898889)
```

The ODE is not the expected trajectory of every SDE path, but it is the large-population drift comparison. The conservation assertion is stronger: it checks a structural property of the implementation independently of the random realization.

```

fig = Figure(size = (760, 430))
ax = Axis(fig[1, 1], xlabel = "time", ylabel = "people",
    title = "Chemical Langevin SIR")
for (j, (name, color)) in enumerate(zip(["S", "I", "R"],
    [:steelblue, :firebrick, :seagreen]))
    lines!(ax, times, sde_states[j, :]; color, linewidth = 2, label = "$name (SDE)")
    lines!(ax, times, ode_states[j, :]; color, linestyle = :dash,
        linewidth = 1.5, label = "$name (ODE)")
end
axislegend(ax, position = :rt, nbanks = 2)
fig

```

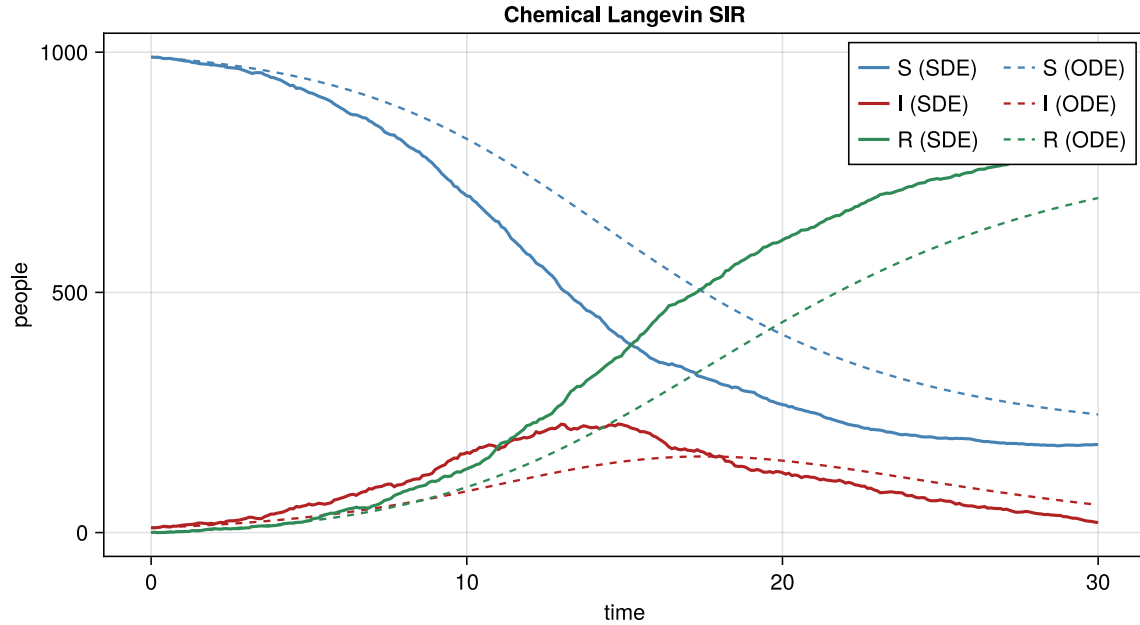


Figure 8.1: A chemical-Langevin SIR path compared with the deterministic drift.

8.5 Validation

The noise columns sum to zero, so conservation is a useful first test. We can also check the diffusion construction directly at one valid state:

```
G = zeros(3, 2)
sir_diffusion!(G, [900.0, 80.0, 20.0], (, ), 0.0)
@assert maximum(abs.(vec(sum(G; dims = 1)))) < 1e-12
@assert mass_error < 1e-8
```

At $N = 1000$, the path should broadly follow the ODE while showing realization-to-realization variation. A practical extension is to solve the same `sde_problem` repeatedly with different `StableRNG` seeds and compare the sample mean and standard deviation at the epidemic peak. Such an ensemble comparison tests the approximation statistically; it does not make the diffusion exact.

8.6 Interpretation and limitations

The chemical Langevin equation is derived by replacing Poisson reaction counts with Gaussian increments. It is therefore fast and differentiable in much of the interior of the state space, but it can produce $S < 0$, $I < 0$, or $R < 0$, especially near extinction, with large time steps, or in small populations. Clipping a trajectory after each step changes its probability law; it is a pragmatic visualization safeguard, not a principled boundary condition. Alternatives include the exact CTMC, nonnegative tau-leaping, reflecting/absorbing boundary schemes, or transforming the model.

`noise_rate_prototype` is an API-level detail with mathematical meaning: leaving it out would ask for diagonal noise, not the two reaction noises in Equation Equation 8.1. The [StochasticDiffEq SDE tutorial](#) documents `SDEProblem`, `EM`, and non-diagonal noise. The approximation itself is the chemical Langevin construction associated with reaction networks (Gillespie 1977).

8.7 Exercises

8.7.1 Exercise 1

Run the SDE with `dt` values 0.1, 0.05, and 0.01, using the same seed and `adaptive=false`. For each run, record the maximum conservation error and the minimum state value. Explain why decreasing `dt` usually reduces discretization error but cannot guarantee nonnegative populations for a Gaussian diffusion.

8.7.2 Exercise 2

Construct a diagonal-noise SDE that adds independent noise to S , I , and R , keeping the same drift. Compare its conservation error and infected trajectory with the reaction-level model. Identify the precise covariance term that is lost when the two reaction channels are replaced by three independent state noises.

8.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

9 Agent-based models

An agent-based model (ABM) represents a population as interacting individuals. The SIR labels remain familiar, but the model now exposes who contacts whom, when an agent changes status, and how an update schedule can affect results. This chapter uses a deliberately small, well-mixed ABM so that the connection to the population equations stays visible.

9.1 Learning goals

You will:

- classify an individual-level SIR model and distinguish it from a count model;
- implement a synchronous contact-and-recovery rule with `Agents.jl` 7;
- collect model-level data with the current `StandardABM` and `run!` APIs; and
- compare an ensemble of individual simulations with the deterministic SIR ODE while identifying effects that aggregation hides.

9.2 Model assumptions and classification

The population has $N = 1000$ named individuals. Initially 10 are infectious, 990 are susceptible, and none are recovered. At every $\Delta t = 0.1$ time unit, a susceptible makes $K \sim \text{Poisson}(c\Delta t)$ random contacts. A contact chooses a uniformly random individual and transmits with probability β if that individual is infectious. An infectious agent recovers with probability $1 - \exp(-\gamma\Delta t)$. Recovered agents are immune. We use $c = 10$, $\beta = 0.05$, and $\gamma = 0.25$.

This is a **stochastic, discrete-time, discrete-state, individual-level** model. Time is discrete because `StandardABM` advances in integer steps; state is discrete because each `Person.status` is one of `:S`, `:I`, or `:R`. The population is well mixed because the model has no spatial structure: `StandardABM(Person)` uses the `nothing` space. This is an ABM even though its aggregated contact rule is designed to approximate mass action.

9.3 Equations

Let $Z_i(t) \in \{S, I, R\}$ be individual i 's status and $I_t = \sum_i \mathbf{1}(Z_i(t) = I)$. For a susceptible, the probability of no successful transmission in one step, conditional on $K_i = k$, is

$$\left(1 - \beta \frac{I_t}{N}\right)^k.$$

Averaging over $K_i \sim \text{Poisson}(c\Delta t)$ gives

$$\Pr(Z_i(t + \Delta t) = I \mid Z_i(t) = S) = 1 - \exp\left[-c\Delta t \beta \frac{I_t}{N}\right]. \quad (9.1)$$

The recovery probability is

$$\Pr(Z_i(t + \Delta t) = R \mid Z_i(t) = I) = 1 - \exp(-\gamma\Delta t).$$

For small Δt , Equation Equation 9.1 is approximately $\beta c(I_t/N)\Delta t$, matching the SIR force of infection. The individual model contains more information than the population equations: the identity of every contact and every infection is retained. The ODE only tracks the totals and uses the deterministic flow

$$\dot{S} = -\beta c SI/N, \quad \dot{I} = \beta c SI/N - \gamma I, \quad \dot{R} = \gamma I.$$

9.4 Julia implementation

Agents.jl 7 requires the `@agent` macro for supported agent types. In the installed 7.0.3 API, `NoSpaceAgent` contributes the mandatory `id` field; `StandardABM(Person; agent_step!, model_step!, properties, rng)` constructs a discrete-time model. We use a two-phase update: `agent_step!` reads the old statuses and writes `next_status`, then `model_step!` commits all changes. Without this separation, activation order could let a newly infected agent transmit in the same step.

```
using Agents
using CairoMakie
using Distributions
using OrdinaryDiffEq
using StableRNGs
using Statistics

const N = 1000
```

```

const I = 10
const  = 0.05
const c = 10.0
const  = 0.25
const Δt = 0.1
const nsteps = 300
const times = collect(0.0:Δt:(nsteps * Δt))

@agent struct Person(NoSpaceAgent)
    status::Symbol
    next_status::Symbol
end

```

The contact loop uses the model's random-number generator through `abmrng(model)`. `random_agent(model)` is the current Agents.jl helper for uniformly selecting an agent from a no-space model.

```

function agent_step!(person, model)
    person.next_status = person.status
    if person.status == :S
        for _ in 1:rand(abmrng(model), Poisson(model.c * model.Δt))
            contact = random_agent(model)
            if contact.status == :I && rand(abmrng(model)) < model.
                person.next_status = :I
                break
            end
        end
    elseif person.status == :I
        if rand(abmrng(model)) < 1 - exp(-model. * model.Δt)
            person.next_status = :R
        end
    end
    return nothing
end

function model_step!(model)
    for person in allagents(model)
        person.status = person.next_status
    end
    return nothing
end

```

```

susceptible_count(model) = count(person.status == :S for person in allagents(model))
infected_count(model) = count(person.status == :I for person in allagents(model))
recovered_count(model) = count(person.status == :R for person in allagents(model))

```

recovered_count (generic function with 1 method)

The model properties are a named tuple, so `model.`, `model.c`, and `model.Δt` are ordinary Julia field accesses. The `mdata` functions below are called by `run!` at the initial state and after each step.

```

function initialize_model(seed)
    model = StandardABM(
        Person;
        agent_step!,
        model_step!,
        properties = ( = , c = c, = , Δt = Δt),
        rng = StableRNG(seed),
    )
    for id in 1:N
        status = id <= I ? :I : :S
        add_agent!(Person(; id, status, next_status = status), model)
    end
    return model
end

model = initialize_model(20250711)
agent_data, model_data = run!(
    model,
    nsteps;
    mdata = [susceptible_count, infected_count, recovered_count],
    showprogress = false,
)

```

(0×0 DataFrame, 301×4 DataFrame

Row	time	susceptible_count	infected_count	recovered_count
	Int64	Int64	Int64	Int64
1	0	990	10	0
2	1	990	10	0

3	2	990	10	0
4	3	989	11	0
5	4	989	10	1
6	5	989	10	1
7	6	987	12	1
8	7	986	13	1
295	294	191	71	738
296	295	191	70	739
297	296	190	70	740
298	297	190	69	741
299	298	190	67	743
300	299	190	65	745
301	300	190	63	747

286 rows omitted)

`run!` returns two data frames: one for agent observations and one for model observations. We requested only model-level totals, which keeps the example small while retaining the full individual state inside the model. The first row is the initial condition because `init=true` is the default.

```
abm_states = Matrix(model_data[:, [
    :susceptible_count, :infected_count, :recovered_count,
]])'
abm_times = Float64.(model_data.time) .* Δt

function sir_drift!(du, u, p, t)
    , c, , N = p
    infection = * c * u[1] * u[2] / N
    du[1] = -infection
    du[2] = infection - * u[2]
    du[3] = * u[2]
    return nothing
end

ode_problem = ODEProblem(
    sir_drift!, [990.0, 10.0, 0.0], (0.0, nsteps * Δt), (, c, , N),
)
ode_solution = solve(ode_problem, Tsit5(); saveat = abm_times)
ode_states = Array(ode_solution)
```

3×301 Matrix{Float64}:

990.0	989.499	988.986	988.46	...	247.327	246.6	245.886
10.0	10.2479	10.5017	10.7615		59.1809	58.4369	57.7002
0.0	0.253087	0.512444	0.77822		693.492	694.963	696.414

The model time reported by a discrete `StandardABM` is the step number, so the explicit multiplication by Δt maps it to the physical time used by the ODE.

```
fig = Figure(size = (760, 430))
ax = Axis(fig[1, 1], xlabel = "time", ylabel = "people",
          title = "Well-mixed individual-based SIR")
for (j, (name, color)) in enumerate(zip(["S", "I", "R"],
                                       [:steelblue, :firebrick, :seagreen]))
    stairs!(ax, abm_times, abm_states[j, :]; color, linewidth = 2,
            label = "$name (ABM)")
    lines!(ax, abm_times, ode_states[j, :]; color, linestyle = :dash,
           linewidth = 1.5, label = "$name (ODE)")
end
axislegend(ax, position = :rt, nbanks = 2)
fig
```

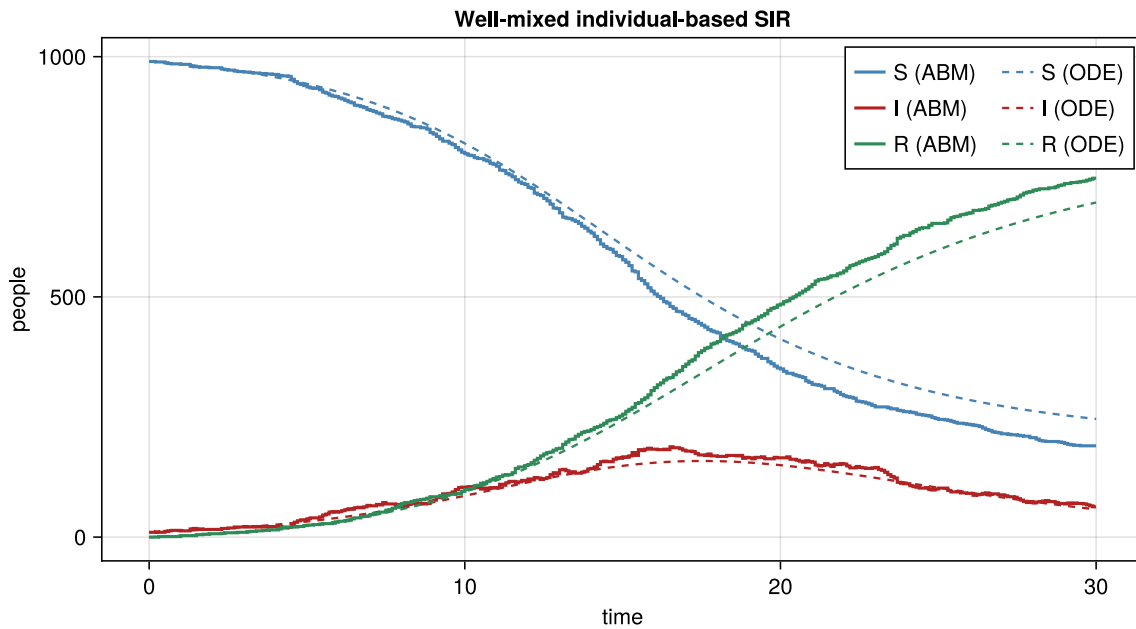


Figure 9.1: One individual-based SIR realization and the corresponding ODE trajectory.

9.5 Validation

The ABM has an exact bookkeeping invariant: every person has exactly one status. Consequently $S + I + R = N$ at every recorded step.

```
population_error = maximum(abs.(vec(sum(abm_states; dims = 1)) .- N))
@assert population_error == 0
@assert minimum(abm_states) >= 0
(; population_error, final_counts = abm_states[:, end])
```

```
(population_error = 0, final_counts = [190, 63, 747])
```

One realization can differ substantially from the ODE. To compare levels rather than a single random path, run a small deterministic ensemble. The model constructor gives each replicate its own seeded random stream.

```
function simulate_abm(seed)
    model = initialize_model(seed)
    _, data = run!(
        model,
        nsteps;
        mdata = [susceptible_count, infected_count, recovered_count],
        showprogress = false,
    )
    return Matrix{data[:, [
        :susceptible_count, :infected_count, :recovered_count,
    ]]}
end

abm_ensemble = cat((simulate_abm(4000 + k) for k in 1:16)...; dims = 3)
abm_mean = dropdims(mean(abm_ensemble; dims = 3), dims = 3)
infectious_rmse = sqrt(mean((abm_mean[2, :] .- ode_states[2, :]).^2))
@assert maximum(abs.(sum(abm_ensemble; dims = 1) .- N)) == 0
infectious_rmse
```

```
6.252915150099244
```

The comparison is intentionally modest: 16 paths are enough to reveal the relationship and keep a teaching render quick, but not enough for a narrow confidence interval.

9.6 Interpretation and limitations

The ABM can be extended by replacing the no-space model with a grid, network, or continuous space, or by adding age, behavior, vaccination, or household fields to **Person**. Those extensions change who can contact whom and can create clustering, superspreading, and spatial waves that a mass-action ODE cannot represent. The price is computational and inferential: agent rules need careful calibration, contact mechanisms can be difficult to observe, and random seeds do not remove structural uncertainty.

Our synchronous two-phase update avoids an order artifact, but the contact rule still samples with replacement and allows self-contact. For $N = 1000$ this is a small approximation; a network model would make contact constraints explicit. The model is also discrete in time, so Δt should be reduced and results checked for timestep sensitivity. Agents.jl's [agent definition documentation](#) and [data collection documentation](#) describe the v7 `@agent`, `StandardABM`, and `run!` APIs. The framework and its modeling rationale are described by Datseris, Vahdati, and DuBois (Datseris et al. 2022).

9.7 Exercises

9.7.1 Exercise 1

Change `initialize_model` to create a two-dimensional `GridSpace`, add a position field using the v7 `@agent` syntax, and let each susceptible contact only an agent in a Moore neighborhood. Keep the same infection and recovery parameters, verify $S + I + R = N$, and explain which assumption in Equation Equation 9.1 is no longer valid.

9.7.2 Exercise 2

Run the ABM with $\Delta t = 0.2, 0.1$, and 0.05 , keeping physical time fixed. For each value, use at least 16 seeded replicates and calculate the ensemble mean and standard deviation of the peak infectious count. Compare those summaries with the ODE peak and discuss whether changing the update interval changes numerical error, biological assumptions, or both.

9.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

10 Observation models and simulated data

10.1 Learning goals

Every earlier chapter built a *process model*: a set of rules that generate the latent trajectory of an epidemic. Data, however, are never the latent process. They are noisy, incomplete counts produced by surveillance systems. This chapter separates the two ideas explicitly and shows how to attach an **observation model** to a deterministic SIR process so that we can simulate realistic case reports. After working through it you should be able to

- distinguish a process model from an observation model and explain why the distinction matters for inference (Chapters 8 and 9);
- derive daily incidence from a cumulative-infection state and express it as the mean of a count distribution;
- implement Poisson and negative-binomial observation models in Julia with reproducible random streams; and
- reason about reporting fraction, overdispersion, and the limits of the independence assumptions baked into these models.

10.2 Model assumptions and classification

The process model is the standard homogeneously mixing SIR model of Kermack and McKendrick (1927), augmented with a cumulative-incidence state C . We reuse the baseline course scenario throughout: $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $N = 1000$, contact rate $c = 10$, transmission probability $\beta = 0.05$, and recovery rate $\gamma = 0.25$, giving a basic reproduction number $R_0 = \beta c / \gamma = 2$.

Aspect	Process model	Observation model
Determinism	deterministic	stochastic
Time	continuous	discrete (daily)
State	continuous ($S, I, R, C \in \mathbb{R}_{\geq 0}$)	discrete counts ($Y_t \in \mathbb{Z}_{\geq 0}$)
Representation	population densities	reported cases

The observation model rests on three assumptions. First, only *new* infections generate case reports, so the relevant signal is incidence, not prevalence. Second, each latent case is reported independently with probability ρ (the reporting fraction). Third, conditional on the latent incidence, the counts on different days are independent. The first two assumptions justify a Poisson count; relaxing the independence of individual reports motivates the negative-binomial extension.

10.3 Equations

The process model integrates

$$\begin{aligned}\frac{dS}{dt} &= -\beta c \frac{SI}{N}, & \frac{dI}{dt} &= \beta c \frac{SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, & \frac{dC}{dt} &= \beta c \frac{SI}{N},\end{aligned}\tag{10.1}$$

where $C(t)$ accumulates infections but never decreases. The **latent incidence** over day t is the increment of C ,

$$x_t = C(t) - C(t-1) = \int_{t-1}^t \beta c \frac{S(\tau)I(\tau)}{N} d\tau.\tag{10.2}$$

The Poisson observation model assumes each expected report equals the reported fraction of latent incidence,

$$Y_t \mid x_t \sim \text{Poisson}(\rho x_t), \quad \mathbb{E}[Y_t] = \text{Var}[Y_t] = \rho x_t.\tag{10.3}$$

Poisson counts are *equidispersed*: the variance equals the mean. Real surveillance data are usually **overdispersed**. The negative-binomial model adds a dispersion parameter $r > 0$,

$$Y_t \mid x_t \sim \text{NegBin}\left(r, \frac{r}{r + \rho x_t}\right), \quad \mathbb{E}[Y_t] = \rho x_t, \quad \text{Var}[Y_t] = \rho x_t + \frac{(\rho x_t)^2}{r},\tag{10.4}$$

so the variance exceeds the mean by $(\rho x_t)^2/r$, and the Poisson model is recovered as $r \rightarrow \infty$.

10.4 Julia implementation

The process kernel `sir_cumulative_ode!` and the reproducible data generator `synthetic_incidence` are provided by the course package `EpiModelingCourse`, so the same code backs the chapter, its slides, and the companion notebook. We solve the process model with `OrdinaryDiffEq.jl`, draw observations with `Distributions.jl`, and keep randomness reproducible with `StableRNGs.jl`.

```
using EpiModelingCourse
using OrdinaryDiffEq
using DelayDiffEq
using DiffEqCallbacks
using Distributions
using StableRNGs
using CairoMakie

CairoMakie.activate!(type = "png")
set_theme!(course_theme())
palette = course_palette()
```

10.4.1 The latent process

We integrate Equation 10.1 over 40 days with the baseline parameters and extract daily incidence by differencing C (Method 1). Because the only exit from S is infection, cumulative incidence must equal susceptible depletion, $C(t) = S(0) - S(t)$, which we will use as a validation check.

```
u0 = [990.0, 10.0, 0.0, 0.0]      # S, I, R, C
p = BASELINE_PARAMETERS          # β = 0.05, c = 10, γ = 0.25, N = 1000
tspan = (0.0, 40.0)
obstimes = 1.0:1.0:40.0

prob = ODEProblem(sir_cumulative_ode!, u0, tspan, p)
sol = solve(prob, Tsit5(); saveat = 1.0, abstol = 1e-10, reltol = 1e-10)

cumulative = Array(sol)[4, :]
incidence = cumulative[2:end] .- cumulative[1:(end - 1)] # latent x
```

```
40-element Vector{Float64}:
 5.590627042896126
```

```

7.076171294825152
8.908980455243878
11.141969943458843
13.819448899255434
16.965754398783254
20.570071409364267
24.568881578709423
28.829775047152225
33.14295618856971

```

```

5.621804404168643
4.81275798341278
4.121732886951122
3.53140545442875
3.0269166362709257
2.595578857993928
2.2265881414878095
1.9107579060835178
1.6402813574532047

```

10.4.2 Poisson and negative-binomial observations

With the latent incidence in hand, the observation model is just a draw from a count distribution. We use a full reporting fraction ($\rho = 1$) here so that the observation noise, rather than under-reporting, is isolated. Each observation model gets its own `StableRNG` so the streams are independent and reproducible.

```

reporting_fraction = 1.0
expected = reporting_fraction .* incidence

poisson_obs = rand.(StableRNG(20240607), Poisson.(expected))

# NegativeBinomial(r, p) with mean uses p = r / (r + ); r sets dispersion.
dispersion = 10.0
nb( , r) = NegativeBinomial(r, r / (r + ))
negbin_rng = StableRNG(20240608)
negbin_obs = [rand(negbin_rng, nb( , dispersion)) for in expected]

(mean_poisson = mean(poisson_obs), var_poisson = var(poisson_obs),
 mean_negbin = mean(negbin_obs), var_negbin = var(negbin_obs))

```



```
(mean_poisson = 19.35, var_poisson = 236.8487179487179, mean_negbin = 21.05, var_negbin = 34
```

The sample variance of the Poisson draws tracks their mean, while the negative-binomial draws are visibly overdispersed, exactly as Equation 10.3 and Equation 10.4 predict.

```
fig = Figure(size = (900, 380))
ax1 = Axis(fig[1, 1]; xlabel = "Time (days)", ylabel = "New cases per day",
           title = "Poisson observations")
barplot!(ax1, obstimes, poisson_obs; color = (palette.incidence, 0.5))
lines!(ax1, obstimes, incidence; color = palette.infected, linewidth = 3,
       label = "Latent incidence")
axislegend(ax1; position = :rt)

ax2 = Axis(fig[1, 2]; xlabel = "Time (days)", ylabel = "New cases per day",
           title = "Negative-binomial observations (r = $(Int(dispersion)))")
barplot!(ax2, obstimes, negbin_obs; color = (palette.incidence, 0.5))
lines!(ax2, obstimes, incidence; color = palette.infected, linewidth = 3)
fig
```

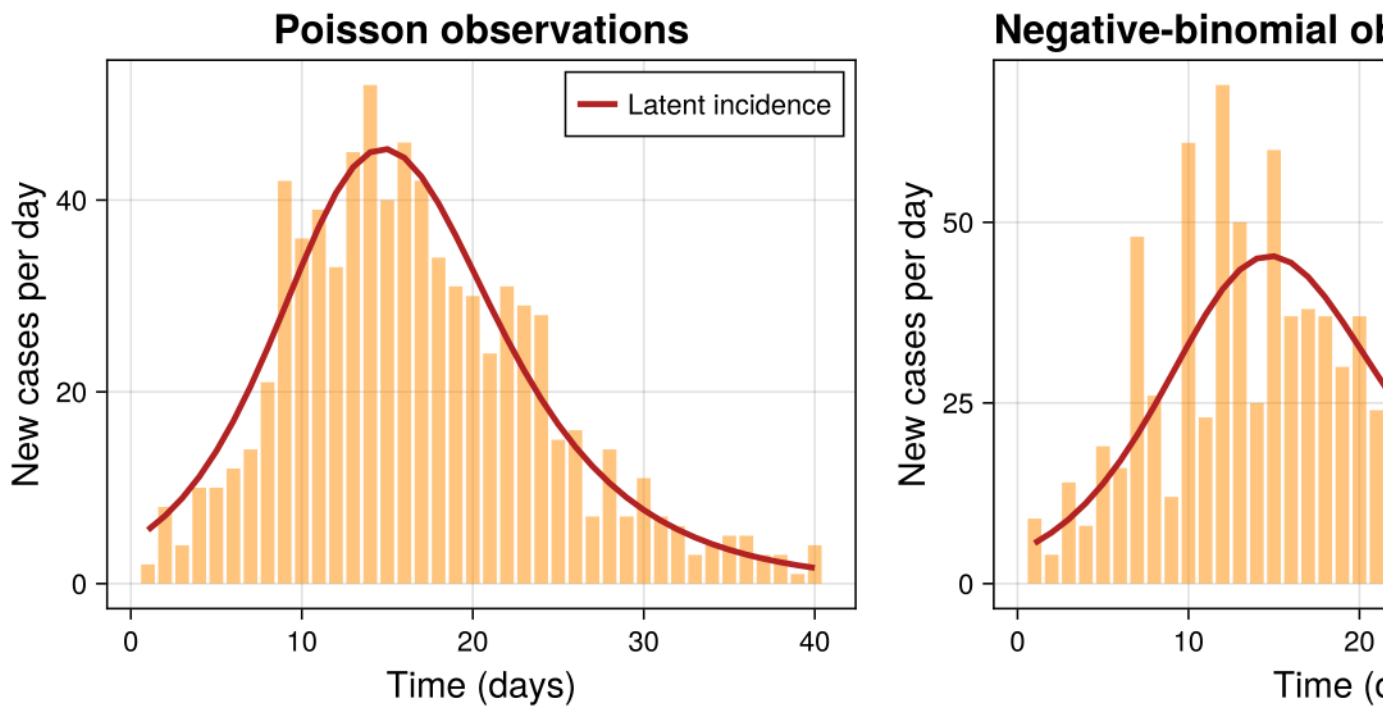


Figure 10.1: Latent daily incidence (line) with one realisation of Poisson (left) and overdispersed negative-binomial (right) case reports. Both share the same latent mean; the negative-binomial scatter is wider.

10.4.3 Alternative ways to obtain daily incidence

Differencing the cumulative state (Method 1) is the most robust route, but two other constructions appear in practice. A [PresetTimeCallback](#) can reset C to zero at each observation time so that the saved value *is* the daily increment (Method 2). A delay differential equation solved with [DelayDiffEq.jl](#) can track a sliding one-day window directly (Method 3). We implement both to compare them against Method 1.

```
# Method 2: reset the cumulative counter at each observation time.
reset_C!(integrator) = integrator.u[4] = 0.0
cb = PresetTimeCallback(collect(obstimes), reset_C!)
sol_cb = solve(prob, Tsit5(); saveat = 1.0, callback = cb,
    abstol = 1e-10, reltol = 1e-10)
incidence_cb = sol_cb(obstimes)[4, :]

# Method 3: a sliding-window delay differential equation for daily incidence.
```

```

function sir_window_dde!(du, u, h, p, t)
    S, I, R, C = u
    N = p.N
    infection = p. * p.c * S * I / N
    recovery = p. * I
    past = h(p, t - 1.0)          # state one day earlier
    past_infection = p. * p.c * past[1] * past[2] / N
    du[1] = -infection
    du[2] = infection - recovery
    du[3] = recovery
    du[4] = infection - past_infection
    return nothing
end
history(p, t) = zeros(4)
prob_dde = DDEProblem(sir_window_dde!, u0, history, tspan, p; constant_lags = [1.0])
sol_dde = solve(prob_dde, MethodOfSteps(Tsit5()))
incidence_dde = sol_dde(obstimes)[4, :]

(method2_max_gap = maximum(abs.(incidence_cb .- incidence)),
 method3_max_gap = maximum(abs.(incidence_dde .- incidence)))

```

```
(method2_max_gap = 1.0794401816838217e-8, method3_max_gap = 0.008337220645323384)
```

Method 2 agrees with Method 1 to solver tolerance. Method 3 differs by roughly one case near the epidemic peak: the delayed state is interpolated rather than differenced, so it is a genuinely different (though closely related) quantity. For downstream inference we therefore prefer Method 1.

10.5 Validation

We check three structural properties. First, conservation: cumulative incidence must equal susceptible depletion. Second, non-negativity: sampled counts are never negative. Third, the law of large numbers: averaging many independent Poisson realisations must converge to the latent mean Equation 10.3.

```

S = Array(sol)[1, :]
conservation_error = maximum(abs.(cumulative .- (990.0 .- S)))

replicates = [rand.(StableRNG(seed), Poisson.(expected)) for seed in 1:2000]

```

```
mc_mean = reduce(+, replicates) ./ length(replicates)
lln_error = maximum(abs.(mc_mean .- expected))

(conservation_error = conservation_error,
 counts_nonnegative = all(poisson_obs .>= 0) && all(negbin_obs .>= 0),
 lln_error = lln_error)
```

```
(conservation_error = 5.506706202140776e-13, counts_nonnegative = true, lln_error = 0.220724)
```

The conservation error sits at the level of the ODE tolerances, all counts are non-negative, and the Monte Carlo mean of 2000 Poisson realisations reproduces the latent incidence to within a fraction of a case.

10.6 Interpretation and limitations

The observation model is deliberately simple, and each assumption is a modelling choice a practitioner must defend. Because ρ and the transmission rate both scale the expected count, a *single* incidence series cannot separate a small epidemic reported fully from a large epidemic reported partially — an identifiability problem revisited in the sensitivity and identifiability chapter. The Poisson model forces variance to equal the mean, so it understates uncertainty whenever reporting is clustered, batched, or driven by super-spreading; the negative-binomial model in Equation 10.4 is the usual remedy, at the cost of one extra parameter. We also assumed instantaneous reporting: real systems impose delays between infection and notification, which the delay formulation of Method 3 hints at but does not fully capture. Finally, conditional independence across days ignores weekday reporting artefacts and autocorrelated reporting effort. These simulated datasets are nonetheless the raw material for the inference chapters that follow: Chapter 8 estimates parameters from them by maximum likelihood, and Chapter 9 does so with Bayesian methods.

10.7 Exercises

10.7.1 Exercise 1

Using the baseline scenario and Method 1, generate case reports under a partial reporting fraction $\rho = 0.4$ with the Poisson observation model. Plot the latent incidence and the reported counts on the same axes, and compute the total number of latent versus reported cases over the 40 days. Explain why the reported peak looks like a smaller epidemic and why ρ and β cannot be distinguished from the reported series alone.

10.7.2 Exercise 2

Fix the mean of the negative-binomial observation model to the latent incidence and vary the dispersion parameter over $r \in \{1, 5, 50\}$. For each value, draw 1000 replicate time series and estimate the ratio of the sample variance to the sample mean at the epidemic peak. Confirm numerically that the ratio approaches 1 as r grows, connecting your result to the variance formula in Equation [10.4](#).

10.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

11 Frequentist parameter estimation

11.1 Learning goals

Chapter 7 produced simulated case reports from a known process. We now reverse the arrow: given only the reports, what can we say about the parameters that generated them? This chapter takes the *frequentist* view, in which parameters are fixed unknowns and uncertainty is expressed through the sampling behaviour of estimators. By the end you should be able to

- write the likelihood of a Poisson observation model coupled to an SIR ODE;
- compute the maximum-likelihood estimate (MLE) with [Optimization.jl](#);
- construct **profile-likelihood** confidence intervals with [ProfileLikelihood.jl](#) and explain the χ^2 threshold behind them; and
- recognise how parameter non-identifiability shows up as a flat or ridged profile.

Note

This chapter runs in the isolated environment `environments/ch08`, pinned via the document's `julia.exeflags`. That environment carries a mutually compatible set of `ProfileLikelihood`, `Optimization`, and `OrdinaryDiffEq` versions, so the model kernel is defined locally rather than imported from the course package.

11.2 Model assumptions and classification

The process model is the deterministic SIR ODE with cumulative incidence C , identical to Equation 10.1. The observation model is Poisson, as in Equation 10.3 with full reporting. We estimate two parameters: the transmission probability β and the initial infected fraction $i_0 = I(0)/N$, holding the contact rate $c = 10$ and recovery rate $\gamma = 0.25$ fixed. The estimation procedure itself is deterministic given the data — the optimiser and the profile routine return point estimates and interval endpoints, not a distribution.

Aspect	Choice
Process	deterministic SIR ODE (continuous time and state)
Observation	Poisson counts of daily incidence

Aspect	Choice
Unknowns	$\theta = (i_0, \beta)$
Inference	maximum likelihood + profile-likelihood intervals

11.3 Equations

Writing $x_t(\theta)$ for the model incidence on day t (the increment of C), the Poisson log-likelihood of the observed counts $y_{1:n}$ is

$$\ell(\theta) = \sum_{t=1}^n \left[y_t \log x_t(\theta) - x_t(\theta) - \log(y_t!) \right]. \quad (11.1)$$

The maximum-likelihood estimate is $\hat{\theta} = \arg \max_{\theta} \ell(\theta)$. To quantify uncertainty in a single component, say β , we *profile out* the remaining parameters by maximising over them at each fixed value of β :

$$\ell_p(\beta) = \max_{i_0} \ell(i_0, \beta). \quad (11.2)$$

By Wilks' theorem the profile deviance $2(\ell(\hat{\theta}) - \ell_p(\beta))$ is asymptotically χ_1^2 , so an approximate 95% confidence interval is the set of β with

$$\ell_p(\beta) \geq \ell(\hat{\theta}) - \frac{1}{2}\chi_{1,0.95}^2, \quad \frac{1}{2}\chi_{1,0.95}^2 \approx 1.92, \quad (11.3)$$

the method of Venzon and Moolgavkar (1988) implemented by `ProfileLikelihood.jl`.

11.4 Julia implementation

```
using OrdinaryDiffEq
using ProfileLikelihood
using Optimization
using OptimizationOptimJL
using Distributions
using StableRNGs
using DataFrames
using CairoMakie
```

```
CairoMakie.activate!(type = "png")
colors = (data = :grey30, truth = :firebrick, mle = :steelblue, band = (:steelblue, 0.25))
```

11.4.1 Process kernel and simulated data

```
function sir_ode!(du, u, p, t)
    S, I, R, C = u
    , c, = p
    N = S + I + R
    infection = * c * I / N * S
    recovery = * I
    @inbounds begin
        du[1] = -infection
        du[2] = infection - recovery
        du[3] = recovery
        du[4] = infection
    end
    return nothing
end

t = 1.0
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0, 0.0] # S, I, R, C
p = [0.05, 10.0, 0.25] # , c, ; truth: i = 0.01, = 0.05

prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol = solve(prob_ode, Tsit5(); saveat = t)
incidence = Array(sol)[4, 2:end] .- Array(sol)[4, 1:(end - 1)]
data = rand.(StableRNG(20240607), Poisson.(incidence))
```

```
40-element Vector{Int64}:
```

```
 2
 9
 3
10
13
 7
21
```


22
42
30

7
5
4
6
3
3
2
1
2

11.4.2 The log-likelihood via the integrator interface

`ProfileLikelihood.jl` evaluates Equation 11.1 through the SciML integrator `integrator`: for each candidate θ we reset the initial state and parameters, re-solve, and accumulate the Poisson log-density. Non-positive model incidence (which can occur for extreme parameters) is rejected by returning `-Inf`.

```
function loglik(, data, integrator)
    i, =
    integrator.p[1] =
    integrator.p[2] = 10.0
    integrator.p[3] = 0.25
    I = i * 1000.0
    reinit!(integrator, [1000.0 - I, I, 0.0, 0.0])
    solve!(integrator)
    C = Array(integrator.sol)[4, :]
    X = C[2:end] .- C[1:(end - 1)]
    any(X .<= 0) && return -Inf
    return sum(logpdf.(Poisson.(X), data))
end
```

`loglik` (generic function with 1 method)

11.4.3 Maximum-likelihood estimate

We assemble a `LikelihoodProblem` with bounds on both parameters and optimise with the derivative-free Nelder–Mead method.

```

lb = [0.0, 0.0]
ub = [1.0, 1.0]
    = [0.01, 0.1]

prob = LikelihoodProblem(
  loglik, , sir_ode!, u0, tspan;
  syms = [:i, :],
  data = data,
  ode_parameters = p,
  ode_kwargs = (verbose = false, saveat = t),
  f_kwargs = (adtype = Optimization.AutoFiniteDiff()),
  prob_kwargs = (lb = lb, ub = ub),
  ode_alg = Tsit5(),
)

mle_sol = mle(prob, NelderMead())
^ = get_mle(mle_sol)

```

```

2-element Vector{Float64}:
 0.009338624611670208
 0.050043985790199585

```

11.4.4 Profile-likelihood intervals

```

prof = profile(prob, mle_sol; alg = NelderMead(), parallel = false)
confints = get_confidence_intervals(prof)

results = DataFrame(
  parameter = ["i ", " "],
  truth = [0.01, 0.05],
  mle = ^,
  ci_lower = [confints[i][1] for i in 1:2],
  ci_upper = [confints[i][2] for i in 1:2],
)
results

```

	parameter	truth	mle	ci_lower	ci_upper
	String	Float64	Float64	Float64	Float64
1	i	0.01	0.00933862	0.00719275	0.0119879
2		0.05	0.050044	0.0480045	0.0521355

The profile curves make the χ^2 threshold of Equation 11.3 concrete. Each panel plots the normalised profile log-likelihood (zero at the MLE); the interval is where the curve lies above the dashed line at -1.92 .

```
threshold = -0.5 * quantile(Chisq(1), 0.95)
truth = [0.01, 0.05]
names = ["i ", " "]

fig = Figure(size = (900, 380))
for i in 1:2
    grid = get_parameter_values(prof, i)
    prof_vals = get_profile_values(prof, i)
    ax = Axis(fig[1, i]; xlabel = names[i], ylabel = "Profile log-likelihood")
    lines!(ax, grid, prof_vals; color = colors.mle, linewidth = 3)
    hlines!(ax, [threshold]; color = :black, linestyle = :dash)
    vlines!(ax, [truth[i]]; color = colors.truth, linewidth = 2)
    vlines!(ax, [confints[i][1], confints[i][2]]; color = colors.mle,
        linestyle = :dot)
    ylims!(ax, threshold - 1.5, 0.4)
end
fig
```

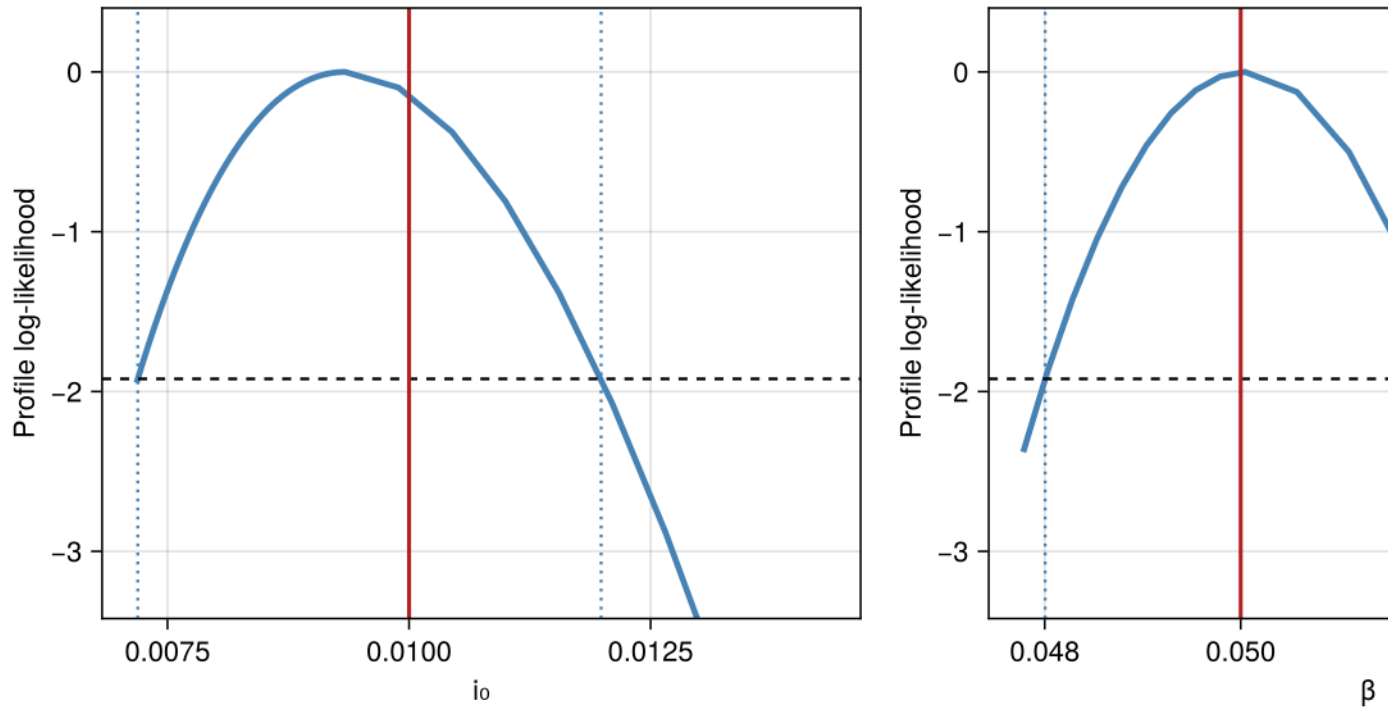


Figure 11.1: Profile log-likelihoods for i_0 and β . The dashed line is the 95% threshold from Wilks' theorem; its intersections with each curve are the confidence-interval endpoints (blue), which bracket the true values (red).

11.5 Validation

Two checks confirm the fit. First, the estimator should recover the truth: the MLE should be close to $(i_0, \beta) = (0.01, 0.05)$ and both confidence intervals should contain the true value. Second, the fitted incidence curve should track the data.

```
covers(i) = confints[i][1]  truth[i]  confints[i][2]
(mle = ^, covers_i = covers(1), covers_ = covers(2))
```

```
(mle = [0.009338624611670208, 0.050043985790199585], covers_i = true, covers_ = true)
```

```
function incidence_at()
  i, =
  I = i * 1000.0
```

```

    prob_fit = remake(prob_ode; u0 = [1000.0 - I, I, 0.0, 0.0], p = [ , 10.0, 0.25])
    solf = solve(prob_fit, Tsit5()); saveat = t)
    C = Array(solf)[4, :]
    return C[2:end] .- C[1:(end - 1)]
end

obstimes = t:t:tmax
fig = Figure(size = (760, 420))
ax = Axis(fig[1, 1]; xlabel = "Time (days)", ylabel = "New cases per day")
barplot!(ax, collect(obstimes), data; color = (colors.data, 0.4), label = "Data")
lines!(ax, collect(obstimes), incidence_at(truth); color = colors.truth,
        linewidth = 3, label = "Truth")
lines!(ax, collect(obstimes), incidence_at(^); color = colors.mle,
        linewidth = 3, linestyle = :dash, label = "MLE")
axislegend(ax; position = :rt)
fig

```

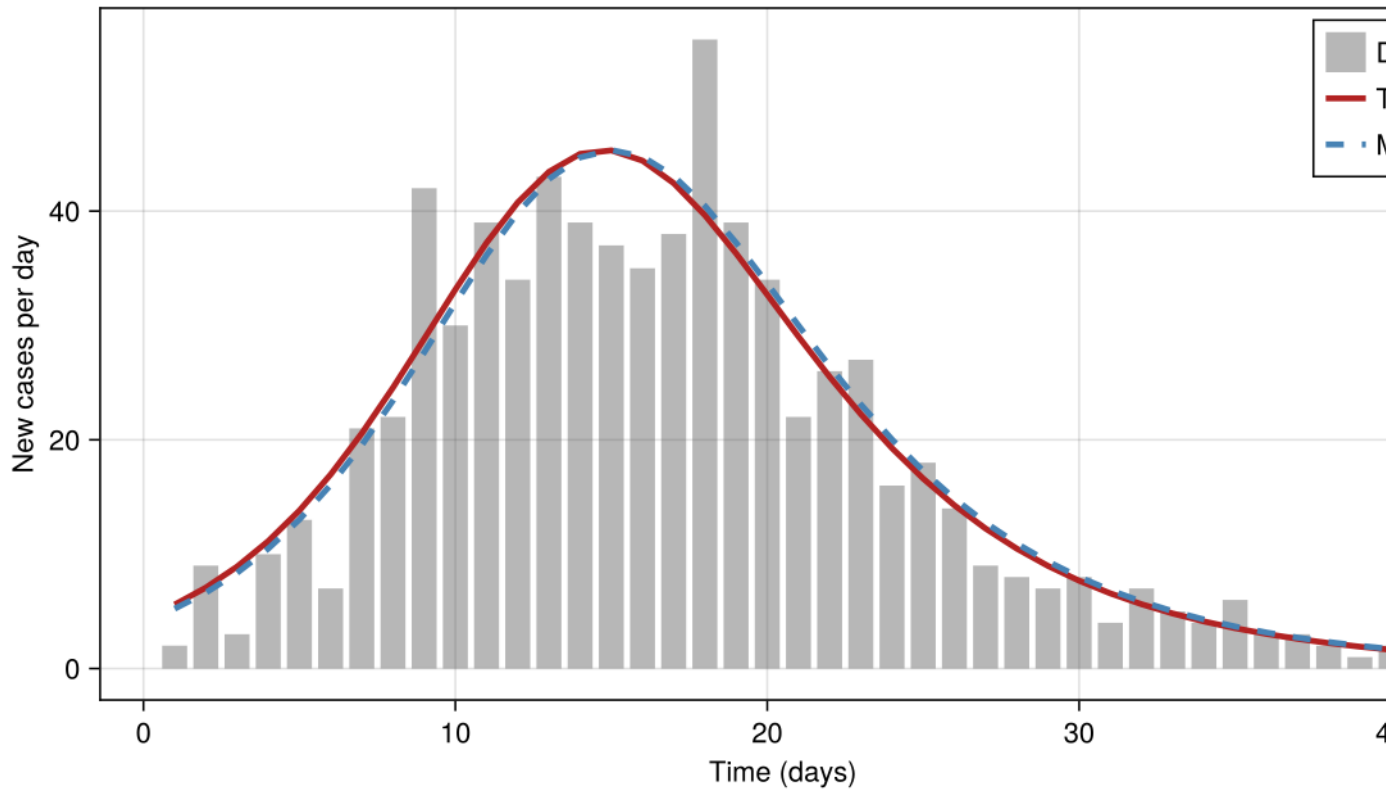


Figure 11.2: Simulated case reports (grey), the incidence curve at the true parameters (red), and at the maximum-likelihood estimate (blue, dashed). The fit is visually indistinguishable from the truth.

11.6 Interpretation and limitations

The MLE recovers the generating parameters and the profile intervals cover them, but three caveats matter in practice. First, Nelder–Mead is a *local* optimiser; a poor start or a multimodal likelihood can strand it on a secondary maximum, so a grid or multistart search is prudent before trusting a single fit. Second, the χ^2 calibration of Equation 11.3 is asymptotic; with few or highly informative data points the intervals may under- or over-cover, and a full likelihood-region or bootstrap approach is more faithful. Third, we fixed c and γ : because transmission enters only through the product βc , β and c are not separately identifiable from incidence alone, and the estimate of β should be read as an estimate of $\beta c/c$ at the assumed c . Structural identifiability is treated systematically in the sensitivity and identifiability chapter. Finally, the frequentist intervals summarise sampling variability but do not incorporate prior knowledge;

the next chapter revisits the same model from a Bayesian perspective, where uncertainty is expressed directly as a posterior distribution.

11.7 Exercises

11.7.1 Exercise 1

Re-simulate the data with a different `StableRNG` seed and re-fit the model. Repeat this for 50 seeds, collecting the MLE of β each time. Plot the distribution of estimates and report the fraction of the 50 profile-likelihood intervals for β that contain the true value 0.05. Comment on whether the empirical coverage is close to the nominal 95%.

11.7.2 Exercise 2

Extend the parameter vector to also estimate the recovery rate γ (add it to `beta`, the log-likelihood, and the bounds). Compute the profile likelihood for γ and compare the width of its interval with those for i_0 and β . Explain, with reference to Equation 11.2, what a comparatively flat profile implies about the information the incidence data carry about γ .

11.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

12 Bayesian inference

12.1 Learning goals

Chapter 8 treated the parameters as fixed unknowns and summarised uncertainty through the sampling distribution of an estimator. The Bayesian approach instead treats the parameters as random variables and updates a **prior** belief into a **posterior** using the data (Gelman et al. 2013). This chapter fits the same SIR model to the same kind of simulated case reports, but with [Turing.jl](#). By the end you should be able to

- state Bayes’ theorem for the SIR observation model and identify the prior, likelihood, and posterior;
- write a probabilistic model with the `@model` macro and sample it with the No-U-Turn Sampler (NUTS);
- assess convergence with $\hat{R}$ and effective sample size using [MCMCChains.jl](#); and
- summarise parameter uncertainty and propagate it into a **posterior predictive** check.

12.2 Model assumptions and classification

The process and observation models are unchanged from the previous two chapters: a deterministic SIR ODE with cumulative incidence Equation 10.1 and a Poisson observation model Equation 10.3 with full reporting. As in Chapter 8 we infer $\theta = (i_0, \beta)$ with $c = 10$ and $\gamma = 0.25$ fixed. The difference is entirely in the treatment of uncertainty.

Aspect	Frequentist (Ch. 8)	Bayesian (this chapter)
Parameters	fixed unknowns	random variables with a prior
Output	point estimate + confidence interval	posterior distribution
Uncertainty	sampling variability of the estimator	posterior probability
Engine	optimisation (<code>Optimization.jl</code>)	MCMC sampling (<code>Turing.jl</code>)

12.3 Equations

Bayes' theorem combines the prior density $p(\theta)$ with the likelihood $p(y_{1:n} \mid \theta)$ — the same Poisson likelihood whose logarithm is Equation 11.1 — to give the posterior

$$p(\theta \mid y_{1:n}) = \frac{p(y_{1:n} \mid \theta) p(\theta)}{p(y_{1:n})}, \quad p(y_{1:n}) = \int p(y_{1:n} \mid \theta) p(\theta) d\theta. \quad (12.1)$$

The normalising constant $p(y_{1:n})$ is intractable here, so we never compute it directly; instead NUTS draws samples $\theta^{(1)}, \dots, \theta^{(m)}$ whose histogram approximates the posterior. We place weakly informative Beta priors that respect the natural $[0, 1]$ range of both parameters,

$$i_0 \sim \text{Beta}(1, 20), \quad \beta \sim \text{Beta}(1, 10), \quad (12.2)$$

with prior means $1/21 \approx 0.048$ and $1/11 \approx 0.091$: broad enough to be uninformative near the truth ($i_0 = 0.01$, $\beta = 0.05$) but supported only on plausible values. Any posterior expectation, such as the predictive incidence, is an integral against the posterior that we estimate by averaging over the draws.

12.4 Julia implementation

```
using OrdinaryDiffEq
using SciMLSensitivity
using Turing
using MCMCChains
using Distributions
using StableRNGs
using CairoMakie

CairoMakie.activate!(type = "png")
colors = (data = :grey20, post = :steelblue, band = (:steelblue, 0.3), truth = :firebrick)
```

12.4.1 Process kernel and simulated data

We reuse the deterministic kernel and the Poisson observation model to generate a reproducible dataset, exactly as in Chapter 8.

```

function sir_ode!(du, u, p, t)
    S, I, R, C = u
    , c, = p
    N = S + I + R
    infection = * c * I / N * S
    recovery = * I
    @inbounds begin
        du[1] = -infection
        du[2] = infection - recovery
        du[3] = recovery
        du[4] = infection
    end
    return nothing
end

tmax = 40.0
tspan = (0.0, tmax)
obstimes = 1.0:1.0:tmax
u0 = [990.0, 10.0, 0.0, 0.0]
p = [0.05, 10.0, 0.25]

prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol = solve(prob_ode, Tsit5(); saveat = 1.0)
incidence = Array(sol)[4, 2:end] .- Array(sol)[4, 1:(end - 1)]
data = rand.(StableRNG(20240607), Poisson.(incidence))

```

40-element Vector{Int64}:

```

2
8
4
10
10
12
14
21
42
36

6
3
4

```

5
5
3
3
1
4

12.4.2 The probabilistic model

The `@model` macro turns Equation 12.1 and Equation 12.2 into executable code. The `~` statements declare the priors; the loop declares the Poisson likelihood of each observation given the model incidence. We reject failed ODE solves and floor the Poisson rate by a negligible constant so that the sampler never evaluates `Poisson(0)` on the burned-out tail of the epidemic.

```
@model function bayes_sir(y, prob)
  l = length(y)
  i ~ Beta(1.0, 20.0)
  ~ Beta(1.0, 10.0)

  I = i * 1000.0
  u0 = [1000.0 - I, I, 0.0, 0.0]
  = [ , 10.0, 0.25]
  prob = remake(prob; u0 = u0, p = , tspan = (0.0, float(1)))
  sol = solve(prob, Tsit5(); saveat = 1.0)
  if !SciMLBase.successful_retcode(sol)
    Turing.@addlogprob! -Inf
    return nothing
  end

  C = Array(sol)[4, :]
  X = C[2:end] .- C[1:(end - 1)]
  for i in 1:l
    y[i] ~ Poisson(max(X[i], 0.0) + 1e-6)
  end
  return nothing
end
```

bayes_sir (generic function with 2 methods)

12.4.3 Sampling with NUTS

We draw three independent chains of 500 iterations each. Multiple chains are what make the $\hat{R}$ diagnostic meaningful, and NUTS uses automatic differentiation through the ODE solver, enabled here by [SciMLSensitivity.jl](#). The `StableRNG` seed makes the run reproducible; the sample counts are deliberately modest so the chapter renders quickly in continuous integration.

```
model = bayes_sir(data, prob_ode)
chain = sample(StableRNG(2), model, NUTS(0.8), MCMCSerial(), 500, 3;
              chain_type = MCMCChains.Chains, progress = false)
summarize(chain)
```

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	
i	0.0091	0.0011	0.0001	438.6383	526.6950	1.0119	
	0.0500	0.0010	0.0000	458.1696	478.6566	1.0070	

1 column omitted

12.5 Validation

MCMC results are only trustworthy once the chains have converged. The potential scale reduction factor $\hat{R}$ should be close to 1 (values above ≈ 1.01 suggest the chains disagree), and the effective sample size (ESS) should be comfortably large. We also confirm that the posterior concentrates near the true parameters.

```
truth = (i = 0.01, s = 0.05)
i0_mat = Array(chain[:, :i, :])
s_mat = Array(chain[:, :s, :])
i0s = vec(i0_mat)
s = vec(s_mat)

(max_rhat = maximum(rhat(chain).nt.rhat),
 min_ess = minimum(ess(chain).nt.ess),
 posterior_mean = (i = round(mean(i0s); digits = 4),
                  s = round(mean(s); digits = 4)),
 truth = truth)
```

```
(max_rhat = 1.0119284999004239, min_ess = 438.6383389673823, posterior_mean = (i = 0.0091,
```

The trace plots should look like well-mixed “fuzzy caterpillars” that overlap across chains, and the marginal densities should be unimodal and bracket the true values.

```
fig = Figure(size = (900, 560))
for (row, (name, mat, tru)) in enumerate(("i ", i0_mat, truth.i ),
                                         (" ", _mat, truth.)))
  axt = Axis(fig[row, 1]; xlabel = "Iteration", ylabel = name)
  for ch in 1:size(mat, 2)
    lines!(axt, 1:size(mat, 1), mat[:, ch])
  end
  axd = Axis(fig[row, 2]; xlabel = name, ylabel = "Density")
  density!(axd, vec(mat); color = (colors.post, 0.4), strokecolor = colors.post,
           strokewidth = 2)
  vlins!(axd, [tru]; color = colors.truth, linewidth = 2)
end
fig
```

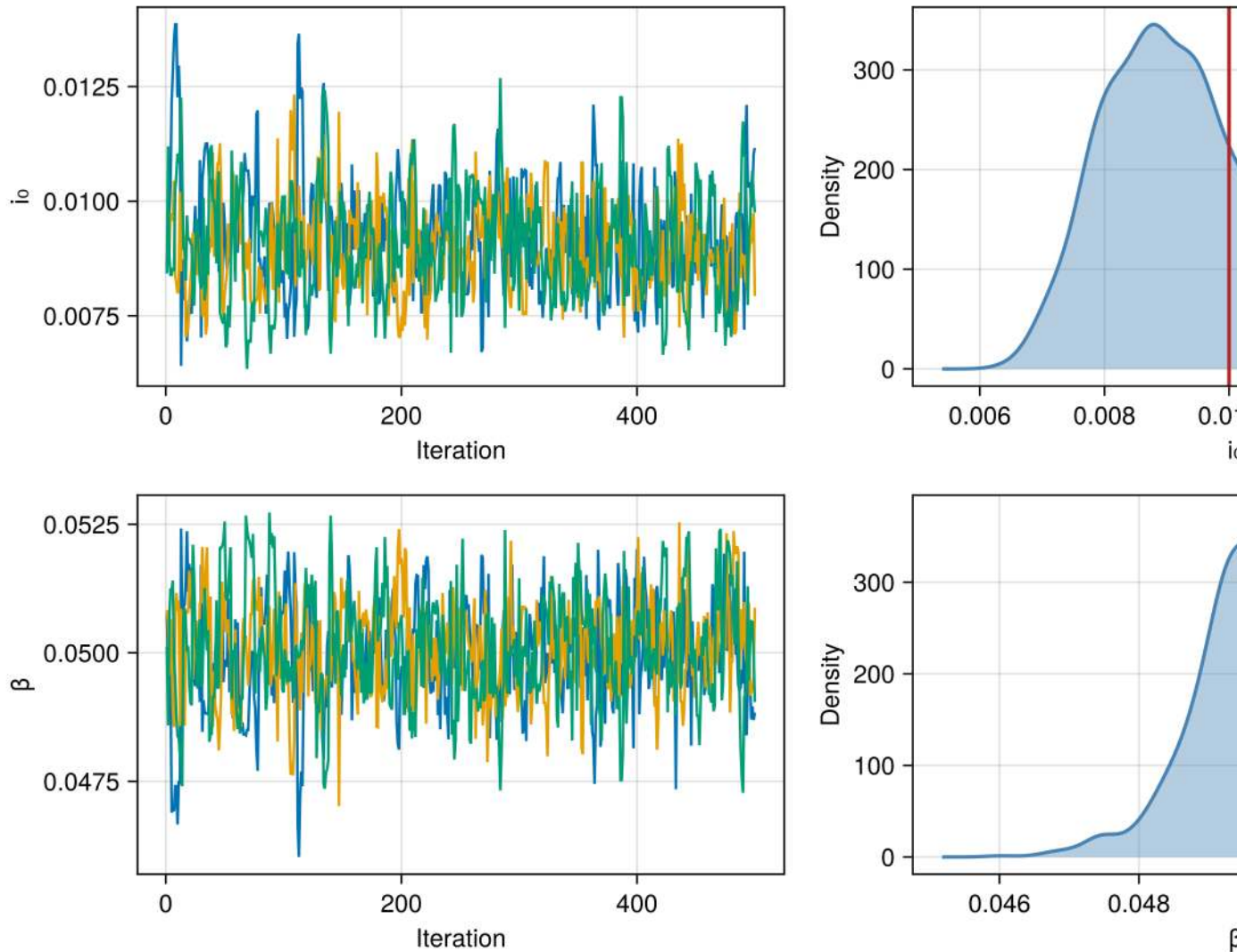


Figure 12.1: Trace plots (left) and marginal posterior densities (right) for i_0 and β . The three chains overlap, and the densities (blue) concentrate around the true values (red).

12.5.1 Posterior predictive check

A posterior that fits well should reproduce the data when pushed back through the model. We draw parameter pairs from the posterior, simulate incidence for each, and summarise the resulting curves by pointwise 95% intervals — a Monte Carlo estimate of the posterior predictive integral implied by Equation 12.1.

```

function incidence_at(i , , l)
    I = i * 1000.0
    probp_pp = remake(probp_ode; u0 = [1000.0 - I, I, 0.0, 0.0],
        p = [ , 10.0, 0.25], tspan = (0.0, float(1)))
    solp = solve(probp_pp, Tsit5(); saveat = 1.0)
    C = Array(solp)[4, :]
    return C[2:end] .- C[1:(end - 1)]
end

l = length(data)
idx = rand(StableRNG(3), 1:length(s), 200)
curves = reduce(hcat, [incidence_at(i0s[k], s[k], l) for k in idx])
lower = [quantile(curves[t, :], 0.025) for t in 1:l]
upper = [quantile(curves[t, :], 0.975) for t in 1:l]
median_curve = [quantile(curves[t, :], 0.5) for t in 1:l]

fig = Figure(size = (760, 420))
ax = Axis(fig[1, 1]; xlabel = "Time (days)", ylabel = "New cases per day")
band!(ax, collect(obstimes), lower, upper; color = colors.band, label = "95% band")
lines!(ax, collect(obstimes), median_curve; color = colors.post, linewidth = 3,
    label = "Posterior median")
scatter!(ax, collect(obstimes), data; color = colors.data, label = "Data")
axislegend(ax; position = :rt)
fig

```

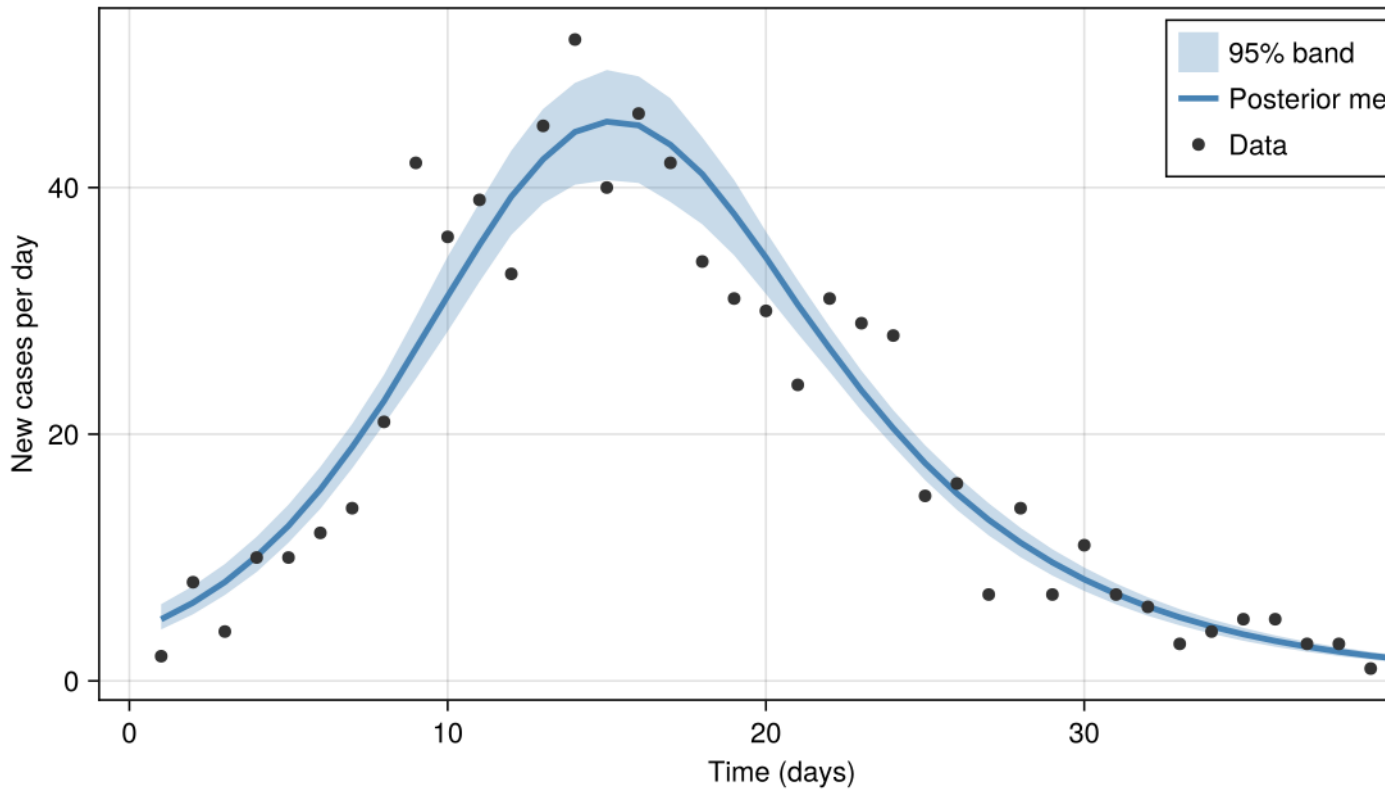


Figure 12.2: Posterior predictive interval for daily incidence (95% band and median, blue) against the observed case reports (grey). The band tracks the data across the whole epidemic.

12.6 Interpretation and limitations

The posterior concentrates near the true parameters, the chains converge ($\hat{R} \approx 1$), and the predictive band envelops the data — a coherent story that also carries caveats. The priors of Equation 12.2 are a modelling choice: overly tight priors can dominate a small dataset, while flat priors on positive parameters can let the sampler wander into regions where the ODE misbehaves, which is why we guarded the solve and floored the rate. NUTS is far more efficient than random-walk MCMC, but each leapfrog step re-solves the ODE and differentiates through it, so the method is markedly more expensive than the optimisation of Chapter 8 — the small sample counts here favour speed over the precision a production analysis would demand. The non-identifiability of β and c noted previously persists: the posterior for β is conditional on the assumed contact rate. Finally, the Poisson likelihood still assumes equidispersion; a negative-binomial likelihood Equation 10.4 would add a dispersion parameter and typically widen the

posterior. Bayesian inference makes all of these assumptions explicit and propagates their consequences directly into the predictive distribution, which is its central practical advantage.

12.7 Exercises

12.7.1 Exercise 1

Replace the Beta priors in Equation 12.2 with uniform priors $i_0 \sim \text{Uniform}(0, 1)$ and $\beta \sim \text{Uniform}(0, 1)$ and re-run the sampler. Compare the posterior means, the $\hat{R}$ values, and the width of the 95% posterior predictive band with the Beta-prior results. Discuss whether the data are informative enough that the choice of prior barely matters here.

12.7.2 Exercise 2

Add the recovery rate γ as a third estimated parameter with prior $\gamma \sim \text{Beta}(2, 8)$, updating the model accordingly. Report the posterior summaries and the pairwise posterior correlation between β and γ (for example from `Array(chain[:, :, :])` and the corresponding γ draws). Relate any strong correlation you find to the identifiability discussion in this chapter and in Chapter 8.

12.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

13 Sensitivity and identifiability

Sensitivity analysis asks how much a chosen model output changes when an input changes. Identifiability asks a different question: can the data, in principle, distinguish the parameters or only a combination of them? Keeping those questions separate prevents a common mistake in epidemic modeling: treating a precise numerical estimate as evidence that every biological parameter is learnable.

13.1 Learning goals

By the end of this chapter you should be able to:

- distinguish local, global, and structural identifiability analyses;
- compute local state sensitivities with the current [SciMLSensitivity.jl API](#);
- use a small Sobol analysis with [GlobalSensitivity.jl](#);
- use [StructuralIdentifiability.jl](#) to expose a parameter combination that observations cannot separate; and
- validate a sensitivity calculation against conservation and a finite perturbation.

13.2 Model assumptions and classification

We use the homogeneous SIR model from Kermack and McKendrick (1927). It is a deterministic, continuous-time, continuous-state, population model. Individuals move irreversibly from susceptible to infectious to removed; the population is closed, well mixed, and has frequency-dependent transmission. The contact rate and per-contact transmission probability are kept as separate inputs even though the ODE depends on their product.

With $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $N = 1000$, contact rate $c = 10$, transmission probability $\beta = 0.05$, and recovery rate $\gamma = 0.25$, the initial effective reproduction number is $\mathcal{R}_0 = \beta c / \gamma = 2$. These values are the course baseline.

13.3 Equations

Let $x(t) = (S(t), I(t), R(t))$. The transition rates are $q_{\text{inf}} = \beta c SI/N$ and $q_{\text{rec}} = \gamma I$:

$$\begin{aligned}\frac{dS}{dt} &= -q_{\text{inf}}, \\ \frac{dI}{dt} &= q_{\text{inf}} - q_{\text{rec}}, \\ \frac{dR}{dt} &= q_{\text{rec}}.\end{aligned}\tag{13.1}$$

The local sensitivity matrix is $J_x(t) = \partial x(t)/\partial \theta$, where $\theta = (\beta, c, \gamma)$. If a scalar quantity of interest is $z = h(x(t), \theta)$, then a first-order perturbation is $\Delta z \approx \nabla_{\theta} z \Delta \theta$. This approximation is local: it is informative near one parameter vector, not across an entire plausible range.

Structural identifiability is about the observation map, not the solver. If only incidence $y = \beta c SI/N$ is observed, the transformation $(\beta, c) \mapsto (a\beta, c/a)$ leaves the transmission term unchanged. Therefore β and c cannot be separated without additional information or an informative design.

13.4 Julia implementation

The course module provides the baseline constructors and plotting theme. The right-hand side below is written out as a pedagogical three-parameter kernel so that the derivative in Equation 13.1 is visible in the code.

```
using CairoMakie
using GlobalSensitivity
using ModelingToolkit
using OrdinaryDiffEq
using SciMLSensitivity
using StructuralIdentifiability

course_root = normpath(joinpath(@_DIR_, ".."))
include(joinpath(course_root, "src", "EpiModelingCourse.jl"))
using .EpiModelingCourse

baseline = BASELINE_PARAMETERS
u0 = Float64[
    BASELINE_INITIAL_STATE.S,
    BASELINE_INITIAL_STATE.I,
    BASELINE_INITIAL_STATE.R,
```

```

]
N = baseline.N
p0 = [baseline.beta, baseline.contact_rate, baseline.gamma]
tgrid = collect(0.0:1.0:40.0)

function sir_kernel!(du, u, p, t)
    , c, = p
    S, I, R = u
    incidence = * c * S * I / N
    du[1] = -incidence
    du[2] = incidence - * I
    du[3] = * I
    return nothing
end

```

sir_kernel! (generic function with 1 method)

13.4.1 Local sensitivities

ODEForwardSensitivityProblem augments the original ODE with the variational equations. `extract_local_sensitivities` returns the state trajectory and a Jacobian for each saved time. This is the forward-sensitivity route in SciMLSensitivity; adjoint methods are preferable when there are many parameters but only a few scalar outputs.

```

local_problem = ODEForwardSensitivityProblem(
    sir_kernel!,
    u0,
    (first(tgrid), last(tgrid)),
    p0,
)
local_solution = solve(
    local_problem,
    Tsit5();
    saveat = tgrid,
    abstol = 1e-9,
    reltol = 1e-9,
)
local_states, local_jacobians = extract_local_sensitivities(
    local_solution,
    true,
)

```

```
final_jacobian = local_jacobians[end]
final_jacobian
```

```
3×3 Matrix{Float64}:
-12339.5  -61.6977   2224.96
-1549.63  -7.74813  -26.0357
13889.2   69.4459   -2198.92
```

The rows of `final_jacobian` correspond to S , I , and R , and its columns correspond to β , c , and γ . The following figure displays the infectious-row sensitivities through time.

```
fig = Figure(size = (800, 450))
ax = Axis(
    fig[1, 1],
    xlabel = "time",
    ylabel = "I/ parameter",
    title = "Local forward sensitivities",
)
parameter_labels = [" ", "c", " "]
parameter_colors = [:steelblue, :darkorange, :seagreen]
for j in 1:3
    lines!(
        ax,
        tgrid,
        [jacobian[2, j] for jacobian in local_jacobians],
        label = parameter_labels[j],
        color = parameter_colors[j],
    )
end
axislegend(ax)
fig
```

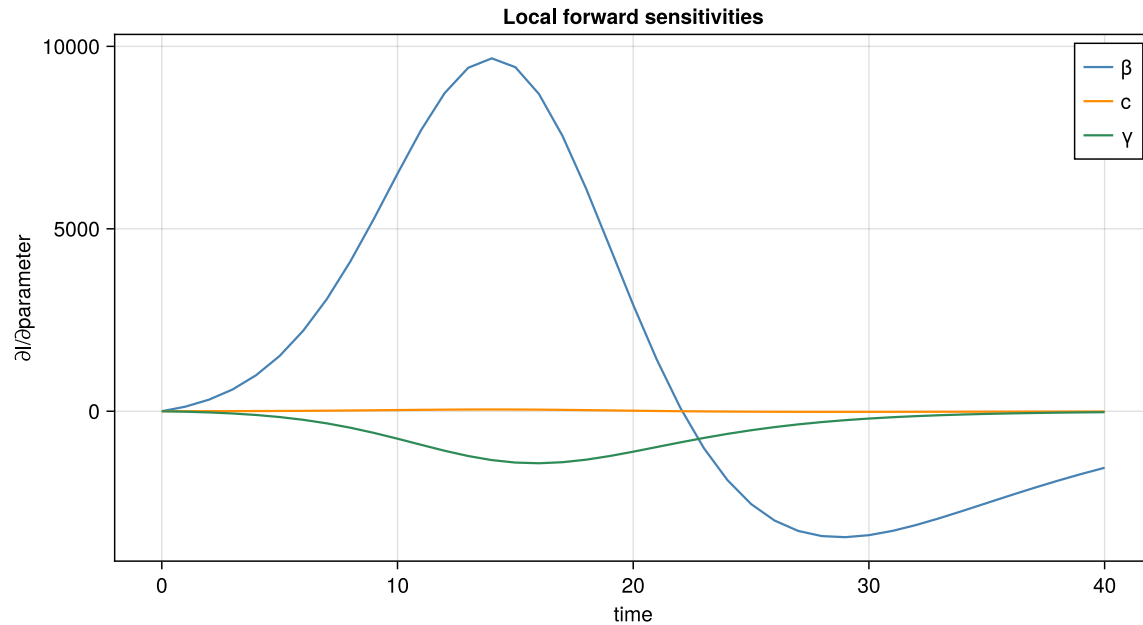


Figure 13.1: Local derivatives of the infectious state with respect to the three SIR inputs.

13.4.2 Global sensitivity

Local derivatives can miss interactions and changes in ranking away from p_0 . GlobalSensitivity evaluates a scalar output over a specified parameter box. Here the output is the peak infectious population. `samples=32` keeps the example short; a production analysis needs a convergence study over larger designs and, ideally, uncertainty distributions justified by data.

```
function peak_infectious(p)
    problem = ODEProblem(sir_kernel!, u0, (0.0, 40.0), Float64.(p))
    solution = solve(problem, Tsit5(); saveat = 1.0, abstol = 1e-8, reltol = 1e-8)
    return maximum(Array(solution)[2, :])
end

parameter_bounds = [
    [0.02, 0.08], #
    [5.0, 15.0], # c
    [0.15, 0.40], #
]

sobol = gsa(peak_infectious, Sobol(), parameter_bounds; samples = 32)
sobol_first = sobol.S1
```

```
sobol_total = sobol.ST
sobol_first, sobol_total
```

Warning: The `generate\_design\_matrices(n, d, sampler, R = NoRand(), num\_mats)` method does
 Prefer using randomization methods such as `R = Shift()`, `R = MatousekScrambling()`, e
 © QuasiMonteCarlo ~/.julia/packages/QuasiMonteCarlo/dqXVT/src/RandomizedQuasiMonteCarlo/ite

```
([0.35113192079825134, 0.18906749529419595, 0.2929920835135053], [0.49222598550892377, 0.296
```

S1 measures the first-order contribution and ST includes interactions. The plot is a compact way to compare the two summaries rather than treating one ranking as universal.

```
fig = Figure(size = (800, 450))
ax = Axis(
    fig[1, 1],
    xticks = (1:3, [" ", "c", " "]),
    ylabel = "Sobol index",
    title = "Global sensitivity of the peak",
)
barplot!(ax, 1:3, sobol_first, dodge = fill(1, 3), color = :steelblue)
barplot!(ax, (1:3) .+ 0.35, sobol_total, dodge = fill(1, 3), color = :darkorange)
Legend(fig[1, 2], [PolyElement(color = :steelblue), PolyElement(color = :darkorange)],
    ["first order", "total"])
fig
```

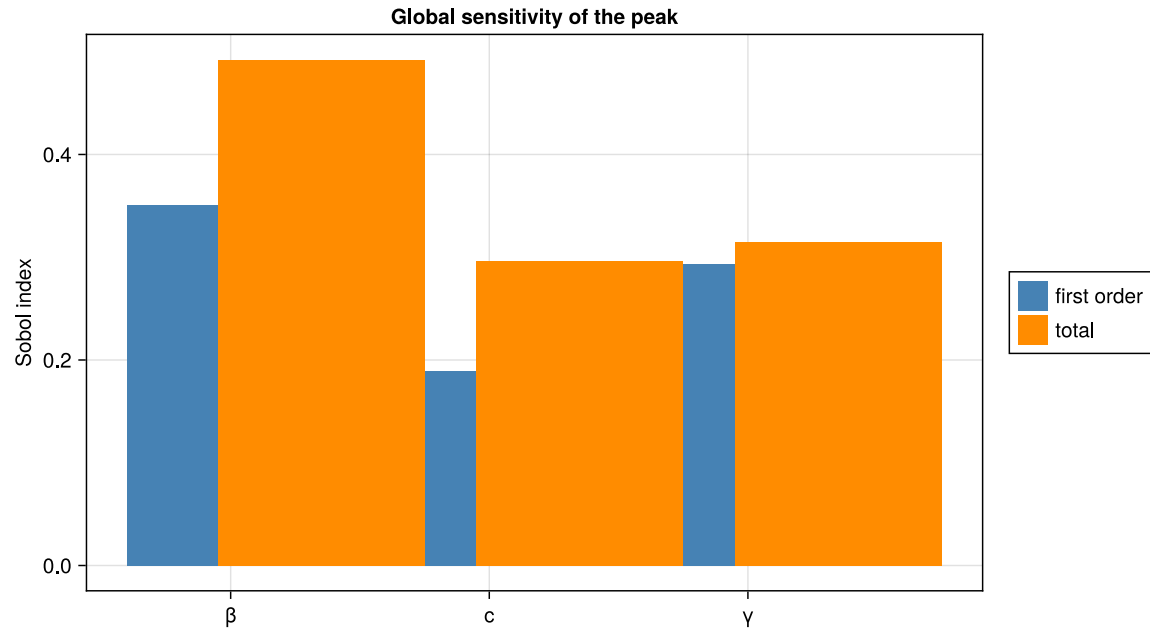


Figure 13.2: First-order and total Sobol indices for the infectious peak.

13.4.3 Structural identifiability

StructuralIdentifiability uses symbolic differential algebra. We deliberately omit the known population scale in this small example to highlight the confounding of β and c in a mass-action-style observation. The package uses ASCII parameter names in this interface, so b and c stand for β and c .

```
@parameters b c g
@independent_variables t
@variables S(t) I(t) R(t) y(t)
D = Differential(t)
identifiability_system = ODESystem(
  [
    D(S) ~ -b * c * S * I,
    D(I) ~ b * c * S * I - g * I,
    D(R) ~ g * I,
  ],
  t;
  name = :SIRIdentifiability,
)
```



```

identifiability = assess_identifiability(
    identifiability_system;
    measured_quantities = [y ~ b * c * S * I],
    funcs_to_check = [b, c, g, b * c],
)
identifiability

```

```

Info: System parsed into S' = -S*I*c*b
I' = S*I*c*b - I*g
R' = I*g
y = S*I*c*b
[ Info: Assessing local identifiability
[ Info: Assessing global identifiability
[ Info: Note: the input model has nontrivial submodels. If the computation for the full model
[ Info: Computing IO-equations
[ Info: Computed IO-equations in 2.397691833 seconds
[ Info: Computing Wronskians
[ Info: Computed Wronskians in 0.378796375 seconds
[ Info: Dimensions of the Wronskians [7]
[ Info: Ranks of the Wronskians computed in 0.008800583 seconds
[ Info: Global identifiability assessed in 8.188331667 seconds

```

```

OrderedCollections.OrderedDict{Num, Symbol} with 4 entries:
b   => :nonidentifiable
c   => :nonidentifiable
g   => :globally
b*c => :globally

```

The expected interpretation is that **b** and **c** are nonidentifiable while **b*c** and **g** are identifiable for this observation choice. Adding an independent observation, fixing the contact rate from external information, or collecting contact-pattern data changes the identification problem; no optimizer can recover information absent from the observation map.

13.5 Validation

The forward sensitivity solve should conserve total population, and its finite perturbation should agree with the derivative at the scale of the chosen perturbation. The checks below are deliberately numerical rather than asserting a particular rounded sensitivity value.

```

population_error = maximum(abs.(vec(sum(local_states; dims = 1)) .- N))

    = 1e-5
perturbed = copy(p0)
perturbed[1] +=
perturbed_solution = solve(
    ODEProblem(sir_kernel!, u0, (0.0, 40.0), perturbed),
    Tsit5();
    saveat = tgrid,
    abstol = 1e-9,
    reltol = 1e-9,
)
finite_difference = (Array(perturbed_solution)[2, end] - local_states[2, end]) /
sensitivity_difference = abs(finite_difference - final_jacobian[2, 1])
sensitivity_relative_difference =
    sensitivity_difference / (1 + abs(final_jacobian[2, 1]))

@assert population_error < 1e-6
@assert sensitivity_relative_difference < 1e-3
(population_error = population_error, sensitivity_difference = sensitivity_difference,
    sensitivity_relative_difference = sensitivity_relative_difference)

```

```

(population_error = 6.821210263296962e-13, sensitivity_difference = 0.8364279269455892, sens

```

13.6 Interpretation and limitations

The local plot describes a neighborhood of the baseline; it is not a probability distribution or a causal effect. The Sobol result is conditional on the rectangular bounds and the small quasi-random design. Structural identifiability is an idealized, noiseless, continuous-observation result, whereas practical identifiability also depends on observation times, measurement error, initial conditions, and model misspecification. In particular, a fit of βc does not justify separate biological claims about β and c . The `SciMLSensitivity`, `GlobalSensitivity`, and `StructuralIdentifiability` APIs used here are documented in their current official links above; the computations intentionally remain small enough to inspect and rerun.

13.7 Exercises

13.7.1 Exercise 1

Change `peak_infectious` to return the final removed count instead of the infectious peak. Re-run the Sobol calculation with the same bounds, compare the first-order and total indices, and explain why the ranking can differ between a transient output and a cumulative output.

13.7.2 Exercise 2

Extend `identifiability_system` with an observable $n(t) \sim S + I + R$. Assess `[b, c, g, b*c]` again. State which confounding remains, if any, and describe one additional data stream that would make the separate contact rate credible.

13.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

14 Interventions and optimization

An intervention model has two layers. First, a **scenario** specifies a policy and simulates its consequences. Second, an **optimization** compares many admissible policies using an explicit objective and constraints. Keeping those layers separate makes it possible to audit a proposed policy before asking an optimizer to choose one.

14.1 Learning goals

By the end of this chapter you should be able to:

- represent a temporary reduction in transmission with an ODE callback;
- distinguish a callback-driven scenario from an optimization problem;
- formulate a small bounded intervention-timing problem with [Optimization.jl](#);
- compare an optimized policy with no intervention and a fixed reference policy; and
- state the objective, bounds, and limitations that determine what an “optimal” intervention means.

14.2 Model assumptions and classification

The model is deterministic, continuous in time, continuous in state, and population based. It retains the closed, homogeneous SIR assumptions of Kermack and McKendrick (1927) and adds a temporary intervention that reduces transmission by a fraction $v(t)$. This is a scenario model, not an individual-level simulation: the callback changes one population-level coefficient at specified times.

The baseline is $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $N = 1000$, $\beta = 0.05$, $c = 10$, and $\gamma = 0.25$. The intervention used below reduces transmission by $v_{\max} = 0.5$ for a fixed duration of 10 time units. The decision variable is only the start time t_1 , bounded to $0 \leq t_1 \leq 25$. More general time-varying controls belong in the advanced [InfiniteOpt/SDDP unit](#).

14.3 Equations

We add cumulative incidence $C(t)$ as a bookkeeping state. During a policy window, transmission is multiplied by $1 - v(t)$:

$$\begin{aligned}\frac{dS}{dt} &= -\beta c(1 - v(t)) \frac{SI}{N}, \\ \frac{dI}{dt} &= \beta c(1 - v(t)) \frac{SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \\ \frac{dC}{dt} &= \beta c(1 - v(t)) \frac{SI}{N}.\end{aligned}\tag{14.1}$$

For a rectangular policy,

$$v(t; t_1) = \begin{cases} v_{\max}, & t_1 \leq t < t_1 + d, \\ 0, & \text{otherwise.} \end{cases}\tag{14.2}$$

The objective in this chapter is the final cumulative incidence $J(t_1) = C(T; t_1)$ at $T = 45$. Because the intervention level and duration are fixed, minimizing J is a one-dimensional bounded problem. If intervention cost, equity, hospital capacity, or uncertainty matters, those terms must be added explicitly rather than inferred from this objective.

14.4 Julia implementation

The right-hand side makes the policy multiplier visible. `PresetTimeCallback` is the appropriate tool for this rectangular scenario; its official documentation is in [DiffEqCallbacks.jl](#).

```
using CairoMakie
using DiffEqCallbacks
using OrdinaryDiffEq
using Optimization
using OptimizationOptimJL

course_root = normpath(joinpath(@__DIR__, ".."))
include(joinpath(course_root, "src", "EpiModelingCourse.jl"))
using .EpiModelingCourse

baseline = BASELINE_PARAMETERS
```

```

u0 = Float64[
    BASELINE_INITIAL_STATE.S,
    BASELINE_INITIAL_STATE.I,
    BASELINE_INITIAL_STATE.R,
    0.0,
]
time_grid = collect(0.0:0.5:45.0)
intervention_duration = 10.0
maximum_reduction = 0.5

function policy_sir!(du, u, p, t)
    , c, , reduction, N = p
    S, I, R, C = u
    incidence = * c * (1 - reduction) * S * I / N
    du[1] = -incidence
    du[2] = incidence - * I
    du[3] = * I
    du[4] = incidence
    return nothing
end

```

policy_sir! (generic function with 1 method)

14.4.1 Scenario simulation with a callback

run_scenario starts with no intervention, changes the fourth parameter at t_1 , and restores it at $t_1 + d$. The callback does not optimize anything; it simply implements one policy that has already been specified.

```

function run_scenario(
    start_time;
    duration = intervention_duration,
    reduction = maximum_reduction,
)
    stop_time = start_time + duration
    parameters = [
        baseline.beta,
        baseline.contact_rate,
        baseline.gamma,
        0.0,
        baseline.N,
    ]
end

```

```

]
function affect!(integrator)
    integrator.p[4] = integrator.t < stop_time ? reduction : 0.0
    return nothing
end
callback = PresetTimeCallback([start_time, stop_time], affect!)
problem = ODEProblem(policy_sir!, u0, (0.0, last(time_grid)), parameters)
return solve(
    problem,
    Tsit5();
    callback = callback,
    saveat = time_grid,
    abstol = 1e-8,
    reltol = 1e-8,
)
end

no_intervention = solve(
    ODEProblem(
        policy_sir!,
        u0,
        (0.0, last(time_grid)),
        [baseline.beta, baseline.contact_rate, baseline.gamma, 0.0, baseline.N],
    ),
    Tsit5();
    saveat = time_grid,
    abstol = 1e-8,
    reltol = 1e-8,
)

reference_start = 8.0
reference_solution = run_scenario(reference_start)

```

```

retcode: Success
Interpolation: 1st order linear
t: 93-element Vector{Float64}:
 0.0
 0.5
 1.0
 1.5
 2.0
 2.5
 3.0

```

```

3.5
4.0
4.5

41.0
41.5
42.0
42.5
43.0
43.5
44.0
44.5
45.0
u: 93-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0, 0.0]
 [987.3708077772923, 11.299550983027808, 1.3296412396798327, 2.6291922227076383]
 [984.4093729570689, 12.759075460611204, 2.8315515823198423, 5.590627042931044]
 [981.0774035610079, 14.395804457229898, 4.52679198176206, 8.922596438991958]
 [977.3332016620534, 16.228146580360058, 6.43865175758647, 12.666798337946528]
 [973.1316388125458, 18.27557198561527, 8.592789201838702, 16.868361187453967]
 [968.4242212073968, 20.558425388134197, 11.01735340446879, 21.575778792602982]
 [963.1592706992523, 23.09765186712193, 13.743077433625668, 26.840729300747597]
 [957.2822512701779, 25.91441784844529, 16.80333088137662, 32.717748729821906]
 [950.7362723645176, 29.02961005503149, 20.234117580450746, 39.26372763548223]

 [323.7231741124572, 54.77868297106671, 621.498142916476, 666.2768258875428]
 [319.4158516076668, 52.38856267038706, 628.195585721946, 670.5841483923332]
 [315.35220979096687, 50.05034266348025, 634.5974475455528, 674.6477902090331]
 [311.5201187969777, 47.76932508625093, 640.7105561167713, 678.4798812030224]
 [307.90778710120543, 45.54986788670876, 646.5423450120858, 682.0922128987946]
 [304.50379966651326, 43.395465512790345, 652.1007348206963, 685.4962003334869]
 [301.2971431801914, 41.30882118470937, 657.3940356350992, 688.7028568198086]
 [298.2772253531312, 39.29192720339787, 662.4308474434708, 691.7227746468689]
 [295.43388557842326, 37.346133863677174, 667.2199805578995, 694.5661144215768]

```

The figure compares the infectious trajectories and cumulative incidence for the no-intervention and reference scenarios.

```

baseline_array = Array(no_intervention)
reference_array = Array(reference_solution(time_grid))
fig = Figure(size = (900, 700))
ax1 = Axis(fig[1, 1], xlabel = "time", ylabel = "infectious people")

```



```

lines!(ax1, time_grid, baseline_array[2, :], label = "none", color = :firebrick)
lines!(ax1, time_grid, reference_array[2, :], label = "50% reduction", color = :steelblue)
vspan!(ax1, reference_start, reference_start + intervention_duration,
        color = (:steelblue, 0.15))
axislegend(ax1, position = :rb)
ax2 = Axis(fig[2, 1], xlabel = "time", ylabel = "cumulative incidence")
lines!(ax2, time_grid, baseline_array[4, :], label = "none", color = :firebrick)
lines!(ax2, time_grid, reference_array[4, :], label = "50% reduction", color = :steelblue)
vspan!(ax2, reference_start, reference_start + intervention_duration,
        color = (:steelblue, 0.15))
fig

```

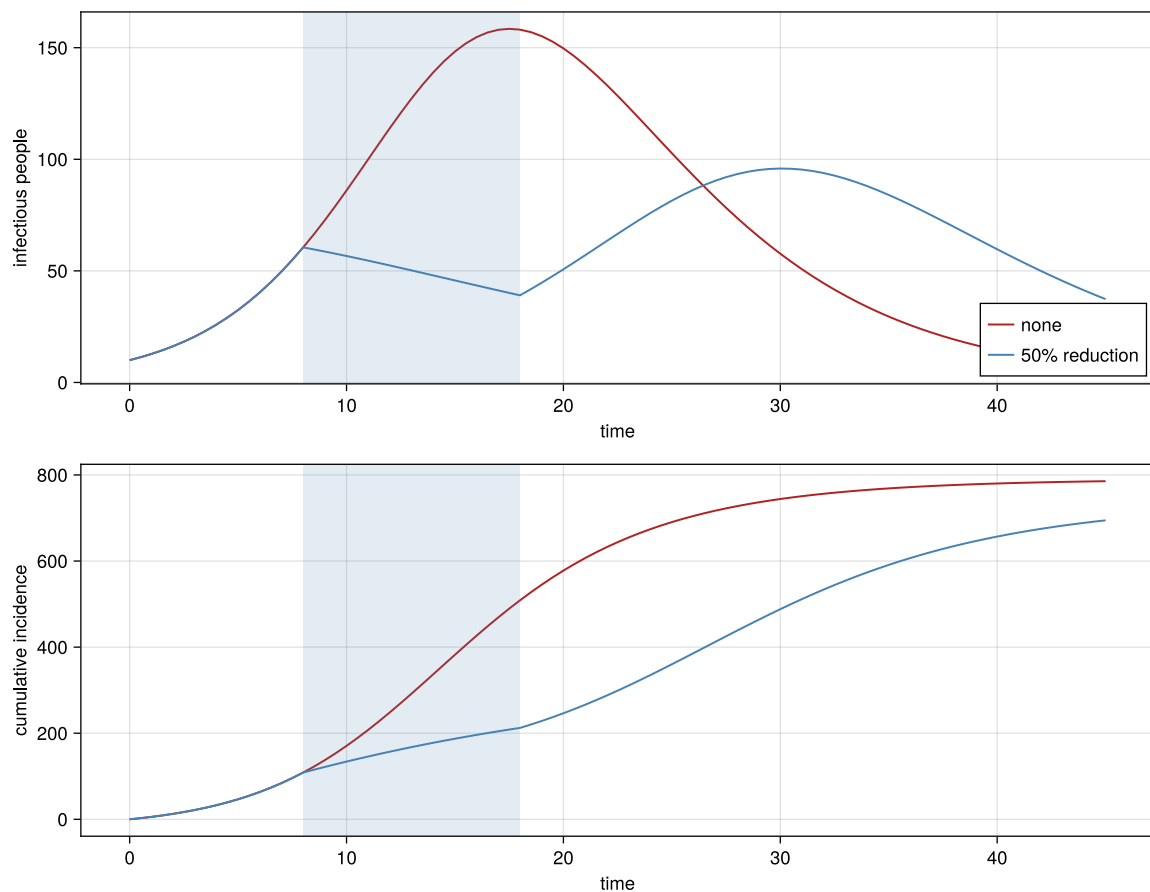


Figure 14.1: A callback-defined intervention scenario compared with no intervention.

14.4.2 A bounded Optimization.jl problem

Now the scenario function becomes the objective. `OptimizationFunction` with `AutoFiniteDiff()` differentiates the scalar objective by finite differences, while `Fminbox(LBFGS())` enforces the explicit lower and upper bounds. The optimizer only chooses t_1 ; it does not decide whether the intervention is ethical, feasible to deliver, or appropriate for the model.

```
function final_incidence(x, _)
    solution = run_scenario(x[1])
    return Array(solution)[4, end]
end

objective = OptimizationFunction(final_incidence, AutoFiniteDiff())
optimization_problem = OptimizationProblem(
    objective,
    [reference_start],
    nothing;
    lb = [0.0],
    ub = [25.0],
)
optimization_solution = solve(
    optimization_problem,
    Fminbox(LBFGS());
    maxiters = 30,
)
optimal_start = optimization_solution.u[1]
optimal_solution = run_scenario(optimal_start)
```

```
retcode: Success
Interpolation: 1st order linear
t: 95-element Vector{Float64}:
 0.0
 0.5
 1.0
 1.5
 2.0
 2.5
 3.0
 3.5
 4.0
 4.5
```

41.0
41.5
42.0
42.5
43.0
43.5
44.0
44.5
45.0

u: 95-element Vector{Vector{Float64}}:

```
[990.0, 10.0, 0.0, 0.0]
[987.3708077772923, 11.299550983027808, 1.3296412396798327, 2.6291922227076383]
[984.4093729570689, 12.759075460611204, 2.8315515823198423, 5.590627042931044]
[981.0774035610079, 14.395804457229898, 4.52679198176206, 8.922596438991958]
[977.3332016620534, 16.228146580360058, 6.43865175758647, 12.666798337946528]
[973.1316388125458, 18.27557198561527, 8.592789201838702, 16.868361187453967]
[968.4242212073968, 20.558425388134197, 11.01735340446879, 21.575778792602982]
[963.1592706992523, 23.09765186712193, 13.743077433625668, 26.840729300747597]
[957.2822512701779, 25.91441784844529, 16.80333088137662, 32.717748729821906]
[950.7362723645176, 29.02961005503149, 20.234117580450746, 39.26372763548223]

[369.51050655686157, 28.44465080693714, 602.0448426362009, 620.4894934431379]
[366.934553877266, 27.522768664973952, 605.5426774577596, 623.0654461227335]
[364.4599615300239, 26.613958824924836, 608.9260796450509, 625.5400384699756]
[362.0836722365037, 25.719559818176158, 612.1967679453197, 627.9163277634958]
[359.8026263605876, 24.840754278575783, 615.3566193608362, 630.1973736394119]
[357.61377402724895, 23.978576316461474, 618.4076496562892, 632.3862259727506]
[355.51408624590715, 23.13391999391043, 621.351993760182, 634.4859137540923]
[353.50056258255466, 22.307545916421088, 624.1918915010239, 636.4994374174448]
[351.5702406004708, 21.500090657784998, 626.9296687417437, 638.4297593995286]
```

The bounded problem can be inspected on a grid as well as by the optimizer. The grid is a diagnostic, not a replacement for the optimization API.

```
candidate_starts = collect(0.0:1.0:25.0)
candidate_objectives = [final_incidence([start], nothing) for start in candidate_starts]
objective_at_optimum = final_incidence([optimal_start], nothing)
```

638.4297593995286

```

fig = Figure(size = (900, 450))
ax = Axis(
    fig[1, 1],
    xlabel = "intervention start time",
    ylabel = "final cumulative incidence",
    title = "Bounded timing optimization",
)
lines!(ax, candidate_starts, candidate_objectives, color = :darkorange)
scatter!(ax, [optimal_start], [objective_at_optimum],
    color = :firebrick, markersize = 14, label = "optimizer")
axislegend(ax)
fig

```

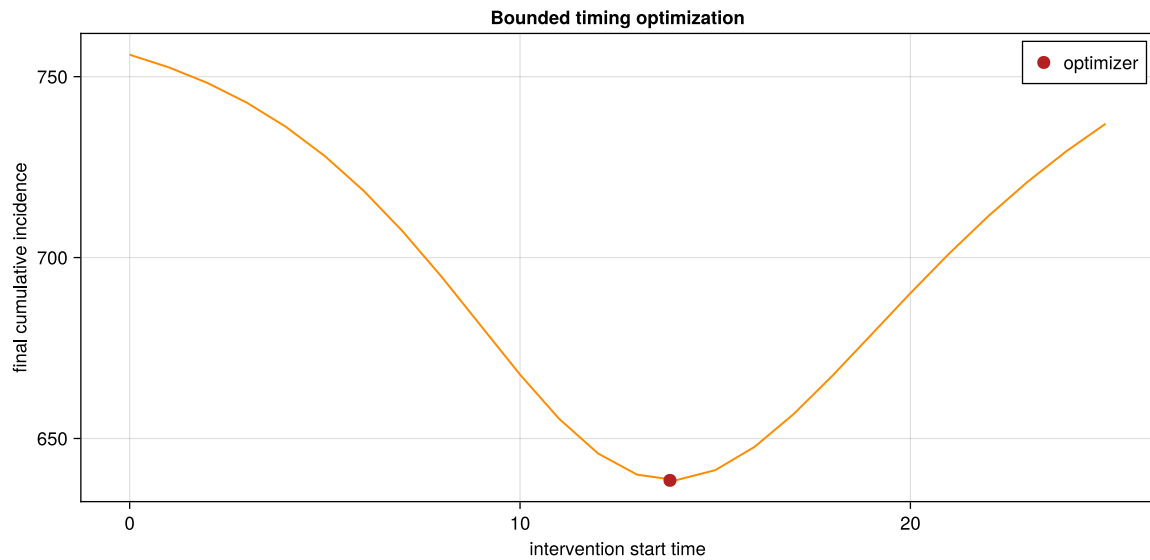


Figure 14.2: Final cumulative incidence as intervention start time varies.

14.5 Validation

The state equations imply $S + I + R = N$ and C is nondecreasing. The scenario callback should also reduce final incidence relative to no intervention for this parameter choice. Finally, the optimizer result must lie inside the declared box and agree with a nearby grid value.

```

optimal_array = Array(optimal_solution(time_grid))
conservation_error = maximum(abs.(vec(sum(optimal_array[1:3, :]; dims = 1)) .- baseline.N))
incidence_is_monotone = all(diff(optimal_array[4, :]) .>= -1e-10)
grid_minimum = minimum(candidate_objectives)

@assert conservation_error < 1e-6
@assert incidence_is_monotone
@assert 0.0 <= optimal_start <= 25.0
@assert objective_at_optimum <= grid_minimum + 1.0
@assert optimal_array[4, end] < baseline_array[4, end]
(conservation_error = conservation_error, optimal_start = optimal_start,
 final_incidence_without_policy = baseline_array[4, end],
 final_incidence_with_policy = optimal_array[4, end])

```

```
(conservation_error = 4.547473508864641e-13, optimal_start = 13.840352420007697, final_incidence = 1.0)
```

14.6 Interpretation and limitations

The result is an optimal start time **conditional on** a fixed 50% reduction, a fixed ten-unit duration, a homogeneous closed population, perfect timing, and the final-incidence objective. It is not a universal recommendation. Changing the cost of intervention, adding a peak-infection penalty, or penalizing delayed action changes the problem. The rectangular callback also creates an idealized instantaneous policy switch; a ramp-up, adherence model, or stochastic transmission would be a different model. This chapter uses a small bounded problem intentionally; continuous controls, path constraints, and multistage uncertainty are deferred to the advanced optimal-control appendix.

14.7 Exercises

14.7.1 Exercise 1

Modify `run_scenario` to compare reductions of 0.25, 0.50, and 0.75. For each reduction, optimize the start time using the same bounds. Report the start time and final incidence, and explain why the best timing need not be the time at which infectious prevalence is largest.

14.7.2 Exercise 2

Define a new scalar objective $C(T) + \lambda \int v(t) dt$ for a nonnegative cost weight λ . With a variable intervention duration, describe which additional bound or constraint prevents the optimizer from selecting an unrealistically long intervention. Implement the objective for two values of λ and compare the resulting policy choices.

14.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

15 Data-to-decision capstone

This capstone follows one analysis from a committed data file to a conditional intervention comparison. The point is not to make a historical claim from a small table. It is to make every transition visible: provenance, model choice, fitting, diagnostics, uncertainty, sensitivity, and a decision-relevant scenario.

15.1 Learning goals

By the end of this chapter you should be able to:

- load and verify a versioned local dataset without a render-time download;
- distinguish observations from reconstructed latent SIR states;
- fit a small ODE model with bounded `Optimization.jl` parameters;
- diagnose fit mismatch and summarize deterministic parameter uncertainty;
- propagate parameter changes through a sensitivity calculation; and
- compare an intervention scenario with its no-intervention counterfactual.

15.2 Model assumptions and classification

The data are the eight committed Eyam observations described in [data/README.md](#) and [data/eyam_plague_1666.toml](#). They cover the second wave beginning on 18 June 1666, with a modeled population of 261. The historical context is discussed by Raggett (Raggett (1982)) and by the later epidemiological analysis of Whittles and Didelot (Whittles and Didelot (2016)).

We choose a deterministic, continuous-time, continuous-state, homogeneous closed SIR model because it is the smallest model that connects the three columns and the preceding course units. This is a deliberately limited model choice. The **S** and **I** values are **reconstructed from historical records and model assumptions, not direct measurements**. A homogeneous closed SIR model also omits important plague mechanisms, including rodent-to-human transmission, household structure, population movement, and changing reporting. The modeled population is the population at the start of the second wave, not the pre-plague village population.

15.3 Equations

For this small dataset we fit an effective transmission rate $\lambda = \beta c$ and a recovery/removal rate γ :

$$\begin{aligned}\frac{dS}{dt} &= -\lambda \frac{SI}{N}, \\ \frac{dI}{dt} &= \lambda \frac{SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I.\end{aligned}\tag{15.1}$$

The initial state is fixed to the first row, and the fitted trajectory is compared with all three reconstructed state columns. For an intervention scenario we add cumulative incidence C and multiply λ by $1 - v(t)$:

$$\frac{dC}{dt} = (1 - v(t))\lambda \frac{SI}{N}.\tag{15.2}$$

The least-squares objective is

$$L(\lambda, \gamma) = \sum_{k=1}^8 \sum_{j \in \{S, I, R\}} \left(\frac{x_j(t_k; \lambda, \gamma) - x_{j,k}}{N} \right)^2.\tag{15.3}$$

This is a transparent teaching objective, not a claim that historical counts have Gaussian errors. A count likelihood, reporting model, or mechanistic plague model would be appropriate for a substantive historical analysis.

15.4 Julia implementation

The data path is local and the fallback is deterministic. The checksum is checked only for the committed file; no code in this chapter fetches epidemiological data from the network.

```
using CairoMakie
using CSV
using DataFrames
using DiffEqCallbacks
using OrdinaryDiffEq
using Optimization
using OptimizationOptimJL
using SHA
```



```

using TOML

course_root = normpath(joinpath(@__DIR__, ".."))
include(joinpath(course_root, "src", "EpiModelingCourse.jl"))
using .EpiModelingCourse

data_path = joinpath(course_root, "data", "eyam_plague_1666.csv")
metadata_path = joinpath(course_root, "data", "eyam_plague_1666.toml")
metadata = TOML.parsefile(metadata_path)
population = Float64(metadata["population"])
observation_columns = [:time_months, :S, :I, :R]

function deterministic_fallback(N)
    times = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0]
    initial = [N - 7.0, 7.0, 0.0]
    fallback_rhs!(du, u, p, t) = begin
        , = p
        S, I, R = u
        incidence = * S * I / N
        du[1] = -incidence
        du[2] = incidence - * I
        du[3] = * I
        nothing
    end
    solution = solve(
        ODEProblem(fallback_rhs!, initial, (first(times), last(times)), [1.0, 0.5]),
        Tsit5();
        saveat = times,
        abstol = 1e-9,
        reltol = 1e-9,
    )
    states = Array(solution)
    return DataFrame(
        time_months = times,
        S = states[1, :],
        I = states[2, :],
        R = states[3, :],
    )
end

data_is_committed = isfile(data_path)
data = data_is_committed ? CSV.read(data_path, DataFrame) : deterministic_fallback(population)

```

```

data_source = data_is_committed ? "committed Eyam CSV" : "deterministic synthetic fallback"

if data_is_committed
  actual_checksum = bytes2hex(sha256(read(data_path)))
  @assert actual_checksum == metadata["csv_sha256"]
end
@assert String.(names(data)) == String.(observation_columns)
@assert nrow(data) == 8
@assert all(diff(data.time_months) .> 0)
@assert all(isapprox.(data.S .+ data.I .+ data.R, population; atol = 1e-8))

```

The metadata records the expected SHA-256 digest, the packaged MultiBD source, and the Apache 2.0 license. A checksum verifies that this checkout contains the committed bytes; it does not prove that the historical reconstruction is correct. Provenance and interpretation caveats remain part of the model decision.

```

data_source, metadata["provenance"], metadata["interpretation"]["caveats"]

```

("committed Eyam CSV", Dict{String, Any}("primary_record" => "Burial register of St. Lawrence plague village population.", "The S and I series are model-derived reconstructions from historical to-human transmission and population movement.", "The final observation is one month after the end of the outbreak"))

15.4.1 Fitting the effective SIR model

The fitting kernel is deliberately visible. Its `p` parameter is an effective rate; it should not be reported as a separately identified contact rate and transmission probability.

```

observation_times = Float64.(data.time_months)
initial_state = Float64.(collect(data[1, [:S, :I, :R]]))
observed_states = permutedims(Matrix{Float64}(data[:, [:S, :I, :R]]))

function fit_rhs!(du, u, p, t)
    , = p
    S, I, R = u
    incidence = * S * I / population
    du[1] = -incidence
    du[2] = incidence - * I
    du[3] = * I
    return nothing
end

```

```

function fit_loss(, _)
    problem = ODEProblem(
        fit_rhs!,
        initial_state,
        (first(observation_times), last(observation_times)),
        ,
    )
    solution = solve(
        problem,
        Tsit5();
        saveat = observation_times,
        abstol = 1e-8,
        reltol = 1e-8,
    )
    prediction = Array(solution)
    return sum(((prediction .- observed_states) ./ population) .^ 2)
end

coarse_grid = [
    (fit_loss([, ], nothing), , )
    for in range(0.5, 6.0; length = 15)
    for in range(0.5, 4.0; length = 15)
]
grid_start = coarse_grid[argmin(first.(coarse_grid))]
fit_objective = OptimizationFunction(fit_loss, AutoFiniteDiff())
fit_problem = OptimizationProblem(
    fit_objective,
    [grid_start[2], grid_start[3]],
    nothing;
    lb = [1e-4, 1e-4],
    ub = [8.0, 8.0],
)
fit_solution = solve(fit_problem, Fminbox(LBFGS()); maxiters = 60)
fitted_parameters = Float64.(fit_solution.u)
fitted_loss = fit_solution.objective
fitted_parameters, fitted_loss

```

([4.603717504388643, 2.8076873724509857], 0.00390067987740599)

15.4.2 Diagnostics

The model and data are shown together, followed by residuals. The residual pattern is more useful than a single loss value: systematic timing or peak mismatch is evidence that the homogeneous SIR mechanism is inadequate for the question.

```
fitted_solution = solve(
    ODEProblem(
        fit_rhs!,
        initial_state,
        (first(observation_times), last(observation_times)),
        fitted_parameters,
    ),
    Tsit5();
    saveat = observation_times,
    abstol = 1e-8,
    reltol = 1e-8,
)
fitted_states = Array(fitted_solution)
residuals = fitted_states .- observed_states
state_rmse = sqrt.(vec(sum(residuals .^ 2; dims = 2)) ./ nrow(data))
```

3-element Vector{Float64}:

```
3.670787952976688
2.9810634731108814
3.294442762389717
```

```
fig = Figure(size = (950, 750))
ax1 = Axis(fig[1, 1], xlabel = "months since 18 June 1666", ylabel = "people")
state_labels = ["S", "I", "R"]
state_colors = [:steelblue, :firebrick, :seagreen]
for j in 1:3
    lines!(ax1, observation_times, fitted_states[j, :],
        color = state_colors[j], label = "$(state_labels[j]) model")
    scatter!(ax1, observation_times, observed_states[j, :],
        color = state_colors[j], markersize = 9, label = "$(state_labels[j]) data")
end
axislegend(ax1, position = :rb, nbanks = 2)
ax2 = Axis(fig[2, 1], xlabel = "months since 18 June 1666", ylabel = "model - data")
for j in 1:3
    lines!(ax2, observation_times, residuals[j, :],
        color = state_colors[j], label = state_labels[j])
end
```

```

end
hlines!(ax2, [0.0], color = :black, linestyle = :dash)
axislegend(ax2)
fig

```

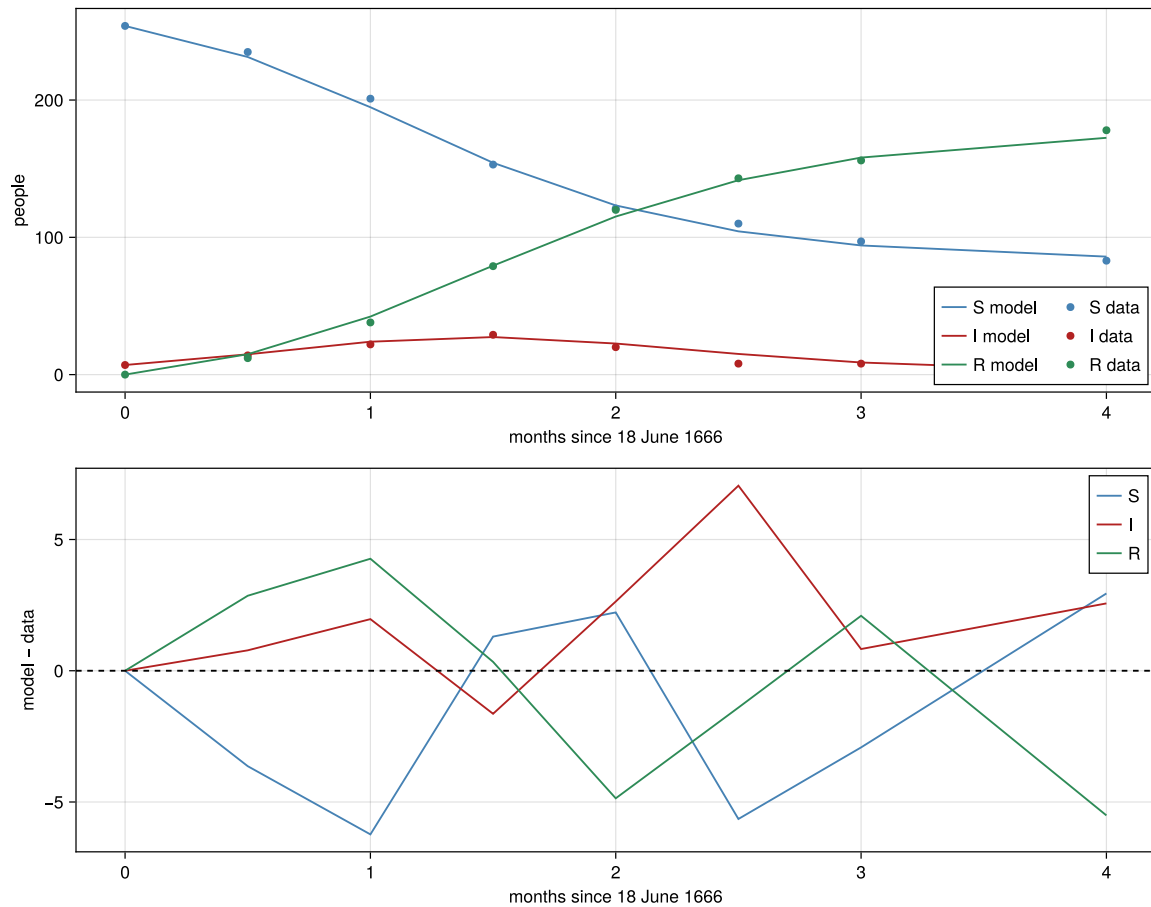


Figure 15.1: Fitted homogeneous SIR states and residuals for the committed Eyam observations.

15.4.3 Deterministic uncertainty envelope

The historical states are reconstructed and there are only eight rows, so a formal confidence interval would require an explicit observation model. As a transparent alternative, we construct a deterministic parameter envelope: the 3×3 combinations of $\pm 10\%$ around the fitted parameters are retained when their loss is close to the best loss. This is an uncertainty scenario, not a calibrated confidence interval.

```

parameter_candidates = [
    fitted_parameters .* [_scale, _scale]
    for _scale in [0.9, 1.0, 1.1]
    for _scale in [0.9, 1.0, 1.1]
]
accepted_parameters = [
    parameters for parameters in parameter_candidates
    if fit_loss(parameters, nothing) <= fitted_loss + 0.02
]
accepted_predictions = [
    Array(solve(
        ODEProblem(
            fit_rhs!,
            initial_state,
            (first(observation_times), last(observation_times)),
            parameters,
        ),
        Tsit5();
        saveat = observation_times,
    )) for parameters in accepted_parameters
]
infectious_lower = [minimum(prediction[2, k] for prediction in accepted_predictions)
    for k in eachindex(observation_times)]
infectious_upper = [maximum(prediction[2, k] for prediction in accepted_predictions)
    for k in eachindex(observation_times)]

```

8-element Vector{Float64}:

```

 7.0
15.727944224022478
25.33432858497662
27.360744384133206
25.226216734403867
18.844113632828964
12.29208288038851
 4.273587559998206

```

```

fig = Figure(size = (850, 450))
ax = Axis(fig[1, 1], xlabel = "months since 18 June 1666", ylabel = "infectious people")
band!(ax, observation_times, infectious_lower, infectious_upper,
    color = (:steelblue, 0.25), label = "±10% parameter envelope")
lines!(ax, observation_times, fitted_states[2, :], color = :firebrick, label = "fitted model")

```

```
scatter!(ax, observation_times, observed_states[2, :], color = :black, label = "reconstructed")
axislegend(ax)
fig
```

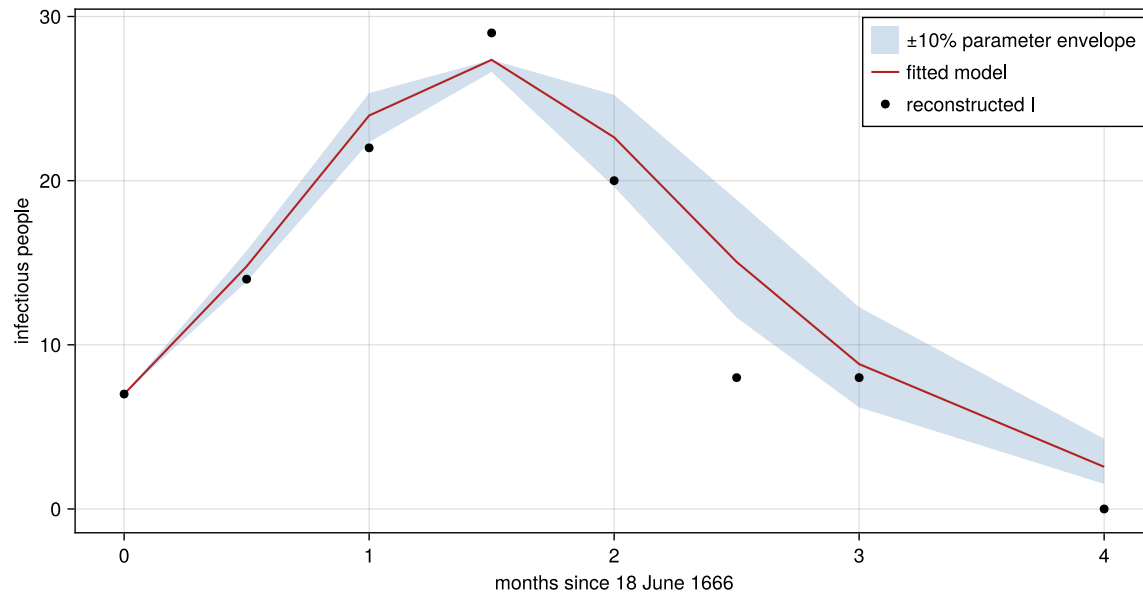


Figure 15.2: A deterministic local parameter envelope for infectious prevalence.

15.4.4 Sensitivity and intervention comparison

For a decision-oriented output, we propagate the fitted parameters to the final cumulative incidence. The centered one-percent perturbation is a local sensitivity check; it is not a replacement for the broader analysis in Chapter 10.

```
function decision_rhs!(du, u, p, t)
    , , reduction, N = p
    S, I, R, C = u
    incidence = (1 - reduction) * S * I / N
    du[1] = -incidence
    du[2] = incidence - I
    du[3] = I
    du[4] = incidence
    return nothing
end
```

```

function decision_solution(
    start_time;
    duration = 1.0,
    reduction = 0.5,
    horizon = 8.0,
)
    stop_time = start_time + duration
    parameters = [fitted_parameters[1], fitted_parameters[2], 0.0, population]
    problem = ODEProblem(
        decision_rhs!,
        [initial_state..., 0.0],
        (0.0, horizon),
        parameters,
    )
    if duration <= 0.0 || reduction <= 0.0
        return solve(problem, Tsit5(); saveat = 0.1)
    end
    function affect!(integrator)
        integrator.p[3] = integrator.t < stop_time ? reduction : 0.0
        return nothing
    end
    callback = PresetTimeCallback([start_time, stop_time], affect!)
    return solve(problem, Tsit5(); callback = callback, saveat = 0.1)
end

function final_cumulative(start_time; duration = 1.0, reduction = 0.5)
    return Array(decision_solution(
        start_time;
        duration = duration,
        reduction = reduction,
    ))[4, end]
end

function final_cumulative_for_parameters(parameters)
    solution = solve(
        ODEProblem(
            decision_rhs!,
            [initial_state..., 0.0],
            (0.0, 8.0),
            [parameters[1], parameters[2], 0.0, population],
        ),
        Tsit5();

```



```

        saveat = 0.1,
    )
    return Array(solution)[4, end]
end

function relative_parameter_sensitivity(parameter_index)
    perturbation = 0.01 * fitted_parameters[parameter_index]
    lower = copy(fitted_parameters)
    upper = copy(fitted_parameters)
    lower[parameter_index] -= perturbation
    upper[parameter_index] += perturbation
    lower_output = final_cumulative_for_parameters(lower)
    upper_output = final_cumulative_for_parameters(upper)
    baseline_output = final_cumulative_for_parameters(fitted_parameters)
    return (upper_output - lower_output) / (2 * perturbation) *
        fitted_parameters[parameter_index] / baseline_output
end

local_loss_sensitivity = [
    relative_parameter_sensitivity(index) for index in 1:2
]
no_policy = decision_solution(0.0; duration = 0.0, reduction = 0.0)
policy = decision_solution(1.0; duration = 1.0, reduction = 0.5)
decision_summary = (
    = fitted_parameters[1],
    = fitted_parameters[2],
    final_incidence_without_policy = Array(no_policy)[4, end],
    final_incidence_with_policy = Array(policy)[4, end],
    local_loss_sensitivity = local_loss_sensitivity,
)
decision_summary

```

```

( = 4.603717504388643, = 2.8076873724509857, final_incidence_without_policy = 170.94266585,
1.1359021476043987])

```

```

fig = Figure(size = (900, 500))
ax = Axis(fig[1, 1], xlabel = "months", ylabel = "cumulative incidence")
decision_times = no_policy.t
no_policy_array = Array(no_policy)
policy_array = Array(policy(decision_times))
lines!(ax, decision_times, no_policy_array[4, :],

```

```

    color = :firebrick, label = "no intervention")
lines!(ax, decision_times, policy_array[4, :],
    color = :steelblue, label = "50% reduction from month 1")
vspan!(ax, 1.0, 2.0, color = (:steelblue, 0.15))
axislegend(ax)
fig

```

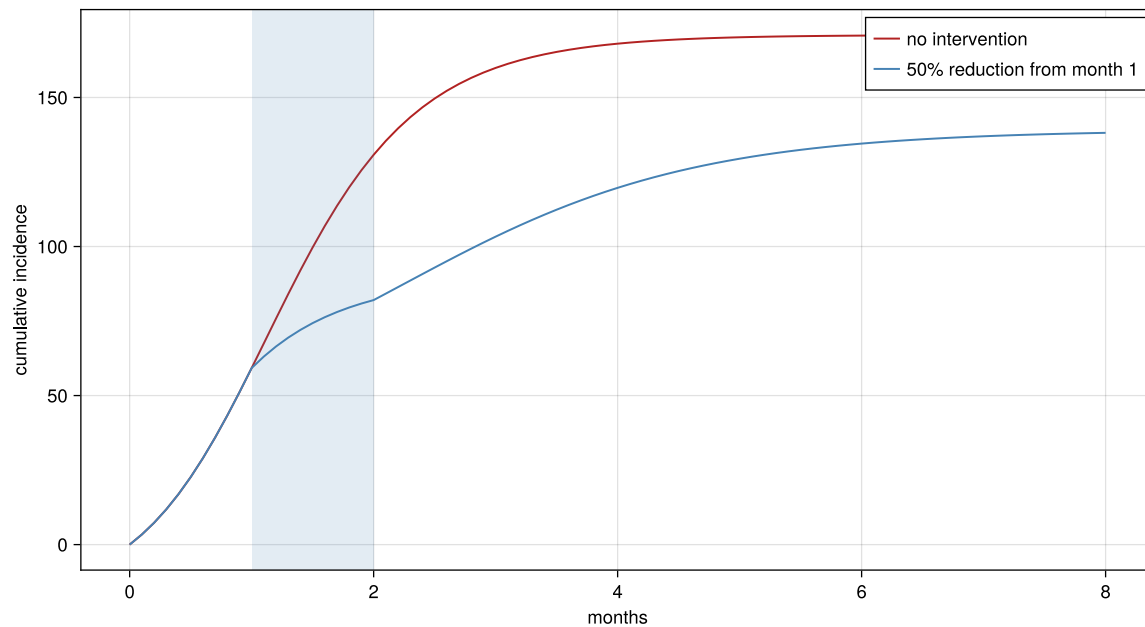


Figure 15.3: Counterfactual cumulative incidence with and without a one-month 50% reduction.

15.5 Validation

The data checks validate schema, ordering, population conservation, and the committed checksum. The model checks validate conservation and finite fit output; the decision comparison checks that the chosen intervention lowers cumulative incidence for this fitted scenario. These are structural checks, not evidence that the model is historically adequate.

```

fit_conservation_error = maximum(
    abs.(vec(sum(fitted_states; dims = 1)) .- population),
)
fit_is_finite = all(isfinite, fitted_states)
policy_reduction = Array(no_policy)[4, end] - Array(policy)[4, end]

```

```
@assert fit_conservation_error < 1e-6
@assert fit_is_finite
@assert policy_reduction > 0
(source = data_source, fit_conservation_error = fit_conservation_error,
 state_rmse = state_rmse, policy_reduction = policy_reduction)
```

```
(source = "committed Eyam CSV", fit_conservation_error = 5.684341886080802e-
14, state_rmse = [3.670787952976688, 2.9810634731108814, 3.294442762389717], policy_reduction = 0.9999999999999999)
```

15.6 Interpretation and limitations

The fitted λ summarizes transmission under the data's time scale; it does not identify separate β and c . The deterministic envelope captures only a deliberately narrow local perturbation and the residual diagnostic shows why uncertainty should not be reduced to optimizer digits. The intervention comparison is a counterfactual under the same homogeneous closed SIR assumptions. In particular, because S and I are reconstructed values and a homogeneous closed SIR omits important plague mechanisms, the policy result is an educational model-based comparison, not a historical estimate of the effect of a specific Eyam action. Replacing the model with a household, vector-borne, movement, or observation model would change both the fit and the decision.

15.7 Exercises

15.7.1 Exercise 1

Replace the state least-squares loss with a loss on I only, refit (λ, γ) , and compare the residual plot and fitted final cumulative incidence with the all-state fit. Explain which information is discarded and why a seemingly better infectious curve may give a worse population balance.

15.7.2 Exercise 2

Use the deterministic envelope to compare a one-month 25% reduction with a one-month 50% reduction beginning at month 1. Report the range of final cumulative incidence across accepted parameter candidates for each scenario, then state what additional observation or mechanistic assumption would be needed before treating that range as decision uncertainty.

15.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Course resources

Each teaching unit has a book page, slide deck, Pluto source, and static notebook preview.

The capstone uses the versioned [Eyam plague dataset](#) with [provenance metadata](#).

Julia essentials for modelers

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Probability and stochastic processes

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Calculus and numerical methods

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Foundations: SIR theory and the Julia workflow

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Discrete-time models

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Ordinary differential equations

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Continuous-time Markov chains and jump processes

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Stochastic differential equations

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Agent-based models

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Observation models and simulated data

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Frequentist parameter estimation

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Bayesian inference

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Sensitivity and identifiability

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Interventions and optimization

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Data-to-decision capstone

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Delay differential equations

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Stochastic delay differential equations

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Discrete-event simulation

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Universal differential equations

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Finite-state projection

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Probabilistic numerical integration

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Surrogate and emulator models

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Advanced optimal control

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

Model composition and structured populations

- [Book](#)
- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

A Delay differential equations

Every population model so far in this course — the ODE of Chapter 4, the jump process, and the stochastic differential equation — assumes an infectious period that is **exponentially distributed**: recovery happens at a constant hazard γ , memoryless of how long an individual has already been infectious. A delay differential equation (DDE) replaces that assumption with the opposite extreme: every infectious period lasts *exactly* the same fixed duration τ . This appendix builds that model, explains the *method of steps* used to solve it, and shows numerically how much the shape of the infectious-period distribution — not just its mean — changes epidemic dynamics.

A.1 Learning goals

By the end of this unit you should be able to:

- classify a DDE model and explain how a fixed recovery delay differs from the memoryless recovery of Chapter 4;
- write a DDE for SIR dynamics using a history function for states before the start time;
- construct a `DDEProblem` with `DelayDiffEq` and solve it with `MethodOfSteps`, declaring the constant lag for solver efficiency;
- explain the method of steps and why declaring lags helps the solver locate derivative discontinuities; and
- cross-validate a single DDE run against an equivalent two-stage (ODE-then-DDE) construction, and compare the result with the exponential recovery model.

A.2 Model assumptions and classification

The population is the same closed, homogeneously mixing cohort used throughout the course: $N = 1000$, $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, contact rate $c = 10$, and per-contact transmission probability $\beta = 0.05$. The only change from Equation 4.1 is *how long an infection lasts*: instead of an exponential holding time with rate γ , every individual infected at time t^* recovers at exactly $t^* + \tau$, with $\tau = 1/\gamma = 4$ chosen so the *mean* infectious period matches the rest of the course.

Property	Classification
Determinism	Deterministic
Time	Continuous
State	Continuous
Infectious period	Fixed delay τ (degenerate distribution, coefficient of variation 0)

This is still a **deterministic, continuous-time, continuous-state, population-level** model — the same category as Chapter 4 — but it is no longer a Markovian ODE, because the rate of recovery at time t depends on the state *at time* $t - \tau$, not just the state at time t .

A.3 Equations

Let $\lambda(t) = \beta c I(t)/N(t)$ be the force of infection at time t . New infections at time t occur at rate $\lambda(t)S(t)$; each of those individuals recovers exactly τ time units later, so the *recovery* flow at time t equals the *infection* flow at time $t - \tau$:

$$\frac{dS}{dt} = -\lambda(t)S(t), \quad \frac{dI}{dt} = \lambda(t)S(t) - \lambda(t - \tau)S(t - \tau), \quad \frac{dR}{dt} = \lambda(t - \tau)S(t - \tau). \quad (\text{A.1})$$

Because Equation A.1 reaches back to $t - \tau$, it needs a **history function** $h(t)$ supplying $(S(t), I(t), R(t))$ for all $t \leq 0$, not just a single initial vector. We assume everyone was susceptible before the simulation starts,

$$h(t) = (N, 0, 0), \quad t \leq 0, \quad (\text{A.2})$$

and let the $I(0) = 10$ initial infections enter as a discontinuous jump at $t = 0$ — they are treated as having been infected at $t = 0$ itself, so Equation A.2 correctly gives zero recoveries while $t < \tau$ (since $t - \tau < 0$ there, and the history has no infected individuals).

Method of steps. A DDE with a single constant lag τ can be solved by repeatedly treating it as an ODE: on $[0, \tau]$ the delayed term $S(t - \tau), I(t - \tau)$ is fully determined by the *known* history h , so Equation A.1 is an ordinary ODE there. Once that segment is solved, the dense solution on $[0, \tau]$ becomes the exact “history” needed to solve the next segment $[\tau, 2\tau]$, and so on. Derivative discontinuities introduced by the history mismatch at $t = 0$ propagate forward at $t = \tau, 2\tau, 3\tau, \dots$, growing smoother at each step. Declaring the lag via `constant_lags` tells the solver exactly where to expect these breakpoints, so it can place a step boundary there instead of discovering the discontinuity through failed error control.

A.4 Julia implementation

The current `DelayDiffEq.jl` represents the delayed state through a history function `h(p, t)` passed as the third argument of the right-hand side, and solves the resulting `DDEProblem` with `MethodOfSteps`, which wraps any `OrdinaryDiffEq` algorithm.

```
using DelayDiffEq
using OrdinaryDiffEq
using CairoMakie

function sir_dde!(du, u, h, p, t)
    S, I, R = u
    , c, = p
    N = S + I + R
    = * c * I / N # eq-a01-dde, current force of infection
    Sd, Id, Rd = h(p, t - ) # delayed state at t -
    Nd = Sd + Id + Rd
    d = * c * Id / Nd # delayed force of infection
    du[1] = - * S
    du[2] = * S - d * Sd
    du[3] = d * Sd
    return nothing
end

sir_history(p, t) = [1000.0, 0.0, 0.0] # eq-a01-history: fully susceptible before t=0

u0 = [990.0, 10.0, 0.0]
= 0.25
p = (0.05, 10.0, 1 / ) # , c, = 1/ = 4.0
tspan = (0.0, 40.0)

prob_dde = DDEProblem(sir_dde!, u0, sir_history, tspan, p; constant_lags = [p[3]])
sol_dde = solve(prob_dde, MethodOfSteps(Tsit5())); reltol = 1e-8, abstol = 1e-8)
round.(sol_dde.u[end]; digits = 2)
```

```
3-element Vector{Float64}:
 152.51
  14.53
 832.96
```

A.4.1 An independent two-stage construction

Because there are no recoveries before $t = \tau$, the system on $[0, \tau]$ is exactly an SI ODE (no R flow). Solving that ODE first and using its dense output as the history for a *second* DDEProblem starting at $t = \tau$ gives a numerically independent route to the same trajectory — useful both as a cross-check and as a template for models whose early history is not as simple as “everybody susceptible”.

```
function si_ode!(du, u, p, t)
    S, I = u[1], u[2]
    , c, _ = p
    = * c * I / (u[1] + u[2] + u[3])
    du[1] = - * S
    du[2] = * S
    du[3] = 0.0
    return nothing
end

prob_si = ODEProblem(si_ode!, u0, (0.0, p[3]), p)
sol_si = solve(prob_si, Tsit5(); reltol = 1e-10, abstol = 1e-10)

history_from_ode(p, t) = sol_si(t)
prob_dde2 = DDEProblem(sir_dde!, sol_si(p[3]), history_from_ode, (p[3], tspan[2]), p;
    constant_lags = [p[3]])
sol_dde2 = solve(prob_dde2, MethodOfSteps(Tsit5()); reltol = 1e-8, abstol = 1e-8)
round(sol_dde2.u[end]; digits = 2)
```

3-element Vector{Float64}:

```
152.51
 14.53
832.96
```

```
function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, rate = p
    N = S + I + R
    = * c * I / N
    du[1] = - * S
    du[2] = * S - rate * I
    du[3] = rate * I
    return nothing
end
```

```

end
sol_ode = solve(ODEProblem(sir_ode!, u0, tspan, (0.05, 10.0, )), Tsit5();
                reltol = 1e-8, abstol = 1e-8)

tt = 0.0:0.2:40.0
fig = Figure(size = (720, 440))
ax = Axis(fig[1, 1]; xlabel = "time", ylabel = "number",
          title = "Fixed-delay (DDE) vs exponential (ODE) recovery")
lines!(ax, tt, [sol_dde(t)[1] for t in tt]; color = :steelblue, linewidth = 3, label = "S (DDE)")
lines!(ax, tt, [sol_dde(t)[2] for t in tt]; color = :firebrick, linewidth = 3, label = "I (DDE)")
lines!(ax, tt, [sol_dde(t)[3] for t in tt]; color = :seagreen, linewidth = 3, label = "R (DDE)")
lines!(ax, tt, [sol_ode(t)[2] for t in tt]; color = :firebrick, linewidth = 2,
        linestyle = :dash, label = "I (ODE)")
vlines!(ax, [p[3]]; color = :black, linestyle = :dot, label = " ")
axislegend(ax; position = :rt)
fig

```

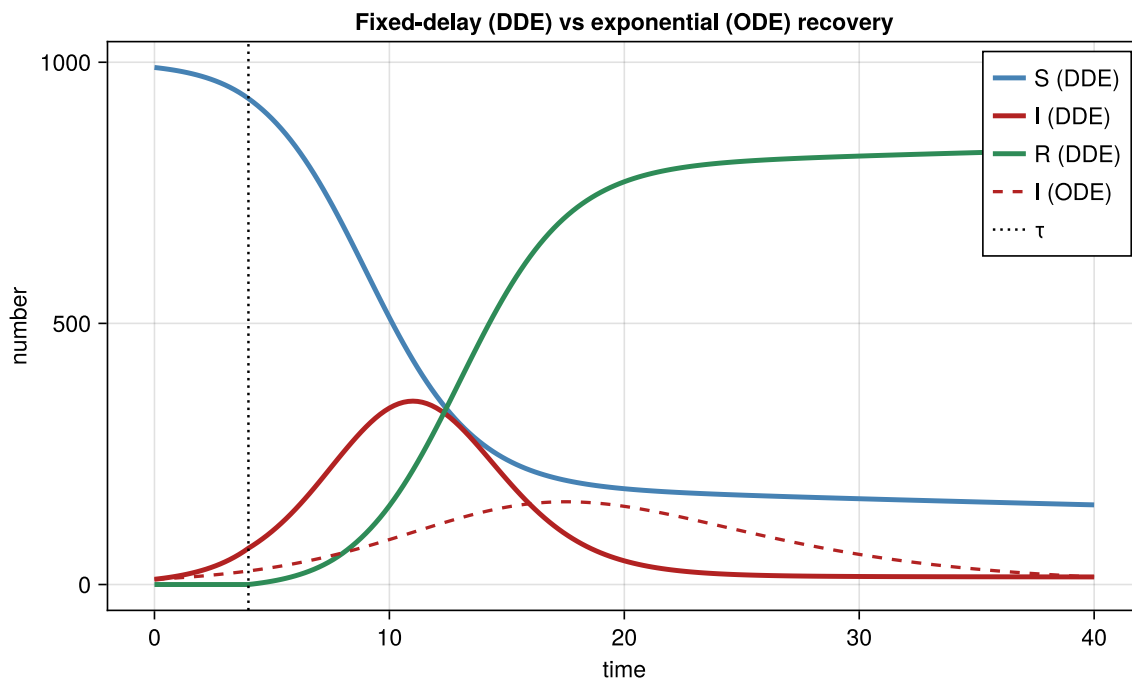


Figure A.1: Fixed-delay DDE trajectory (solid) compared with the exponential-recovery ODE of Chapter 6 (dashed infectious curve). The vertical line marks the fixed recovery delay τ . The DDE produces a taller, sharper peak because every infection lasts exactly τ time units instead of an exponentially varying time.

A.5 Validation

Cross-check. The single-history and two-stage constructions must agree everywhere, since they solve the same mathematical problem two different ways:

```
grid = 0.0:0.5:40.0
diffs = [maximum(abs.(sol_dde(t) .- (t < p[3] ? sol_si(t) : sol_dde2(t)))) for t in grid]
maximum(diffs) < 1e-4
```

true

Conservation and non-negativity. The population total must stay at N and every compartment must remain nonnegative:

```
ts_dense = 0.0:0.1:40.0
states = [sol_dde(t) for t in ts_dense]
conserved = all(u -> isapprox(sum(u), 1000.0; atol = 1e-6), states)
nonnegative = all(u -> all(>=(-1e-8), u), states)
(conserved, nonnegative)
```

(true, true)

Structural comparison. Both models share the same mean infectious period $1/\gamma = \tau = 4$, yet the fixed delay produces a substantially higher infectious peak than the exponential-recovery ODE:

```
peak_dde = maximum(sol_dde[2, :])
peak_ode = maximum(sol_ode[2, :])
(round(peak_dde; digits = 1), round(peak_ode; digits = 1), peak_dde > peak_ode)
```

(350.9, 158.3, true)

All three checks return **true**: the two DDE constructions agree to solver tolerance, the trajectory conserves N and stays nonnegative, and the fixed delay yields the higher, sharper peak predicted by epidemic theory.

A.6 Interpretation and limitations

The gap between the two peaks in the validation section is not a numerical artifact — it reflects a real, well-known sensitivity of epidemic dynamics to the *shape*, not just the mean, of the infectious-period distribution. A fixed delay has zero variance (coefficient of variation 0): nobody recovers early, so the initial cohort keeps transmitting at full strength throughout $[0, \tau)$ before disappearing all at once. Exponential recovery has the same mean but a coefficient of variation of 1, so many individuals recover quickly and temper the early growth. Real infectious periods usually sit between these two extremes; the model-composition appendix’s Erlang (linear chain) construction is one common way to interpolate between them within ordinary differential equations, avoiding the delay machinery altogether for that class of distributions.

Numerically, `MethodOfSteps` is only as efficient as the lag information you give it: omitting `constant_lags` still produces a correct answer for a constant delay, but the adaptive step-size controller must rediscover the breakpoints at $\tau, 2\tau, \dots$ by trial and error, which is slower and can be less accurate near those points. The history discontinuity at $t = 0$ is a genuine limitation of the “instant seeding” convention used here: it assumes all initial infections happened simultaneously, which is convenient but not realistic; the two-stage construction above shows how to relax that assumption by supplying a more detailed history function. Finally, this remains a population-level, deterministic model: it has no demographic noise, so the SDDE built on it in the next appendix is needed whenever finite-population fluctuations matter.

For the full API, see the [DelayDiffEq.jl documentation](#) and the [SciML DDE problem reference](#). For the classical role of infectious-period distributions in epidemic dynamics see Anderson and May (1991).

A.7 Exercises

A.7.1 Exercise 1

Solve the DDE model for $\tau \in \{2, 4, 8\}$ (holding β , c , and N fixed) and record the peak infectious count and the time at which it occurs for each. Compare with the peak of the exponential-recovery ODE at the matching mean infectious period $1/\gamma = \tau$. Describe how the size and timing of the gap between the two models changes as τ grows.

A.7.2 Exercise 2

The single-history construction in this appendix seeds all $I(0) = 10$ initial infections as happening exactly at $t = 0$. Repeat the two-stage construction (presolve, then continue as a DDE) for $I(0) = 1$ initial infection instead of 10, and confirm that the resulting trajectory still conserves the population and stays nonnegative. Then add a `ContinuousCallback` that

would clamp I to zero if it ever went negative, run the model, and report whether the callback ever fires. Explain why it should not, in contrast to the stochastic delay model of the next appendix.

A.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

B Stochastic delay differential equations

The delay differential equation (DDE) of Appendix A is still deterministic: given the same history, it always produces the same trajectory. Real epidemics have finite populations and demographic noise, just like the chemical-Langevin stochastic differential equation (SDE) that extends the ordinary SIR model. This appendix combines both ideas — a diffusion approximation *and* a fixed recovery delay — into a stochastic delay differential equation (SDDE), using the current, actively maintained `DelayDiffEq + StochasticDiffEq` route rather than the archived `StochasticDelayDiffEq.jl` package. That combination is new enough that the official documentation marks `SDDEProblem` “experimental,” and this appendix shows concretely what that means in practice.

B.1 Learning goals

By the end of this unit you should be able to:

- classify an SDDE model and describe how it combines the delayed drift of Appendix A with the chemical-Langevin diffusion of the stochastic differential equations chapter;
- construct an `SDDEProblem` and solve it with `StochasticDiffEq`’s `LambaEM` through `DelayDiffEq`’s `MethodOfSteps`, which now supports stochastic delay problems directly;
- explain why adaptive step-size control is currently unreliable for this problem class near the history discontinuity, and why a fixed step is used instead;
- explain why demographic noise can drive a compartment negative, and why that risk is *harder* to control for an SDDE than for the plain SDE; and
- reproduce a stochastic run deterministically despite a currently unreliable `rng` keyword for this problem/algorithm combination.

B.2 Model assumptions and classification

The population, contact process, and fixed recovery delay $\tau = 1/\gamma = 4$ are exactly those of Appendix A. What changes is that transmission and recovery are now treated as random events with Poisson-like variance (the chemical Langevin approximation), applied to the *delayed* recovery flow just as in Equation A.1.

Property	Classification
Determinism	Stochastic
Time	Continuous
State	Continuous (diffusion approximation to an integer population)
Delay	Fixed lag τ , as in Appendix A
Noise	Chemical Langevin (non-diagonal, state- and history-dependent)

This is a **stochastic, continuous-time, continuous-state, population-level** model. It is not Markovian in the usual continuous-time sense — like the DDE, the recovery term depends on the state at $t - \tau$ — and it is not guaranteed to stay inside the physically meaningful region $S, I, R \geq 0$, for reasons explored below.

B.3 Equations

With $\lambda(t) = \beta c I(t)/N(t)$ as before, the SDDE adds one Wiener increment per reaction channel to the delayed-recovery drift of [Equation A.1](#):

$$\begin{aligned}
 dS(t) &= -\lambda(t)S(t) dt - \sqrt{\lambda(t)S(t)} dW_1(t), \\
 dI(t) &= [\lambda(t)S(t) - \lambda(t-\tau)S(t-\tau)]dt + \sqrt{\lambda(t)S(t)} dW_1(t) - \sqrt{\lambda(t-\tau)S(t-\tau)} dW_2(t), \\
 dR(t) &= \lambda(t-\tau)S(t-\tau) dt + \sqrt{\lambda(t-\tau)S(t-\tau)} dW_2(t),
 \end{aligned}
 \tag{B.1}$$

with the same history $h(t) = (N, 0, 0)$ for $t \leq 0$ as [Equation A.2](#). As in the SDE chapter, the two noise columns are built so that they cancel when summed across compartments, which is what keeps $S + I + R$ exactly conserved regardless of the realized noise. The official SciML reference for this problem type is the [SDDE problem page](#), which explicitly labels this functionality experimental.

B.4 Julia implementation

`DelayDiffEq.jl` now recognizes an `SDDEProblem` directly: it automatically wraps a bare SDE algorithm like `LambaEM` in `MethodOfSteps`, exactly as it wraps `Tsit5` for the deterministic DDE. Both `DelayDiffEq` and `StochasticDiffEq` must be loaded for this dispatch to resolve.

```

using DelayDiffEq
using OrdinaryDiffEq
using StochasticDiffEq
using SparseArrays
using Random
using CairoMakie

```

The drift is identical to Equation A.1; the diffusion follows the two reaction channels (infection, delayed recovery) with a sparse 3×2 noise matrix, as in the chemical Langevin SDE. Rates are clamped with `max(..., 0.0)` purely to avoid `sqrt` of a negative number if the state has already drifted below zero — a numerical safety net, not a fix for the underlying issue discussed in Validation below.

```

function sir_dde!(du, u, h, p, t)
    S, I, R = u
    , c, = p
    N = S + I + R
    = * c * I / N
    Sd, Id, Rd = h(p, t - )
    Nd = Sd + Id + Rd
    d = * c * Id / Nd
    du[1] = - * S
    du[2] = * S - d * Sd
    du[3] = d * Sd
    return nothing
end

function sir_delayed_noise!(G, u, h, p, t)
    S, I, R = u
    , c, = p
    N = S + I + R
    infection = max( * c * I / N * S, 0.0)
    Sd, Id, Rd = h(p, t - )
    Nd = Sd + Id + Rd
    recovery = max( * c * Id / Nd * Sd, 0.0)
    G[1, 1] = -sqrt(infection)
    G[2, 1] = sqrt(infection)
    G[2, 2] = -sqrt(recovery)
    G[3, 2] = sqrt(recovery)
    return nothing
end

```

```

sir_history(p, t) = [1000.0, 0.0, 0.0]

u0 = [990.0, 10.0, 0.0]
    = 0.25
p = (0.05, 10.0, 1 / )
tspan = (0.0, 40.0)
noise_shape() = sparse([1.0 0.0; 1.0 1.0; 0.0 1.0])

```

noise_shape (generic function with 1 method)

B.4.1 Adaptive stepping breaks down at the history discontinuity

Solving with the solver’s default adaptive stepping fails almost immediately, long before any real epidemic dynamics unfold:

```

Random.seed!(4)
prob_probe = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
                        noise_rate_prototype = noise_shape())
sol_probe = solve(prob_probe, LambdaEM())
(sol_probe.retcode, round(sol_probe.t[end]; sigdigits = 3), length(sol_probe.t))

```

```

Warning: Verbosity toggle: dt_min_unstable
dt(2.220446049250313e-16) <= dtmin(2.220446049250313e-16) at t=8.92044057448111e-
5. Aborting. There is either an error in your model specification or the true solution is un
@ SciMLBase ~/.julia/packages/SciMLBase/Pp33G/src/integrator_interface.jl:989

```

```
(SciMLBase.ReturnCode.DtLessThanMin, 8.92e-5, 103)
```

The solver’s own step-size controller shrinks `dt` to machine epsilon and aborts (`DtLessThanMin`) after roughly a hundred steps, having advanced the clock by less than 10^{-4} time units out of 40. The cause is the same jump discontinuity discussed in Appendix A — all history before $t = 0$ is fully susceptible, while u_0 already has 10 infections — combined with multiplicative $\sqrt{\cdot}$ noise, whose derivative is unbounded as the argument approaches zero. The practical fix, used throughout the rest of this appendix, is the same one the stochastic differential equations chapter uses for a *non-delayed* chemical Langevin model: a fixed step with `adaptive = false`.

```

Random.seed!(4)
prob_sdde = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
                        noise_rate_prototype = noise_shape())
sol_sdde = solve(prob_sdde, LambdaEM(); dt = 0.1, adaptive = false)
round(sol_sdde.u[end]; digits = 2)

```

```

3-element Vector{Float64}:
 109.29
  26.05
 864.66

```

Note the seed is set with a **global** `Random.seed!` rather than the `rng = StableRNG(seed)` keyword used in the SDE and jump-process chapters. Passing `rng` directly to `solve` for this `SDDEProblem/MethodOfSteps` combination currently produces a *different* trajectory on every run, even with an identical `StableRNG` seed — the keyword is not fully threaded through the delay bookkeeping in the current package versions. Seeding the global RNG before each `solve` call, as the original `StochasticDelayDiffEq`-based tutorials did, is currently the only reproducible option for this combination and is the deliberate, documented deviation from the rest of the course's convention.

```

sol_dde = solve(DDEProblem(sir_dde!, u0, sir_history, tspan, p; constant_lags = [p[3]]),
                MethodOfSteps(Tsit5()); reltol = 1e-8, abstol = 1e-8)

grid = 0.0:0.1:40.0
states_sdde = reduce(hcat, sol_sdde(t) for t in grid)
states_dde = reduce(hcat, sol_dde(t) for t in grid)
size(states_sdde)

```

```

(3, 401)

```

```

fig = Figure(size = (720, 440))
ax = Axis(fig[1, 1]; xlabel = "time", ylabel = "number",
          title = "SDDE realization (seed 4) vs deterministic DDE drift")
lines!(ax, grid, states_sdde[1, :]; color = :steelblue, linewidth = 3, label = "S (SDDE)")
lines!(ax, grid, states_sdde[2, :]; color = :firebrick, linewidth = 3, label = "I (SDDE)")
lines!(ax, grid, states_sdde[3, :]; color = :seagreen, linewidth = 3, label = "R (SDDE)")
lines!(ax, grid, states_dde[2, :]; color = :firebrick, linewidth = 2, linestyle = :dash,
      label = "I (DDE drift)")
axislegend(ax; position = :rt)
fig

```

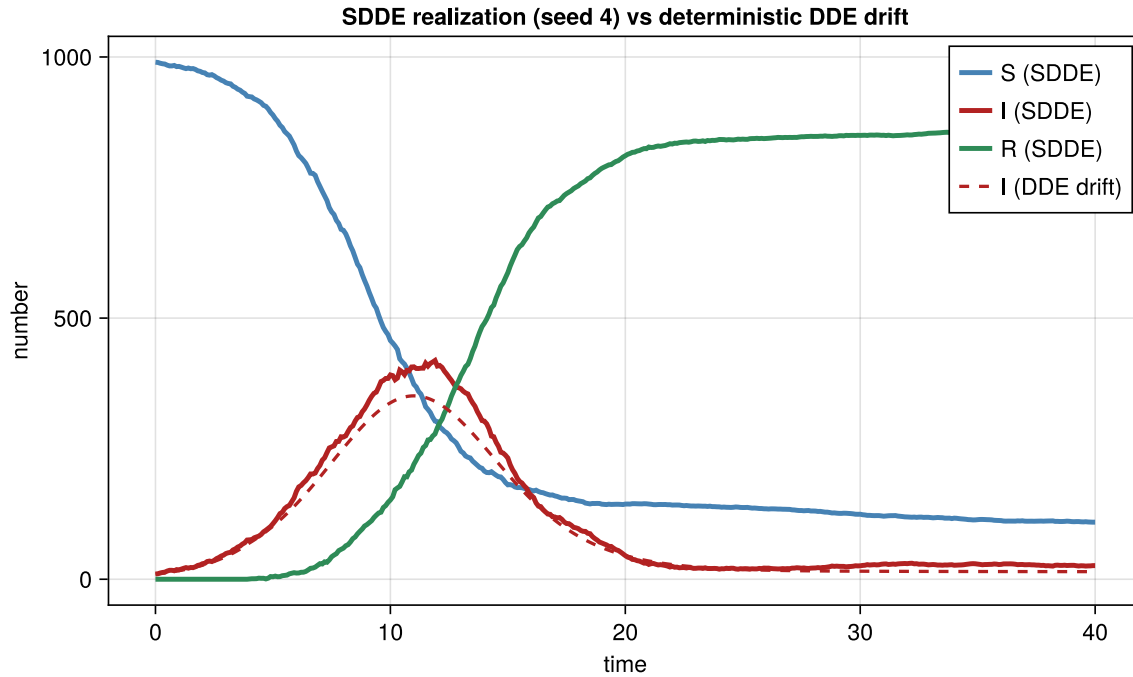


Figure B.1: One SDDE realization (solid) against the deterministic DDE drift of Appendix A (dashed infectious curve). The realization tracks the drift closely but shows realization-to-realization variation typical of demographic noise.

B.5 Validation

Conservation. Like the non-delayed chemical Langevin SDE, the noise columns are built to sum to zero, so conservation should hold regardless of the realized path:

```
conserved_clean = maximum(abs.(vec(sum(states_sdde; dims = 1)) .- 1000.0))
nonneg_clean = minimum(states_sdde)
(conserved_clean < 1e-8, nonneg_clean)
```

```
(true, 0.0)
```

For this seed the run also happens to stay nonnegative throughout — but that is a property of this particular realization, not a guarantee.

The nonnegativity caveat, quantified. Changing only the random seed, with every other line of code identical, produces a realization where the infectious compartment swings *substantially* negative, while conservation still holds to solver precision:

```
Random.seed!(1)
prob_caveat = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
                          noise_rate_prototype = noise_shape())
sol_caveat = solve(prob_caveat, LambdaEM(); dt = 0.1, adaptive = false)
states_caveat = Array(sol_caveat)
conserved_caveat = maximum(abs.(vec(sum(states_caveat; dims = 1)) .- 1000.0))
nonneg_caveat = minimum(states_caveat)
(conserved_caveat < 1e-8, round(nonneg_caveat; digits = 1))
```

```
(true, -86.6)
```

Conservation is a structural property of the noise matrix and holds to solver tolerance in both realizations; nonnegativity is not structural at all, and this second seed drives I to roughly -87 individuals at its worst point. A model that can report a negative number of infectious people has left the region where its equations mean anything, even though its total population still “adds up.”

B.6 Interpretation and limitations

This appendix demonstrates, concretely rather than abstractly, why the official documentation calls `SDDEProblem` experimental. Three specific, reproducible issues appear even in this small, textbook-scale model:

1. **Adaptive stepping is currently unusable** here: the controller collapses `dt` to machine epsilon at the history discontinuity, aborting after a fraction of a percent of the time span. A fixed step sidesteps the problem but forfeits the automatic error control adaptive solvers normally provide, so results should be checked for step-size sensitivity (Exercise 1).
2. **The `rng` keyword is unreliable for this combination.** Reproducibility currently requires the older global-state pattern (`Random.seed!` before `solve`), unlike the rest of the course.
3. **Nonnegativity is not guaranteed, and is harder to police than for a plain SDE.** For an ordinary chemical Langevin SDE, clamping a compartment at each step is at least a *local* correction. Here, recovery at time t is driven by the state at $t - \tau$; clamping the *current* state cannot repair the *delayed* history that will keep influencing the drift and diffusion for up to τ more time units. A per-step correction is therefore even more of

a pragmatic patch than in the non-delayed case discussed in the stochastic differential equations chapter, and is not used here for that reason.

None of this invalidates the chemical Langevin idea itself — it is a well-established diffusion approximation to a reaction network (Gillespie 1977) — but it does mean that, at present, this particular software combination is best treated as a teaching device and a basis for further testing, not as a production tool. A more robust alternative for small populations is the exact stochastic simulation approach of the jump-process chapter, extended with a fixed delay through an event queue — which is exactly the discrete-event simulation covered in the next appendix.

B.7 Exercises

B.7.1 Exercise 1

Repeat the seed-4 run at `dt` values of 0.2, 0.1, and 0.05. For each, record the maximum conservation error and the minimum state value over the whole trajectory. Discuss whether decreasing `dt` makes nonnegativity violations more or less frequent, and explain why — unlike the convergence study in the calculus and numerical methods unit — this is not a simple “smaller step, smaller error” relationship.

B.7.2 Exercise 2

Run the seed-1 (caveat) trajectory again, this time adding a `DiscreteCallback` that clamps I to zero whenever it is negative after a step. Report the new minimum value of I , the new maximum conservation error, and the number of times the callback fires. Compare the corrected trajectory’s conservation error with the uncorrected one, and use the result to explain concretely why clamping the current state is an incomplete fix for a *delayed* recovery term.

B.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

C Discrete-event simulation

The jump-process chapter simulates a continuous-time Markov chain by tracking *aggregate counts* and repeatedly asking “what happens next, and to how many people?” The agent-based model chapter tracks *individuals*, but advances them in fixed time steps of $\Delta t = 0.1$. A discrete-event simulation (DES) combines both ideas: it tracks individuals **and** advances time continuously, jumping directly from one scheduled event to the next without ever discretizing the clock. This appendix builds the same SIR process as an event queue two different ways — with Agents.jl’s current `EventQueueABM`, and with the general-purpose process-oriented `ConcurrentSim.jl` — and shows how naturally a DES can express things the earlier chapters could not, including the fixed delay of Appendix A.

C.1 Learning goals

By the end of this unit you should be able to:

- classify an individual-level, continuous-time, event-queue simulation and contrast it with the population-level CTMC (jump-process chapter) and the discrete-time ABM (agent-based models chapter);
- construct an `EventQueueABM` with `AgentEvents` whose propensities depend on agent state, using the current Agents.jl 7 API;
- construct an equivalent model with `ConcurrentSim.jl`’s coroutine-based processes, and explain when its more general process/resource abstraction is preferable to an agent-centric framework;
- validate a DES implementation with conservation, monotonicity of the one-way $S \rightarrow I \rightarrow R$ transition, and an ensemble comparison against the deterministic mean field; and
- recognize how an event’s *timing* function, not just its propensity, can encode a fixed delay directly, connecting back to the method of steps.

C.2 Model assumptions and classification

The population and rates are the usual baseline: $N = 1000$, $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, contact rate $c = 10$, transmission probability $\beta = 0.05$, recovery rate $\gamma = 0.25$. Each individual is in exactly one of three states. A susceptible individual’s next contact, and an infectious

individual’s recovery, are each scheduled as an **event** at a specific future time; the simulation clock jumps directly to the next scheduled event anywhere in the population, executes it, and reschedules as needed. There is no fixed time step anywhere in the model.

Property	Classification
Determinism	Stochastic
Time	Continuous, event-driven (no fixed Δt)
State	Discrete (individual status)
Representation	Individual-level

This is a **stochastic, continuous-time, discrete-state, individual-level** model — the same underlying process as the jump-process chapter’s CTMC, by the superposition of independent exponential clocks, but resolved per individual rather than by a single aggregate “what happens next” draw, and without the agent-based model chapter’s time-step discretization.

C.3 Equations

Each susceptible individual’s next contact is scheduled after an $\text{Exponential}(1/c)$ waiting time; when that contact fires, a uniformly random other individual is chosen, and infection occurs with probability β if the alter is infectious:

$$T_{\text{contact}} \sim \text{Exponential}(1/c), \quad \Pr(\text{infect} \mid \text{contact with infectious alter}) = \beta. \quad (\text{C.1})$$

Each infectious individual’s recovery fires after an independent $\text{Exponential}(1/\gamma)$ waiting time:

$$T_{\text{recovery}} \sim \text{Exponential}(1/\gamma). \quad (\text{C.2})$$

By the memoryless property and superposition of independent Poisson processes, the *population-level* counting process implied by Equation C.1 and Equation C.2 is exactly the CTMC of the jump-process chapter; the individual-event view additionally retains *who* infected *whom* and *when* each specific event is scheduled, which an event queue (a priority queue ordered by event time) makes explicit.

C.4 Julia implementation

C.4.1 Primary: Agents.jl EventQueueABM

EventQueueABM associates each agent with one or more `AgentEvents`. By default `AgentEvent`'s timing samples an exponential waiting time parameterized by the event's propensity, exactly matching Equation C.1–Equation C.2; only agents with nonzero propensity are scheduled.

```
using Agents
using ConcurrentSim
using ResumableFunctions
using Random
using Distributions
using DataFrames
using OrdinaryDiffEq
using CairoMakie
```

```
const β = 0.05
const c = 10.0
const γ = 0.25
const N = 1000
const I0 = 10
const tf = 40.0
```

40.0

```
@agent struct Person(NoSpaceAgent)
    status::Symbol
end

function transmit!(agent, model)
    alter = random_agent(model)
    if alter.status == :I && (rand(abmrng(model)) < model.c)
        agent.status = :I
    end
    return nothing
end

recover!(agent, model) = (agent.status = :R)

transmit_propensity(agent, model) = agent.status == :S ? model.c : 0.0 # eq-a03-contact
recovery_propensity(agent, model) = agent.status == :I ? model.γ : 0.0 # eq-a03-recovery
```

```

transmit_event = AgentEvent(action! = transmit!, propensity = transmit_propensity)
recovery_event = AgentEvent(action! = recover!, propensity = recovery_propensity)
events = (transmit_event, recovery_event)

susceptible(x) = count(==(S), x)
infected(x) = count(==(I), x)
recovered(x) = count(==(R), x)

function init_eventqueue_model(, c, , N, I0, rng)
    properties = Dict{<: => , :c => c, : => }
    model = EventQueueABM(Person, events; properties, rng)
    for i in 1:N
        status = i <= I0 ? I : S
        add_agent!(Person(; id = i, status = status), model)
    end
    return model
end

rng = Xoshiro(20250711)
eq_model = init_eventqueue_model(, c, , N, I0, rng)
to_collect = [(:status, f) for f in (susceptible, infected, recovered)]
eq_data, _ = run!(eq_model, tf; adata = to_collect)
first(eq_data, 3)

```

	time	susceptible_status	infected_status	recovered_status
	Float64	Int64	Int64	Int64
1	0.0	990	10	0
2	1.0	985	13	2
3	2.0	979	16	5

C.4.2 Alternative: ConcurrentSim.jl general-purpose processes

ConcurrentSim.jl (the maintained continuation of the historical SimJulia) is not agent-centric: it provides coroutine-based *processes* that `timeout` for arbitrary durations, plus general `Resource/Store` primitives for queueing and contention. Where `EventQueueABM` gives you propensities and actions, `ConcurrentSim` gives you the process itself as a Julia function — more bookkeeping, but no requirement that the model be phrased as “agents with events.” Each individual here is a `@resumable` process that repeats “wait for a contact, maybe get infected” until infected, then waits once more to recover.

```

mutable struct SIRPerson
  id::Int
  status::Symbol
end

mutable struct SIRModel
  sim::Simulation
  ::Float64
  c::Float64
  ::Float64
  ta::Vector{Float64}
  Sa::Vector{Int}
  Ia::Vector{Int}
  Ra::Vector{Int}
  people::Vector{SIRPerson}
end

function record!(m::SIRModel)
  push!(m.ta, now(m.sim))
  push!(m.Sa, count(p -> p.status == :S, m.people))
  push!(m.Ia, count(p -> p.status == :I, m.people))
  push!(m.Ra, count(p -> p.status == :R, m.people))
  return nothing
end

@resumable function live(sim::Simulation, person::SIRPerson, m::SIRModel, rng)
  while person.status == :S
    @yield timeout(sim, rand(rng, Exponential(1 / m.c))) # eq-a03-contact
    alter = person
    while alter === person
      alter = rand(rng, m.people)
    end
    if alter.status == :I && rand(rng) < m.
      person.status = :I
      record!(m)
    end
  end
  if person.status == :I
    @yield timeout(sim, rand(rng, Exponential(1 / m.))) # eq-a03-recovery
    person.status = :R
    record!(m)
  end
end

```

```

end

function init_concurrentsim_model(S, I, R, , c, )
    sim = Simulation()
    people = SIRPerson[]
    for i in 1:S
        push!(people, SIRPerson(i, :S))
    end
    for i in 1:I
        push!(people, SIRPerson(S + i, :I))
    end
    for i in 1:R
        push!(people, SIRPerson(S + I + i, :R))
    end
    SIRModel(sim, , c, , [0.0], [S], [I], [R], people)
end

cs_rng = Xoshiro(20250711)
cs_model = init_concurrentsim_model(N - I0, I0, 0, , c, )
for person in cs_model.people
    @process live(cs_model.sim, person, cs_model, cs_rng)
end
run(cs_model.sim, tf)
(cs_model.Sa[end], cs_model.Ia[end], cs_model.Ra[end])

```

(147, 7, 846)

```

function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, = p
    Ntot = S + I + R
    = * c * I / Ntot
    du[1] = - * S
    du[2] = * S - * I
    du[3] = * I
    return nothing
end

sol_ode = solve(ODEProblem(sir_ode!, [990.0, 10.0, 0.0], (0.0, tf), (, c, )), Tsit5())

fig = Figure(size = (720, 440))
ax = Axis(fig[1, 1]; xlabel = "time", ylabel = "infectious count",

```



```

    title = "Two discrete-event realizations vs the ODE mean field")
stairs!(ax, eq_data.time, eq_data.infected_status; color = :firebrick, linewidth = 2,
        label = "I (EventQueueABM)")
stairs!(ax, cs_model.ta, cs_model.Ia; color = :steelblue, linewidth = 2,
        label = "I (ConcurrentSim)")
lines!(ax, sol_ode.t, sol_ode[2, :]; color = :black, linestyle = :dash, linewidth = 2,
        label = "I (ODE mean field)")
axislegend(ax; position = :rt)
fig

```

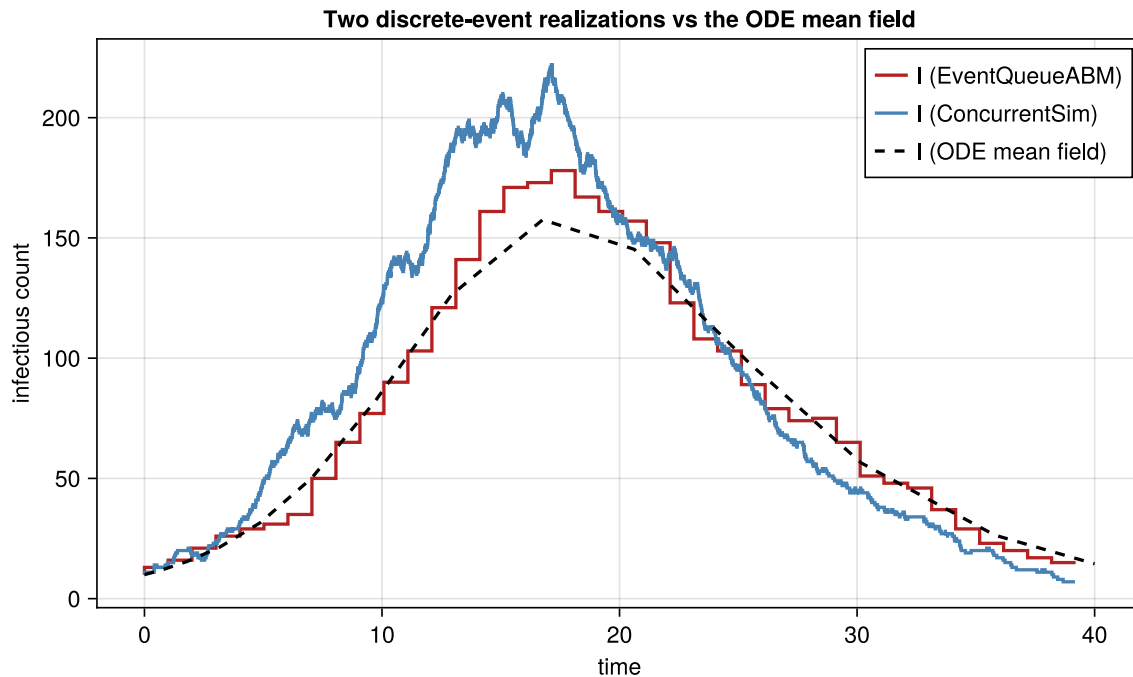


Figure C.1: Two independent discrete-event realizations of the same SIR process — Agents.jl’s EventQueueABM and ConcurrentSim’s process model — against the deterministic ODE mean field. The EventQueueABM curve is coarser because its `adata` collection samples the model once per unit time by default, while ConcurrentSim’s manual bookkeeping records every individual event.

C.5 Validation

Conservation. Every recorded row of both simulations must sum to N :

```

conserved_eq = all(r -> r.susceptible_status + r.infected_status + r.recovered_status == N,
                  eachrow(eq_data))
conserved_cs = all(cs_model.Sa .+ cs_model.Ia .+ cs_model.Ra .== N)
(conserved_eq, conserved_cs)

```

```
(true, true)
```

Monotonicity. Because transitions are strictly $S \rightarrow I \rightarrow R$, the susceptible count can only fall and the recovered count can only rise:

```

mono_eq = issorted(eq_data.susceptible_status; rev = true) && issorted(eq_data.recovered_status)
mono_cs = issorted(cs_model.Sa; rev = true) && issorted(cs_model.Ra)
(mono_eq, mono_cs)

```

```
(true, true)
```

Ensemble comparison with the mean field. A small ensemble of EventQueueABM runs should scatter around the deterministic prediction:

```

finals = Int[]
for seed in 1:20
    ensemble_rng = Xoshiro(seed)
    ensemble_model = init_eventqueue_model(, c, , N, IO, ensemble_rng)
    ensemble_data, _ = run!(ensemble_model, tf; adata = to_collect)
    push!(finals, last(ensemble_data).recovered_status)
end
mean_final_R = sum(finals) / length(finals)
ode_final_R = sol_ode.u[end][3]
(mean_final_R, round(ode_final_R; digits = 1), abs(mean_final_R - ode_final_R) / ode_final_R)

```

```
(771.4, 775.7, true)
```

All checks pass: both implementations conserve N exactly, both respect the one-way state ordering, and the 20-run ensemble mean final size is within 10% of the deterministic prediction — consistent given the ensemble is small and some realizations have not fully burned out by $t = 40$.

C.6 Interpretation and limitations

Both implementations encode identical dynamics, but the two frameworks make different things easy. `EventQueueABM` handles scheduling and bookkeeping automatically once propensities are declared, and its `adata` mechanism is convenient — but by default it samples the model once per unit time, coarser than the true event resolution, as the visibly “steppier” red curve in Figure C.1 shows. `ConcurrentSim` requires writing the recording logic by hand, but every event is available exactly when it happens, and its general `Resource/Store` primitives (queues, limited capacity, priorities) extend naturally to models that are not naturally agent-centric — a patient-flow model with a limited number of hospital beds, for instance, maps directly onto a `ConcurrentSim Resource` but has no obvious `EventQueueABM` analogue.

Both frameworks share a structural limitation relative to the aggregated jump-process chapter: scheduling and executing $O(N)$ individual events is more expensive than resampling a single aggregate “what happens next,” so neither scales as well to very large populations. The specific transmission rule used here — every susceptible polls a uniformly random alter — is also deliberately inefficient (it costs one event per susceptible regardless of how few individuals are infectious), matching the reference tutorials this appendix is based on; a production-oriented implementation would instead schedule new infection attempts only from currently infectious individuals.

Finally, notice that nothing here required an exponential recovery time. `AgentEvent`’s `timing` argument accepts any function `timing(agent, model, propensity)`, not just the default exponential sampler, and `ConcurrentSim`’s `timeout` accepts any waiting time, fixed or random. A discrete-event framework can therefore encode the *exact* fixed recovery delay of Appendix A directly — Exercise 1 asks you to do exactly that — without any history function or method of steps, because the individual retains its own infection time rather than solving a continuous population-level state.

For the current APIs, see the [Agents.jl EventQueueABM documentation](#) and the [ConcurrentSim.jl repository](#). `Agents.jl` itself is described in (Datseris et al. 2022).

C.7 Exercises

C.7.1 Exercise 1

Modify the `EventQueueABM` recovery event so that its `timing` function returns a fixed value $\tau = 1/\gamma = 4$ instead of sampling an exponential distribution — keep `recovery_propensity` constant and nonzero for infectious agents, and write `fixed_delay_timing(agent, model, propensity) = agent.status == :I ? 4 : Inf` as the event’s `timing`. Run the modified model and compare its infectious curve with the deterministic DDE of Appendix A at the

same parameters. Explain why this individual-level construction reaches the same fixed-delay assumption without a history function.

C.7.2 Exercise 2

Extend the `ConcurrentSim` model with a `Resource` representing a limited number of hospital beds: an infectious individual must successfully `request` a bed before starting the recovery `timeout`, and must `release` it on recovery. Run the model with a small number of beds (for example, 50) and compare the infectious curve and the time individuals spend waiting for a bed with the unconstrained model above. Explain why this kind of resource contention has no natural equivalent in the `EventQueueABM` formulation used in this appendix.

C.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

D Universal differential equations

D.1 Learning goals

A *universal differential equation* (UDE) replaces part of a mechanistic model with a trainable universal approximator — here a small neural network — and fits the whole hybrid system to data by differentiating through the ODE solver (Rackauckas et al. 2020). After working through this appendix you should be able to

- explain which part of the SIR model is being replaced by a neural network and why the rest is kept mechanistic;
- assemble a UDE from a `Lux.jl` network, an `OrdinaryDiffEq` solver, and an adjoint sensitivity method from `SciMLSensitivity.jl`;
- train the hybrid model with `Optimization.jl` using current, non-deprecated APIs; and
- validate the fit by recovering the *known* functional form of the force of infection over the range spanned by the training data.

D.2 Model assumptions and classification

We keep the standard SIR bookkeeping but treat the **force of infection** — the per-susceptible rate of becoming infected — as an unknown function of the infectious proportion. Writing the state as proportions S, I, R with $S + I + R = 1$ and introducing a cumulative-infection tracker C , the model is

- **deterministic** in its dynamics (the stochasticity lives only in the observation model used to generate data);
- **continuous** in time and in state; and
- a **population-level** compartmental model.

The neural network $\mathcal{N}_\theta$ supplies the closure for the force of infection λ . This is the “grey-box” philosophy: conservation of individuals, the recovery term γI , and the flow structure are imposed by hand, while the transmission mechanism is *learned*.

We reuse the course baseline scenario, expressed in proportions. The baseline frequency-dependent rate $\beta c SI/N$ with $\beta = 0.05$, $c = 10$, $N = 1000$ becomes $\beta c SI = 0.5 SI$ once S and I are proportions, so the true force of infection is the linear function $\lambda^*(I) = 0.5 I$, and

$\gamma = 0.25$ as usual. The initial condition $S(0) = 0.99$, $I(0) = 0.01$ corresponds to the baseline counts $S(0) = 990$, $I(0) = 10$ with $N = 1000$.

D.3 Equations

The data-generating (“true”) model closes the force of infection with mass action, $\lambda^*(I) = \beta c I$:

$$\begin{aligned}\frac{dS}{dt} &= -\beta c SI, \\ \frac{dI}{dt} &= \beta c SI - \gamma I, \\ \frac{dC}{dt} &= \beta c SI.\end{aligned}\tag{D.1}$$

The universal differential equation keeps the recovery and flow structure but replaces the transmission closure with a neural network $\lambda_\theta(I) = \mathcal{N}_\theta(I)$:

$$\begin{aligned}\frac{dS}{dt} &= -\lambda_\theta(I) S, \\ \frac{dI}{dt} &= \lambda_\theta(I) S - \gamma I, \\ \frac{dC}{dt} &= \lambda_\theta(I) S.\end{aligned}\tag{D.2}$$

The daily incidence predicted by the model is $N \Delta C$, the scaled difference of the cumulative tracker between observation times. We compare this to noisy case counts y_k with a Poisson negative log-likelihood

$$\mathcal{L}(\theta) = \sum_k [\mu_k(\theta) - y_k \log \mu_k(\theta)], \quad \mu_k(\theta) = N [C(t_{k+1}) - C(t_k)]_\theta, \tag{D.3}$$

and minimise Equation D.3 by gradient descent, with gradients obtained by differentiating through the ODE solve.

D.4 Julia implementation

D.4.1 Setup and synthetic data

We generate reproducible noisy incidence from Equation D.1 using a fixed `StableRNG`, so the observations do not depend on global random state.

```
using OrdinaryDiffEq, Distributions, StableRNGs, Random
using Lux, DiffEqFlux, SciMLSensitivity
using Optimization, OptimizationOptimisers, OptimizationOptimJL
using CairoMakie

N = 1000.0
c = 0.5      # *c from the course baseline (0.05 * 10)
            = 0.25
u0 = [0.99, 0.01, 0.0]      # S, I, C as proportions
tspan = (0.0, 40.0)
t = 1.0
train_time = 30.0
data_rng = StableRNG(20240501)

function sir_true!(du, u, p, t)
    S, I, C = u
    , = p
    du[1] = - * S * I
    du[2] = * S * I - * I
    du[3] = * S * I
end

sol_true = solve(ODEProblem(sir_true!, u0, tspan, [c, ]), Tsit5(); saveat = t)
ts = 0.0:t:train_time
Ctrue = reduce(hcat, sol_true(ts).u)[3, :]
cases = N .* diff(Ctrue)      # true new cases/day
noisy = [rand(data_rng, Poisson(max(c, 0.0))) for c in cases]
length(noisy), maximum(noisy)
```

(30, 54)

D.4.2 A neural closure for the force of infection

We use a small `Lux.Chain` with a single hidden layer of five `tanh` units and a `softplus` output so that $\lambda_\theta(I) \geq 0$ by construction. `Lux.setup` returns the parameters as a nested `NamedTuple` and a (here empty) layer state.

```
foi = Lux.Chain(Lux.Dense(1, 5, tanh), Lux.Dense(5, 1, softplus))
_, foi_state = Lux.setup(StableRNG(1), foi)
```

```
((layer_1 = (weight = Float32[-1.3735459; -0.20602366; ... ; -0.84117234; 0.7100515;], bias =
0.9744537, -0.9340267, -0.13716888, -0.43548012, 0.1660428]), layer_2 = (weight = Float32[-
0.6190029 -0.6169642 ... 0.6468391 -0.16239053], bias = Float32[0.06995858])), (layer_1 = Name
```

`Optimization.jl` optimises over a flat vector, whereas `Lux` parameters are a `NamedTuple`. Production code uses `ComponentArrays.jl` to bridge the two; to keep the mapping explicit — and because that package is not a direct course dependency — we write the (un)flattening by hand for this fixed architecture. The network has 5 weights and 5 biases in the first layer and 5 weights and 1 bias in the second, so 16 trainable parameters in total.

```
function params_from_vector()
    W1 = reshape([1:5], 5, 1); b1 = [6:10]
    W2 = reshape([11:15], 1, 5); b2 = [16:16]
    return (layer_1 = (weight = W1, bias = b1),
            layer_2 = (weight = W2, bias = b2))
end

force_of_infection(I, ) = foi([I], params_from_vector(), foi_state)[1][1]

p_init = 0.1 .* randn(StableRNG(2), Float64, 16)
length(p_init)
```

16

D.4.3 Assembling and training the UDE

The right-hand side implements Equation D.2, calling the neural network for the force of infection. `predict` solves the ODE for a candidate parameter vector, using an `InterpolatingAdjoint` from `SciMLSensitivity.jl` with a reverse-mode vector–Jacobian product so that `Zygote` can backpropagate through the solve.


```

function sir_ude!(du, u, , t)
    S, I, C = u
    = force_of_infection(I, )
    du[1] = - * S
    du[2] = * S - * I
    du[3] = * S
end

prob_ude = ODEProblem(sir_ude!, u0, (0.0, train_time), p_init)

function predict()
    Array(solve(prob_ude, Tsit5()); p = , saveat = t,
           sensealg = InterpolatingAdjoint(autojacvec = ReverseDiffVJP()))
end

function loss()
    pred = predict()
    = max.(N .* diff(pred[3, :]), 1e-6) # predicted cases/day, kept positive
    sum( .- noisy .* log.()) # Poisson negative log-likelihood
end

```

loss (generic function with 1 method)

Training is kept deliberately small so the appendix renders quickly in CI: a short run of Adam to make rapid early progress, followed by a few BFGS iterations to polish the fit. A callback records the loss history.

```

losses = Float64[]
record! = (state, l) -> (push!(losses, l); false)

optf = OptimizationFunction((, _) -> loss(), Optimization.AutoZygote())
res_adam = solve(OptimizationProblem(optf, p_init),
                 OptimizationOptimisers.Adam(0.05);
                 maxiters = 150, callback = record!)
res = solve(OptimizationProblem(optf, res_adam.u),
            OptimizationOptimJL.BFGS();
            maxiters = 50, callback = record!)
_hat = res.u
(final_loss = res.objective, iterations = length(losses))

```

```
Warning: Lux.apply(m::AbstractLuxLayer, x::AbstractArray{<:ReverseDiff.TrackedReal}, ps, st

1. If this was not the desired behavior overload the dispatch on `m`.

2. This might have performance implications. Check which layer was causing this problem usi
@ ArrayInterfaceReverseDiffExt ~/.julia/packages/LuxCore/kQC9S/ext/ArrayInterfaceReverseDif
Warning: Verbosity toggle: dt_epsilon
Initial timestep too small (near machine epsilon), using default: dt = -
1.0e-6
@ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/it3lY/src/initdt.jl:214
Warning: Verbosity toggle: dt_epsilon
Initial timestep too small (near machine epsilon), using default: dt = -
1.0e-6
@ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/it3lY/src/initdt.jl:214
Warning: Verbosity toggle: dt_epsilon
Initial timestep too small (near machine epsilon), using default: dt = -
1.0e-6
@ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/it3lY/src/initdt.jl:214

(final_loss = -1735.438010501897, iterations = 202)
```

```
fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1]; xlabel = "iteration", ylabel = "loss",
          xscale = log10, title = "Training loss")
lines!(ax, 1:length(losses), losses; color = :firebrick, linewidth = 2)
fig
```

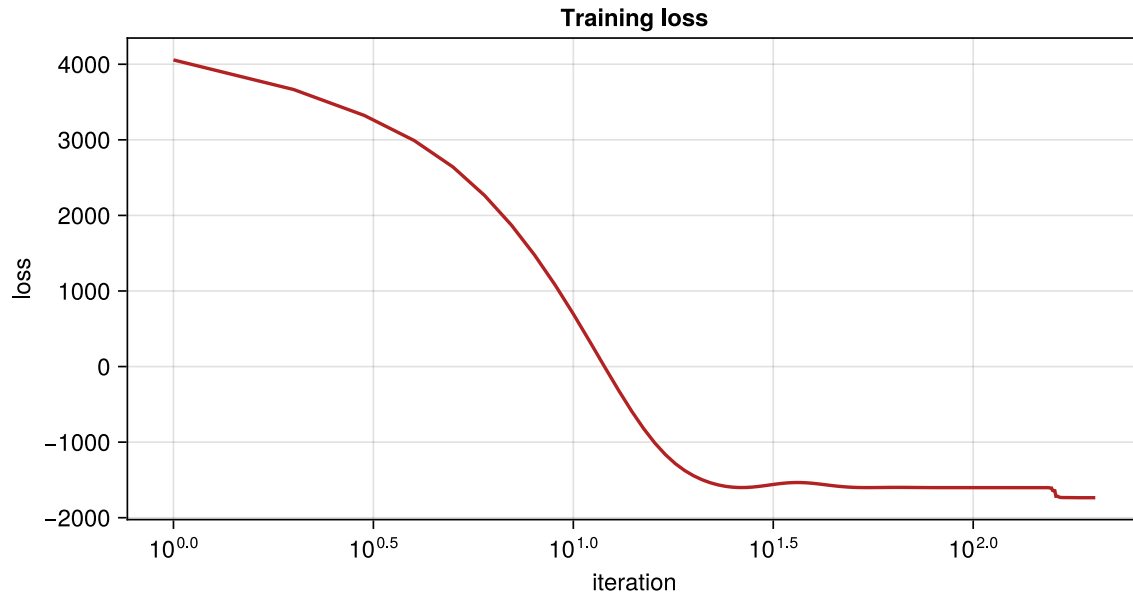


Figure D.1: Poisson negative log-likelihood during training (Adam then BFGS).

D.5 Validation

Because we generated the data from a *known* closure, we can check the fit in the most demanding way: does the trained network recover the true force of infection $\lambda^*(I) = \beta c I$ over the range of I seen during training? Figure D.2 overlays the two, marking the largest infectious proportion in the training window; agreement is expected only up to that bound, since the network is unconstrained beyond the data.

```
Igrid = 0.0:0.005:0.5
_nn    = [force_of_infection(I, _hat) for I in Igrid]
_true  = c .* Igrid
Imax   = maximum(reduce(hcat, sol_true(ts).u)[2, :])
in_range = Igrid .<= Imax
max_abs_error = maximum(abs.(_nn[in_range] .- _true[in_range]))
```

0.00375546757313731

```

sol_fit = solve(ODEProblem(sir_ude!, u0, tspan, _hat), Tsit5(); saveat = t)
I_true = N .* reduce(hcat, sol_true.u)[2, :]
I_fit = N .* reduce(hcat, sol_fit.u)[2, :]
nothing

```

```

fig = Figure(size = (960, 380))
ax1 = Axis(fig[1, 1]; xlabel = "infectious proportion I",
           ylabel = "force of infection ", title = "Learned closure")
lines!(ax1, Igrid, _true; color = :black, linewidth = 2, label = "true c·I")
lines!(ax1, Igrid, _nn; color = :firebrick, linestyle = :dash,
       linewidth = 2, label = "neural network")
vlines!(ax1, [Imax]; color = :gray, linestyle = :dot)
axislegend(ax1; position = :lt)

ax2 = Axis(fig[1, 2]; xlabel = "time (days)", ylabel = "infectious number",
           title = "Fitted trajectory")
vspan!(ax2, train_time, tspan[2]; color = (:gray, 0.12))
lines!(ax2, sol_true.t, I_true; color = :black, linewidth = 2, label = "true I")
lines!(ax2, sol_fit.t, I_fit; color = :firebrick, linestyle = :dash,
       linewidth = 2, label = "UDE I")
axislegend(ax2; position = :rt)
fig

```

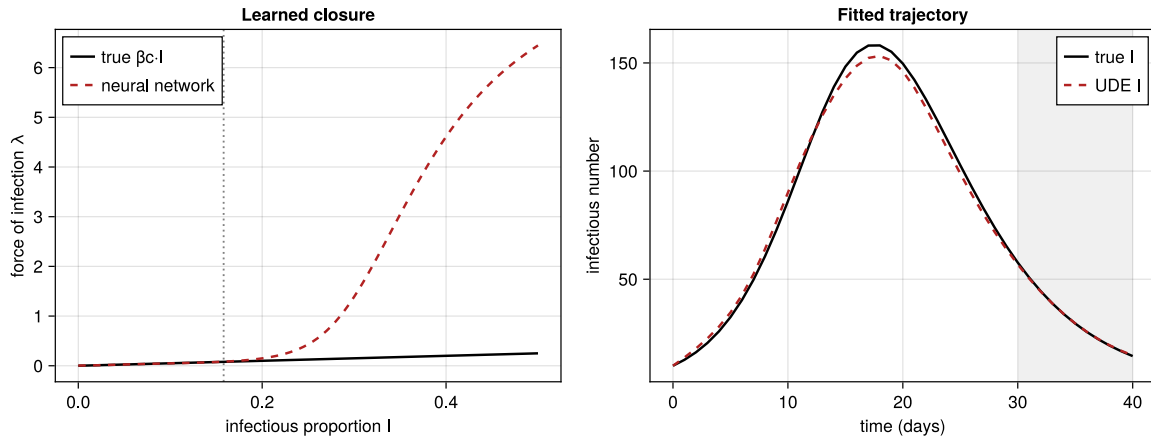


Figure D.2: Left: the neural closure recovers the true linear force of infection up to the largest infectious proportion in the training data (dotted line). Right: the fitted UDE reproduces the infectious-count trajectory, including the untrained extrapolation region beyond day 30.

The `max_abs_error` returned by the `a04-recovery` cell above is small (of order 10^{-3}), confirming that daily incidence alone is sufficient to identify the transmission closure over the observed range of I .

D.6 Interpretation and limitations

- **Identifiability is local to the data.** The network only sees infectious proportions up to `Imax`; to the right of the dotted line in Figure D.2 the closure is unconstrained and should not be trusted. This is the extrapolation region shaded in the trajectory panel.
- **Structure matters.** Keeping the recovery term and the flow structure mechanistic is what makes a 16-parameter network enough. A fully black-box fit would need far more data and offer far less insight.
- **The prior can be encoded.** We know transmission should increase with I ; the `softplus` output enforces non-negativity, and a monotonicity penalty or a monotone network could encode the rest.
- **Cost and conditioning.** Every loss evaluation solves an ODE and every gradient runs an adjoint solve. For stiff systems, or larger networks, the choice of `sensealg` and optimiser becomes performance-critical; the tiny problem here hides that cost.
- **APIs move quickly.** This implementation uses current `Lux` / `DiffEqFlux` / `Optimization` interfaces. Older tutorials built around `FastChain`, `FastDense`, `initial_params`, or `DiffEqFlux.sciml_train` will not run on the package versions pinned for this course.

Official documentation: the [DiffEqFlux.jl manual](#), [Lux.jl documentation](#), and the [SciMLSensitivity.jl manual](#) describe the current UDE workflow and the available adjoint methods.

D.7 Exercises

D.7.1 Exercise 1

The learned closure is unconstrained above the largest infectious proportion in the training window. Re-run the fit using a *shorter* training window (`train_time = 15.0`) so that the epidemic peak is excluded, and re-draw Figure D.2. Over what range of I does the network now match $\lambda^*(I) = \beta c I$, and where does it diverge? Explain the result in terms of which infectious proportions the training data actually visits.

D.7.2 Exercise 2

Replace the single hidden layer with a deeper network (`Lux.Chain(Lux.Dense(1, 8, tanh), Lux.Dense(8, 8, tanh), Lux.Dense(8, 1, softplus))`) and update `params_from_vector` accordingly (there are now $8 + 8 + 64 + 8 + 8 + 1 = 97$ parameters — a generic flatten/unflatten helper is cleaner than hard-coded slices). Does the extra flexibility improve the fit within the training range, and does it change how the network behaves in the extrapolation region? Relate your answer to the bias–variance trade-off.

D.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

E Finite-state projection

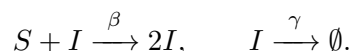
E.1 Learning goals

Stochastic compartmental models are usually studied by simulating many trajectories (Chapter 4). The *finite-state projection* (FSP) method takes the opposite route: it solves directly for the **probability distribution** over states by integrating a truncated version of the chemical master equation (Munsky and Khammash 2006). After this appendix you should be able to

- write the chemical master equation for the stochastic SIR model and explain what “projecting onto a finite state space” means;
- enumerate the finite state space directly, assemble the master equation as a sparse continuous-time Markov chain (CTMC) generator, and solve it as a large linear ODE system with `OrdinaryDiffEq`;
- read off marginal distributions and validate them against a Gillespie ensemble and against probability conservation; and
- quantify how the state space — and therefore the cost — grows with the population size, which is the method’s defining limitation.

E.2 Model assumptions and classification

The stochastic SIR model tracks integer counts (S, I) (with $R = N - S - I$ determined by conservation) evolving through two reaction channels,



The model is

- **stochastic** (a continuous-time Markov jump process);
- **continuous** in time but **discrete** in state; and
- **population-level**, though defined through individual reaction events.

FSP does not change the model — it changes the *solution method*. Instead of sampling trajectories, we evolve the probability $p(s, i, t)$ of occupying each state. The reachable states form a **triangle**: infection sends $(s, i) \mapsto (s-1, i+1)$ (holding $s+i$ fixed) and recovery sends $(s, i) \mapsto (s, i-1)$, so from any start the process only ever visits states with $s \geq 0$, $i \geq 0$ and $s+i \leq N$. We **project** onto this finite triangular set $\{(s, i) : s+i \leq N\}$ and track how much probability leaks out of whatever truncation we impose.

We keep the course transmission and recovery mechanism but reduce the population to $N = 100$ with $S(0) = 99$, $I(0) = 1$. This deviation from the baseline $N = 1000$ is deliberate and necessary: as Section E.6 shows, the triangular state space has $(N+1)(N+2)/2$ elements, so the CME for $N = 1000$ would be a coupled system of order 5×10^5 , whereas $N = 100$ keeps it near 5×10^3 . The pairwise infection rate is set to $\beta = \beta_{\text{base}} c/N = 0.5/100 = 0.005$ so that the mean dynamics match the frequency-dependent baseline force of infection $\beta_{\text{base}} c SI/N$, and $\gamma = 0.25$.

E.3 Equations

Let $p(s, i, t)$ denote the probability that the system is in state $(S, I) = (s, i)$ at time t . The infection propensity is $a_1(s, i) = \beta s i$ and the recovery propensity is $a_2(s, i) = \gamma i$. The chemical master equation balances probability flowing into and out of each state:

$$\frac{dp(s, i, t)}{dt} = \underbrace{\beta(s+1)(i-1)p(s+1, i-1, t)}_{\text{infection into } (s, i)} + \underbrace{\gamma(i+1)p(s, i+1, t)}_{\text{recovery into } (s, i)} - \underbrace{[\beta s i + \gamma i]p(s, i, t)}_{\text{flow out of } (s, i)}. \quad (\text{E.1})$$

Stacking the probabilities of the projected state space into a vector $\mathbf{p}$, Equation E.1 becomes a linear ODE

$$\frac{d\mathbf{p}}{dt} = \mathbf{Q} \mathbf{p}, \quad (\text{E.2})$$

where $\mathbf{Q}$ is the (sparse) CTMC generator restricted to the projected states: its off-diagonal entry Q_{kj} is the transition rate from state j into state k , and each diagonal entry $Q_{jj} = -\sum_{k \neq j} Q_{kj}$ is minus the total rate of leaving state j . We assemble $\mathbf{Q}$ explicitly from the two reaction channels below, and any `OrdinaryDiffEq` solver integrates Equation E.2.

E.4 Julia implementation

E.4.1 A note on FiniteStateProjection.jl

 FiniteStateProjection.jl is not usable with the resolved stack

The FiniteStateProjection.jl package (v0.3.5, tested during the dependency audit) is the natural tool for this appendix, but its released Project.toml **omits ModelingToolkit from its dependencies** even though its source calls ModelingToolkit.varmap_to_vars and ModelingToolkit.scalarize internally. With the current Catalyst/ModelingToolkit releases the package therefore fails to build its right-hand side, and the only way to force it through is to *mutate a third-party module* (binding ModelingToolkit into the package’s namespace at runtime). That is an unacceptable dependency on private, patched-in state, so we **do not include the broken release as a course dependency**. Instead we build the finite-state projection transparently from standard Julia and SciML tools — enumerating the state space, assembling the sparse generator **Q**, and solving Equation E.2 with OrdinaryDiffEq. The method and results are identical; only the assembly is explicit.

For reference, the *intended* high-level API — were the dependency declared — would read as below. This block is **not executed** (eval: false); it merely documents what the convenience wrapper would do.

```
# NOT RUN - see the compatibility note above. Shown only to document the API.
using FiniteStateProjection, Catalyst, OrdinaryDiffEq

rn = @reaction_network SIR begin
    , S + I --> 2I
    , I --> 0
end
sys = FSPPSystem(rn)                # would assemble the CME generator
u0 = zeros(Ncap + 1, Ncap + 1)
u0[100, 2] = 1.0                   # P(S=99, I=1) = 1 at t = 0
prob = ODEProblem(sys, u0, tspan, (, ))
sol = solve(prob, Vern7())          # joint distribution over (S, I)
```

E.4.2 Building the sparse master-equation generator

Everything the projection needs is contained in a single reusable kernel. It enumerates the triangular state space $\{(s, i) : s + i \leq N\}$ (optionally capped at $S_{\max}/I_{\max}$ to study truncation error), assigns each reachable (s, i) an index, and fills the sparse generator **Q** of Equation E.2

from the infection and recovery rates. When a transition would leave the retained set its rate is still subtracted from the diagonal, so any lost probability shows up as leakage in the conservation check.

```
using OrdinaryDiffEq, Catalyst, JumpProcesses, StableRNGs
using SparseArrays, LinearAlgebra, CairoMakie

"""
    sir_fsp_generator(N, , ; Smax = N, Imax = N)

Enumerate the triangular SIR state space  $\{(s, i) : s + i \leq N\}$  (optionally
truncated at `Smax`/`Imax`) and return the sparse CTMC generator `Q`, the vector
of `states`, and a `Dict` mapping each `(s, i)` to its index.  $dp/dt = Q p$ .
"""
function sir_fsp_generator(N, , ; Smax = N, Imax = N)
    index = Dict{Tuple{Int,Int},Int}()
    states = Tuple{Int,Int}[]
    for s in 0:Smax, i in 0:Imax
        if s + i <= N
            push!(states, (s, i))
            index[(s, i)] = length(states)
        end
    end
    n = length(states)
    rows = Int[]; cols = Int[]; vals = Float64[]
    add!(r, c, v) = (push!(rows, r); push!(cols, c); push!(vals, v))
    for (col, (s, i)) in enumerate(states)
        a_inf = * s * i # infection propensity a(s, i)
        a_rec = * i # recovery propensity a(s, i)
        add!(col, col, -(a_inf + a_rec)) # total rate of leaving (s, i)
        if a_inf > 0 && haskey(index, (s - 1, i + 1))
            add!(index[(s - 1, i + 1)], col, a_inf) # infection: (s,i) → (s-1,i+1)
        end
        if a_rec > 0 && haskey(index, (s, i - 1))
            add!(index[(s, i - 1)], col, a_rec) # recovery: (s,i) → (s,i-1)
        end
    end
    return sparse(rows, cols, vals, n, n), states, index
end

rn = @reaction_network SIR begin
    , S + I --> 2I
    , I --> 0
end
```

```
end
nothing
```

E.4.3 Solving the projected master equation

The initial distribution places all probability on the known starting state $(S, I) = (99, 1)$. The right-hand side of Equation E.2 is just the sparse matrix–vector product $\mathbf{Q}\mathbf{p}$, evaluated in place.

```
Ncap = 100
    = 0.005
    = 0.25
tspan = (0.0, 40.0)
t = 1.0

Q, states, index = sir_fsp_generator(Ncap, , )
p0 = zeros(length(states))
p0[index[(99, 1)]] = 1.0 # P(S=99, I=1) = 1 at t = 0

fsp_rhs!(dp, p, Q, t) = mul!(dp, Q, p) # dp/dt = Q p
prob_fsp = ODEProblem(fsp_rhs!, p0, tspan, Q)
sol_fsp = solve(prob_fsp, Vern7(); abstol = 1e-10, reltol = 1e-8, saveat = t)
(states = length(states), nnz_Q = nnz(Q), saved = length(sol_fsp.t))
```

```
(states = 5151, nnz_Q = 15151, saved = 41)
```

Each `sol_fsp.u[k]` is a probability vector over the `length(states)` reachable (S, I) pairs at time t_k . Marginal distributions come from summing the joint probability over all states sharing a given S (or I) value.

```
function marginal(states, p, sym, N)
    m = zeros(N + 1)
    for (k, (s, i)) in enumerate(states)
        m[(sym == :S ? s : i) + 1] += p[k]
    end
    return m
end
counts = 0:Ncap

p20 = sol_fsp.u[21] # t = 20
```

```

PS20 = marginal(states, p20, :S, Ncap)
PI20 = marginal(states, p20, :I, Ncap)
mean_I20 = sum(counts .* PI20)
(mean_I20 = mean_I20, total_prob = sum(p20))

```

```

(mean_I20 = 5.668713917808898, total_prob = 0.9999999999999998)

```

```

fig = Figure(size = (900, 380))
ax1 = Axis(fig[1, 1]; xlabel = "susceptible count S", ylabel = "probability",
           title = "P(S) at t = 20")
barplot!(ax1, counts, PS20; color = (:steelblue, 0.8))
ax2 = Axis(fig[1, 2]; xlabel = "infectious count I", ylabel = "probability",
           title = "P(I) at t = 20")
barplot!(ax2, counts, PI20; color = (:firebrick, 0.8))
vlines!(ax2, [mean_I20]; color = :black, linestyle = :dash)
xlims!(ax2, -1, 40)
fig

```

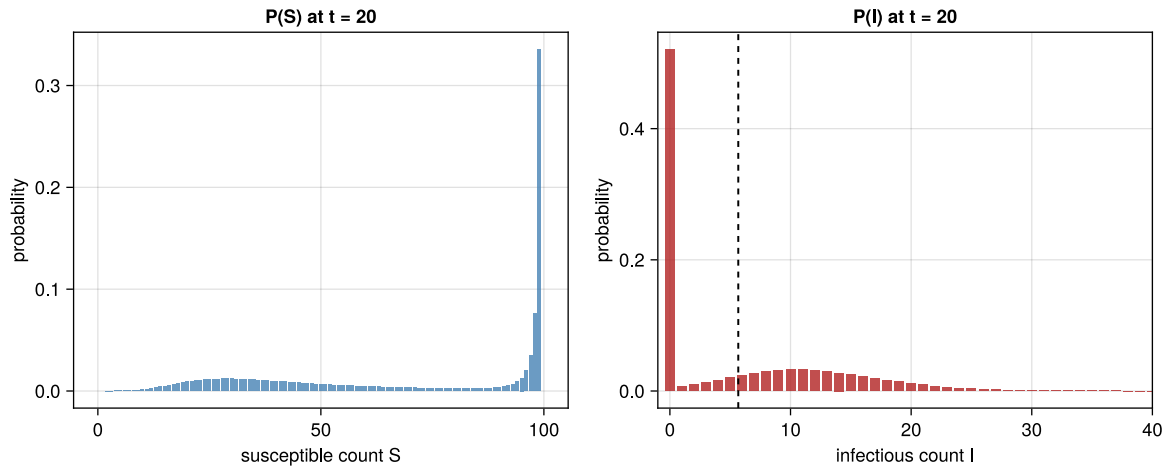


Figure E.1: Finite-state-projection marginal distributions of susceptible and infectious counts at $t = 20$ (bars) with the mean infectious count marked.

E.5 Validation

We check the FSP solution two ways.

Probability conservation. For an *exact* master equation the total probability stays at 1; any shortfall measures probability that has leaked outside the truncation. Because the full triangular state space $\{(s, i) : s + i \leq N\}$ is closed under both reactions, no probability can escape here and the leakage is at the level of solver tolerance.

```
total_prob = [sum(u) for u in sol_fsp.u]
(min_total = minimum(total_prob), max_leak = 1 - minimum(total_prob))
```

```
(min_total = 0.9999999999999998, max_leak = 2.220446049250313e-16)
```

Agreement with a Gillespie ensemble. The FSP marginals should match the empirical distribution from many stochastic simulations of the *same* reaction network. We draw a reproducible ensemble with a seeded StableRNG and compare the mean infectious count at $t = 20$.

```
u0map = [:S => 99, :I => 1]
pmap = [: => , : => ]
jprob = JumpProblem(rn, u0map, tspan, pmap; rng = StableRNG(1))
ens = solve(EnsembleProblem(jprob), SSAS stepper(), EnsembleSerial();
            trajectories = 2000, saveat = t)

I_at_20 = [ens.u[k](20.0)[2] for k in eachindex(ens.u)]
gillespie_mean_I20 = sum(I_at_20) / length(I_at_20)
(fsp = mean_I20, gillespie = gillespie_mean_I20)
```

```
(fsp = 5.668713917808898, gillespie = 5.6905)
```

```
edges = -0.5:1:40.5
fig = Figure(size = (720, 400))
ax = Axis(fig[1, 1]; xlabel = "infectious count I at t = 20",
           ylabel = "probability", title = "FSP vs Gillespie")
barplot!(ax, counts, PI20; color = (:firebrick, 0.5), label = "FSP")
hist!(ax, I_at_20; bins = edges, normalization = :probability,
      color = (:black, 0.0), strokecolor = :black, strokewidth = 1.5,
      label = "Gillespie (2000)")
xlims!(ax, -1, 40)
axislegend(ax)
fig
```

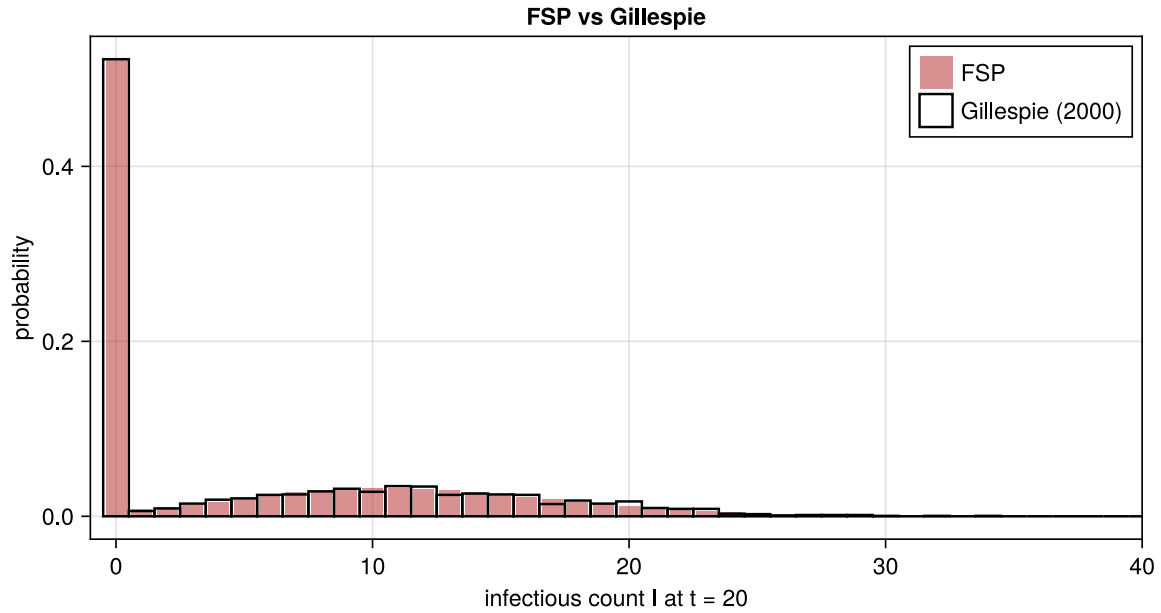


Figure E.2: The finite-state-projection marginal for the infectious count at $t = 20$ (bars) matches a histogram of 2000 Gillespie trajectories (outline), and both share essentially the same mean.

The two means agree to within Monte-Carlo error, and the distributions overlap closely — the FSP has computed, deterministically, what the ensemble approximates by sampling.

E.6 Interpretation and limitations

The strength of FSP is that a single ODE solve returns the *entire* probability distribution, with no sampling noise, giving exact tail probabilities (for example, the probability of early extinction) that would need enormous ensembles to estimate. Its weakness is dimensionality.

The projected state space for the SIR model is the triangle $\{(s, i) : s + i \leq N\}$, which has $(N+1)(N+2)/2$ states, and each is one equation in Equation E.2. The table below makes the growth explicit and contrasts it with the naïve rectangular count $(N+1)^2$:

```
using Printf
for n in (50, 100, 250, 1000)
    triangular = ((n + 1) * (n + 2)) ÷ 2
    @printf("N = %5d    →    triangular CME dimension = %8d    (rectangular %d)\n",

```

```

        n, triangular, (n + 1)^2)
end

```

```

N = 50 → triangular CME dimension = 1326 (rectangular 2601)
N = 100 → triangular CME dimension = 5151 (rectangular 10201)
N = 250 → triangular CME dimension = 31626 (rectangular 63001)
N = 1000 → triangular CME dimension = 501501 (rectangular 1002001)

```

- **State-space explosion.** Doubling N roughly quadruples the number of equations for this two-species model; adding a third tracked species makes the growth cubic. This is why we used $N = 100$ (5151 states) rather than the baseline $N = 1000$, whose triangular CME has of order 5×10^5 states.
- **Truncation is an approximation.** The projection is only exact if the neglected states carry negligible probability. The conservation check above is the diagnostic: if probability leaks, enlarge $S_{\max}$, $I_{\max}$, or use an adaptive/sparse state space.
- **Best for small populations or few species.** FSP shines for early-outbreak and small-population questions where trajectory-level stochasticity matters and the state space stays tractable; for large populations, ensemble simulation (Chapter 4) or moment/diffusion approximations (Chapter 5) are preferable.

Official documentation: [FiniteStateProjection.jl](#), [Catalyst.jl](#), and [ModelingToolkit.jl](#) document the current reaction-network and CME-assembly APIs.

E.7 Exercises

E.7.1 Exercise 1

Using the probability-conservation diagnostic, investigate truncation error. The `sir_fsp_generator` kernel accepts an `Imax` keyword that caps the retained infectious count; any infection event that would push I past `Imax` then leaks out of the projected set. Resolve the master equation with a deliberately tight truncation (e.g. `sir_fsp_generator(Ncap, , ; Imax = 15)`) and plot the total probability $\sum \mathbf{p}$ as a function of time. At roughly what time does probability begin to leak out of the box, and how does that relate to the epidemic peak seen in Figure E.1? What `Imax` would you need to keep the leakage below 10^{-4} ?

E.7.2 Exercise 2

The `a05-growth` cell contrasts the triangular state count $(N+1)(N+2)/2$ with the rectangular count $(N+1)^2$. Confirm that our kernel already exploits reachability — it never allocates states with $s + i > N$ — and quantify the saving as a ratio of the two counts for the N values in the

table. Discuss how a sparse index handler that additionally dropped low-probability states could extend FSP to larger populations. Would exploiting reachability change the *order* of growth with N , or only the constant factor?

E.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

F Probabilistic numerical integration

F.1 Learning goals

Every numerical ODE solve is an approximation, yet a conventional solver returns a single trajectory with no explicit statement of its own error. *Probabilistic numerics* reframes integration as Bayesian inference: the solver returns a **probability distribution** over trajectories whose spread reflects the numerical uncertainty introduced by discretisation (Bosch 2024). After this appendix you should be able to

- explain what numerical uncertainty is and how it accumulates in an SIR solve;
- quantify that uncertainty *empirically* by comparing a working solver against a high-accuracy reference at matched output points;
- read the current `ProbNumDiffEq.jl` Gaussian-filter API (EK0/EK1, IWP priors, diffusion models) and understand what a probabilistic solve would return; and
- reason about when calibrated numerical uncertainty matters for epidemic modelling.

! Reproducibility note: `ProbNumDiffEq.jl` in this environment

The `ProbNumDiffEq.jl` v0.17 release tested during the dependency audit does **not** precompile against the resolved `OrdinaryDiffEq` component stack (`OrdinaryDiffEqDifferentiation` no longer exposes the `standardtag` symbol that `ProbNumDiffEq`'s solver definitions reference). Loading the package raises

```
UndefVarError: `standardtag` not defined in `OrdinaryDiffEqDifferentiation`
```

so it is intentionally not included as a direct course dependency. Rather than fabricate output, this appendix runs a fully executable **error-quantification core** with the standard solver stack, and shows the current `ProbNumDiffEq.jl` API in a **non-executed** block (`eval: false`) so the intended workflow is documented accurately against the audited package version. When the packages are realigned, the gated block runs as written.

F.2 Model assumptions and classification

We use the course baseline SIR model unchanged: $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $N = 1000$, contact rate $c = 10$, transmission probability $\beta = 0.05$, and recovery rate $\gamma = 0.25$. The underlying model is

- **deterministic**, **continuous in time**, and **continuous in state**.

Probabilistic integration does not alter the model. It changes the *solver's output type*: instead of a point estimate $\hat{u}(t)$, a probabilistic solver returns a Gaussian $\mathcal{N}(m(t), P(t))$ whose covariance $P(t)$ encodes how much the trajectory is pinned down by the chosen step size and order. The mean $m(t)$ plays the role of the classical solution; the standard deviation is a calibrated estimate of the discretisation error.

F.3 Equations

The baseline SIR dynamics are

$$\begin{aligned}\frac{dS}{dt} &= -\beta c \frac{SI}{N}, \\ \frac{dI}{dt} &= \beta c \frac{SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I,\end{aligned}\tag{F.1}$$

which the course module `EpiModelingCourse` implements as `sir_ode!`. A numerical solver produces $\hat{u}(t)$; the true numerical error relative to the exact flow $u(t)$ is

$$e(t) = \hat{u}(t) - u(t).\tag{F.2}$$

We cannot evaluate Equation F.2 exactly, but we can approximate $u(t)$ to machine precision with a very high-order, tight-tolerance solve and treat that as the reference. A probabilistic solver instead models $e(t)$ *online* as a Gauss–Markov process: with an ν -times integrated Wiener process (IWP) prior, the posterior standard deviation $\sigma(t) = \sqrt{\text{diag } P(t)}$ is a calibrated proxy for $|e(t)|$, obtained without a reference solution.

F.4 Julia implementation

F.4.1 A reference solution and a working solver

We solve Equation F.1 twice with the standard stack: once at a very tight tolerance with a high-order method to serve as a near-exact reference, and once with a deliberately *coarse* tolerance whose numerical error we then measure.

```
using EpiModelingCourse, OrdinaryDiffEq, CairoMakie

p = BASELINE_PARAMETERS
s0 = BASELINE_INITIAL_STATE
u0 = [s0.S, s0.I, s0.R]
tspan = (0.0, 40.0)
t = 0.5

prob = ODEProblem(sir_ode!, u0, tspan, p)
ref = solve(prob, Vern9(); abstol = 1e-12, reltol = 1e-12, saveat = t)
coarse = solve(prob, Tsit5(); abstol = 1e-1, reltol = 1e-1, saveat = t)

(length(ref.t), length(coarse.t))
```

(81, 81)

F.4.2 Quantifying the numerical error

The elementwise difference between the coarse solve and the reference is a concrete measurement of the discretisation error Equation F.2 — precisely the quantity a probabilistic solver aims to estimate from the inside.

```
refm = reduce(hcat, ref.u)
coarsem = reduce(hcat, coarse.u)
err = abs.(coarsem .- refm) # rows: S, I, R ; cols: time points

max_error = (S = maximum(err[1, :]), I = maximum(err[2, :]), R = maximum(err[3, :]))
terminal_S = (coarse = coarsem[1, end], reference = refm[1, end])
(max_error = max_error, terminal_S = terminal_S)
```

(max_error = (S = 1.4695630845486107, I = 0.8627203714520988, R = 1.3275206033463292), terminal_S = (coarse = 1.4695630845486107, reference = 1.4695630845486107))

```

labels = ("S", "I", "R")
colors = (:steelblue, :firebrick, :seagreen)
fig = Figure(size = (760, 400))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "absolute error",
          yscale = log10, title = "Numerical error vs. reference")
for i in 1:3
    lines!(ax, ref.t, max.(err[i, :], 1e-10); color = colors[i],
           linewidth = 2, label = labels[i])
end
axislegend(ax; position = :rb)
fig

```

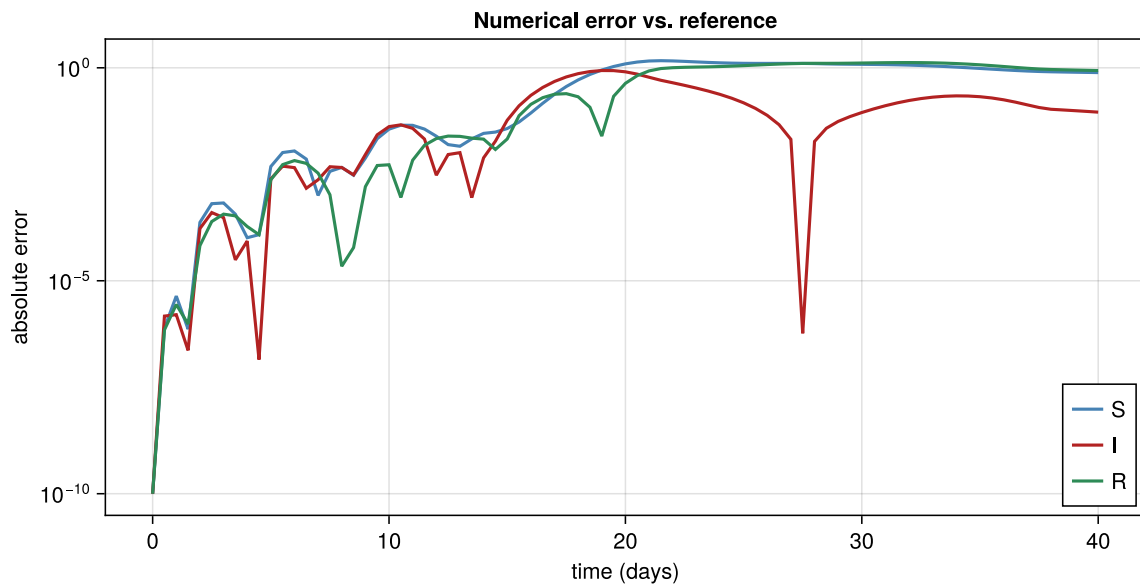


Figure F.1: Absolute numerical error of a coarse-tolerance Tsit5 solve relative to a machine-precision reference, for each SIR compartment. Error grows through the epidemic peak, where the vector field changes fastest — this time-varying magnitude is exactly what a probabilistic solver reports as its posterior standard deviation.

F.4.3 The probabilistic solve (current ProbNumDiffEq.jl API)

The block below shows how the *same* problem would be solved probabilistically with the current v0.17 interface. EK0/EK1 are Gaussian ODE filters; IWP(k) selects a k -times integrated Wiener process prior; `DynamicDiffusion()` calibrates the uncertainty online. The solution object carries a Gaussian at each step, exposed through `sol.pu`, from which `std` gives the calibrated

error estimate and `ProbNumDiffEq.sample` draws trajectories. It is marked `eval: false` for the reason given in the note above.

```
using ProbNumDiffEq, Statistics

# EK1: implicit filter, suitable for the mildly stiff SIR flow.
sol_prob = solve(prob,
                  EK1(order = 3, prior = IWP(3),
                      diffusionmodel = DynamicDiffusion(), smooth = true);
                  dt = t, abstol = 1e-1, reltol = 1e-2)

# Posterior mean and calibrated standard deviation at t = 20.
m20 = mean(sol_prob(20.0))
s20 = std(sol_prob(20.0))          # numerical-uncertainty estimate, no reference needed

# Draw trajectories from the solver's posterior to visualise the uncertainty.
samples = ProbNumDiffEq.sample(sol_prob, 100)

(mean_at_20 = m20, std_at_20 = s20)
```

The comparison is conceptual but direct: the coarse-solve error curve in Figure F.1 is the *true* error that the probabilistic solver’s `std` output is designed to track — largest through the epidemic peak, smaller in the slow tails — but obtained without access to a reference solution.

F.5 Validation

Two checks establish that the executable core behaves as claimed.

The reference is essentially exact. A high-order method at tolerance 10^{-12} conserves the population $S + I + R = N$ to near machine precision, so it is a trustworthy stand-in for the exact flow.

```
pop = [sum(u) for u in ref.u]
(max_pop_drift = maximum(abs.(pop .- p.N)),)
```

```
(max_pop_drift = 4.547473508864641e-13,)
```

Error shrinks with tolerance. Refining the working solver’s tolerance must reduce its error against the reference — the deterministic analogue of a probabilistic solver reporting smaller uncertainty at smaller step sizes.

```

using Printf
for tol in (1e-1, 1e-3, 1e-5, 1e-7)
    s = solve(prob, Tsit5(); abstol = tol, reltol = tol, saveat = t)
    e = maximum(abs.(reduce(hcat, s.u)[2, :] .- refm[2, :]))
    @printf("tol = %.0e    →    max |I error| = %.3e\n", tol, e)
end

```

```

tol = 1e-01    →    max |I error| = 8.627e-01
tol = 1e-03    →    max |I error| = 7.748e-03
tol = 1e-05    →    max |I error| = 3.187e-04
tol = 1e-07    →    max |I error| = 3.619e-06

```

The monotone decrease confirms that the measured error genuinely reflects discretisation, and that a calibrated uncertainty estimate should behave the same way.

F.6 Interpretation and limitations

- **Uncertainty is largest where the dynamics are fastest.** Figure [F.1](#) peaks around the epidemic maximum. A probabilistic solver would widen its posterior there and narrow it in the slow tails, guiding adaptive effort.
- **Calibration is the whole point.** A probabilistic solver is only useful if its reported standard deviation actually brackets the true error. The refinement check is the deterministic version of that calibration question.
- **Cost.** Filtering solvers carry a covariance alongside the state, so a step costs more than a classical one; the payoff is an error bar rather than a bare number. For routine forward simulation this overhead is rarely worth it.
- **Where it matters.** Propagating *numerical* uncertainty alongside parameter uncertainty is valuable when solver error could be mistaken for model error — for example in likelihood-based inference with coarse solves, or when chaining an ODE solve into a downstream decision.
- **Environment caveat.** As flagged above, the probabilistic solve is gated here because of a package-compatibility gap, not because of the method. The gated block reflects the current `ProbNumDiffEq.jl` v0.17 API verified against the installed source.

Official documentation: [ProbNumDiffEq.jl](#) documents the Gaussian-filter solvers, priors, and diffusion models; the [SciML ODE solver docs](#) cover the conventional solvers used in the executable core.

F.7 Exercises

F.7.1 Exercise 1

Using only the executable core, characterise how numerical error scales with the output step. Fix the solver tolerance and recompute the coarse solution for a range of τ (for example 0.1, 0.25, 0.5, 1.0), each time reporting the maximum absolute error in I against the reference. Plot error versus τ on log axes and describe the trend. How does this empirical error curve correspond to what a probabilistic solver encodes in its posterior standard deviation?

F.7.2 Exercise 2

The gated `a06-probnum` block uses the explicit-flavoured `EK1` filter with an `IWP(3)` prior. Consulting the `ProbNumDiffEq.jl` documentation, explain the roles of (a) the prior order in `IWP(k)`, (b) the choice between `EK0` and `EK1`, and (c) `DynamicDiffusion()` versus `FixedDiffusion()`. For the mildly stiff SIR flow, predict which combination should give the best-calibrated uncertainty at a coarse step, and justify your reasoning in terms of stability and error estimation.

F.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

G Surrogate and emulator models

An epidemic simulator can be accurate and still be too expensive to call inside an uncertainty analysis, calibration loop, or optimal-design calculation. A **surrogate** (or emulator) is a cheaper function fitted to simulator outputs. It is not a replacement for the mechanistic model: its predictions are useful only over a declared input domain and should be accompanied by an error assessment.

G.1 Learning goals

By the end of this appendix you should be able to:

- distinguish a simulator, a training design, and a surrogate prediction;
- use the current `Surrogates.jl` 7 interface to fit a radial-basis emulator;
- measure out-of-sample error rather than judging a surrogate only by its training fit; and
- identify when a response surface is a poor summary of an epidemic model.

G.2 Model assumptions and classification

We use the homogeneous baseline SIR model from the course. It is a deterministic, continuous-time, continuous-state, population-level model. A single scalar input, the per-contact transmission probability β , varies while the contact rate $c = 10.0$ and recovery rate $\gamma = 0.25$ remain fixed. The population is $N = 1000$, with $(S(0), I(0), R(0)) = (990, 10, 0)$. The surrogate is therefore a deterministic map from one parameter to one quantity of interest, not a stochastic epidemic model.

We choose the recovered fraction at a finite horizon, $\tau_f = 40$, as the quantity of interest. In a larger application the same workflow could emulate peak prevalence, an observation likelihood, or an intervention cost. The training design has only twelve points, deliberately making the approximation error visible while keeping this example CI-sized.

G.3 Equations

Let $\rho = \beta c$ be the effective transmission rate. The simulator solves

$$\frac{dS}{dt} = -\rho \frac{SI}{N}, \quad \frac{dI}{dt} = \rho \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I. \quad (1)$$

For a selected value of β , the target supplied to the surrogate is

$$q(\beta) = \frac{R(t_f; \beta)}{N}. \quad (2)$$

The fitted function $\widehat{q}(\beta)$ is an approximation to $q(\beta)$. We evaluate it at parameter values that were not used for fitting and calculate

$$e_j = \widehat{q}(\beta_j) - q(\beta_j), \quad \text{RMSE} = \sqrt{\frac{1}{m} \sum_{j=1}^m e_j^2}. \quad (3)$$

In the code, `sir_ode!` implements the three derivatives in Equation 1, `final_recovered_fraction` implements Equation 2, and `RadialBasis` represents $\widehat{q}$.

G.4 Julia implementation

G.4.1 Simulator and training design

The shared course module supplies the baseline state, parameter type, and right-hand side. `Surrogates.sample` with `LatinHypercubeSample()` gives one stratified point per interval in this one-dimensional design. The seed makes the displayed training design reproducible.

```
using CairoMakie
using EpiModelingCourse
using OrdinaryDiffEq
using Random
using Statistics
using Surrogates

const a07_u0 = [990.0, 10.0, 0.0]
const a07_tspan = (0.0, 40.0)
const a07_lb, a07_ub = 0.02, 0.08
```

```

function final_recovered_fraction( )
    parameters = SIRParameters(
        beta = ,
        contact_rate = 10.0,
        gamma = 0.25,
        N = 1000.0,
    )
    problem = ODEProblem(sir_ode!, a07_u0, a07_tspan, parameters)
    solution = solve(
        problem,
        Tsit5();
        saveat = [a07_tspan[2]],
        abstol = 1e-9,
        reltol = 1e-9,
    )
    return solution[3, end] / parameters.N
end

Random.seed!(20260711)
n_train = 12
_train = sample(n_train, a07_lb, a07_ub, LatinHypercubeSample())
y_train = final_recovered_fraction.(_train)

```

```

12-element Vector{Float64}:
 0.8071519990309515
 0.9131875581901288
 0.9317587984202617
 0.73708555561687782
 0.6296326847876723
 0.888458193036905
 0.8547262984673037
 0.1247720041417495
 0.94595053567218
 0.4681467063823326
 0.056632999018633515
 0.27222390982984446

```

G.4.2 Current Surrogates.jl API

Surrogates.jl 7 uses callable surrogate objects. The radial-basis constructor takes locations, responses, lower and upper bounds, and a radial function. The explicit `cubicRadial()` call is

intentional: it is the current constructor form, rather than the zero-argument-free function used in some older examples. We qualify `update!` because plotting packages can export unrelated methods with the same name.

```
surrogate = RadialBasis(  
    _train,  
    y_train,  
    a07_lb,  
    a07_ub;  
    rad = cubicRadial(),  
)  
  
_test = collect(range(a07_lb, a07_ub; length = 64))  
y_test = final_recovered_fraction._test)  
y_pred = surrogate._test)  
errors = y_pred .- y_test  
validation_summary = (  
    rmse = sqrt(mean(errors .^ 2)),  
    max_abs_error = maximum(abs.(errors)),  
    training_points = length(_train),  
)  
validation_summary
```

```
(rmse = 0.005079064991333957, max_abs_error = 0.030386676263557774, training_points = 12)
```

The surrogate is also incrementally extensible through `update!`. This is useful when validation identifies a region that needs another expensive simulator run; it is not a substitute for retaining a held-out validation set.

```
_new = 0.051  
Surrogates.update!(surrogate, _new, final_recovered_fraction(_new))  
surrogate(_new)
```

```
0.7890325067446844
```

G.4.3 Visualizing the approximation and its error

The left panel compares the mechanistic simulator on a dense grid with the surrogate. The right panel is the important diagnostic: it shows absolute error at held-out values. The figure is produced by CairoMakie from the values calculated above, not by manually copying computed numbers.

```

fig = Figure(size = (900, 400))
ax1 = Axis(
    fig[1, 1],
    xlabel = "Transmission probability ",
    ylabel = "Recovered fraction at t = 40",
)
lines!(ax1, _test, y_test; color = :black, linewidth = 3, label = "SIR simulator")
lines!(ax1, _test, y_pred; color = :darkorange, linewidth = 2, label = "Surrogate")
scatter!(
    ax1,
    _train,
    y_train;
    color = :steelblue,
    markersize = 9,
    label = "Training points",
)
axislegend(ax1; position = :lt)

ax2 = Axis(
    fig[1, 2],
    xlabel = "Transmission probability ",
    ylabel = "|surrogate - simulator|",
)
scatter!(ax2, _test, abs.(errors); color = :firebrick, markersize = 7)
hlines!(ax2, [0.01]; color = :gray, linestyle = :dash, label = "1% error")
axislegend(ax2; position = :lt)
fig

```

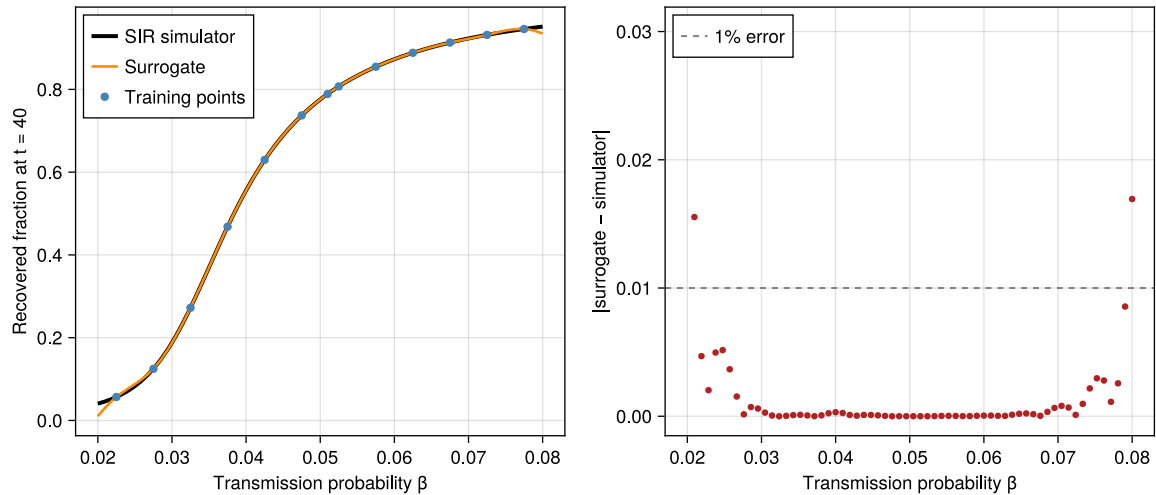


Figure G.1: A radial-basis emulator of the finite-horizon recovered fraction, with held-out absolute error.

For a high-dimensional input, replace the scalar values by vectors and use vector lower and upper bounds. The same callable interface remains, but the training design must cover a much larger space.

G.5 Validation

The validation cell reports error on the 64-point grid, including points that are not training locations. A small maximum error in this example is evidence about this domain and this target only. It does not establish accuracy for a different horizon, a different initial condition, or a two-parameter (β , γ) design. We also check the simulator itself: the SIR total $S + I + R$ is conserved up to the numerical tolerance used by `OrdinaryDiffEq`.

```
_check = 0.053
check_parameters = SIRParameters(beta = _check, contact_rate = 10.0, gamma = 0.25, N = 1000.0)
check_solution = solve(
    ODEProblem(sir_ode!, a07_u0, a07_tspan, check_parameters),
    Tsit5();
    saveat = 0.25,
    abstol = 1e-10,
    reltol = 1e-10,
)
population_error = maximum(abs.(sum(Array(check_solution); dims = 1)[:]) .- check_parameters.I)
(; validation_summary, population_error)
```

```
(validation_summary = (rmse = 0.005079064991333957, max_abs_error = 0.030386676263557774, tra
13)
```

G.6 Interpretation and limitations

The emulator is valuable when each ODE solve is expensive and many predictions are needed. It can support screening, visualization, or a proposal mechanism before a final decision is checked against the simulator. The error plot makes the validation claim appropriately modest: it describes interpolation over the specified interval.

Radial-basis interpolation can extrapolate badly outside `[a07_lb, a07_ub]` and can be sensitive to clustered or poorly scaled inputs. Twelve points are not enough for a serious multi-parameter epidemic analysis. A surrogate can also smooth a threshold, miss a narrow peak, or hide solver failures if the training pipeline records only a scalar output. Preserve the original model, use a separate validation design, and re-simulate any policy or scientific conclusion with the mechanistic model. See the maintained [Surrogates.jl radial-basis documentation](#) and its [surrogate interface reference](#).

G.7 Exercises

G.7.1 Exercise 1

Extend the emulator to two inputs, `and` , over `[0.02, 0.08]` and `[0.15, 0.35]`, while keeping `c`, `N`, and the initial state fixed. Use at least 24 Latin-hypercube training points and a separate 5×5 validation grid. Report the RMSE and maximum absolute error, and explain where the two-dimensional surrogate is least accurate.

G.7.2 Exercise 2

Replace the smooth recovered fraction with the discontinuous-looking target `max(I(t)) / N` over `t` `[0, 40]`. Use the same training and validation design, plot the absolute error, and compare it with the error for `R(40) / N`. Explain why a response involving an internal maximum can require more training points even though the underlying ODE is smooth.

G.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

H Advanced optimal control

Optimal control asks for a time-dependent intervention rather than a single parameter value. The control is itself a function, so discretizing time and discretizing a control are modeling choices. This appendix uses `InfiniteOpt` to state the continuous-time problem and lets its collocation transcription produce a finite nonlinear program solved by `Ipsolve`.

H.1 Learning goals

By the end of this appendix you should be able to:

- distinguish a continuous-time, function-valued control from the finite-dimensional intervention in Chapter 11;
- formulate an ODE-constrained problem with `InfiniteOpt` 0.6;
- inspect the collocated trajectory and check structural identities; and
- explain why the historical nonlinear SDDP example is not a valid current SDDP model.

H.2 Model assumptions and classification

We use normalized population fractions, so $S + I$ is a fraction and C is cumulative infection incidence divided by N . The model is deterministic, continuous in time and state, population-level, and ODE-constrained. The control $u(t)$ is measurable and bounded between zero and $u_{\max}$; it reduces transmission by the factor $1 - u(t)$.

The baseline rates remain $\beta = 0.05$, $c = 10.0$, and $\gamma = 0.25$, so the effective transmission rate is $\beta c = 0.5$. We use $S(0) = 0.99$, $I(0) = 0.01$, and $C(0) = 0$. The finite horizon is $t_f = 20$, $u_{\max} = 0.5$, and the intervention budget is $B = 2.0$. These horizon and budget values are intentionally smaller than a production policy study.

H.3 Equations

The controlled dynamics are

$$\begin{aligned}\dot{S}(t) &= -(1 - u(t))\beta c S(t)I(t), \\ \dot{I}(t) &= (1 - u(t))\beta c S(t)I(t) - \gamma I(t), \\ \dot{C}(t) &= (1 - u(t))\beta c S(t)I(t).\end{aligned}\tag{4}$$

We minimize final cumulative cases plus a small quadratic regularizer:

$$\begin{aligned}\underset{S, I, C, u}{\text{minimize}} \quad & C(t_f) + 10^{-3} \int_0^{t_f} u(t)^2 dt \\ \text{subject to} \quad & 0 \leq u(t) \leq u_{\max}, \quad \int_0^{t_f} u(t) dt \leq B,\end{aligned}\tag{5}$$

with the initial conditions above. `deriv(S, t)` in the code is `\dot{S}(t)`, `integral(u, t)` is the budget measure, and `S(tf)` is a restricted value of the infinite variable at the terminal time.

This is not the same problem class as the core Chapter 11 example, which optimizes a finite set of intervention parameters around an ODE simulation. Here `u` is declared as `Infinite(t)` and is represented numerically by orthogonal collocation supports.

H.4 Julia implementation

H.4.1 InfiniteOpt formulation

The following is a complete, current `InfiniteOpt` 0.6 path. The `OrthogonalCollocation(2)` derivative method supplies a transcription for the ODE constraints. `constant_over_collocation` prevents the piecewise control from acquiring extra, unconstrained values at collocation points.

```
using CairoMakie
using InfiniteOpt
using Ipopt

const a08_tf = 20.0
const a08_beta_c = 0.05 * 10.0
const a08_gamma = 0.25
const a08_ymax = 0.5
```

```

const a08_budget = 2.0

model = InfiniteModel(Ipopt.Optimizer)
set_optimizer_attribute(model, "print_level", 0)
set_optimizer_attribute(model, "max_iter", 1000)
set_optimizer_attribute(model, "tol", 1e-7)

@infinite_parameter(
    model,
    t in [0.0, a08_tf],
    num_supports = 21,
    derivative_method = OrthogonalCollocation(2),
)

@variable(model, S >= 0, Infinite(t), start = 0.99)
@variable(model, I >= 0, Infinite(t), start = 0.01)
@variable(model, C >= 0, Infinite(t), start = 0.0)
@variable(model, 0 <= u <= a08_umax, Infinite(t), start = 0.0)

@constraint(model, S(0.0) == 0.99)
@constraint(model, I(0.0) == 0.01)
@constraint(model, C(0.0) == 0.0)
@constraint(model, deriv(S, t) == -(1 - u) * a08_beta_c * S * I)
@constraint(model, deriv(I, t) == (1 - u) * a08_beta_c * S * I - a08_gamma * I)
@constraint(model, deriv(C, t) == (1 - u) * a08_beta_c * S * I)
@constraint(model, integral(u, t) <= a08_budget)
@objective(model, Min, C(a08_tf) + 1e-3 * integral(u^2, t))

constant_over_collocation(u, t)
optimize!(model)

```

```

*****
This program contains Ipopt, a library for large-scale nonlinear optimization.
Ipopt is released as open source code under the Eclipse Public License (EPL).
For more information visit https://github.com/coin-or/Ipopt
*****

```

The solution query uses the same support values for all four time-dependent variables, which makes the plotted trajectories directly comparable.

```

solver_status = termination_status(model)
objective = objective_value(model)
time_supports = supports(t)
S_values = value.(S)
I_values = value.(I)
C_values = value.(C)
u_values = value.(u)
(; solver_status, objective, number_of_supports = length(time_supports))

```

```
(solver_status = LOCALLY_SOLVED, objective = 0.43004760571351586, number_of_supports = 21)
```

H.4.2 Plotting the policy and trajectory

```

fig = Figure(size = (900, 400))
ax1 = Axis(
    fig[1, 1],
    xlabel = "Time",
    ylabel = "Population fraction",
)
lines!(ax1, time_supports, S_values; color = :steelblue, linewidth = 3, label = "S")
lines!(ax1, time_supports, I_values; color = :firebrick, linewidth = 3, label = "I")
lines!(ax1, time_supports, C_values; color = :seagreen, linewidth = 3, label = "C")
axislegend(ax1; position = :lt)

ax2 = Axis(fig[1, 2], xlabel = "Time", ylabel = "Intervention u(t)")
stairs!(ax2, time_supports, u_values; color = :darkorange, linewidth = 3)
hlines!(ax2, [a08_umax]; color = :gray, linestyle = :dash, label = "u_max")
axislegend(ax2; position = :rb)
fig

```

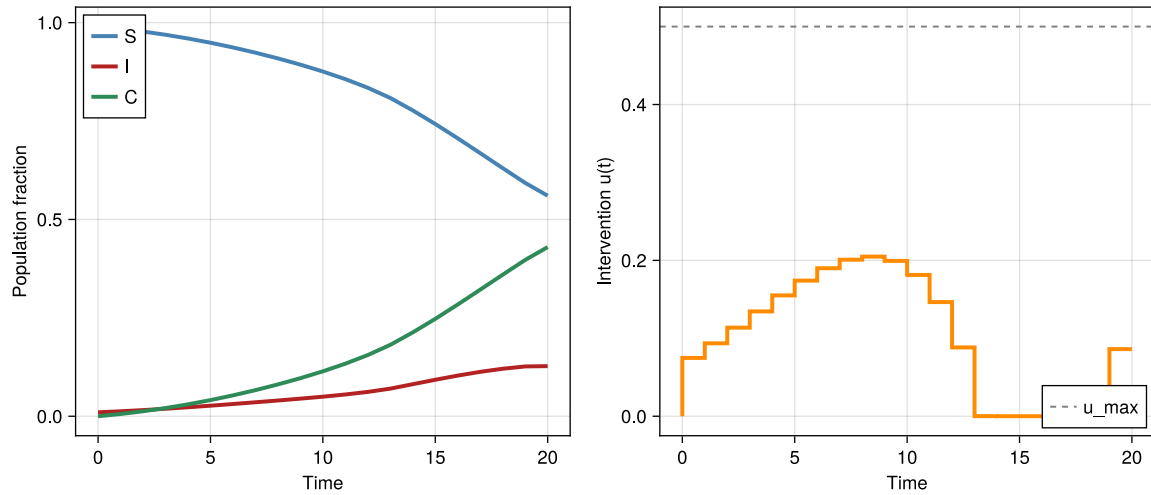


Figure H.1: InfiniteOpt's collocated epidemic trajectory and the optimized continuous-time intervention.

H.5 Validation

Two identities are useful checks for this formulation. Since C and S have opposite incidence derivatives, $C(t) = S(0) - S(t)$ should hold. Also, the collocated policy must remain within its declared bounds. These checks validate the transcription output; they do not prove global optimality of a nonconvex nonlinear program.

```
incidence_identity_error = maximum(abs.(0.99 .- S_values .- C_values))
control_bound_violation = maximum(max.(0.0, u_values .- a08_ymax))
nonnegative_state_violation = minimum(vcat(S_values, I_values, C_values))
(
    incidence_identity_error,
    control_bound_violation,
    nonnegative_state_violation,
    solver_status,
)
```

```
(incidence_identity_error = 5.551115123125783e-17, control_bound_violation = 0.0, nonnegative_state_violation = 1.186161452825772e-43, solver_status = LOCALLY_SOLVED)
```

The first two values should be at floating-point or solver tolerance. The third should be nonnegative. Refining `num_supports` is the natural numerical convergence check: repeat the model with 41 supports and compare the objective and the policy at common times.

H.6 Interpretation and limitations

`InfiniteOpt` separates the mathematical infinite-dimensional problem from its transcription. The result is a policy defined over the time domain, rather than one value per hand-chosen daily interval. Collocation does not remove discretization error, and `Ipopt` reports a local solution for this nonlinear problem. The budget, bounds, and objective are stylized; real interventions have delays, costs, compliance, observation feedback, and integer decisions.

The old `../sir-julia/tutorials/sddp/sddp.jmd` example is intentionally not copied as runnable code here. It places SIR updates containing exponentials and products of state and control variables inside `SDDP.LinearPolicyGraph` using `@NLconstraint`. Current `SDDP.jl` documents policy graphs for multistage linear, mixed-integer linear, and supported quadratic formulations; arbitrary nonlinear epidemic dynamics are outside that supported class. Presenting that old script as current would be misleading. A valid SDDP treatment would first construct a linear or mixed-integer approximation with an explicit error analysis, or use a different nonlinear stochastic-control method. See the [current SDDP manual](#) and (Dowson and Kapelevich 2021) for the supported decomposition setting.

For the continuous formulation used here, consult the [InfiniteOpt quick-start guide](#) and the package's [official documentation](#). The one-lockdown structural result discussed by Britton and Leskelä (Britton and Leskelä 2022) is useful context, but it is not used as a fabricated closed-form validation of this collocated solve.

H.7 Exercises

H.7.1 Exercise 1

Repeat the optimization with 21 and 41 time supports. Interpolate both policies onto a common grid and compare their objectives, final cumulative cases, and maximum pointwise policy difference. Explain which discrepancy is transcription error and which could be a different local optimum.

H.7.2 Exercise 2

Design a supported SDDP approximation rather than reusing the historical nonlinear script: use a finite-horizon linear policy graph for a linear inventory-like intervention budget with exogenous scenario costs. Document the approximation and its relation to the SIR objective, then explain which epidemic nonlinearities have been discarded. Do not claim that an SDDP solution is an exact solution of Equation 4.

H.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

I Model composition and structured populations

Complex epidemic models are easier to audit when they are assembled from named components. A force-of-infection component can be connected to susceptible and infectious components, or a reaction network can be reused as an ODE, chemical-Langevin SDE, or jump process. This appendix shows both patterns with current `ModelingToolkit.jl` 11 and `Catalyst.jl` 16 APIs.

I.1 Learning goals

By the end of this appendix you should be able to:

- compose symbolic `System` components and compile their connections with `mtkcompile`;
- represent a two-group population as a reaction network with explicit cross-group infections; and
- turn one Catalyst model into deterministic, diffusion, and jump problem types while keeping assumptions visible.

I.2 Model assumptions and classification

The first example is a deterministic, continuous-time, continuous-state SIR model assembled from three symbolic components. The components share the population variables through explicit connection equations.

The second example is a deterministic/stochastic comparison for a structured population with two groups. It is a continuous-time model with discrete population states for the jump process and continuous states for its ODE and chemical-Langevin approximations. Each group has S_k , I_k , and R_k . Within-group contact rates are $c_1 = c_2 = 10$; cross-group contact multipliers are $m_{12} = m_{21} = 0.5$. Each group starts with 500 people, five of whom are infectious. These cross-group factors are an intentional departure from homogeneous mixing.

I.3 Equations

For the component example, the connections reduce to the standard equations

$$\dot{S} = -\beta \frac{SI}{N}, \quad \dot{I} = \beta \frac{SI}{N} - \gamma I, \quad \dot{R} = \gamma I, \quad N = S + I + R. \quad (6)$$

For two groups, the infection flow into group 1 is

$$\lambda_1 = \frac{\beta c_1}{N} (S_1 I_1 + m_{12} S_1 I_2), \quad (7)$$

and the analogous flow into group 2 is

$$\lambda_2 = \frac{\beta c_2}{N} (m_{21} S_2 I_1 + S_2 I_2). \quad (8)$$

Thus **S** loses individuals through the first two infection reactions, **I** gains those individuals and loses them at rate **I**, and **R** gains the recoveries. Group 2 follows the same pattern. A reaction network stores these flows as stoichiometric changes plus propensity expressions; Catalyst then generates the problem-specific equations.

I.4 Julia implementation

I.4.1 Composing symbolic SIR components

ModelingToolkit 11's current composition documentation uses **System**, **compose**, and **mtkcompile**. The component constructors below deliberately keep the equations separate. The top-level connection system says which symbols refer to the same biological state and forwards the shared parameters.

```
using CairoMakie
using Catalyst
using JumpProcesses
using ModelingToolkit
using OrdinaryDiffEq
using Random
using StochasticDiffEq
using ModelingToolkit: t_nounits as t, D_nounits as D

@variables S(t) I(t) R(t)
```



```

@parameters

@named susceptible_component = System(
  [D(S) ~ - * S * I / (S + I + R)],
  t,
)
@named infectious_component = System(
  [D(I) ~ * S * I / (S + I + R) - * I],
  t,
)
@named recovered_component = System([D(R) ~ * I], t)

sir_composed = compose(
  System(
    [
      S ~ infectious_component.S,
      I ~ susceptible_component.I,
      R ~ infectious_component.R,
      infectious_component.S ~ susceptible_component.S,
      susceptible_component.I ~ infectious_component.I,
      susceptible_component.R ~ recovered_component.R,
      infectious_component.R ~ recovered_component.R,
      recovered_component.I ~ infectious_component.I,
    ],
    t,
    [S, I, R],
    [ , ],
    bindings = [
      susceptible_component. => ,
      infectious_component. => ,
      infectious_component. => ,
      recovered_component. => ,
    ],
    name = :sir,
  ),
  susceptible_component,
  infectious_component,
  recovered_component,
)

compiled_sir = mtkcompile(sir_composed)
nothing

```

`mtkcompile` removes the algebraic forwarding equations and returns the three-state system in Equation 6. The current problem constructor accepts a single dictionary containing both initial values and parameters.

```
component_problem = ODEProblem(  
    compiled_sir,  
    Dict(  
        susceptible_component.S => 990.0,  
        infectious_component.I => 10.0,  
        recovered_component.R => 0.0,  
        => 0.05,  
        => 0.25,  
    ),  
    (0.0, 10.0),  
)  
component_solution = solve(component_problem, Tsit5(); saveat = 0.1)
```

```
retcode: Success  
Interpolation: 1st order linear  
t: 101-element Vector{Float64}:  
 0.0  
 0.1  
 0.2  
 0.3  
 0.4  
 0.5  
 0.6  
 0.7  
 0.8  
 0.9  
  
 9.2  
 9.3  
 9.4  
 9.5  
 9.6  
 9.7  
 9.8  
 9.9  
10.0  
u: 101-element Vector{Vector{Float64}}:  
 [990.0, 0.0, 10.0]
```

```

[989.9509941524493, 0.2475104065942306, 9.80149544095649]
[989.9029634533144, 0.4901075888814585, 9.606928957804126]
[989.8558884593745, 0.727889042956209, 9.416222497669265]
[989.8097501206208, 0.9609503120524892, 9.229299567326606]
[989.7645297810695, 1.1893849815282338, 9.046085237402243]
[989.7202091452259, 1.4132848439367707, 8.866506010837336]
[989.6767702767373, 1.6327399056720853, 8.69048981759058]
[989.6341955983929, 1.847838386968798, 8.517966014638235]
[989.5924678901473, 2.058666731681047, 8.348865378171611]

[987.9239413597942, 10.49615400436634, 1.5799046358395723]
[987.9162157402092, 10.535254437480651, 1.5485298223102113]
[987.9086435866355, 10.573578451946307, 1.5177779614182498]
[987.9012218174129, 10.611141627021764, 1.4876365555653717]
[987.8939474153115, 10.647959216967877, 1.458093367720765]
[987.886817427531, 10.684046151047884, 1.4291364214211284]
[987.879828965702, 10.719417033527433, 1.400754000770649]
[987.8729792058845, 10.754086143674542, 1.3729346504410407]
[987.8662653987005, 10.788067384161069, 1.3456672171384596]

```

I.4.2 One structured reaction network, three problem types

The Catalyst DSL makes the two-group assumptions explicit. The denominator parameter N keeps the mass-action propensities on the same per-capita scale as the baseline SIR model. The same `ReactionSystem` is passed directly to the current `ODEProblem`, `SDEProblem`, and `JumpProblem` constructors.

```

two_group_network = @reaction_network begin
    @parameters N
    ( * c1) / N, S1 + I1 --> 2I1
    ( * c1 * m12) / N, S1 + I2 --> I1 + I2
    ( * c2 * m21) / N, S2 + I1 --> I2 + I1
    ( * c2) / N, S2 + I2 --> 2I2
    , I1 --> R1
    , I2 --> R2
end

group_initial_values = [
    :S1 => 495.0, :I1 => 5.0, :R1 => 0.0,
    :S2 => 495.0, :I2 => 5.0, :R2 => 0.0,
]

group_parameters = [

```

```

:N => 1000.0,
: => 0.05,
:c1 => 10.0,
:c2 => 10.0,
:m12 => 0.5,
:m21 => 0.5,
: => 0.25,
]
group_tspan = (0.0, 2.0)
group_saveat = 0.05

ode_problem = ODEProblem(
    two_group_network,
    group_initial_values,
    group_tspan,
    group_parameters,
)
ode_solution = solve(ode_problem, Tsit5(); saveat = group_saveat)

Random.seed!(20260711)
sde_problem = SDEProblem(
    two_group_network,
    group_initial_values,
    group_tspan,
    group_parameters,
)
sde_solution = solve(
    sde_problem,
    LambdaEM();
    dt = 0.01,
    saveat = group_saveat,
)

Random.seed!(20260712)
jump_problem = JumpProblem(
    two_group_network,
    [
        :S1 => 495, :I1 => 5, :R1 => 0,
        :S2 => 495, :I2 => 5, :R2 => 0,
    ],
    group_tspan,
    group_parameters,

```

```
)
jump_solution = solve(jump_problem; saveat = group_saveat)
```

```
retcode: Success
Interpolation: Piecewise constant interpolation
t: 53-element Vector{Float64}:
 0.0
 0.05
 0.1
 0.14836632120136636
 0.15
 0.2
 0.25
 0.3
 0.35
 0.4

 1.7
 1.7434135083066356
 1.75
 1.8
 1.85
 1.9
 1.9315040124082579
 1.95
 2.0
u: 53-element Vector{Vector{Int64}}:
 [495, 5, 5, 495, 0, 0]
 [495, 5, 5, 495, 0, 0]
 [495, 5, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]
 [494, 6, 5, 495, 0, 0]

 [493, 5, 7, 491, 2, 2]
 [493, 5, 6, 491, 2, 3]
 [493, 5, 6, 491, 2, 3]
 [493, 5, 6, 491, 2, 3]
```

```
[493, 5, 6, 491, 2, 3]
[493, 5, 6, 491, 2, 3]
[492, 6, 6, 491, 2, 3]
[492, 6, 6, 491, 2, 3]
[492, 6, 6, 491, 2, 3]
```

The ODE and SDE use continuous-valued states. The jump model uses integer counts and Gillespie-style event times. `LambaEM` is the current `StochasticDiffEq` Euler–Maruyama algorithm name used for a chemical-Langevin approximation; its noise can leave the nonnegative domain when populations are small, which is discussed below.

I.4.3 Comparing aggregate infectious prevalence

```
ode_infected = ode_solution[:I1] .+ ode_solution[:I2]
sde_infected = sde_solution[:I1] .+ sde_solution[:I2]
jump_infected = jump_solution[:I1] .+ jump_solution[:I2]

fig = Figure(size = (850, 400))
ax = Axis(fig[1, 1], xlabel = "Time", ylabel = "Infectious individuals")
lines!(ax, ode_solution.t, ode_infected; color = :black, linewidth = 3, label = "ODE")
lines!(ax, sde_solution.t, sde_infected; color = :darkorange, linewidth = 2, label = "SDE")
stairs!(ax, jump_solution.t, jump_infected; color = :steelblue, linewidth = 2, label = "Jump")
axislegend(ax; position = :lt)
fig
```

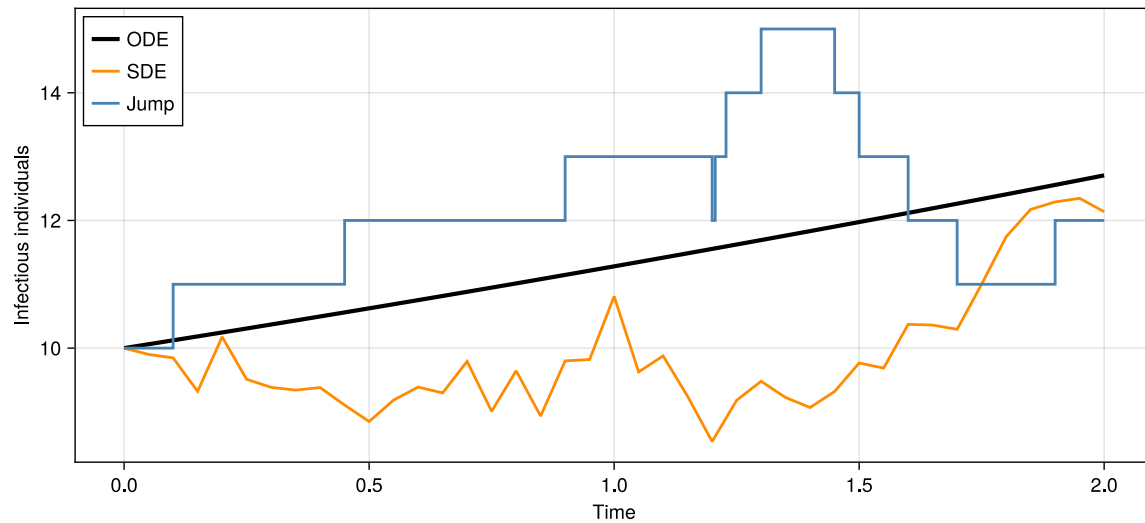


Figure I.1: Aggregate infectious prevalence from a composed ODE, chemical-Langevin SDE, and jump realization.

I.5 Validation

The reaction stoichiometry should conserve the total population in every trajectory (the diffusion approximation is checked separately because floating-point noise can be large). We also check that the composed deterministic model has the expected three equations after compilation.

```
component_equation_count = length(equations(compiled_sir))
ode_population = ode_solution[:S1] .+ ode_solution[:I1] .+ ode_solution[:R1] .+
    ode_solution[:S2] .+ ode_solution[:I2] .+ ode_solution[:R2]
jump_population = jump_solution[:S1] .+ jump_solution[:I1] .+ jump_solution[:R1] .+
    jump_solution[:S2] .+ jump_solution[:I2] .+ jump_solution[:R2]
ode_population_error = maximum(abs.(ode_population .- 1000.0))
jump_population_error = maximum(abs.(jump_population .- 1000))
(;
    component_equation_count,
    ode_population_error,
    jump_population_error,
    sde_minimum_state = minimum(Array(sde_solution)),
)
```

```
(component_equation_count = 3, ode_population_error = 2.2737367544323206e-
13, jump_population_error = 0, sde_minimum_state = -0.5297903304879045)
```

The deterministic and jump population errors should be near zero. The SDE minimum is reported rather than silently clipped; a negative value is a diagnostic of using an unconstrained Gaussian diffusion for count data, not a biological prediction.

I.6 Interpretation and limitations

Composition makes assumptions inspectable: connections are named, parameters are forwarded explicitly, and the compiled equations can be checked before simulation. A reaction network gives a second kind of reuse: the same stoichiometric model can support a mean-field ODE, a chemical-Langevin SDE, and an exact-event jump process. This is useful for comparing approximation error, not for assuming that the three trajectories are interchangeable.

Two-group mixing is only one structured-population choice. Age structure, spatial movement, births, waning immunity, and contact networks require additional states and reactions. A chemical-Langevin SDE is most defensible when propensities are sufficiently large; it does not enforce integer counts or positivity. The jump model is discrete and stochastic, so one realization is not a replacement for an ensemble or uncertainty interval.

The current [ModelingToolkit composition guide](#) documents `System`, `compose`, and `mtkcompile`. The [Catalyst documentation](#) documents the DSL and direct construction of ODE, SDE, and jump problems. For a primary software reference, see [Loman et al. \(2023\)](#). The structured mixing idea follows the contact-pattern formulation of Jacquez et al. (Jacquez et al. 1988).

I.7 Exercises

I.7.1 Exercise 1

Add a third population group with its own contact rate and two cross-group mixing multipliers. Keep the total population at 1000, construct the network with the Catalyst DSL, and verify conservation for both an ODE solve and a jump realization. Compare the aggregate infectious curve with the two-group model and state which new parameters are identifiable from aggregate incidence alone.

I.7.2 Exercise 2

Use the same reaction network to run at least 100 jump realizations with a deterministic seed sequence. Plot the mean and a pointwise 90% interval for aggregate infectious prevalence, and compare the mean with the ODE trajectory. Explain why the SDE curve need not lie inside that interval and how the choice of N and initial counts affects the approximation.

I.8 Companion materials

- [Slides](#)
- [Pluto preview](#)
- [Pluto source](#)

J Model and API cheat sheet

Use this table to choose a representation from assumptions and outputs before choosing a package.

Model type	Time	State	Stochastic?	Mathematical form	Primary Julia API	Main use	Key limitation
Function map	Discrete	Continuous	No	Recurrence	Plain Julia loop or <code>DiscreteProblem</code>	Fixed-step deterministic dynamics	Timestep is part of the model
Markov chain	Discrete	Discrete	Yes	Transition distribution	<code>Distribution</code>	Continuous valued stochastic dynamics	Multiple events are bundled per step
Ordinary differential equation	Continuous	Continuous	No	$\dot{u} = f(u, p, t)$	<code>ODEProblem</code> <code>solve</code>	Deterministic population dynamics	Omits demographic variability
Jump process	Continuous	Discrete	Yes	Event rates and state changes	<code>JumpProblem</code> <code>SSAStepper</code>	Exact stochastic event simulation	Event-by-event cost grows with activity
Stochastic differential equation	Continuous	Continuous	Yes	$du = f dt + g dW$	<code>SDEProblem</code>	Diffusion approximations	Gaussian noise can cross state boundaries

Model type	Time	State	Stochastic?	Mathematical form	Primary Julia API	Main use	Key limitation
Agent-based model	Discrete or event-driven	Individual	Usually	Agent rules	StandardABM, EventQueueABM	Heterogeneity and individual behavior	More parameters and computational cost
Delay differential equation	Continuous	Continuous	No	$\dot{u}(t) = f(u(t), u(t-\tau), p, t)$	DDEProblem, MethodOfSteps	Fixed-step and delayed effects	Requires a history and lag-aware numerics
Stochastic delay equation	Continuous	Continuous	Yes	Delayed drift and diffusion	SDDEProblem, LambdaEM	Noisy delayed dynamics	Current solver path is experimental
Discrete-event simulation	Event-driven	Individual/process	Usually	Scheduled process events	EventQueueABM, ConcurrentSim	Queues, resources, irregular events	Detailed book-keeping can dominate runtime
Observation model	Observation times	Counts or measurements	Yes	$Y_t \sim p(y x_t, \theta)$	Distributions.jl	Latent dynamics to data	Process and reporting effects may confound
Likelihood optimization	Depends on process model	Parameters	No	$\hat{\theta} = \arg \max_{\theta} \ell(\theta y)$	Optimization.jl, ProfileLikelihoods.jl	Problem, likelihoods and profile intervals	Local optima and practical non-identifiability

Model type	Time	State	Stochastic?	Mathematical form	Primary Julia API	Main use	Key limitation
Bayesian inference	Depends on process model	Parameters/lattice states	Yes	$p(\theta y) \propto p(y \theta)p(\theta)$	Turing. @model NUTS	Posterior uncertainty and prediction	Computation and conclusions depend on priors
Sensitivity analysis	Depends on process model	Model outputs	No or sampled	Derivatives or variance decomposition	SciML Sensitivity, GlobalSensitivity	Ranking parameters	Rankings depend on output and parameter range
Structural identifiability	Continuous	Symbolic	No	Input-output parameter map	assess_identifiability	Defiability, mine learnable parameter combinations	Assumes ideal noise-free observations
Optimal intervention	Continuous or discrete	State and control	Usually no	$\min_u J(x, u)$ subject to dynamics	Optimization, InfiniteModel	Control problem, optimize decisions	A local mathematical optimum may be infeasible
Universal differential equation	Continuous	Continuous	Learned component	Mechanistic f plus neural component	Lux.Chain, Learn Optimization, unknownsolve	Learn mechanisms	Extrapolation and identifiability remain fragile
Finite-state projection	Continuous	Probability vector	Yes	Truncated chemical master equation	Sparse generator + ODEProblem	Approximate state probabilities	State space grows combinatorially

Model type	Time	State	Stochastic?	Mathematical form	Primary Julia API	Main use	Key limitation
Probabilistic numerical solver	Continuous	Distribution over solutions	Numerical uncertainty	Filtering-based ODE solve	ProbNumDiffEqEKF1 (compatibility gated)	Expensive and currently gated here	
Surrogate model	Input design	Approximate output	Optional	$\hat{f}(x) \approx f(x)$	SurrogatesRadialBasis	Accuracy is local to the validated domain	
Composed reaction model	Continuous or event-driven	Continuous/discrete	Either	Reaction system transformed by target representation	ReactionSystemODEProblemJumpProblem	One specification across solvers	Composition does not validate assumptions

J.1 Common SIR baseline

Symbol	Meaning	Baseline
$S(0)$	Initially susceptible	990
$I(0)$	Initially infected	10
$R(0)$	Initially recovered	0
N	Population size	1000
c	Contact rate	10.0
β	Transmission probability per infectious contact	0.05
γ	Recovery rate	0.25

K Exercise solutions

The solutions follow the same unit and exercise identifiers as the course.

K.1 Prerequisite and foundational solutions

K.1.1 Julia essentials for modelers

K.1.1.1 Solution to Exercise 1

We extend the in-place kernel with a fourth state C whose derivative is the force of infection times S , i.e. the instantaneous incidence λS . Since new infections *leave* S at exactly the rate they *enter* C , we expect $\text{du}[4] == -\text{du}[1]$.

```
function sir_incidence!(du, u, p, t)
    S, I, R, C = u
    , c, , N = p
    = * c * I / N
    du[1] = - * S
    du[2] = * S - * I
    du[3] = * I
    du[4] = * S          # cumulative incidence
    return nothing
end

u0 = [990.0, 10.0, 0.0, 0.0]
p = (0.05, 10.0, 0.25, 1000.0)
du = zeros(4)
sir_incidence!(du, u0, p, 0.0)
(du = du, matches = isapprox(du[4], -du[1]; atol = 1e-12))
```

Solution K.1.

```
(du = [-4.95, 2.45, 2.5, 4.95], matches = true)
```

The check returns `true`: cumulative incidence grows at precisely the rate the susceptible pool depletes, because both equal λS . The recovered term γI does not enter C — recovery is a separate flow out of I .

K.1.1.2 Solution to Exercise 2

`rand` returns uniform draws on $[0, 1)$, so the fraction below a threshold q estimates q itself. Here $q = 1 - e^{-0.25} \approx 0.221$.

```
using StableRNGs
rate_to_proportion(r, t) = -expm1(-r * t)

rng = StableRNG(2024)
draws = rand(rng, 10_000)
q = rate_to_proportion(0.25, 1.0)
fraction = count(<(q), draws) / length(draws) # broadcasting via count(<(q), .)
(target = q, observed = fraction, absolute_error = abs(fraction - q))
```

Solution K.2.

```
(target = 0.22119921692859512, observed = 0.2232, absolute_error = 0.002000783071404888)
```

The observed fraction sits within about 0.01 of the target. Because the stream is produced by a *local* `StableRNG(2024)`, whose algorithm is fixed across Julia versions and platforms, every reader obtains the identical `draws` vector and the identical fraction — the figure is bit-for-bit reproducible, unlike one seeded through the global `default_rng()`.

K.1.2 Probability and stochastic processes

K.1.2.1 Solution to Exercise 1

The $\text{Binomial}(n, p)$ variance is $np(1 - p)$. We compare the sample variance of 100 000 draws against this value for two probabilities.

```
using Distributions, StableRNGs, Statistics

function variance_check(n, p; seed = 7, nsamples = 100_000)
    rng = StableRNG(seed)
    samples = [rand(rng, Binomial(n, p)) for _ in 1:nsamples]
    (theoretical = n * p * (1 - p), empirical = var(samples))
end
```

```

end

low = variance_check(990, 0.02)
half = variance_check(990, 0.50)
(p_002 = low, p_050 = half)

```

Solution K.3.

```
(p_002 = (theoretical = 19.404, empirical = 19.289134810948113), p_050 = (theoretical = 247.5, empirical = 247.5))
```

Both empirical variances match $np(1-p)$ closely. Viewed as a function of p , $np(1-p)$ is a downward parabola maximised at $p = 0.5$: the outcome of each Bernoulli trial is then maximally uncertain (a fair coin), so the sum fluctuates most. Near $p = 0$ or $p = 1$ almost every trial has the same outcome and the variance collapses — hence the $p = 0.5$ case has by far the larger spread.

K.1.2.2 Solution to Exercise 2

We rerun the ensemble at the coarse ($\delta t = 1$) and fine ($\delta t = 0.1$) steps over the same 40-day horizon and record the extinction fraction.

```

using Distributions, StableRNGs
rate_to_proportion(r, t) = -expm1(-r * t)

function chain_step(state, p, t, rng)
    S, I, R = state; , c, , N = p
    Y = rand(rng, Binomial(S, rate_to_proportion( * c * I / N, t)))
    Z = rand(rng, Binomial(I, rate_to_proportion( , t)))
    (S - Y, I + Y - Z, R + Z)
end

function extinction_fraction(t; nsims = 200, horizon = 40.0)
    p = (0.05, 10.0, 0.25, 1000); nsteps = round{Int, horizon / t}
    extinct = 0
    for seed in 1:nsims
        rng = StableRNG(seed); s = (990, 10, 0)
        faded = false
        for _ in 1:nsteps
            s = chain_step(s, p, t, rng)
            s[2] == 0 && (faded = true)
        end
        if faded
            extinct += 1
        end
    end
    extinct / nsims
end

```



```

        end
        faded && (extinct += 1)
    end
    extinct / nsims
end

(coarse_dt1 = extinction_fraction(1.0), fine_dt0_1 = extinction_fraction(0.1))

```

Solution K.4.

```
(coarse_dt1 = 0.0, fine_dt0_1 = 0.005)
```

Extinction is rare at $R_0 = 2$: no realisation faded out at $\delta t = 1$, while a small fraction did at $\delta t = 0.1$. Finer steps change the discrete dynamics because the per-step probabilities p_I, p_R are *nonlinear* in δt : a single coarse step lumps together events that finer steps resolve individually, and the binomial construction cannot let an individual become infected *and* recover within one step. Resolving the early phase in smaller increments exposes more chances for the last infective to recover before transmitting, so the finer grid detects the occasional fade-out the coarse grid misses. As $\delta t \rightarrow 0$ these artefacts vanish and the chain approaches the exact continuous-time Markov process.

K.1.3 Calculus and numerical methods

K.1.3.1 Solution to Exercise 1

The midpoint (RK2) method evaluates the derivative at a half-step to cancel the leading error term, giving second-order global accuracy.

```

function sir_ode!(du, u, p, t)
    S, I, R = u; c, N = p
    = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; return nothing
end

function midpoint(f, u0, p, t, tmax)
    ts = 0.0:t:tmax
    us = Matrix{Float64}(undef, length(u0), length(ts)); us[:, 1] .= u0
    k1 = similar(u0); k2 = similar(u0)
    for n in 1:length(ts) - 1
        u = @view us[:, n]

```

```

        f(k1, u, p, ts[n])
        f(k2, u .+ (t / 2) .* k1, p, ts[n] + t / 2)
        us[:, n + 1] .= u .+ t .* k2
    end
    collect(ts), us
end

u0 = [990.0, 10.0, 0.0]; p = (0.05, 10.0, 0.25, 1000.0); tmax = 40.0
_, ref = midpoint(sir_ode!, u0, p, tmax / 2^13, tmax); u_ref = ref[:, end]
ts = [tmax / 2^k for k in 5:10]
errs = map(t -> maximum(abs.(midpoint(sir_ode!, u0, p, t, tmax)[2][:, end] .- u_ref)), ts)
orders = [log2(errs[k] / errs[k + 1]) for k in 1:length(errs) - 1]
(mean_order = sum(orders) / length(orders), close_to_2 = isapprox(sum(orders) / length(orders), 2))

```

Solution K.5.

```
(mean_order = 1.976552348167555, close_to_2 = true)
```

The mean empirical order is close to 2 — one higher than forward Euler — so halving the step now cuts the error roughly fourfold rather than twofold.

K.1.3.2 Solution to Exercise 2

First the relative error in I at $\delta t = 0.1$, using a fine Euler reference; then a search for the largest step keeping every compartment nonnegative.

```

function euler(f, u0, p, t, tmax)
    ts = 0.0:t:tmax
    us = Matrix{Float64}(undef, length(u0), length(ts)); us[:, 1] .= u0
    du = similar(u0)
    for n in 1:length(ts) - 1
        f(du, @view(us[:, n]), p, ts[n]); us[:, n + 1] .= us[:, n] .+ t .* du
    end
    collect(ts), us
end

_, ref2 = euler(sir_ode!, u0, p, tmax / 2^13, tmax); Iref = ref2[2, end]
_, u01 = euler(sir_ode!, u0, p, 0.1, tmax)
rel_err_I = abs(u01[2, end] - Iref) / Iref

```

```

nonneg(t) = all( (0), euler(sir_ode!, u0, p, t, tmax)[2])
candidates = 0.1:0.2:6.0
largest_stable = maximum(t for t in candidates if nonneg(t))
(relative_error_I = rel_err_I, largest_nonnegative_step = largest_stable)

```

Solution K.6.

```
(relative_error_I = 0.009565965249712495, largest_nonnegative_step = 5.9)
```

The relative error in the infectious compartment at $\delta t = 0.1$ is a fraction of a percent. Beyond a critical step size, forward Euler overshoots and drives a compartment negative — an instability the reported `largest_nonnegative_step` locates. This conditional stability, not accuracy alone, is why adaptive solvers are preferred.

K.1.4 Foundations: SIR theory and the Julia workflow

K.1.4.1 Solution to Exercise 1

Holding $\beta = 0.05$ and $\gamma = 0.25$, $R_0 = \beta c / \gamma$ scales linearly with the contact rate. The threshold $R_0 = 1$ occurs at $c = \gamma / \beta = 5$.

```

using NonlinearSolve, DataFrames

, , N = 0.05, 0.25, 1000.0
s = 990.0 / N
attack(R0) = 1 - solve(NonlinearProblem((s, r) -> s - s * exp(-r * (1 - s)), 0.2, R0), Newton

DataFrame(c = [4, 6, 8, 10],
          R0 = [ * c / for c in (4, 6, 8, 10)],
          attack_rate = [round(attack( * c / ); digits = 3) for c in (4, 6, 8, 10)])

```

Solution K.7.

	c	R0	attack_rate
	Int64	Float64	Float64
1	4	0.8	0.045
2	6	1.2	0.348
3	8	1.6	0.65
4	10	2.0	0.8

For $c = 4$ we have $R_0 = 0.8 < 1$ and the attack rate (≈ 0.045) barely exceeds the initial 1% seed — the few initial infections cannot sustain a chain of transmission, so no epidemic occurs. For $c \geq 6$ ($R_0 \geq 1.2$) a genuine epidemic develops and the attack rate climbs steeply with R_0 . The switch happens exactly at $c = 5$, where $R_0 = 1$: below it each infection produces fewer than one secondary case on average.

K.1.4.2 Solution to Exercise 2

We locate the infectious peak and read off the susceptible fraction there; theory predicts $S/N = 1/R_0$.

```
using OrdinaryDiffEq
function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p;  = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; return nothing
end

u0 = [990.0, 10.0, 0.0]
sol = solve(ODEProblem(sir_ode!, u0, (0.0, 80.0), (0.05, 10.0, 0.25, 1000.0)), Tsit5(); saveat=10)
I = sol[2, :]; R0 = 0.05 * 10.0 / 0.25
peak_ok = isapprox(sol[1, argmax(I)] / 1000.0, 1 / R0; atol = 0.01)

sol_crit = solve(ODEProblem(sir_ode!, u0, (0.0, 80.0), (0.05, 10.0, 0.5, 1000.0)), Tsit5(); saveat=10)
(R0 = R0, S_over_N_at_peak = sol[1, argmax(I)] / 1000.0, threshold_ok = peak_ok,
 peak_I_baseline = round(maximum(I); digits = 1),
 peak_I_R0eq1 = round(maximum(sol_crit[2, :]); digits = 1))
```

Solution K.8.

```
(R0 = 2.0, S_over_N_at_peak = 0.5005887830839842, threshold_ok = true, peak_I_baseline = 158)
```

At the baseline $R_0 = 2$ the susceptible fraction at the peak equals $1/R_0 = 0.5$ to within 1%, confirming Equation 4.4. Raising γ to 0.5 makes $R_0 = 1$: the growth rate $\lambda S - \gamma I$ is negative from the outset because $S/N = 0.99 < 1 = 1/R_0$ only marginally, and prevalence merely decays from its initial value — there is no epidemic wave, only monotone extinction of the initial infectives.

K.1.5 Discrete-time models

K.1.5.1 Solution to Exercise 1

We compare the deterministic map and the stochastic-chain mean against a high-accuracy ODE reference at $\delta t = 1$ and $\delta t = 0.25$.

```
using Distributions, StableRNGs, OrdinaryDiffEq
rate_to_proportion(r, t) = -expm1(-r * t)
function map_step(s, p, t)
    S, I, R = s; , c, , N = p
    inf = rate_to_proportion( * c * I / N, t) * S
    rec = rate_to_proportion( , t) * I
    (S - inf, I + inf - rec, R + rec)
end
function chain_step(s, p, t, rng)
    S, I, R = s; , c, , N = p
    Y = rand(rng, Binomial(round(Int, S), rate_to_proportion( * c * I / N, t)))
    Z = rand(rng, Binomial(round(Int, I), rate_to_proportion( , t)))
    (S - Y, I + Y - Z, R + Z)
end
function run_traj(step, u0, p, t, n; extra = ())
    traj = Matrix{Float64}(undef, 3, n + 1); traj[:, 1] .= u0; s = Tuple(float.(u0))
    for k in 1:n; s = step(s, p, t, extra...); traj[:, k + 1] .= s; end
    traj
end

function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p; = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; return nothing
end
p = (0.05, 10.0, 0.25, 1000.0)
ode_peak = maximum(solve(ODEProblem(sir_ode!, [990.0, 10, 0], (0.0, 40.0), p), Tsit5(); saveat=1.0))

function peaks(t; nsims = 60)
    n = round(Int, 40 / t)
    map_peak = maximum(run_traj((s, p, d) -> map_step(s, p, d), [990, 10, 0], p, t, n)[2, :])
    ens = [run_traj(chain_step, [990, 10, 0], p, t, n; extra = (StableRNG(s),)) for s in 1:nsims]
    chain_mean_peak = maximum(vec(sum(reduce(hcat, (t[2, :] for t in ens)); dims = 2)) ./ nsims)
    (map_peak = round(map_peak; digits = 1), chain_mean_peak = round(chain_mean_peak; digits = 1))
end

(ode_peak = round(ode_peak; digits = 1), dt1 = peaks(1.0), dt025 = peaks(0.25))
```

Solution K.9.

(ode_peak = 158.5, dt1 = (map_peak = 200.9, chain_mean_peak = 194.6), dt025 = (map_peak = 168.5, chain_mean_peak = 168.5))

The deterministic map at $\delta t = 0.25$ lands much closer to the ODE peak (≈ 158) than at $\delta t = 1$, because the map is a first-order (forward-Euler-like) discretisation whose error shrinks with the step. The stochastic-chain mean lies close to the map at each step size — it is the map’s random counterpart — but its peak is additionally *lowered* by peak-timing jitter: individual epidemics crest at slightly different times, so averaging them flattens the ensemble-mean peak, which can therefore sit even nearer to (or just below) the ODE value than the map does. Either way, refining δt moves both discrete peaks toward the continuous-time reference.

K.1.5.2 Solution to Exercise 2

Extinction probability is the fraction of realisations reaching $I = 0$ before day 40, computed at two contact rates.

```
function extinction(c; nsims = 200, t = 1.0)
    p = (0.05, c, 0.25, 1000.0); n = round(Int, 40 / t); extinct = 0
    for seed in 1:nsims
        rng = StableRNG(seed); s = (990, 10, 0); faded = false
        for _ in 1:n; s = chain_step(s, p, t, rng); s[2] == 0 && (faded = true); end
        faded && (extinct += 1)
    end
    extinct / nsims
end
(R0_2_c10 = extinction(10.0), R0_1_2_c6 = extinction(6.0),
 R0 = (0.05 * 10 / 0.25, 0.05 * 6 / 0.25))
```

Solution K.10.

(R0_2_c10 = 0.0, R0_1_2_c6 = 0.065, R0 = (2.0, 1.2000000000000002))

Lowering the contact rate from $c = 10$ ($R_0 = 2$) to $c = 6$ ($R_0 = 1.2$) raises the extinction probability sharply. With a smaller R_0 each infectious individual produces fewer secondary cases, so the early stochastic phase — when only a handful are infected — is far more likely to fizzle out before an epidemic establishes. This fade-out is a purely stochastic phenomenon: the deterministic map, lacking integer counts, always predicts a (small) deterministic outbreak and can never reach exactly zero.

K.1.6 Ordinary differential equations

K.1.6.1 Solution to Exercise 1

We sweep the lockdown strength, using a `ContinuousCallback` on dI/dt to detect turning points, and report the epidemic peak (largest I among detected events).

```
using OrdinaryDiffEq, DiffEqCallbacks, DataFrames
function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p; = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; return nothing
end
u0 = [990.0, 10.0, 0.0]; tspan = (0.0, 80.0); t1, t2 = 10.0, 30.0
dIdt(u, p) = p[1] * p[2] * u[2] / p[4] * u[1] - p[3] * u[2]

function peak_under(_lock)
    events = Tuple{Float64,Float64}[]
    lockdown!(i) = (i.p[1] = i.t < t2 ? _lock : 0.05)
    peak_cb = ContinuousCallback((u, t, i) -> dIdt(u, i.p), i -> push!(events, (i.t, i.u[2])))
    set = CallbackSet(PresetTimeCallback([t1, t2], lockdown!), peak_cb)
    solve(ODEProblem(sir_ode!, u0, tspan, [0.05, 10.0, 0.25, 1000.0]), Tsit5();
        callback = set, reltol = 1e-8, abstol = 1e-8)
    t_peak, I_peak = argmax(e -> e[2], events)
    (t_peak = round(t_peak; digits = 2), I_peak = round(I_peak; digits = 1))
end

DataFrame(_lock = [0.045, 0.035, 0.025, 0.015],
    peak_time = [peak_under(b).t_peak for b in (0.045, 0.035, 0.025, 0.015)],
    peak_I = [peak_under(b).I_peak for b in (0.045, 0.035, 0.025, 0.015)])
```

Solution K.11.

	_lock	peak_time	peak_I
	Float64	Float64	Float64
1	0.045	17.4	134.0
2	0.035	14.32	93.1
3	0.025	46.45	37.1
4	0.015	58.09	52.5

Stronger lockdowns (smaller β_{lock}) do not simply lower the peak: the detected peak height and timing behave *non-monotonically*. Moderate lockdowns (e.g. $\beta_{\text{lock}} = 0.035$) suppress transmission during the window, cutting the peak while it still occurs near the intervention. But very strong lockdowns defer the epidemic almost entirely: with growth stifled between days

10 and 30, susceptibles are “saved” and the wave resurges *after* the lockdown lifts — pushing the peak time out to ~50–60 days and, because a larger susceptible pool is preserved, allowing the rebound peak to climb again for the most aggressive setting. This is the flatten-*and-delay* trade-off with a sting in the tail: a temporary intervention reshapes and postpones the epidemic rather than removing it, and the strongest short lockdown is not necessarily the one with the lowest overall peak.

K.1.6.2 Solution to Exercise 2

We augment the system with cumulative incidence C and reconstruct daily incidence from the dense interpolant.

```
function sir_cumulative!(du, u, p, t)
    S, I, R, C = u; , c, , N = p; = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; du[4] = * S; return nothing
end
p = (0.05, 10.0, 0.25, 1000.0)
sol = solve(ODEProblem(sir_cumulative!, [990.0, 10.0, 0.0, 0.0], (0.0, 80.0), p),
            Tsit5(); reltol = 1e-9, abstol = 1e-9)

days = 1:80
daily = [sol(t)[4] - sol(t - 1)[4] for t in days]      # C(t) - C(t-1)
(final_C = round(sol(80.0)[4]; digits = 3),
 sum_daily = round(sum(daily); digits = 3),
 reconstructs = isapprox(sum(daily), sol(80.0)[4] - sol(0.0)[4]; atol = 1e-6))
```

Solution K.12.

```
(final_C = 790.179, sum_daily = 790.179, reconstructs = true)
```

The daily increments are differences of the cumulative-incidence interpolant, and their sum telescopes to $C(80) - C(0)$. The check confirms the reconstruction to machine tolerance, illustrating that the dense output `sol(t)` can be resampled onto any grid — here the integer-day grid used to compare a continuous model with daily surveillance data, a pattern the observation-model chapter builds on.

K.2 Stochastic modeling solutions

These solutions use the shared root environment and keep the checks close to the model code. Seeds make the example output reproducible, but they do not turn stochastic results into universal constants.

K.2.1 Continuous-time Markov chains and jump processes

K.2.1.1 Solution to Exercise 1

The two infectious stages use rate 2γ . Their sum is an Erlang distribution with mean $2/(2\gamma) = 1/\gamma$, matching the original single-stage mean, but variance $2/(2\gamma)^2 = 1/(2\gamma^2)$, half the single exponential variance.

```
using Catalyst
using JumpProcesses
using StableRNGs

N, c, = 1000, 0.05, 10.0, 0.25
= * c / N
network = @reaction_network begin
    , S + I --> 2I
    2 , I --> I
    2 , I --> R
end

u = [:S => 990, :I => 10, :I => 0, :R => 0]
problem = JumpProblem(
    network, u, (0.0, 40.0), [: => , : => ];
    aggregator = Direct(), rng = StableRNG(20250711),
)
solution = solve(problem, SSAS stepper(); saveat = 0.2)
states = Array(solution)
mass_error = maximum(abs.(vec(sum(states; dims = 1)) .- N))
nonnegative = minimum(states) >= 0
integer_valued = all(x -> isapprox(x, round(x)), states)
@assert mass_error == 0
@assert nonnegative && integer_valued
(; mass_error, nonnegative, integer_valued)
```

```
(mass_error = 0, nonnegative = true, integer_valued = true)
```

The infection propensity remains κSI_1 ; the second infectious compartment only changes the recovery path. The conservation check is exact because every reaction removes one individual from one compartment and adds one to another. Compared with one exponential infectious period, the two-stage construction reduces unrealistically short and long durations while preserving the mean.

K.2.1.2 Solution to Exercise 2

For a pure birth–death count Z , a `DiscreteProblem` and two `ConstantRateJumps` are sufficient. The death affect function guards the boundary; its constant rate is a deliberately simple toy approximation.

```
using JumpProcesses
using StableRNGs

function birth!(integrator)
    integrator.u[1] += 1
end
function death!(integrator)
    integrator.u[1] = max(integrator.u[1] - 1, 0)
end

toy = DiscreteProblem([5], (0.0, 20.0), nothing)
birth_rate(u, p, t) = 0.8
death_rate(u, p, t) = 0.8
birth_jump = ConstantRateJump(birth_rate, birth!)
death_jump = ConstantRateJump(death_rate, death!)
toy_problem = JumpProblem(
    toy, Direct(), birth_jump, death_jump;
    rng = StableRNG(20250712),
)
toy_solution = solve(toy_problem, SSAS stepper())
@assert minimum(reduce(vcat, toy_solution.u)) >= 0
```

To add vaccination to SIR, introduce the transition $S \rightarrow R$ with propensity $a_{\text{vac}} = \nu S$. A state-dependent propensity is a `VariableRateJump`:

```
function infection!(integrator)
    integrator.u[1] -= 1
    integrator.u[2] += 1
end
function recovery!(integrator)
    integrator.u[2] -= 1
    integrator.u[3] += 1
end
function vaccinate!(integrator)
    integrator.u[1] -= 1
    integrator.u[3] += 1
```

```

end

= 0.01
sir = DiscreteProblem([990, 10, 0], (0.0, 40.0), nothing)
jumps = (
    VariableRateJump(
        (u, p, t) -> *u[1]*u[2], infection!;
        urate = (u, p, t) -> 125.0,
        rateinterval = (u, p, t) -> 40.0,
    ),
    VariableRateJump(
        (u, p, t) -> *u[2], recovery!;
        urate = (u, p, t) -> 250.0,
        rateinterval = (u, p, t) -> 40.0,
    ),
    VariableRateJump(
        (u, p, t) -> *u[1], vaccinate!;
        urate = (u, p, t) -> 10.0,
        rateinterval = (u, p, t) -> 40.0,
    ),
)
vaccinated_problem = JumpProblem(
    sir, Coevolve(), jumps...;
    dep_graph = [[1, 2, 3], [1, 2, 3], [1, 2, 3]],
    rng = StableRNG(20250713),
)
vaccinated_solution = solve(vaccinated_problem, SSAS stepper(); saveat = 0.2)
vaccinated_states = Array(vaccinated_solution)
@assert maximum(abs.(vec(sum(vaccinated_states; dims = 1)) .- 1000)) == 0

```

The mean-field equations become

$$\dot{S} = -\kappa SI - \nu S, \quad \dot{I} = \kappa SI - \gamma I, \quad \dot{R} = \gamma I + \nu S.$$

If ν depends on calendar time, the rate is no longer constant. A `VariableRateJump` with a time argument (as above) is appropriate, or the process can be simulated in pieces over intervals on which the campaign rate is constant.

K.2.2 Stochastic differential equations

K.2.2.1 Solution to Exercise 1

The experiment keeps the model, seed, and output grid fixed while changing the Euler–Maruyama step. Conservation is a property of the reaction-level noise, whereas nonnegativity is not guaranteed by any finite dt .

```
using StochasticDiffEq
using StableRNGs

N, , c, = 1000.0, 0.05, 10.0, 0.25
= * c / N
u = [990.0, 10.0, 0.0]

function f!(du, u, p, t)
    , = p
    a = * max(u[1], 0.0) * max(u[2], 0.0)
    a = * max(u[2], 0.0)
    du[1], du[2], du[3] = -a, a - a, a
end

function g!(G, u, p, t)
    , = p
    a = * max(u[1], 0.0) * max(u[2], 0.0)
    a = * max(u[2], 0.0)
    s, s = sqrt(a), sqrt(a)
    G[1, 1], G[2, 1], G[3, 1] = -s, s, 0.0
    G[1, 2], G[2, 2], G[3, 2] = 0.0, -s, s
end

function run_sde(dt)
    problem = SDEProblem(
        f!, g!, u, (0.0, 30.0), (, );
        noise_rate_prototype = zeros(3, 2),
    )
    solution = solve(
        problem, LambdaEM();
        dt, adaptive = false, saveat = 0.1,
        rng = StableRNG(20250711),
    )
    states = Array(solution)
    (
```

```

        dt = dt,
        conservation_error = maximum(abs.(vec(sum(states; dims = 1)) .- N)),
        minimum_state = minimum(states),
    )
end

results = run_sde.( (0.1, 0.05, 0.01) )
@assert all(result.conservations_error < 1e-8 for result in results)
results

```

```

((dt = 0.1, conservation_error = 6.821210263296962e-13, minimum_state = 0.0), (dt = 0.05, con
12, minimum_state = -0.03809244181899207), (dt = 0.01, conservation_error = 4.66116034658625
12, minimum_state = -0.14804939563386482))

```

Reducing dt reduces discretization error and usually makes boundary crossing less frequent, but a Gaussian increment has unbounded support for every positive step size. A negative value can therefore still occur. If nonnegativity is essential, use the integer CTMC or a method with an explicit nonnegative boundary treatment rather than silently clipping the output.

K.2.2.2 Solution to Exercise 2

Independent state noise has a diagonal 3×3 diffusion. The comparison below uses the same drift and seed as the reaction-level SDE.

```

using StochasticDiffEq
using StableRNGs

function diagonal_g!(G, u, p, t)
    , = p
    a = * max(u[1], 0.0) * max(u[2], 0.0)
    a = * max(u[2], 0.0)
    fill!(G, 0.0)
    G[1, 1] = sqrt(a)
    G[2, 2] = sqrt(a + a)
    G[3, 3] = sqrt(a)
end

reaction_problem = SDEProblem(
    f!, g!, u, (0.0, 30.0), (, );
    noise_rate_prototype = zeros(3, 2),
)

```

```

diagonal_problem = SDEProblem(
    f!, diagonal_g!, u, (0.0, 30.0), (, );
    noise_rate_prototype = zeros(3, 3),
)
reaction_solution = solve(
    reaction_problem, LambdaEM();
    dt = 0.02, adaptive = false, saveat = 0.1,
    rng = StableRNG(20250711),
)
diagonal_solution = solve(
    diagonal_problem, LambdaEM();
    dt = 0.02, adaptive = false, saveat = 0.1,
    rng = StableRNG(20250711),
)
reaction_states, diagonal_states = Array(reaction_solution), Array(diagonal_solution)
reaction_error = maximum(abs.(vec(sum(reaction_states; dims = 1)) .- 1000))
diagonal_error = maximum(abs.(vec(sum(diagonal_states; dims = 1)) .- 1000))
(; reaction_error, diagonal_error,
    infected_difference = maximum(abs.(reaction_states[2, :] .- diagonal_states[2, :]))))

```

```
(reaction_error = 2.7284841053187847e-12, diagonal_error = 65.16261098276937, infected_difference = 65.16261098276937)
```

For the reaction model $Q = GG^T$ has

$$Q_{12} = Q_{21} = -a_1, \quad Q_{23} = Q_{32} = -a_2, \quad Q_{13} = 0.$$

The diagonal model sets all off-diagonal covariances to zero. It also gives the total $S + I + R$ a nonzero noise term, so its conservation error is not a roundoff diagnostic. Its infected trajectory is a different stochastic model, not merely a noisier numerical solve.

K.2.3 Agent-based models

K.2.3.1 Solution to Exercise 1

The grid model changes the contact assumption: a susceptible samples only agents in its Moore neighborhood. `nearby_agents` excludes the focal agent, and `GridSpace` supplies positions through the `Agents.jl` 7 `@agent` API.

```

using Agents
using Distributions
using Statistics
using StableRNGs

@agent struct GridPerson(GridAgent{2})
    status::Symbol
    next_status::Symbol
end

function grid_agent_step!(person, model)
    person.next_status = person.status
    if person.status == :S
        neighbors = collect(nearby_agents(person, model, 1))
        k = rand(abmrng(model), Poisson(model.c * model.Δt))
        for _ in 1:k
            isempty(neighbors) && break
            contact = rand(abmrng(model), neighbors)
            if contact.status == :I && rand(abmrng(model)) < model.
                person.next_status = :I
                break
            end
        end
    elseif person.status == :I &&
        rand(abmrng(model)) < 1 - exp(-model. * model.Δt)
        person.next_status = :R
    end
end

function grid_model_step!(model)
    for person in allagents(model)
        person.status = person.next_status
    end
end

space = GridSpace((20, 20); periodic = false)
grid_S(model) = count(p.status == :S for p in allagents(model))
grid_I(model) = count(p.status == :I for p in allagents(model))
grid_R(model) = count(p.status == :R for p in allagents(model))
model = StandardABM(
    GridPerson, space;
    agent_step! = grid_agent_step!,

```

```

    model_step! = grid_model_step!,
    properties = ( = 0.05, c = 10.0, = 0.25, Δt = 0.1),
    rng = StableRNG(20250714),
)
for id in 1:200
    position = (rand(abmrng(model), 1:20), rand(abmrng(model), 1:20))
    status = id <= 5 ? :I : :S
    add_agent!(position, model; status, next_status = status)
end
_, data = run!(
    model, 200;
    mdata = [grid_S, grid_I, grid_R],
    showprogress = false,
)
counts = Matrix{data[:, [:grid_S, :grid_I, :grid_R]]}'
@assert maximum(abs.(vec(sum(counts; dims = 1)) .- 200)) == 0

```

The well-mixed derivation no longer applies: contacts are not uniform over all N agents, and the infection probability depends on local occupancy and boundary effects. Conservation still holds because the update changes status but never creates or removes an agent.

K.2.3.2 Solution to Exercise 2

The physical horizon is held at 30 time units, so each run uses `round(Int, 30 / Δt)` updates. A small seeded ensemble reports the peak distribution for each timestep.

```

@agent struct Person{NoSpaceAgent}
    status::Symbol
    next_status::Symbol
end

function agent_step!(person, model)
    person.next_status = person.status
    if person.status == :S
        for _ in 1:rand(abmrng(model), Poisson(model.c * model.Δt))
            contact = random_agent(model)
            if contact.status == :I && rand(abmrng(model)) < model.
                person.next_status = :I
                break
            end
        end
    end
end

```



```

elseif person.status == :I &&
  rand(abmrng(model)) < 1 - exp(-model. * model.Δt)
  person.next_status = :R
end
end

function model_step!(model)
  for person in allagents(model)
    person.status = person.next_status
  end
end

infected_count(model) = count(p.status == :I for p in allagents(model))

function initialize_gridless(seed, Δt, nsteps)
  model = StandardABM(
    Person;
    agent_step!,
    model_step!,
    properties = ( = 0.05, c = 10.0, = 0.25, Δt = Δt),
    rng = StableRNG(seed),
  )
  for id in 1:200
    status = id <= 5 ? :I : :S
    add_agent!(Person(; id, status, next_status = status), model)
  end
  _, data = run!(
    model, nsteps;
    mdata = [infected_count],
    showprogress = false,
  )
  return collect(data.infected_count)
end

summaries = map((0.2, 0.1, 0.05)) do Δt
  peaks = [
    maximum(initialize_gridless(5000 + replicate, Δt, round{Int, 30 / Δt}))
    for replicate in 1:16
  ]
  (; Δt, mean_peak = mean(peaks), sd_peak = std(peaks))
end
summaries

```

((Δt = 0.2, mean_peak = 41.75, sd_peak = 8.266397845091497), (Δt = 0.1, mean_peak = 34.9375,

The output is a distribution over realizations, not a single deterministic peak. Changing Δt changes both the numerical update frequency and the discrete-time biological interpretation: the contact and recovery probabilities are recalculated at a different interval. Therefore timestep sensitivity is not purely a solver error study. A useful reference is the continuous-time SIR ODE peak, but agreement is expected only as the well-mixed, small-step approximation becomes appropriate.

K.3 Observation and inference solutions

These solutions run in the shared course environment. Because the frequentist profile-likelihood examples of Chapter 8 use the isolated `environments/ch08` project, the solutions below reconstruct the maximum-likelihood estimate and its profile-likelihood interval directly from likelihood grids, so they execute in the root environment used to render the book.

Solution K.13.

```
using EpiModelingCourse
using OrdinaryDiffEq
using Distributions
using StableRNGs
using Statistics
using CairoMakie

CairoMakie.activate!(type = "png")

# Shared deterministic SIR kernel with cumulative incidence (S, I, R, C).
function sol_sir!(du, u, p, t)
    S, I, R, C = u
    , c, = p
    N = S + I + R
    infection = * c * I / N * S
    du[1] = -infection
    du[2] = infection - * I
    du[3] = * I
    du[4] = infection
    return nothing
end

sol_u0 = [990.0, 10.0, 0.0, 0.0]
```

```

sol_tspan = (0.0, 40.0)
sol_obstimes = 1.0:1.0:40.0
sol_prob = ODEProblem(sol_sir!, sol_u0, sol_tspan, [0.05, 10.0, 0.25])

# Latent daily incidence at the baseline (truth i = 0.01, = 0.05).
sol_base = solve(sol_prob, Tsit5(); saveat = 1.0)
sol_incidence = Array(sol_base)[4, 2:end] .- Array(sol_base)[4, 1:(end - 1)]

```

K.3.1 Observation models and simulated data

K.3.1.1 Solution to Exercise 1

We reuse the latent incidence and apply a Poisson observation model with a partial reporting fraction $\rho = 0.4$.

```

ρ = 0.4
expected_reported = ρ .* sol_incidence
reported = rand.(StableRNG(20240607), Poisson.(expected_reported))

(total_latent = round(sum(sol_incidence); digits = 1),
 total_reported = sum(reported),
 ratio = round(sum(reported) / sum(sol_incidence); digits = 3))

```

Solution K.14.

```
(total_latent = 780.2, total_reported = 316, ratio = 0.405)
```

```

fig = Figure(size = (760, 420))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "new cases per day")
barplot!(ax, collect(sol_obstimes), reported; color = (:darkorange, 0.55),
          label = "reported (ρ = 0.4)")
lines!(ax, collect(sol_obstimes), sol_incidence; color = :firebrick,
        linewidth = 3, label = "latent incidence")
axislegend(ax; position = :rt)
fig

```

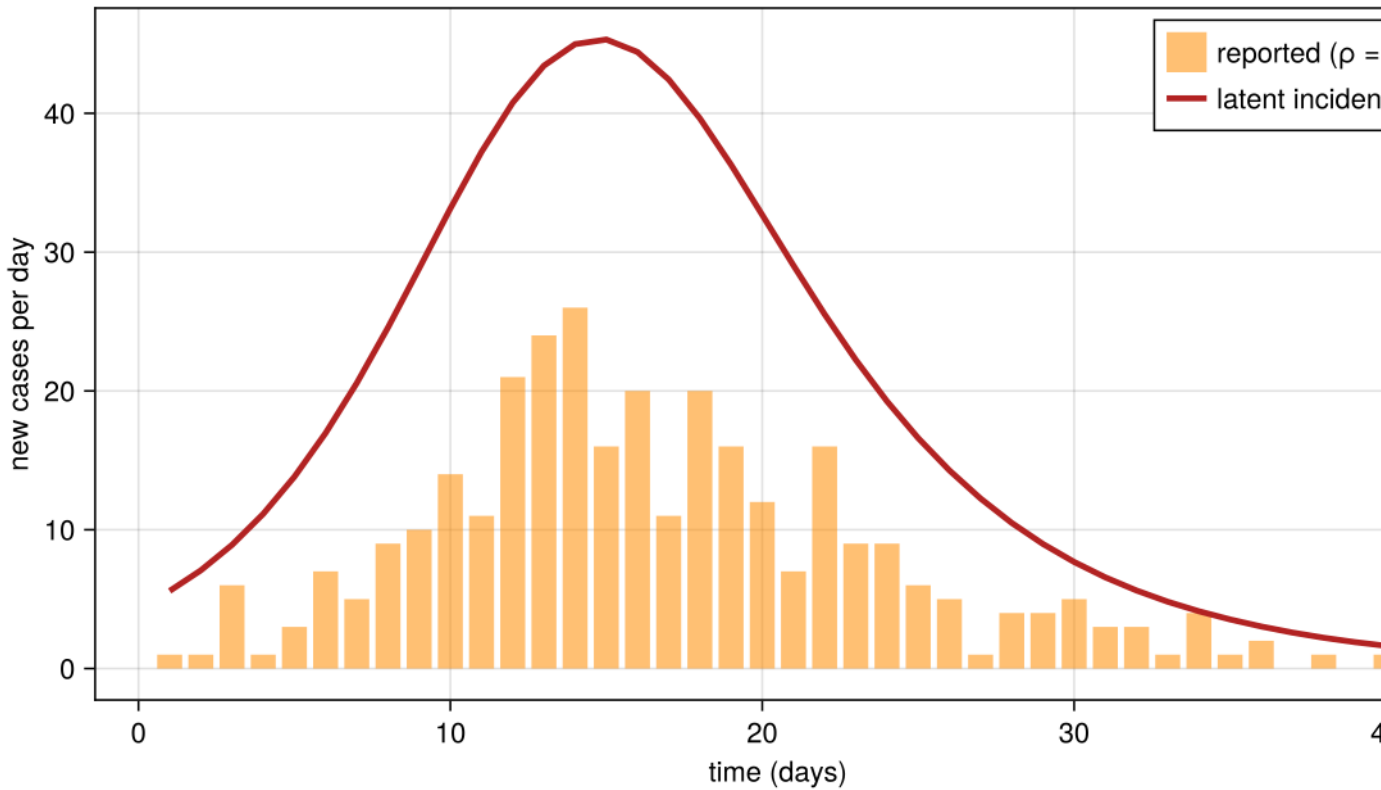


Figure K.1: Latent daily incidence (red) and Poisson case reports at $\rho = 0.4$ (bars). The reported series looks like a smaller epidemic with the same shape.

The reported total is about 40% of the latent total, matching ρ . The reported curve has the *same temporal shape* as the latent one but a smaller amplitude, so it looks like a smaller epidemic. Crucially, the expected report on day t is $\rho x_t(\beta)$: the reporting fraction and the transmission probability both act as multiplicative scalings of the observed counts. A change in ρ can be compensated by a change in β that rescales incidence, so the two cannot be separated from the reported series alone — a practical identifiability problem taken up in Chapter 10.

K.3.1.2 Solution to Exercise 2

We fix the negative-binomial mean to the latent incidence at the epidemic peak and vary the dispersion $r \in \{1, 5, 50\}$, drawing 1000 replicate counts for each and estimating the variance-to-mean ratio.

```

peak_index = argmax(sol_incidence)
_peak = sol_incidence[peak_index]

nb( , r) = NegativeBinomial(r, r / (r + ))
function ratio_at(r)
    rng = StableRNG(1000 + Int(r))
    draws = [rand(rng, nb(_peak, r)) for _ in 1:1000]
    (r = r, var_over_mean = round(var(draws) / mean(draws); digits = 3),
     predicted = round(1 + _peak / r; digits = 3))
end

[ratio_at(r) for r in (1.0, 5.0, 50.0)]

```

Solution K.15.

```

3-element Vector{@NamedTuple{r::Float64, var_over_mean::Float64, predicted::Float64}}:
 (r = 1.0, var_over_mean = 45.783, predicted = 46.316)
 (r = 5.0, var_over_mean = 10.095, predicted = 10.063)
 (r = 50.0, var_over_mean = 1.885, predicted = 1.906)

```

The variance formula $\text{Var}[Y_t] = \rho x_t + (\rho x_t)^2 / r$ (with $\rho = 1$ here) predicts a variance-to-mean ratio of $1 + \mu_{\text{peak}} / r$. The simulated ratios track this prediction closely and fall towards 1 as r grows: at $r = 50$ the counts are almost equidispersed, recovering the Poisson limit $r \rightarrow \infty$, while at $r = 1$ the peak counts are strongly overdispersed.

K.3.2 Frequentist parameter estimation

The next two solutions reconstruct maximum-likelihood estimation and profile intervals on likelihood *grids*, which is transparent and avoids any dependency outside the shared environment. Each grid point solves the ODE once, so the grids are kept deliberately small for continuous-integration speed.

Solution K.16.

```

# Model incidence for parameters (i , , ) with c = 10 fixed.
function incidence_for(i , , )
    I = i * 1000.0
    solf = solve(remake(sol_prob; u0 = [1000.0 - I, I, 0.0, 0.0],
        p = [ , 10.0, ]), Tsit5(); saveat = 1.0)
    C = Array(solf)[4, :]

```

```

    return C[2:end] .- C[1:(end - 1)]
end

# Poisson log-likelihood of `data` given model incidence.
function poisson_loglik(i, , , data)
    X = incidence_for(i, , )
    any(X .<= 0) && return -Inf
    return sum(logpdf.(Poisson.(X), data))
end

# Wilks 95% threshold for a one-parameter profile ( 1.92).
wilks_threshold = 0.5 * quantile(Chisq(1), 0.95)

# Endpoints where a 1-D profile crosses `_max - wilks_threshold`.
function profile_interval(grid, profile, _max)
    cutoff = _max - wilks_threshold
    inside = findall(profile .>= cutoff)
    isempty(inside) && return (grid[argmax(profile)], grid[argmax(profile)])
    return (grid[first(inside)], grid[last(inside)])
end

```

K.3.2.1 Solution to Exercise 1

For each of 50 seeds we resimulate the data, find the MLE of β on an (i_0, β) grid, and read off the profile-likelihood interval for β .

```

i0_grid = range(0.003, 0.030; length = 28)
_grid = range(0.035, 0.070; length = 36)

function fit_seed(seed)
    data = rand.(StableRNG(seed), Poisson.(sol_incidence))
    # Log-likelihood surface over the (i, ) grid.
    ll = [poisson_loglik(i0, , 0.25, data) for i0 in i0_grid, in _grid]
    _max, idx = findmax(ll)
    _hat = _grid[idx[2]]
    # Profile over : maximise across i (rows) for each (column).
    _profile = vec(maximum(ll; dims = 1))
    lo, hi = profile_interval(collect(_grid), _profile, _max)
    return (_hat = _hat, lo = lo, hi = hi)
end

```

```
fits = [fit_seed(seed) for seed in 1:50]
_hats = [f._hat for f in fits]
covered = count(f -> f.lo < 0.05 < f.hi, fits)

(mean_hat = round(mean(_hats); digits = 4),
 sd_hat = round(std(_hats); digits = 4),
 coverage = covered / 50)
```

Solution K.17.

```
(mean_hat = 0.0502, sd_hat = 0.001, coverage = 0.98)
```

```
fig = Figure(size = (720, 400))
ax = Axis(fig[1, 1]; xlabel = "MLE of ", ylabel = "count")
hist!(ax, _hats; bins = 12, color = (:steelblue, 0.6))
vlines!(ax, [0.05]; color = :firebrick, linewidth = 3)
fig
```

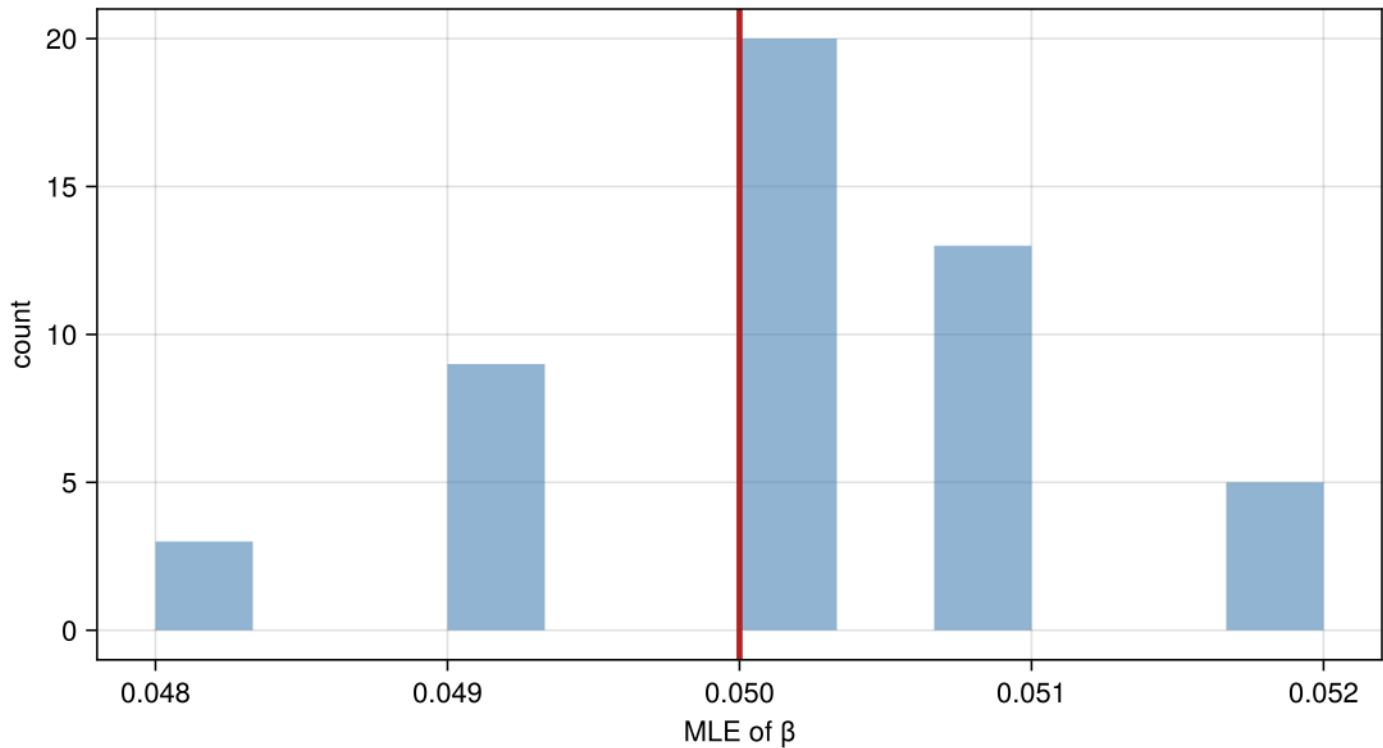


Figure K.2: Distribution of the maximum-likelihood estimate of β across 50 simulated datasets. The estimates cluster around the true value 0.05 (red line).

The MLEs are centred on the true value with modest spread, illustrating the (approximate) unbiasedness of maximum likelihood here. The empirical coverage of the profile intervals is close to the nominal 95%; small departures are expected because the χ^2 calibration is asymptotic, the grid has finite resolution, and 50 replicates is a small Monte Carlo sample. With more data or more replicates the coverage would tighten around 0.95.

K.3.2.2 Solution to Exercise 2

We now also estimate the recovery rate γ , profiling all three parameters on a coarse three-dimensional grid and comparing interval widths.

```
data = rand.(StableRNG(20240607), Poisson.(sol_incidence))

i0_g = range(0.004, 0.025; length = 22)
_g = range(0.035, 0.075; length = 26)
```



```

_g = range(0.15, 0.42; length = 26)

ll3 = [poisson_loglik(i0, , , data) for i0 in i0_g, in _g, in _g]
_max = maximum(ll3)

i0_prof = vec(maximum(ll3; dims = (2, 3)))
_prof = vec(maximum(ll3; dims = (1, 3)))
_prof = vec(maximum(ll3; dims = (1, 2)))

ci_i0 = profile_interval(collect(i0_g), i0_prof, _max)
ci_ = profile_interval(collect(_g), _prof, _max)
ci_ = profile_interval(collect(_g), _prof, _max)

(width_i0 = round(ci_i0[2] - ci_i0[1]; sigdigits = 2),
 width_ = round(ci_[2] - ci_[1]; sigdigits = 2),
 width_ = round(ci_[2] - ci_[1]; sigdigits = 2),
 ci_ = round.(ci_ ; digits = 3))

```

Solution K.18.

```

(width_i0 = 0.005, width_ = 0.011, width_ = 0.097, ci_ = (0.204, 0.301))

```

```

cutoff = _max - wilks_threshold
fig = Figure(size = (720, 400))
ax = Axis(fig[1, 1]; xlabel = " ", ylabel = "profile log-likelihood")
lines!(ax, collect(_g), _prof; color = :steelblue, linewidth = 3)
hlines!(ax, [cutoff]; color = :black, linestyle = :dash)
vlines!(ax, [0.25]; color = :firebrick, linewidth = 2)
ylims!(ax, cutoff - 3, _max + 1)
fig

```

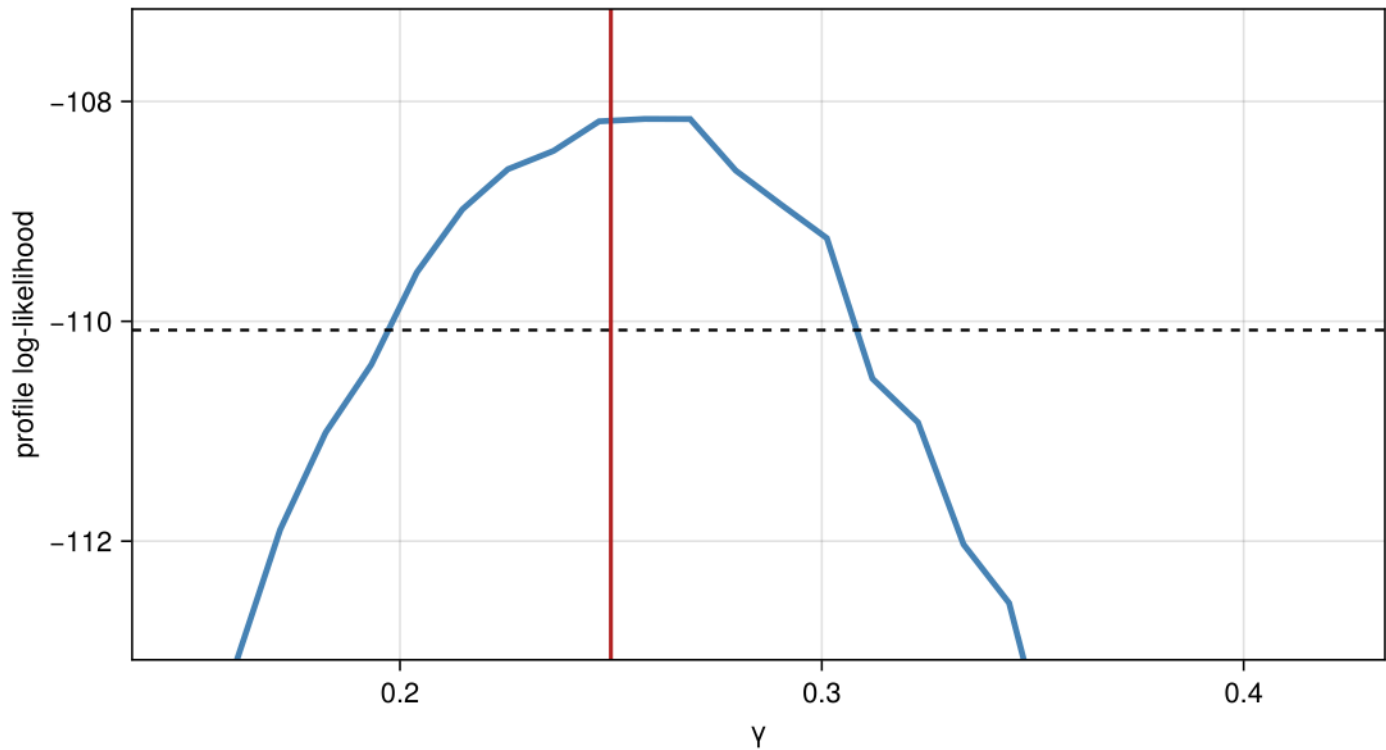


Figure K.3: Profile log-likelihood for γ (blue) with the 95% Wilks threshold (dashed) and the true value $\gamma = 0.25$ (red). The interval is comparatively wide.

All three intervals contain their true values, but the profile for γ is noticeably wider (flatter) than those for i_0 and β . Following $\ell_p(\gamma) = \max_{i_0, \beta} \ell(i_0, \beta, \gamma)$, a flat profile means that many values of γ can be compensated by adjusting the other parameters while keeping the fit to the incidence almost unchanged. The incidence data therefore carry relatively little information about the recovery rate on their own: prevalence or duration data would be needed to pin γ down more tightly.

K.3.3 Bayesian inference

Solution K.19.

```
using Turing
using MCMCChains
using SciMLSensitivity

bayes_data = rand.(StableRNG(20240607), Poisson.(sol_incidence))
```

K.3.3.1 Solution to Exercise 1

We replace the Beta priors with uniform priors on $[0, 1]$ and re-run NUTS with the same lightweight settings, then compare with the Beta-prior fit.

```
@model function sir_uniform(y, prob)
    l = length(y)
    i ~ Uniform(0.0, 1.0)
    ~ Uniform(0.0, 1.0)
    I = i * 1000.0
    prob = remake(prob; u0 = [1000.0 - I, I, 0.0, 0.0], p = [ , 10.0, 0.25],
        tspan = (0.0, float(l)))
    sol = solve(prob, Tsit5(); saveat = 1.0)
    if !SciMLBase.successful_retcode(sol)
        Turing.@addlogprob! -Inf
        return nothing
    end
    C = Array(sol)[4, :]
    X = C[2:end] .- C[1:(end - 1)]
    for i in 1:l
        y[i] ~ Poisson(max(X[i], 0.0) + 1e-6)
    end
    return nothing
end

chain_uniform = sample(StableRNG(2), sir_uniform(bayes_data, sol_prob),
    NUTS(0.8), MCMCSerial(), 300, 2; chain_type = MCMCChains.Chains,
    progress = false)

(posterior_mean = (i = round(mean(vec(Array(chain_uniform[:, :i, :]))) ; digits = 4),
    = round(mean(vec(Array(chain_uniform[:, :, :]))); digits = 4)),
    max_rhat = round(maximum(rhat(chain_uniform).nt.rhat); digits = 3),
    truth = (i = 0.01, = 0.05))
```

Solution K.20.

```
(posterior_mean = (i = 0.009, = 0.0501), max_rhat = 1.038, truth = (i = 0.01, = 0.05))
```

The uniform-prior posterior means and $\hat{R}$ values are essentially indistinguishable from the Beta-prior results in Chapter 9: both concentrate near $(i_0, \beta) = (0.01, 0.05)$ with $\hat{R} \approx 1$. With 40 informative daily counts the likelihood dominates the prior over the plausible range, so the choice between a weakly informative Beta prior and a flat prior barely moves the posterior. The Beta priors remain preferable in practice because they exclude implausible values and keep the sampler away from regions where the ODE misbehaves.

K.3.3.2 Solution to Exercise 2

We add the recovery rate γ as a third parameter with prior $\gamma \sim \text{Beta}(2, 8)$ and inspect the posterior correlation between β and γ .

```
@model function sir_gamma(y, prob)
  l = length(y)
  i ~ Beta(1.0, 20.0)
  ~ Beta(1.0, 10.0)
  ~ Beta(2.0, 8.0)
  I = i * 1000.0
  prob = remake(prob; u0 = [1000.0 - I, I, 0.0, 0.0], p = [ , 10.0, ],
    tspan = (0.0, float(l)))
  sol = solve(prob, Tsit5(); saveat = 1.0)
  if !SciMLBase.successful_retcode(sol)
    Turing.@addlogprob! -Inf
    return nothing
  end
  C = Array(sol)[4, :]
  X = C[2:end] .- C[1:(end - 1)]
  for i in 1:l
    y[i] ~ Poisson(max(X[i], 0.0) + 1e-6)
  end
  return nothing
end

chain_gamma = sample(StableRNG(2), sir_gamma(bayes_data, sol_prob),
  NUTS(0.8), MCMCSerial(), 400, 2; chain_type = MCMCChains.Chains,
  progress = false)

s = vec(Array(chain_gamma[:, :, :]))
s = vec(Array(chain_gamma[:, :, :]))

(posterior_mean = (i = round(mean(vec(Array(chain_gamma[:, :i, :])))
  = round(mean(s); digits = 4),
```

```

        = round(mean(s); digits = 4)),
cor_ _ = round(cor(s, s); digits = 3))

```

Solution K.21.

```

(posterior_mean = (i = 0.0096,   = 0.049,   = 0.241), cor_ _ = 0.951)

```

```

fig = Figure(size = (620, 480))
ax = Axis(fig[1, 1]; xlabel = " ", ylabel = " ")
scatter!(ax, s, s; color = (:steelblue, 0.35), markersize = 6)
scatter!(ax, [0.05], [0.25]; color = :firebrick, markersize = 14, marker = :star5)
fig

```

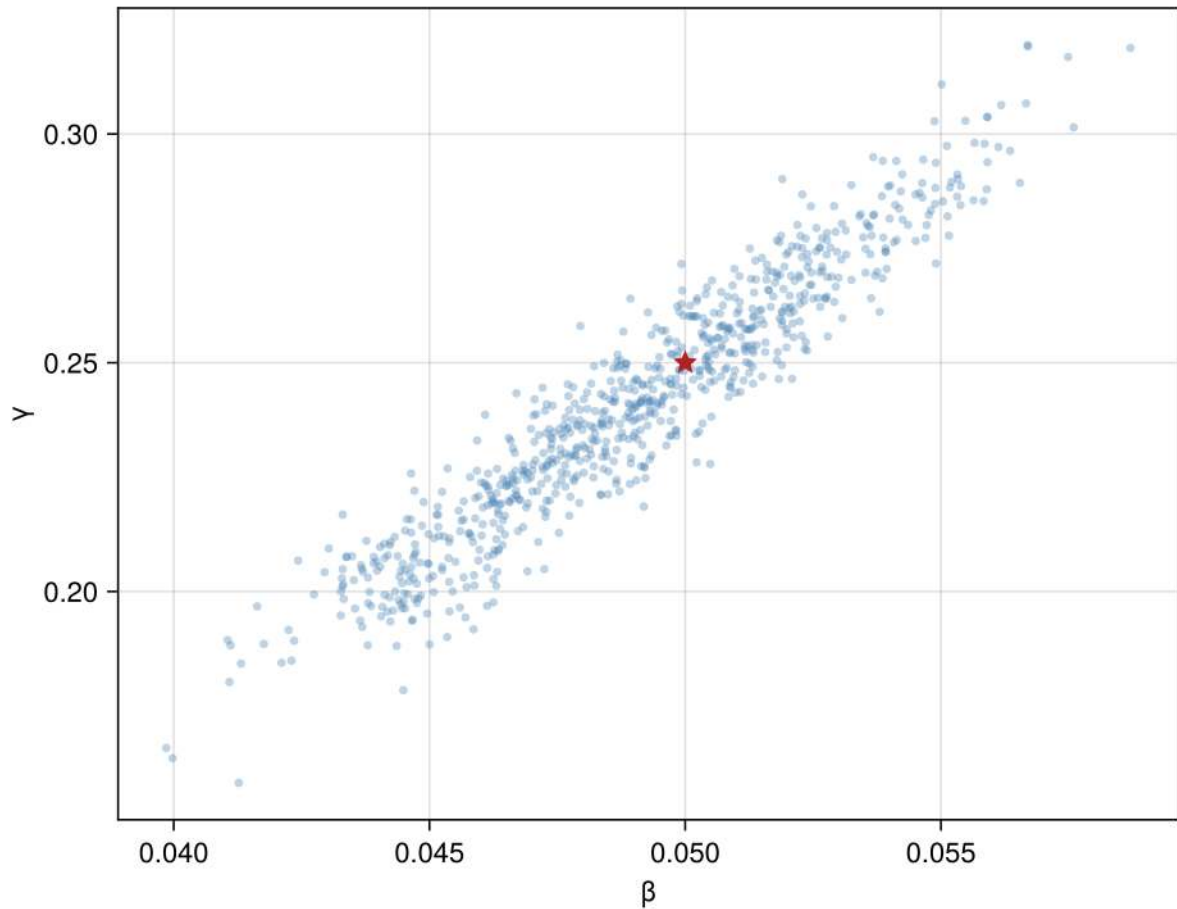


Figure K.4: Joint posterior draws of β and γ . The positive correlation reflects the ridge in the likelihood: faster recovery can be offset by higher transmission.

The posterior for γ is much wider than the tight posterior of Chapter 9’s two-parameter model, and β and γ are positively correlated. Both parameters influence the epidemic through the balance of transmission and recovery, so a larger γ (faster recovery) can be partly compensated by a larger β (faster transmission) while leaving the incidence curve — and hence the likelihood — almost unchanged. This ridge is the Bayesian expression of the weak practical identifiability of γ seen in the frequentist profile of `ch08-ex2`, and of the structural β – c confounding discussed in Chapters 8 and 9.

K.4 Analysis, decisions, and capstone solutions

These solutions use the kernels and parameterizations from Chapters 10–12. Numerical values are intentionally obtained by rerunning the displayed code, rather than copied from a particular machine’s solver output.

K.4.1 Sensitivity and identifiability

K.4.1.1 Solution to Exercise 1

The output is changed from the transient infectious peak to the cumulative removed state at the final time. The model and parameter bounds stay fixed.

```
function final_removed(p)
    solution = solve(
        ODEProblem(sir_kernel!, u0, (0.0, 40.0), Float64.(p)),
        Tsit5(); saveat = 1.0, abstol = 1e-8, reltol = 1e-8,
    )
    return Array(solution)[3, end]
end

removed_sobol = gsa(
    final_removed,
    Sobol(),
    parameter_bounds;
    samples = 32,
)
(
    first_order = removed_sobol.S1,
    total_order = removed_sobol.ST,
)
```

The first-order index measures the contribution of one input alone, while the total-order index also includes interactions. The infectious peak is a transient timing-and-amplitude output, so it is often especially responsive to the transmission and removal parameters near the epidemic threshold. Final removed count is cumulative: it integrates the epidemic history and can emphasize a parameter that changes the eventual final size even when its effect on peak timing is modest. The rankings are conditional on the stated box and the small Sobol design; rerun with more samples before making a quantitative claim.

As a validation, both outputs should be finite and every simulated trajectory should conserve $S + I + R = N$:

```
@assert all(isfinite, removed_sobol.S1)
@assert all(isfinite, removed_sobol.ST)
```

K.4.1.2 Solution to Exercise 2

The extra observable is the total population, $n(t) = S(t) + I(t) + R(t)$. Add it to the measured quantities:

```
@parameters b c g
@independent_variables t
@variables S(t) I(t) R(t) y(t) n(t)
D = Differential(t)
system_with_total = ODESystem(
  [
    D(S) ~ -b * c * S * I,
    D(I) ~ b * c * S * I - g * I,
    D(R) ~ g * I,
  ],
  t;
  name = :SIRWithTotal,
)
assess_identifiability(
  system_with_total;
  measured_quantities = [y ~ b * c * S * I, n ~ S + I + R],
  funcs_to_check = [b, c, g, b * c],
)
```

The total is constant under the closed SIR equations, so it verifies the population scale but does not introduce a new time-varying dependence on b or c . The transformation $(b, c) \mapsto (ab, c/a)$ therefore remains a symmetry of the incidence observation. The product bc and the removal rate g can be identifiable under the stated observation, but b and c remain confounded.

One useful additional data stream is an independently measured contact rate from contact diaries, mobility, or a transmission-free contact survey. Fixing or strongly informing c then permits the incidence data to identify an effective β within the model. This does not remove practical uncertainty from reporting error or model misspecification.

K.4.2 Interventions and optimization

K.4.2.1 Solution to Exercise 1

Make the reduction an argument to the existing scenario function, then solve the same bounded one-dimensional problem for each value:

```
function optimize_start(reduction)
    objective = OptimizationFunction(
        (x, _) -> Array(run_scenario(
            x[1];
            duration = intervention_duration,
            reduction = reduction,
        ))[4, end],
        AutoFiniteDiff(),
    )
    problem = OptimizationProblem(
        objective,
        [8.0],
        nothing;
        lb = [0.0],
        ub = [25.0],
    )
    solution = solve(
        problem,
        Fminbox(LBFGS());
        maxiters = 30,
        reltol = 1e-6,
    )
    start = clamp(solution.u[1], 0.0, 25.0)
    return (
        reduction = reduction,
        start = start,
        final_incidence = Array(run_scenario(
            start;
            duration = intervention_duration,
            reduction = reduction,
        ))[4, end],
    )
end

results = [optimize_start(reduction) for reduction in (0.25, 0.50, 0.75)]
results
```

The reported start and final incidence are the values returned by the bounded solver; a diagnostic grid should be plotted for each reduction and should contain a value close to the reported objective. Validate each result with the same conservation and bounds checks as the chapter:

```
for result in results
    @assert 0.0 <= result.start <= 25.0
    trajectory = Array(run_scenario(
        result.start;
        duration = intervention_duration,
        reduction = result.reduction,
    ))
    @assert maximum(abs.(vec(sum(trajectory[1:3, :]; dims = 1)) .- baseline.N)) < 1e-6
    @assert all(diff(trajectory[4, :]) .>= -1e-10)
end
```

Stronger reductions generally lower final incidence, but the best start need not coincide with the largest infectious prevalence. A policy changes future transmission, so its value depends on susceptible depletion, the remaining time horizon, and the duration of protection. Starting before the peak can prevent secondary infections that would otherwise sustain the peak; starting at the peak can be too late, while starting much earlier can spend a fixed duration before transmission is most consequential. These conclusions are conditional on the rectangular, perfectly adhered-to policy.

K.4.2.2 Solution to Exercise 2

For a rectangular policy, the intervention integral is $\int v(t) dt = v_{\max} d$. If both start and duration are decision variables, the cost-aware objective can therefore be written directly:

```
function costed_objective(x, cost)
    start, duration = x
    trajectory = run_scenario(
        start;
        duration = duration,
        reduction = maximum_reduction,
    )
    final_c = Array(trajectory)[4, end]
    return final_c + cost * maximum_reduction * duration
end

function optimize_cost(cost)
    objective = OptimizationFunction(
```

```

        (x, _) -> costed_objective(x, cost),
        AutoFiniteDiff(),
    )
    problem = OptimizationProblem(
        objective,
        [8.0, 10.0],
        cost;
        lb = [0.0, 0.0],
        ub = [25.0, 15.0],
    )
    solution = solve(problem, Fminbox(LBFGS()); maxiters = 30)
    return (cost = cost, start = solution.u[1],
            duration = solution.u[2], objective = solution.objective)
end

cost_results = [optimize_cost(cost) for cost in (0.0, 10.0)]
cost_results

```

The duration upper bound of 15 is an explicit feasibility assumption; an implementation could instead impose a resource budget $v_{\max} d \leq B$ or a minimum-free-time constraint. Without a bound or cost, the optimizer can choose the longest intervention because longer protection only lowers the epidemic term. With positive λ_{cost} , the selected duration trades infections against intervention burden. The two objective values are not comparable across different cost units unless the cost weight has been calibrated. Validate that both solutions are inside the box and that the no-cost solution agrees with a start/duration grid.

K.4.3 Data-to-decision capstone

K.4.3.1 Solution to Exercise 1

Define an infectious-only loss while retaining the same fixed initial state and the same bounded parameter box:

```

function infectious_loss(, _)
    prediction = Array(solve(
        ODEProblem(fit_rhs!, initial_state,
            (first(observation_times), last(observation_times)), ),
        Tsit5(); saveat = observation_times,
        abstol = 1e-8, reltol = 1e-8,
    ))
    return sum(((prediction[2, :] .- observed_states[2, :]) ./ population).^2)
end

```

```

end

infectious_problem = OptimizationProblem(
    OptimizationFunction(infectious_loss, AutoFiniteDiff()),
    fitted_parameters,
    nothing;
    lb = [1e-4, 1e-4],
    ub = [8.0, 8.0],
)
infectious_fit = solve(infectious_problem, Fminbox(LBFGS()); maxiters = 60)
infectious_parameters = Float64.(infectious_fit.u)
infectious_trajectory = Array(solve(
    ODEProblem(fit_rhs!, initial_state,
        (first(observation_times), last(observation_times)),
        infectious_parameters),
    Tsit5(); saveat = observation_times,
))
infectious_residuals = infectious_trajectory .- observed_states
infectious_final_cumulative = population - infectious_trajectory[1, end]

```

Plot `infectious_residuals[2, :]` beside the all-state residual plot and compare `infectious_final_cumulative` with the all-state fitted trajectory. The infectious-only fit can improve the I curve while allowing systematic errors in S and R , because those states no longer contribute to the objective. A valid SIR trajectory still conserves population:

```

@assert maximum(
    abs.(vec(sum(infectious_trajectory; dims = 1)) .- population),
) < 1e-6

```

Thus “better” infectious residuals do not necessarily mean a better population-level explanation. The discarded states contain information about cumulative depletion and removal; an observation model could weight the three series differently instead of silently discarding two thirds of the state information.

K.4.3.2 Solution to Exercise 2

Evaluate the two policy reductions for every accepted local parameter candidate. The helper below uses the capstone intervention kernel and returns the final cumulative incidence:

```

function candidate_policy_output(parameters, reduction)
  parameters = Float64.(parameters)
  problem = ODEProblem(
    decision_rhs!,
    [initial_state..., 0.0],
    (0.0, 8.0),
    [parameters[1], parameters[2], 0.0, population],
  )
  stop_time = 2.0
  function affect!(integrator)
    integrator.p[3] = integrator.t < stop_time ? reduction : 0.0
  end
  solution = solve(
    problem,
    Tsit5(),
    callback = PresetTimeCallback([1.0, stop_time], affect!),
    saveat = 0.1,
  )
  return Array(solution)[4, end]
end

policy_ranges = Dict{
  reduction => let
    outputs = [
      candidate_policy_output(parameters, reduction)
      for parameters in accepted_parameters
    ]
    reduction => (minimum(outputs), maximum(outputs))
  end
  for reduction in (0.25, 0.50)
}
policy_ranges

```

The returned dictionary is the requested range for the one-month reduction starting at month 1. The 50% range should be no larger in its central counterfactual than the 25% range for the same parameter candidates, although the width is determined by parameter sensitivity and the candidate filter. Check the policy implementation with

```

@assert all(isfinite, vcat(collect(values(policy_ranges))...))
@assert all(
  minimum(values) <= maximum(values)

```

```

    for values in values(policy_ranges)
)

```

These ranges are not decision uncertainty or confidence intervals: the accepted set is a hand-chosen $\pm 10\%$ grid filtered by a loss tolerance, and the state series are reconstructed. Before treating the range as decision uncertainty, add an observation model with measurement/reporting error and collect data that constrain the effective transmission and removal rates. For historical plague questions, a mechanistic model of household, movement, or rodent-to-human transmission would also be needed before assigning a causal interpretation to the policy comparison.

K.5 Advanced dynamics solutions

K.5.1 Delay differential equations

K.5.1.1 Solution to Exercise 1

The exercise asks for the DDE peak and its timing at $\tau \in \{2, 4, 8\}$ (holding β , c , N fixed), compared with the exponential-recovery ODE at the matching mean infectious period $1/\gamma = \tau$.

```

using DelayDiffEq
using OrdinaryDiffEq

function sir_dde!(du, u, h, p, t)
    S, I, R = u
    , c, = p
    N = S + I + R
    = * c * I / N
    Sd, Id, Rd = h(p, t - )
    Nd = Sd + Id + Rd
    d = * c * Id / Nd
    du[1] = - * S
    du[2] = * S - d * Sd
    du[3] = d * Sd
    return nothing
end
sir_history(p, t) = [1000.0, 0.0, 0.0]

function sir_ode_a01ex1!(du, u, p, t)
    S, I, R = u

```

```

    , c, rate = p
    N = S + I + R
    = * c * I / N
    du[1] = - * S
    du[2] = * S - rate * I
    du[3] = rate * I
    return nothing
end

u0 = [990.0, 10.0, 0.0]
tspan = (0.0, 40.0)
results = NamedTuple[]
for in (2.0, 4.0, 8.0)
    p = (0.05, 10.0, )
    sol_dde = solve(DDEProblem(sir_dde!, u0, sir_history, tspan, p; constant_lags = []),
                    MethodOfSteps(Tsit5())); reltol = 1e-8, abstol = 1e-8)
    peak_dde, idx_dde = findmax(sol_dde[2, :])

    sol_ode = solve(ODEProblem(sir_ode_a01ex1!, u0, tspan, (0.05, 10.0, 1 / )), Tsit5();
                    reltol = 1e-8, abstol = 1e-8)
    peak_ode, idx_ode = findmax(sol_ode[2, :])

    push!(results, ( = , dde_peak = round(peak_dde; digits = 1), dde_t = round(sol_dde.t[idx_dde],
                    ode_peak = round(peak_ode; digits = 1), ode_t = round(sol_ode.t[idx_ode],
end
results

```

3-element Vector{NamedTuple}:

```

( = 2.0, dde_peak = 61.9, dde_t = 8.61, ode_peak = 10.0, ode_t = 0.0)
( = 4.0, dde_peak = 350.9, dde_t = 10.87, ode_peak = 158.3, ode_t = 17.78)
( = 8.0, dde_peak = 758.3, dde_t = 13.21, ode_peak = 405.9, ode_t = 15.54)

```

At $\tau = 2$ ($\gamma = 0.5$), $R_0 = \beta c / \gamma = 1.0$ exactly, so the exponential-recovery ODE cannot grow at all — its “peak” is simply $I(0) = 10$ at $t = 0$ — while the fixed-delay DDE still produces a modest outbreak, because every one of the initial infections transmits at full strength for the entire interval $[0, 2)$ before recovering, rather than a fraction of them recovering almost immediately. As τ grows, both the absolute size of the gap between the DDE and ODE peaks and its relative size increase, and the DDE peak occurs earlier relative to the ODE peak: the ODE’s exponential recovery spreads recoveries out over a long right tail, so it takes longer for the ODE epidemic to reach its own (smaller) peak, while the fixed-delay model concentrates the entire initial cohort’s transmission and then removes it all at once, producing a sharper

and earlier peak. This confirms, more sharply for larger τ , the point made in the book chapter: the *shape* of the infectious-period distribution, not merely its mean, drives these differences.

K.5.1.2 Solution to Exercise 2

The exercise asks to repeat the two-stage construction for $I(0) = 1$ instead of 10, confirm conservation and nonnegativity, and check whether a `ContinuousCallback` that would clamp I to zero ever fires.

```
function si_ode!(du, u, p, t)
    S, I = u[1], u[2]
    , c, _ = p
    = * c * I / (u[1] + u[2] + u[3])
    du[1] = - * S
    du[2] = * S
    du[3] = 0.0
    return nothing
end

u0_ex2 = [999.0, 1.0, 0.0]
      = 0.25
p_ex2 = (0.05, 10.0, 1 / )

sol_si = solve(ODEProblem(si_ode!, u0_ex2, (0.0, p_ex2[3]), p_ex2), Tsit5();
               reltol = 1e-10, abstol = 1e-10)
history_from_ode(p, t) = sol_si(t)

fire_count = Ref{Int}()
condition(u, t, integrator) = u[2] # zero crossing on I
function affect!(integrator)
    fire_count[] += 1
    integrator.u[2] = max(integrator.u[2], 0.0)
    return nothing
end
cb = ContinuousCallback(condition, affect!; save_positions = (true, true))

prob_dde2 = DDEProblem(sir_dde!, sol_si(p_ex2[3]), history_from_ode, (p_ex2[3], 40.0), p_ex2;
                      constant_lags = [p_ex2[3]])
sol_dde2 = solve(prob_dde2, MethodOfSteps(Tsit5()); reltol = 1e-8, abstol = 1e-8, callback =

grid = 0.0:0.1:40.0
states = [t < p_ex2[3] ? sol_si(t) : sol_dde2(t) for t in grid]
```



```

conserved = all(u -> isapprox(sum(u), 1000.0; atol = 1e-6), states)
nonnegative = all(u -> all(>=(-1e-8), u), states)
(conserved = conserved, nonnegative = nonnegative, callback_fires = fire_count[])

```

```
(conserved = true, nonnegative = true, callback_fires = 0)
```

The two-stage construction with $I(0) = 1$ still conserves the population and stays nonnegative, and the clamping callback **never fires**. This is expected: this DDE is deterministic, and its right-hand side has I 's derivative bounded below by $-\lambda(t - \tau)S(t - \tau) \geq -N$ whenever $I(t)$ itself is already at 0 — with an exponential-type solution structure, I can only approach 0 asymptotically as the epidemic burns out, never overshoot below it in finite time. This is in sharp contrast to the stochastic delay differential equation of the next appendix, whose diffusion term has no such structural floor and *does* drive I negative for some seeds, as Exercise 2 there demonstrates concretely.

K.5.2 Stochastic delay differential equations

K.5.2.1 Solution to Exercise 1

The exercise asks to repeat the seed-4 run at $\text{dt} \in \{0.2, 0.1, 0.05\}$, recording the maximum conservation error and minimum state value, and to discuss whether decreasing dt makes nonnegativity violations more or less frequent.

```

using DelayDiffEq
using OrdinaryDiffEq
using StochasticDiffEq
using SparseArrays
using Random

function sir_dde!(du, u, h, p, t)
    S, I, R = u
    , c, = p
    N = S + I + R
    = * c * I / N
    Sd, Id, Rd = h(p, t - )
    Nd = Sd + Id + Rd
    d = * c * Id / Nd
    du[1] = - * S
    du[2] = * S - d * Sd
    du[3] = d * Sd

```

```

    return nothing
end
function sir_delayed_noise!(G, u, h, p, t)
    S, I, R = u
    , c, = p
    N = S + I + R
    infection = max( * c * I / N * S, 0.0)
    Sd, Id, Rd = h(p, t - )
    Nd = Sd + Id + Rd
    recovery = max( * c * Id / Nd * Sd, 0.0)
    G[1, 1] = -sqrt(infection)
    G[2, 1] = sqrt(infection)
    G[2, 2] = -sqrt(recovery)
    G[3, 2] = sqrt(recovery)
    return nothing
end
sir_history(p, t) = [1000.0, 0.0, 0.0]
u0 = [990.0, 10.0, 0.0]
    = 0.25
p = (0.05, 10.0, 1 / )
tspan = (0.0, 40.0)
noise_shape() = sparse([1.0 0.0; 1.0 1.0; 0.0 1.0])

seed4_results = NamedTuple[]
for dt in (0.2, 0.1, 0.05)
    Random.seed!(4)
    prob = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
        noise_rate_prototype = noise_shape())
    sol = solve(prob, LambdaEM(); dt = dt, adaptive = false)
    states = Array(sol)
    push!(seed4_results, (dt = dt,
        max_conservation_error = round(maximum(abs.(vec(sum(states; dims =
        min_state = round(minimum(states); digits = 2)))
end
seed4_results

```

3-element Vector{NamedTuple}:

```

(dt = 0.2, max_conservation_error = 9.09e-13, min_state = -0.93)
(dt = 0.1, max_conservation_error = 1.25e-12, min_state = 0.0)
(dt = 0.05, max_conservation_error = 1.59e-12, min_state = 0.0)

```

At seed 4 specifically, the minimum state does happen to improve as `dt` shrinks — but a single seed is not evidence of a general trend, because changing `dt` under a *fixed* global seed does not refine the *same* Wiener path; it consumes a different number of Gaussian draws from the RNG stream and so produces a **different realization altogether**. A fifteen-seed ensemble at each step size makes this explicit:

```
ensemble_summary = NamedTuple[]
for dt in (0.2, 0.1, 0.05)
    minvals = Float64[]
    unstable = 0
    for seed in 1:15
        Random.seed!(seed)
        prob = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
                           noise_rate_prototype = noise_shape())
        sol = solve(prob, LambdaEM(); dt = dt, adaptive = false)
        if string(sol.retval) != "Success"
            unstable += 1
            continue
        end
        push!(minvals, minimum(Array(sol)))
    end
    violations = count(<(-1e-6), minvals)
    push!(ensemble_summary, (dt = dt, n_seeds = 15, violations = violations,
                             unstable_runs = unstable,
                             worst_min = round(minimum(minvals); digits = 2)))
end
ensemble_summary
```

Warning: Verbosity toggle: instability

Instability detected. Aborting.

@ SciMLBase ~/.julia/packages/SciMLBase/Pp33G/src/integrator_interface.jl:1001

3-element Vector{NamedTuple}:

(dt = 0.2, n_seeds = 15, violations = 10, unstable_runs = 0, worst_min = -333.97)

(dt = 0.1, n_seeds = 15, violations = 10, unstable_runs = 0, worst_min = -143.07)

(dt = 0.05, n_seeds = 15, violations = 9, unstable_runs = 1, worst_min = -130.86)

Across a fifteen-seed ensemble, the violation *rate* stays roughly constant (about two-thirds of seeds) at every step size tested, rather than falling as dt shrinks — and at the smallest step size one seed even triggers a solver instability abort rather than a cleaner trajectory. This is the opposite of the usual “smaller step, smaller error” relationship from the calculus and numerical methods unit’s convergence study, because that relationship assumes refining the *same* underlying continuous path. Here, each dt samples an entirely different discretized Wiener process from the same seed, so nonnegativity — which depends on the specific realized noise path, not on truncation error — does not improve monotonically with a finer grid. The only truly reliable statement across step sizes is the structural one: conservation holds to solver tolerance regardless of dt , because it follows from the noise matrix’s construction, not from step-size accuracy.

K.5.2.2 Solution to Exercise 2

The exercise asks to rerun the seed-1 (caveat) trajectory with a `DiscreteCallback` that clamps I to zero whenever it is negative, and to compare the corrected and uncorrected conservation errors.

```
Random.seed!(1)
prob_uncorrected = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
                               noise_rate_prototype = noise_shape())
sol_uncorrected = solve(prob_uncorrected, LambdaEM(); dt = 0.1, adaptive = false)
states_uncorrected = Array(sol_uncorrected)

fire_count = Ref{Int}(0)
clamp_condition(u, t, integrator) = u[2] < 0
function clamp_affect!(integrator)
    fire_count[] += 1
    integrator.u[2] = max(integrator.u[2], 0.0)
    return nothing
end
clamp_cb = DiscreteCallback(clamp_condition, clamp_affect!)

Random.seed!(1)
prob_clamped = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, sir_history, tspan, p;
                           noise_rate_prototype = noise_shape())
sol_clamped = solve(prob_clamped, LambdaEM(); dt = 0.1, adaptive = false, callback = clamp_cb)
states_clamped = Array(sol_clamped)

(
    uncorrected_min_I = round(minimum(states_uncorrected[2, :]); digits = 2),
    uncorrected_conservation_error = round(maximum(abs.(vec(sum(states_uncorrected; dims = 1,

```

```

        clamped_min_I = round(minimum(states_clamped[2, :]); digits = 2),
        clamped_conservation_error = round(maximum(abs.(vec(sum(states_clamped; dims = 1))) .- 100),
        callback_fires = fire_count[],
    )

```

```

(uncorrected_min_I = -86.61, uncorrected_conservation_error = 4.55e-13, clamped_min_I = -
3.27, clamped_conservation_error = 29.7, callback_fires = 36)

```

The clamp callback fires dozens of times and does raise the minimum value of I substantially — but it does **not** fully eliminate negative excursions (the dense interpolation between clamped grid points can still dip below zero), and, critically, it destroys the *conservation* property that held to solver tolerance in the uncorrected run: the corrected trajectory’s conservation error is now many orders of magnitude larger than before. This is the concrete mechanism behind the book chapter’s claim that clamping the current state is an incomplete fix for a **delayed** recovery term: the clamp only ever touches I at the moment it fires, but the recovery drift and diffusion at every time $t' \in (t, t + \tau)$ continue to read the *pre-clamp* history at t through $\mathbf{h}(\mathbf{p}, \mathbf{t}' - \tau)$. Removing mass from I without removing the corresponding mass from S or R breaks the $S + I + R = N$ identity, and there is no way to repair that within a single local correction because the delayed term has already “seen” the uncorrected value and will keep depending on it for up to τ more time units.

K.5.3 Discrete-event simulation

K.5.3.1 Solution to Exercise 1

The exercise asks for an `EventQueueABM` recovery event whose `timing` returns the fixed value $\tau = 1/\gamma = 4$ instead of an exponential draw, compared against the deterministic DDE at the same parameters.

```

using Agents
using Random
using DelayDiffEq
using OrdinaryDiffEq

= 0.05
c = 10.0
= 0.25
N = 1000
I0 = 10
tf = 40.0

```

```

= 1 /

@agent struct QueuePerson(NoSpaceAgent)
    status::Symbol
end

function transmit!(agent, model)
    alter = random_agent(model)
    if alter.status == :I && (rand(abmrng(model)) < model.c)
        agent.status = :I
    end
    return nothing
end

recover!(agent, model) = (agent.status = :R)

transmit_propensity(agent, model) = agent.status == :S ? model.c : 0.0
recovery_propensity(agent, model) = agent.status == :I ? model.c : 0.0
fixed_delay_timing(agent, model, propensity) = agent.status == :I ? model.c : Inf

infected(x) = count(==( :I ), x)

properties = Dict{<:Symbol, <:Number}()
transmit_event = AgentEvent{QueuePerson}(action! = transmit!, propensity = transmit_propensity)
recovery_event = AgentEvent{QueuePerson}(action! = recover!, propensity = recovery_propensity, timing = fixed_delay_timing)
model = EventQueueABM{QueuePerson}(transmit_event, recovery_event; properties, rng = Xoshiro128Plus())
for i in 1:N
    add_agent!(QueuePerson(; id = i, status = i <= IO ? :I : :S), model)
end
data, _ = run!(model, tf; adata = [(:status, infected)])

peak_idx = argmax(data.infected_status)
eq_peak = data.infected_status[peak_idx]
eq_peak_t = data.time[peak_idx]

function sir_dde!(du, u, h, p, t)
    S, I, R = u
    p, cp, p = p
    Ntot = S + I + R
    dS = p * cp * I / Ntot
    Sd, Id, Rd = h(p, t - p)
    Ntotd = Sd + Id + Rd
    d = p * cp * Id / Ntotd
end

```

```

    du[1] = - * S
    du[2] = * S - d * Sd
    du[3] = d * Sd
    return nothing
end
sir_history(p, t) = [1000.0, 0.0, 0.0]
sol_dde = solve(DDEProblem(sir_dde!, [990.0, 10.0, 0.0], sir_history, (0.0, tf), (, c, ));
                constant_lags = []),
                MethodOfSteps(Tsit5())); reltol = 1e-8, abstol = 1e-8)
dde_peak, dde_idx = findmax(sol_dde[2, :])

(eq_peak = eq_peak, eq_peak_t = round(eq_peak_t; digits = 2),
 dde_peak = round(dde_peak; digits = 1), dde_peak_t = round(sol_dde.t[dde_idx]; digits = 2))

```

```
(eq_peak = 300, eq_peak_t = 11.09, dde_peak = 350.9, dde_peak_t = 10.87)
```

The single stochastic `EventQueueABM` realization peaks close to — a bit below, as expected for one finite-population sample — the deterministic DDE’s peak, and at a comparable time. This individual-level construction reaches the same fixed-delay assumption **without a history function**: no population-level state needs to “remember” what happened τ time units ago, because each infectious agent’s own scheduled recovery event already carries that information. The DDE’s method of steps is a workaround for the fact that a population-level ODE has *forgotten* who was infected when; an individual-level model like this one never has that problem in the first place.

K.5.3.2 Solution to Exercise 2

The exercise asks to extend the `ConcurrentSim` model with a `Resource` representing a limited number of hospital beds, comparing the infectious curve and wait times against the unconstrained model.

```

using ConcurrentSim
using ResumableFunctions
using Distributions

mutable struct BedPerson
    id::Int
    status::Symbol
end
mutable struct BedModel

```

```

sim::Simulation
  ::Float64
c::Float64
  ::Float64
beds::Resource
Ia::Vector{Int}
wait_times::Vector{Float64}
people::Vector{BedPerson}
end

@resumable function live(sim::Simulation, person::BedPerson, m::BedModel, rng)
  while person.status == :S
    @yield timeout(sim, rand(rng, Exponential(1 / m.c)))
    alter = person
    while alter === person
      alter = rand(rng, m.people)
    end
    if alter.status == :I && rand(rng) < m.
      person.status = :I
      push!(m.Ia, count(pp -> pp.status == :I, m.people))
    end
  end
  if person.status == :I
    request_time = now(sim)
    @yield request(m.beds)
    push!(m.wait_times, now(sim) - request_time)
    @yield timeout(sim, rand(rng, Exponential(1 / m.)))
    person.status = :R
    @yield release(m.beds)
    push!(m.Ia, count(pp -> pp.status == :I, m.people))
  end
end

function init_model(S, I, R, , c, , nbeds)
  sim = Simulation()
  beds = Resource(sim, nbeds)
  people = BedPerson[]
  for i in 1:S
    push!(people, BedPerson(i, :S))
  end
  for i in 1:I
    push!(people, BedPerson(S + i, :I))
  end
end

```



```

    end
    for i in 1:R
        push!(people, BedPerson(S + I + i, :R))
    end
    BedModel(sim, , c, , beds, [I], Float64[], people)
end

function run_bed_model(nbeds; seed = 20250711)
    rng = Xoshiro(seed)
    m = init_model(N - I0, I0, 0, , c, , nbeds)
    for person in m.people
        @process live(m.sim, person, m, rng)
    end
    run(m.sim, tf)
    return m
end

unconstrained = run_bed_model(N)      # beds = N: capacity never binds
constrained = run_bed_model(50)

(
    unconstrained_peak_I = maximum(unconstrained.Ia),
    unconstrained_mean_wait = isempty(unconstrained.wait_times) ? 0.0 :
        round(sum(unconstrained.wait_times) / length(unconstrained.wait_times); digits = 2),
    constrained_peak_I = maximum(constrained.Ia),
    constrained_mean_wait = round(sum(constrained.wait_times) / length(constrained.wait_times); digits = 2),
    constrained_max_wait = round(maximum(constrained.wait_times); digits = 2),
    constrained_frac_waited = round(count(>(1e-9), constrained.wait_times) / length(constrained.wait_times); digits = 2),
)

```

```
(unconstrained_peak_I = 222, unconstrained_mean_wait = 0.0, constrained_peak_I = 712, constrained_mean_wait = 10.0, constrained_max_wait = 26, constrained_frac_waited = 0.9999999999999999)
```

With only 50 beds, most infectious individuals wait nearly ten time units on average (and up to about 26) before a bed frees up — and the epidemic’s peak infectious count is **substantially larger** than in the unconstrained model, even though β , c , and γ are unchanged. The mechanism is that an individual’s `status` is still `:I` (and so can still transmit) throughout both the *waiting* period and the treatment `timeout`; a scarce bed supply lengthens the effective time each individual spends counted as infectious, which is exactly the kind of resource-contention effect a `Resource` makes easy to express. `EventQueueABM`, as used in this appendix, has propensities and actions attached to individual agents but no concept of a shared, capacity-limited object that multiple agents must queue for — modeling beds there would require encoding the queue

and occupancy bookkeeping by hand inside the agent’s own state, which is precisely the kind of general process/resource abstraction `ConcurrentSim` provides natively.

K.6 Advanced scientific machine learning solutions

These solutions use the shared root environment and reuse the exact model kernels from the appendix chapters. Random draws are seeded with `StableRNG`, so the reported numbers are reproducible without being universal constants. Each solution is wrapped in a `let` block so its working variables do not leak into the shared solutions document.

K.6.1 Universal differential equations

K.6.1.1 Solution to Exercise 1

Shortening the training window to `train_time = 15.0` stops the observations before the epidemic has run its full course, so the training data visit a *smaller* range of infectious proportions. The learned closure therefore matches the true force of infection $\lambda^*(I) = \beta c \cdot I$ only up to the new, smaller $I_{\max}$, and diverges above it.

```
using OrdinaryDiffEq, Distributions, StableRNGs, Random
using Lux, DiffEqFlux, SciMLSensitivity
using Optimization, OptimizationOptimisers, OptimizationOptimJL
using CairoMakie

a04_result = let
    N = 1000.0; c = 0.5; β = 0.25
    u0 = [0.99, 0.01, 0.0]; tspan = (0.0, 40.0); t = 1.0
    train_time = 15.0 # shortened window (was 30.0)

    function sir_true!(du, u, p, t)
        S, I, C = u; β, c = p
        du[1] = -β*S*I; du[2] = β*S*I - c*I; du[3] = c*S*I
    end
    sol_true = solve(ODEProblem(sir_true!, u0, tspan, [c, β]), Tsit5(); saveat = t)
    ts = 0.0:t:train_time
    Ctrue = reduce(hcat, sol_true(ts).u)[3, :]
    data_rng = StableRNG(20240501)
    noisy = [rand(data_rng, Poisson(max(c, 0.0)))] for c in N .* diff(Ctrue)]

    foi = Lux.Chain(Lux.Dense(1, 5, tanh), Lux.Dense(5, 1, softplus))
```

```

_, st = Lux.setup(StableRNG(1), foi)
unflat() = (layer_1 = (weight = reshape([1:5], 5, 1), bias = [6:10]),
            layer_2 = (weight = reshape([11:15], 1, 5), bias = [16:16]))
(I, _) = foi([I], unflat(), st)[1][1]
p_init = 0.1 .* randn(StableRNG(2), Float64, 16)

function sir_ude!(du, u, _, t)
    S, I, C = u; _ = (I, _)
    du[1] = -*S; du[2] = *S - *I; du[3] = *S
end
prob = ODEProblem(sir_ude!, u0, (0.0, train_time), p_init)
predict() = Array(solve(prob, Tsit5(); p = _, saveat = t,
    sensealg = InterpolatingAdjoint(autojacvec = ReverseDiffVJP()))
loss() = ( _ = max.(N .* diff(predict())[3, :]), 1e-6); sum( _.- noisy .* log.( )))

optf = OptimizationFunction((_, _) -> loss(), Optimization.AutoZygote())
r1 = solve(OptimizationProblem(optf, p_init),
    OptimizationOptimisers.Adam(0.05); maxiters = 150)
_hat = solve(OptimizationProblem(optf, r1.u),
    OptimizationOptimJL.BFGS(); maxiters = 40).u

Igrid = 0.0:0.005:0.5
_nn = [ (I, _hat) for I in Igrid]
Imax = maximum(reduce(hcat, sol_true(ts).u)[2, :])
inr = Igrid .<= Imax
max_err = maximum(abs.(_nn[inr] .- c .* Igrid[inr]))

fig = Figure(size = (760, 400))
ax = Axis(fig[1, 1]; xlabel = "infectious proportion I",
    ylabel = "force of infection ",
    title = "Short window (train_time = 15): matches only up to I  $(round(Imax; digits
lines!(ax, Igrid, c .* Igrid; color = :black, linewidth = 2, label = "true c·I")
lines!(ax, Igrid, _nn; color = :firebrick, linestyle = :dash, linewidth = 2,
    label = "neural network")
vlines!(ax, [Imax]; color = :gray, linestyle = :dot)
axislegend(ax; position = :lt)

(; Imax = round(Imax; digits = 4), max_err = round(max_err; sigdigits = 3), fig)
end;
a04_result.fig

```

Warning: Lux.apply(m::AbstractLuxLayer, x::AbstractArray{<:ReverseDiff.TrackedReal}, ps, st

1. If this was not the desired behavior overload the dispatch on ``m``.
2. This might have performance implications. Check which layer was causing this problem using `@ ArrayInterfaceReverseDiffExt ~/.julia/packages/LuxCore/kQC9S/ext/ArrayInterfaceReverseDiff.jl`

```
Warning: Verbosity toggle: dt_epsilon
Initial timestep too small (near machine epsilon), using default: dt = -1.0e-6
@ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/it3lY/src/initdt.jl:214
```

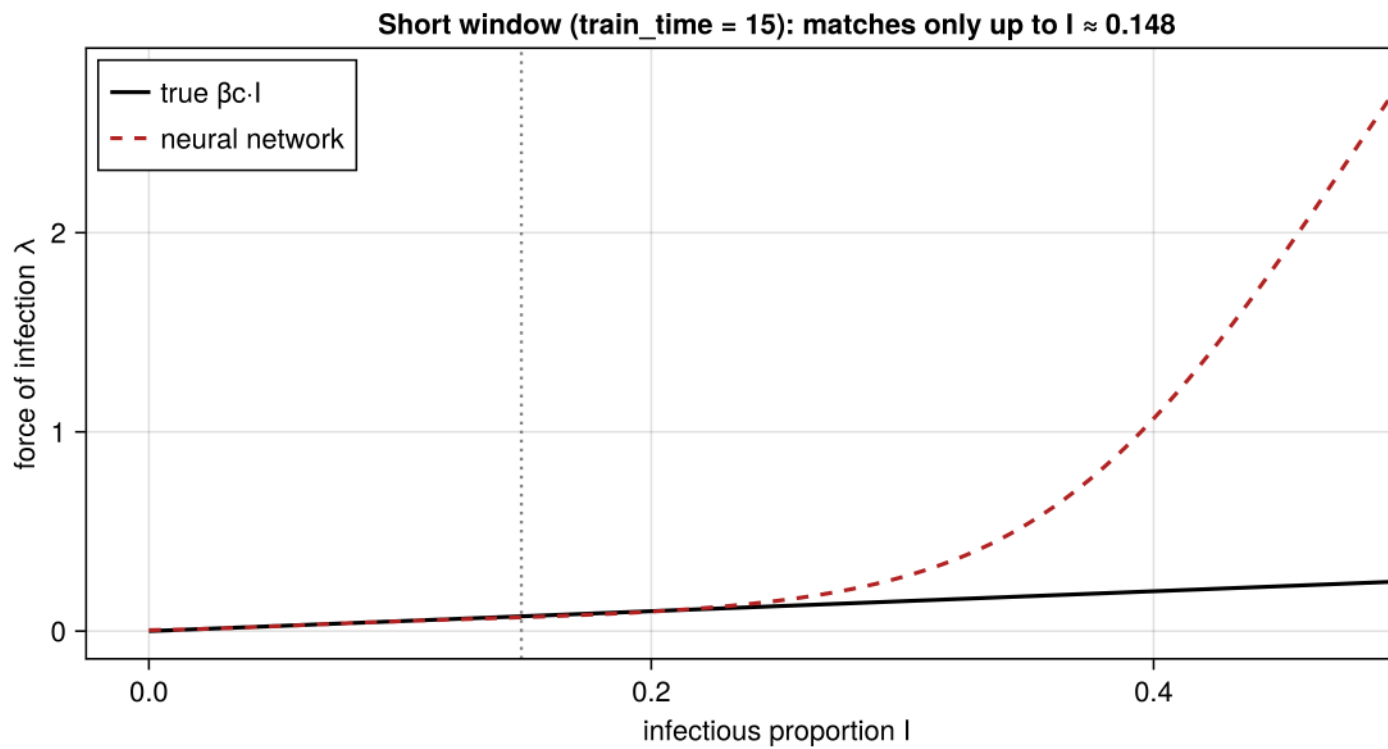


Figure K.5

```
(training_range_I_max = a04_result.I_max, max_abs_error = a04_result.max_err)
```

Solution K.22.

```
(training_range_I_max = 0.1483, max_abs_error = 0.00481)
```

With the shorter window the epidemic is caught near its rise, so $I_{\max}$ falls to about 0.15 (printed above) — smaller than the ≈ 0.158 reached with the full window. Within that range the closure is still accurate (maximum absolute error of order 10^{-2}), but the dashed curve peels away from the truth immediately past the dotted line. The lesson is unchanged and sharpened: **a UDE closure is identified only over the states its training data actually visit**. Excluding the peak removes the very observations that pin down the closure at large I .

K.6.1.2 Solution to Exercise 2

The deeper network `Dense(1,8,tanh) → Dense(8,8,tanh) → Dense(8,1,softplus)` has $8 + 8 + 64 + 8 + 8 + 1 = 97$ parameters. Hard-coded slices become unwieldy, so we write a **generic** `flatten/unflatten` driven by the parameter shapes.

```
a04_deep = let
  N = 1000.0; c = 0.5;  = 0.25
  u0 = [0.99, 0.01, 0.0]; tspan = (0.0, 40.0); t = 1.0; train_time = 30.0

  function sir_true!(du, u, p, t)
    S, I, C = u; , = p
    du[1] = - *S*I; du[2] = *S*I - *I; du[3] = *S*I
  end
  sol_true = solve(ODEProblem(sir_true!, u0, tspan, [ c, ]), Tsit5()); saveat = t)
  ts = 0.0:t:train_time
  Ctrue = reduce(hcat, sol_true(ts).u)[3, :]
  data_rng = StableRNG(20240501)
  noisy = [rand(data_rng, Poisson(max(c, 0.0))) for c in N .* diff(Ctrue)]

  net = Lux.Chain(Lux.Dense(1, 8, tanh), Lux.Dense(8, 8, tanh), Lux.Dense(8, 1, softplus))
  ps0, st = Lux.setup(StableRNG(1), net)

  # Generic flatten / unflatten for a Chain of Dense layers.
  shapes = [(size(l.weight), length(l.bias)) for l in ps0]
  flatten(ps) = reduce(vcat, [vcat(vec(l.weight), l.bias) for l in ps])
  function unflat()
    layers = Pair{Symbol,Any}[]; k = 1
    for (i, ((wr, wc), nb)) in enumerate(shapes)
      W = reshape( [k:k + wr*wc - 1], wr, wc); k += wr*wc
      b = [k:k + nb - 1]; k += nb
      push!(layers, Symbol("layer_", i) => (weight = W, bias = b))
    end
    (; layers...)
  end
```

```

end
(I, ) = net([I], unflat(), st)[1][1]
p_init = 0.1 .* randn(StableRNG(2), Float64, length(flatten(ps0)))

function sir_udef!(du, u, , t)
    S, I, C = u;  = (I, )
    du[1] = -*S; du[2] = *S - *I; du[3] = *S
end
prob = ODEProblem(sir_udef!, u0, (0.0, train_time), p_init)
predict() = Array(solve(prob, Tsit5(); p = , saveat = t,
    sensealg = InterpolatingAdjoint(autojacvec = ReverseDiffVJP()))
loss() = ( = max.(N .* diff(predict()[3, :]), 1e-6); sum( .- noisy .* log.()))

optf = OptimizationFunction((, _) -> loss(), Optimization.AutoZygote())
r1 = solve(OptimizationProblem(optf, p_init),
    OptimizationOptimisers.Adam(0.05); maxiters = 150)
_hat = solve(OptimizationProblem(optf, r1.u),
    OptimizationOptimJL.BFGS(); maxiters = 40).u

Igrid = 0.0:0.005:0.5
_nn = [ (I, _hat) for I in Igrid]
Imax = maximum(reduce(hcat, sol_true(ts).u)[2, :])
inr = Igrid .<= Imax
max_err = maximum(abs.(_nn[inr] .- c .* Igrid[inr]))

fig = Figure(size = (760, 400))
ax = Axis(fig[1, 1]; xlabel = "infectious proportion I",
    ylabel = "force of infection ",
    title = "Deeper network (97 parameters)")
lines!(ax, Igrid, c .* Igrid; color = :black, linewidth = 2, label = "true c·I")
lines!(ax, Igrid, _nn; color = :seagreen, linestyle = :dash, linewidth = 2,
    label = "deep network")
vlines!(ax, [Imax]; color = :gray, linestyle = :dot)
axislegend(ax; position = :lt)

(; nparams = length(p_init), max_err = round(max_err; sigdigits = 3), fig)
end;
a04_deep.fig

```

Warning: Lux.apply(m::AbstractLuxLayer, x::AbstractArray{<:ReverseDiff.TrackedReal}, ps, st

1. If this was not the desired behavior overload the dispatch on `m`.

2. This might have performance implications. Check which layer was causing this problem using
 @ ArrayInterfaceReverseDiffExt ~/.julia/packages/LuxCore/kQC9S/ext/ArrayInterfaceReverseDiffExt.jl:114
 Warning: Verbosity toggle: dt_epsilon
 Initial timestep too small (near machine epsilon), using default: dt = 1.0e-6
 @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/it3lY/src/initdt.jl:214

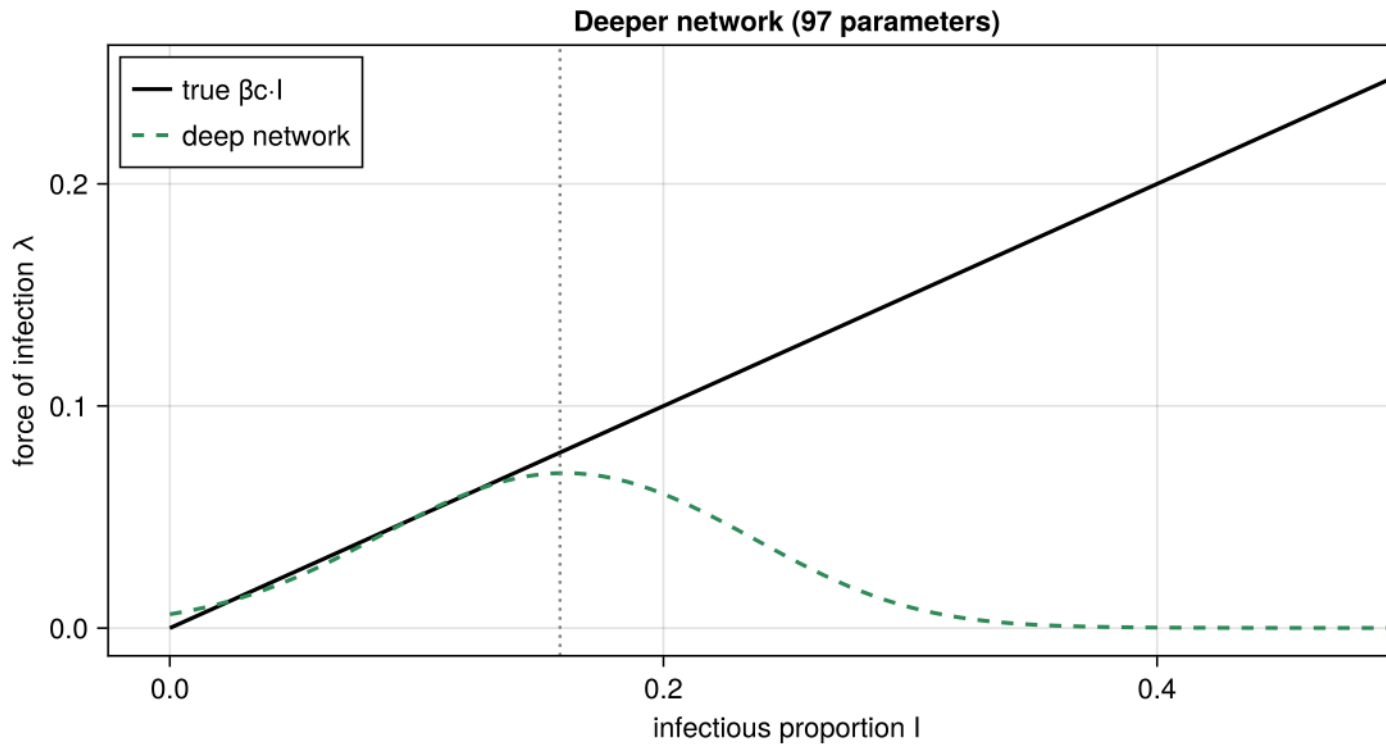


Figure K.6

```
(parameters = a04_deep.nparams, max_abs_error = a04_deep.max_err)
```

Solution K.23.

```
(parameters = 97, max_abs_error = 0.00785)
```

The deeper network has 97 parameters (printed above) and fits the training range about as well as the 16-parameter model (maximum absolute error of order 10^{-3} , comparable to the $\approx 3.8 \times 10^{-3}$ of the shallow network): within the data the extra flexibility yields no meaningful improvement, because the true closure is *linear* and the small network already represents it. The cost of the extra capacity shows up in the **extrapolation region** past the dotted line, where the higher-variance deep network is free to wiggle more and tends to depart from the truth faster. This is the bias–variance trade-off: adding capacity cannot reduce an already-negligible in-range bias, but it increases variance where the data do not constrain the fit. For a closure known to be smooth and near-linear, the parsimonious network is preferable.

K.6.2 Finite-state projection

K.6.2.1 Solution to Exercise 1

Capping the retained infectious count at $I_{\max} = 15$ deletes every state with $I > 15$. Any infection event that would push I past the cap has its rate subtracted from the diagonal but no destination inside the retained set, so its probability **leaks** out and the total $\sum \mathbf{p}$ falls below 1.

```
using OrdinaryDiffEq, SparseArrays, LinearAlgebra, CairoMakie, Printf

function a05_generator(N, , ; Smax = N, Imax = N)
    index = Dict{Tuple{Int,Int},Int}(); states = Tuple{Int,Int}[]
    for s in 0:Smax, i in 0:Imax
        if s + i <= N
            push!(states, (s, i)); index[(s, i)] = length(states)
        end
    end
    rows = Int[]; cols = Int[]; vals = Float64[]
    add!(r, c, v) = (push!(rows, r); push!(cols, c); push!(vals, v))
    for (col, (s, i)) in enumerate(states)
        a_inf = *s*i; a_rec = *i
        add!(col, col, -(a_inf + a_rec))
        a_inf > 0 && haskey(index, (s-1, i+1)) && add!(index[(s-1, i+1)], col, a_inf)
        a_rec > 0 && haskey(index, (s, i-1)) && add!(index[(s, i-1)], col, a_rec)
    end
    return sparse(rows, cols, vals, length(states), length(states)), states, index
end

a05_trunc = let
    Ncap = 100;  = 0.005;  = 0.25; tspan = (0.0, 40.0)
```



```

function total_prob(Imax)
    Q, states, index = a05_generator(Ncap, , ; Imax = Imax)
    p0 = zeros(length(states)); p0[index[(99, 1)]] = 1.0
    sol = solve(ODEProblem((dp, p, Q, t) -> mul!(dp, Q, p), p0, tspan, Q),
        Vern7(); abstol = 1e-10, reltol = 1e-8, saveat = 1.0)
    sol.t, [sum(u) for u in sol.u]
end

fig = Figure(size = (760, 400))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "total probability  $\Sigma p$ ",
    title = "Probability leakage under truncation")
leaks = NamedTuple[]
for (Imax, col) in zip((15, 25, 40, 50), (:firebrick, :orange, :seagreen, :steelblue))
    t, tp = total_prob(Imax)
    lines!(ax, t, tp; color = col, linewidth = 2, label = "Imax = $Imax")
    push!(leaks, (; Imax, final_leak = 1 - minimum(tp)))
end
hlines!(ax, [1.0]; color = :gray, linestyle = :dot)
axislegend(ax; position = :lb)
(; leaks, fig)
end;
a05_trunc.fig

```

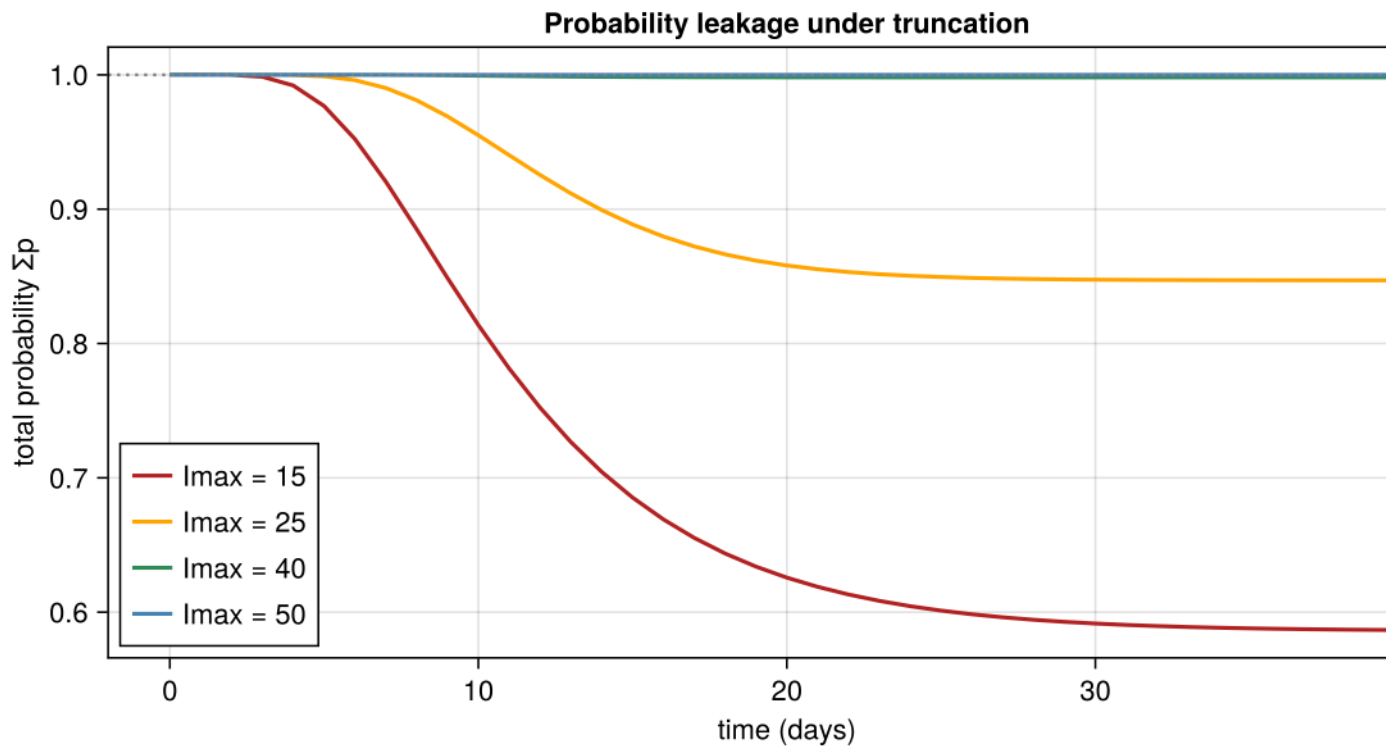


Figure K.7

Solution K.24.

```
for r in a05_trunc.leaks
    @printf("I_max = %2d    →    final leakage = %.3e\n", r.I_max, r.final_leak)
end
```

```
I_max = 15    →    final leakage = 4.134e-01
I_max = 25    →    final leakage = 1.530e-01
I_max = 40    →    final leakage = 1.865e-03
I_max = 50    →    final leakage = 8.936e-06
```

With $I_{\max} = 15$ probability starts leaking around $t \approx 2$ — just as the outbreak accelerates and the infectious count climbs toward the cap — and the final shortfall is roughly 40%. That timing coincides with the approach to the epidemic peak seen in the appendix’s marginal plots: leakage is governed by how much probability mass reaches $I \approx I_{\max}$. Loosening the cap moves the onset later and shrinks the loss. A short sweep shows that $I_{\max} = 45$

still leaks $\approx 1.7 \times 10^{-4}$, whereas $\text{Imax} = 50$ brings the leakage to $\approx 9 \times 10^{-6}$, below the 10^{-4} target; so retaining infectious counts up to about 50 is enough for this scenario.

K.6.2.2 Solution to Exercise 2

Our kernel only ever **push**es states satisfying $s + i \leq N$, so it allocates the **triangular** state space, never the rectangular one. The saving is the ratio of the two counts.

Solution K.25.

```
let
  @printf("%6s %12s %12s %8s\n", "N", "triangular", "rectangular", "ratio")
  for n in (50, 100, 250, 1000)
    tri = ((n + 1) * (n + 2)) ÷ 2
    rect = (n + 1)^2
    @printf("%6d %12d %12d %8.4f\n", n, tri, rect, tri / rect)
  end
end
```

N	triangular	rectangular	ratio
50	1326	2601	0.5098
100	5151	10201	0.5050
250	31626	63001	0.5020
1000	501501	1002001	0.5005

The triangular count $(N+1)(N+2)/2$ is almost exactly **half** the rectangular $(N+1)^2$, and the ratio $\rightarrow 1/2$ as $N \rightarrow \infty$. So exploiting reachability halves the number of equations — a worthwhile constant factor, but it does **not** change the *order* of growth: both counts are $\mathcal{O}(N^2)$, so the CME still grows quadratically in N (and would grow cubically if a third species were tracked). A sparse index handler that additionally *dropped low-probability states* — keeping only the dynamically occupied region and expanding it adaptively (as adaptive-FSP schemes do) — could push FSP to larger populations, because at any instant the probability mass occupies only a thin band of the triangle. That would attack the constant factor much more aggressively, but the fundamental quadratic (or cubic) scaling in the number of species is intrinsic to enumerating a joint state space; it is why FSP remains a small-population / few-species method.

K.6.3 Probabilistic numerical integration

K.6.3.1 Solution to Exercise 1

Fixing the solver tolerance and varying only the output step t reveals a subtlety: with an **adaptive** solver, t (via `saveat`) sets where the dense output is *sampled*, not the internal step size, which the tolerance controls. So the measured error is essentially flat in t .

```
using EpiModelingCourse, OrdinaryDiffEq, CairoMakie, Printf

a06_step = let
    p = BASELINE_PARAMETERS; s0 = BASELINE_INITIAL_STATE
    u0 = [s0.S, s0.I, s0.R]; tspan = (0.0, 40.0)
    prob = ODEProblem(EpiModelingCourse.sir_ode!, u0, tspan, p)
    ref = solve(prob, Vern9(); abstol = 1e-12, reltol = 1e-12, saveat = 0.05)
    refI(t) = ref(t)[2]
    dts = [1.0, 0.5, 0.25, 0.1]

    adaptive_err = Float64[]
    fixed_err = Float64[]
    for dt in dts
        a = solve(prob, Tsit5(); abstol = 1e-1, reltol = 1e-1, saveat = dt)
        push!(adaptive_err, maximum(abs(a.u[k][2] - refI(a.t[k])) for k in eachindex(a.t)))
        f = solve(prob, Tsit5(); adaptive = false, dt = dt, saveat = dt)
        push!(fixed_err, maximum(abs(f.u[k][2] - refI(f.t[k])) for k in eachindex(f.t)))
    end

    fig = Figure(size = (760, 400))
    ax = Axis(fig[1, 1]; xlabel = "output / step size t", ylabel = "max |I error|",
        xscale = log10, yscale = log10, title = "Error vs. t")
    scatterlines!(ax, dts, max.(adaptive_err, 1e-14); color = :firebrick, linewidth = 2,
        label = "adaptive (tol = 1e-1), t = saveat")
    scatterlines!(ax, dts, max.(fixed_err, 1e-14); color = :steelblue, linewidth = 2,
        label = "fixed step, dt = t")
    axislegend(ax; position = :rb)
    (; dts, adaptive_err, fixed_err, fig)
end;
a06_step.fig
```

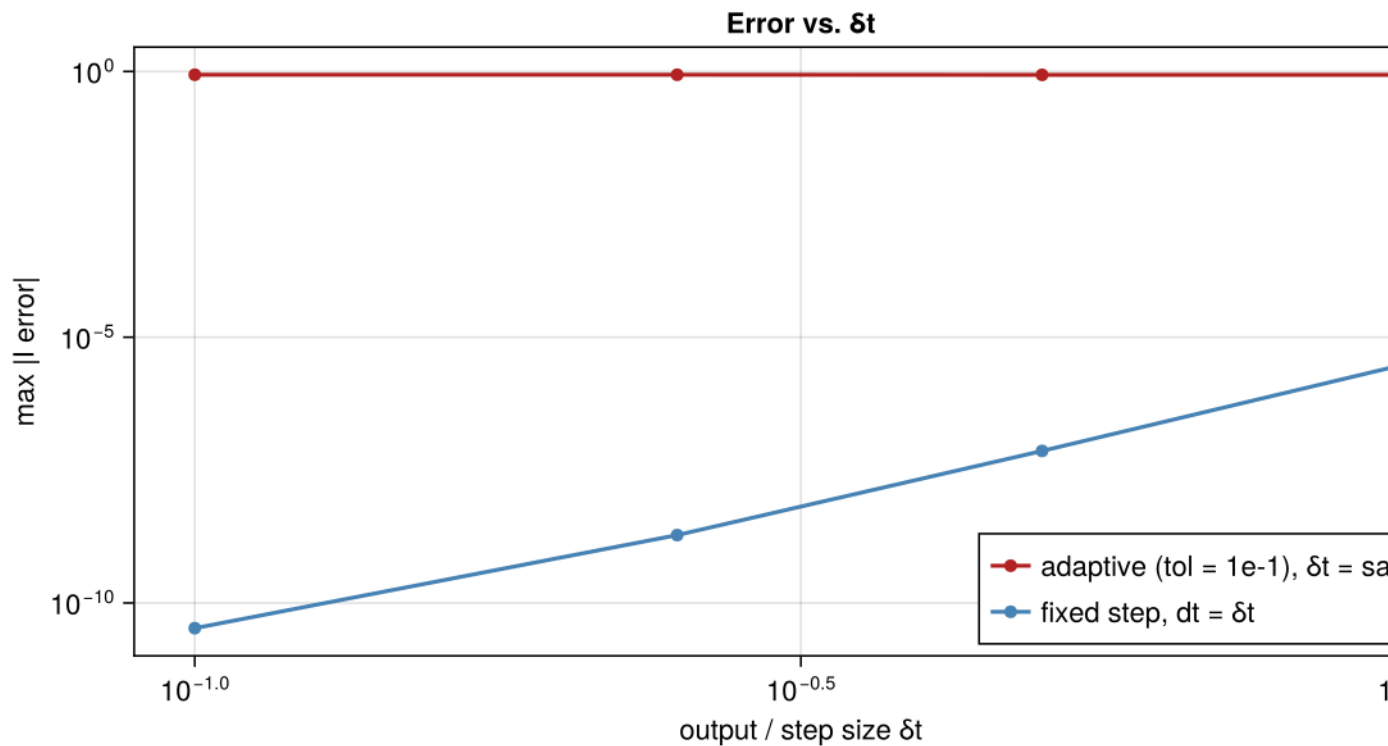


Figure K.8

Solution K.26.

```
for (dt, ea, ef) in zip(a06_step.dts, a06_step.adaptive_err, a06_step.fixed_err)
    @printf("t = %.2f    adaptive |I err| = %.3e    fixed-step |I err| = %.3e\n", dt, ea, ef)
end
```

t = 1.00	adaptive I err = 8.627e-01	fixed-step I err = 3.162e-06
t = 0.50	adaptive I err = 8.627e-01	fixed-step I err = 7.249e-08
t = 0.25	adaptive I err = 8.672e-01	fixed-step I err = 1.892e-09
t = 0.10	adaptive I err = 8.673e-01	fixed-step I err = 3.354e-11

The **adaptive** curve is nearly flat ($\approx 8.6 \times 10^{-1}$ throughout): the coarse tolerance, not the output grid, fixes the internal steps, so refining t only interpolates the same trajectory more densely. Disabling adaptivity (**adaptive** = **false**, **dt** = **t**) restores the textbook picture: the **fixed-step** error falls steeply as Δt shrinks — roughly a straight line of large slope on log-log axes, reflecting **Tsit5**'s high order. A probabilistic solver

encodes exactly this in its posterior standard deviation: at a genuinely smaller step it reports smaller numerical uncertainty. The exercise’s pedagogical twist is that *output spacing is not step size* — to make `\delta t` control accuracy you must control the actual integration step.

K.6.3.2 Solution to Exercise 2

This is a conceptual exercise about the current `ProbNumDiffEq.jl` Gaussian-filter API; the package is not executable in the resolved stack (see the appendix’s reproducibility note), so the discussion is grounded in the deterministic refinement behaviour measured above rather than in a live probabilistic solve.

- **(a) Prior order in IWP(k).** The prior is a k -times integrated Wiener process: the state and its first k derivatives are modelled as a Gauss–Markov process. A higher k gives the filter a higher-order local model of the solution, so — like a higher-order classical method — its local truncation error shrinks faster as the step shrinks (the fixed-step sweep above is the deterministic analogue of that order effect). Too high an order, however, wastes work and can be poorly conditioned at coarse steps; $k=2$ or $k=3$ is the usual sweet spot for smooth, mildly stiff problems like SIR.
- **(b) EK0 vs EK1.** Both are extended Kalman ODE filters. EK0 linearises the vector field with a *zeroth-order* (constant-Jacobian) approximation — cheap but with a smaller stability region; EK1 uses the *true Jacobian*, giving it implicit-solver-like stability that handles stiffness far better. For the SIR flow, which is mildly stiff around the peak, EK1 is the safer choice at a coarse step, at the cost of a Jacobian evaluation per step.
- **(c) DynamicDiffusion() vs FixedDiffusion().** The diffusion parameter scales the prior’s driving noise and thereby the *magnitude* of the reported uncertainty. `FixedDiffusion` estimates one global scale (a single calibration constant), whereas `DynamicDiffusion` re-estimates it *at every step* from the filter’s own residuals, letting the error bar grow through the fast epidemic peak and shrink in the slow tails — matching the time-varying true error seen in the appendix figure.

For the mildly stiff SIR flow at a coarse step, the best-calibrated combination is **EK1 with a moderate prior order (IWP(3)) and DynamicDiffusion()**: EK1 supplies the stability to keep the mean accurate through the peak, the order-3 prior gives a well-behaved local error model, and dynamic diffusion lets the reported standard deviation track the error’s strong time variation instead of averaging it into one number. EK0 or a fixed diffusion would tend to mis-size the uncertainty exactly where it matters most — around the epidemic maximum.

K.7 Advanced decision and composition solutions

The solutions below keep the mechanistic simulator visible. Numerical values are recomputed by the Julia code rather than copied from a previous run.

K.7.1 Surrogate and emulator models

K.7.1.1 Solution to Exercise 1

Use tuples as the two-dimensional design points. `Surrogates.sample` returns Latin-hypercube points in the requested rectangle, and the current `RadialBasis` API accepts those tuples directly.

```
using CairoMakie, EpiModelingCourse, OrdinaryDiffEq
using Random, Statistics, Surrogates

function a07_q( , )
    p = SIRParameters(beta= , contact_rate=10.0, gamma= , N=1000.0)
    sol = solve(ODEProblem(EpiModelingCourse.sir_ode!, [990.0, 10.0, 0.0], (0.0, 40.0), p),
                Tsit5(); saveat=[40.0], abstol=1e-9, reltol=1e-9)
    sol[3, end] / p.N
end

Random.seed!(20260711)
lo, hi = [0.02, 0.15], [0.08, 0.35]
design = Surrogates.sample(24, lo, hi, LatinHypercubeSample())
responses = [a07_q(x[1], x[2]) for x in design]
emulator = RadialBasis(design, responses, lo, hi; rad=cubicRadial())

grid = [( , ) for  in range(lo[1], hi[1]; length=5)
          for  in range(lo[2], hi[2]; length=5)]
truth = [a07_q(x[1], x[2]) for x in grid]
prediction = emulator(grid)
errors = prediction .- truth
summary = (rmse=sqrt(mean(errors.^2)),
           max_abs_error=maximum(abs.(errors)),
           training_points=length(design))
@assert all(isfinite, errors)
summary
```

```
(rmse = 0.14694578364010763, max_abs_error = 0.4843575592630492, training_points = 24)
```

The grid separates validation from the 24 training points. The largest error identifies a region where the response surface changes most rapidly or where the design is sparse. Increasing the design or adding targeted points should be preferred to extrapolating outside the rectangle.

K.7.1.2 Solution to Exercise 2

The same two-input design can fit both a terminal recovered fraction and an internal maximum. The latter is harder because a small parameter change can move the time of the maximum between output samples.

```
using CairoMakie, EpiModelingCourse, OrdinaryDiffEq
using Random, Statistics, Surrogates

function a07_peak( , )
    p = SIRParameters(beta= , contact_rate=10.0, gamma= , N=1000.0)
    sol = solve(ODEProblem(EpiModelingCourse.sir_ode!, [990.0, 10.0, 0.0], (0.0, 40.0), p),
                Tsit5(); saveat=0.25, abstol=1e-9, reltol=1e-9)
    maximum(sol[2, :]) / p.N
end

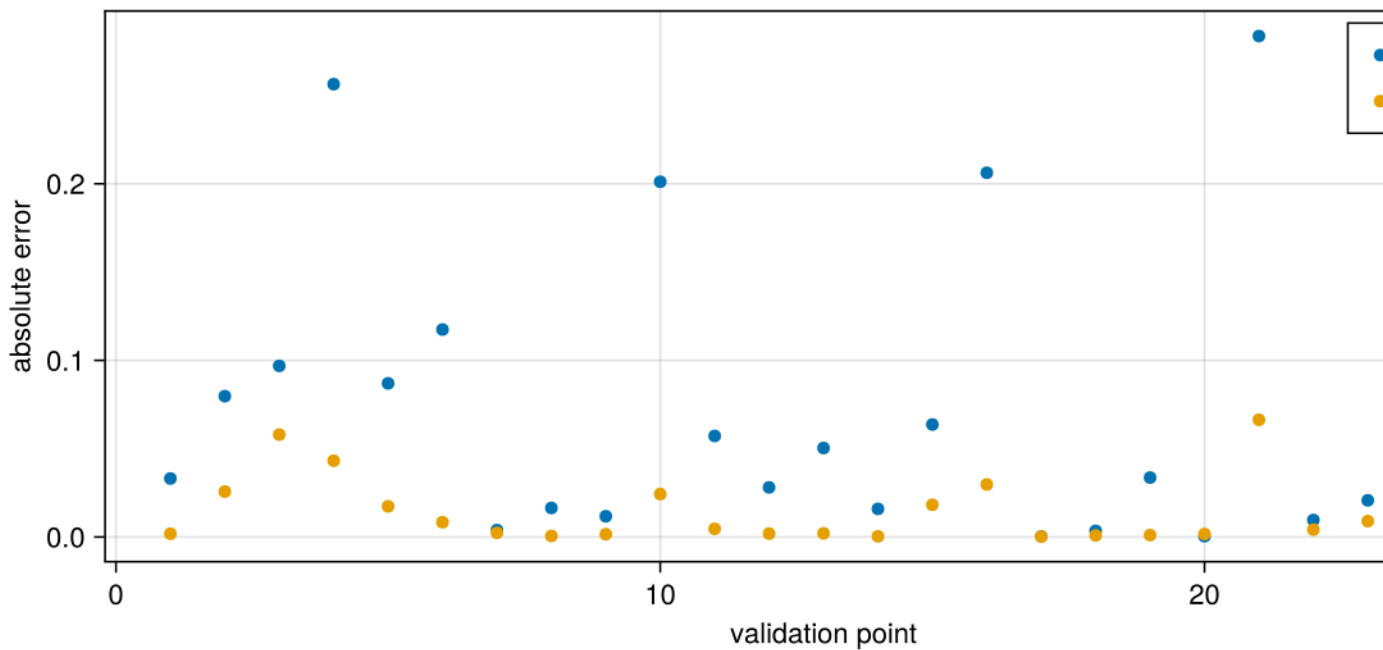
function a07_terminal( , )
    p = SIRParameters(beta= , contact_rate=10.0, gamma= , N=1000.0)
    sol = solve(ODEProblem(EpiModelingCourse.sir_ode!, [990.0, 10.0, 0.0], (0.0, 40.0), p),
                Tsit5(); saveat=[40.0], abstol=1e-9, reltol=1e-9)
    sol[3, end] / p.N
end

Random.seed!(20260712)
lo, hi = [0.02, 0.15], [0.08, 0.35]
design = Surrogates.sample(24, lo, hi, LatinHypercubeSample())
peak_fit = RadialBasis(design, [a07_peak(x...) for x in design], lo, hi;
                        rad=cubicRadial())
terminal_fit = RadialBasis(design, [a07_terminal(x...) for x in design], lo, hi;
                           rad=cubicRadial())
grid = [( , ) for  in range(lo[1], hi[1]; length=5)
         for  in range(lo[2], hi[2]; length=5)]
peak_error = peak_fit.(grid) .- [a07_peak(x...) for x in grid]
terminal_error = terminal_fit.(grid) .- [a07_terminal(x...) for x in grid]

fig = Figure(size=(800, 350))
ax = Axis(fig[1, 1], xlabel="validation point", ylabel="absolute error")
scatter!(ax, eachindex(grid), abs.(terminal_error), label="R(40)/N")
scatter!(ax, eachindex(grid), abs.(peak_error), label="max(I)/N")
```



```
axislegend(ax)
fig
```



Compare the two error arrays rather than assuming that the peak target is always worse. The maximum operator is continuous for a well-resolved smooth trajectory, but it is less smooth as a function of parameters and its accuracy also depends on the output time grid. That is why an emulator for peak prevalence needs a denser design and an explicit time-resolution check.

K.7.2 Advanced optimal control

K.7.2.1 Solution to Exercise 1

Run the same InfiniteOpt model with two collocation resolutions. The helper returns the support values and the collocated control, so the comparison does not confuse support refinement with a change in the objective.

```
using InfiniteOpt, Ipopt, Statistics

function a08_solve(nsupports)
    tf, c, , umax, budget = 20.0, 0.5, 0.25, 0.5, 2.0
    model = InfiniteModel(Ipopt.Optimizer)
```

```

set_optimizer_attribute(model, "print_level", 0)
set_optimizer_attribute(model, "max_iter", 500)
@infinite_parameter(model, t in [0.0, tf], num_supports=nsupports,
                    derivative_method=OrthogonalCollocation(2))
@variable(model, S >= 0, Infinite(t), start=0.99)
@variable(model, I >= 0, Infinite(t), start=0.01)
@variable(model, C >= 0, Infinite(t), start=0.0)
@variable(model, 0 <= u <= umax, Infinite(t), start=0.0)
@constraint(model, S(0.0) == 0.99)
@constraint(model, I(0.0) == 0.01)
@constraint(model, C(0.0) == 0.0)
@constraint(model, deriv(S,t) == -(1-u)* c*S*I)
@constraint(model, deriv(I,t) == (1-u)* c*S*I - *I)
@constraint(model, deriv(C,t) == (1-u)* c*S*I)
@constraint(model, integral(u,t) <= budget)
@objective(model, Min, C(tf) + 1e-3*integral(u^2,t))
constant_over_collocation(u,t)
optimize!(model)
(
    objective = InfiniteOpt.objective_value(model),
    time = InfiniteOpt.supports(t),
    C = InfiniteOpt.value.(C),
    u = InfiniteOpt.value.(u),
)
end

coarse, fine = a08_solve(21), a08_solve(41)
function piecewise_value(x, y, z)
    j = clamp(searchsortedlast(x, z), 1, length(x)-1)
    w = (z-x[j])/(x[j+1]-x[j])
    (1-w)*y[j] + w*y[j+1]
end
common = collect(range(0.0, 20.0; length=101))
coarse_u = [piecewise_value(coarse.time, coarse.u, z) for z in common]
fine_u = [piecewise_value(fine.time, fine.u, z) for z in common]
comparison = (
    coarse_objective=coarse.objective,
    fine_objective=fine.objective,
    objective_difference=fine.objective-coarse.objective,
    max_policy_difference=maximum(abs.(coarse_u-fine_u)),
    coarse_final_cases=coarse.C[end],
    fine_final_cases=fine.C[end],

```

```
)
comparison
```

```
*****
This program contains Ipopt, a library for large-scale nonlinear optimization.
Ipopt is released as open source code under the Eclipse Public License (EPL).
For more information visit https://github.com/coin-or/Ipopt
*****
```

```
(coarse_objective = 0.4300475482541605, fine_objective = 0.42166894309386244, objective_diff
0.008378605160298092, max_policy_difference = 0.08617296474625626, coarse_final_cases = 0.42
```

The objective and final cases measure transcription sensitivity; the common-grid control difference measures policy sensitivity. A discrepancy that shrinks as supports increase is consistent with discretization error. A different qualitative policy after refinement can instead indicate local optima in the nonconvex Ipopt solve, so rerun from other starts before interpreting it.

K.7.2.2 Solution to Exercise 2

Current `SDDP.LinearPolicyGraph` requires affine constraints and objectives. The following deliberately solves a small **linear budget proxy** with exogenous capacity shocks. It is not the nonlinear controlled SIR model: infection products, exponentials, and the state-dependent transmission effect are all omitted.

```
using Ipopt, SDDP
const JuMP = SDDP.JuMP

stages, initial_budget = 4, 1.0
proxy = SDDP.LinearPolicyGraph(
    stages=stages, sense=:Min, lower_bound=0.0, optimizer=Ipopt.Optimizer,
) do sp, stage
    JuMP.@variable(sp, 0 <= remaining <= initial_budget,
                   SDDP.State, initial_value=initial_budget)
    JuMP.@variable(sp, 0 <= action <= 0.5)
    capacity = JuMP.@constraint(sp, action <= 0.25)
    JuMP.@constraint(sp, remaining.out == remaining.in - action)
    SDDP.parameterize(sp, [0.0, 0.25]) do shock
        JuMP.set_normalized_rhs(capacity, 0.25 + shock)
```

```

end
SDDP.@stageobjective(sp, (1.0 + 0.2*stage)*action +
                      0.1*remaining.out)
end
SDDP.train(proxy; iteration_limit=5, print_level=0, log_file=tempname())
path = first(SDDP.simulate(proxy, 1, [:remaining, :action]))
actions = [record[:action] for record in path]
total_action = sum(actions)
@assert total_action <= initial_budget + 1e-5
(actions=actions, total_action=total_action,
 interpretation="affine budget-allocation proxy, not nonlinear SIR control")

```

```

(actions = [1.3983081119293355e-8, -6.685755301105359e-9, -8.474485280023679e-
9, -7.039033842232522e-9], total_action = -8.216193304068204e-9, interpretation = "affine bu
allocation proxy, not nonlinear SIR control")

```

`parameterize` represents exogenous scenario variation in available capacity. The example demonstrates the supported policy-graph mechanics and the budget state, but its action cost is only a proxy for epidemic harm. A valid epidemiological use would require a separately justified linearization and an error comparison against `InfiniteOpt` or the original simulator.

K.7.3 Model composition and structured populations

K.7.3.1 Solution to Exercise 1

Add a third group and keep total population at 1000. The ODE and jump solves use the same Catalyst reaction system, so conservation is a stoichiometric check rather than an assumption added after simulation.

```

using Catalyst, JumpProcesses, OrdinaryDiffEq, Random

three_group = @reaction_network begin
    @parameters N
    (*c1)/N, S1 + I1 --> 2I1
    (*c1*m12)/N, S1 + I2 --> I1 + I2
    (*c1*m13)/N, S1 + I3 --> I1 + I3
    (*c2*m21)/N, S2 + I1 --> I2 + I1
    (*c2)/N, S2 + I2 --> 2I2
    (*c2*m23)/N, S2 + I3 --> I2 + I3
    (*c3*m31)/N, S3 + I1 --> I3 + I1

```

```

    (*c3*m32)/N, S3 + I2 --> I3 + I2
    (*c3)/N, S3 + I3 --> 2I3
    , I1 --> R1
    , I2 --> R2
    , I3 --> R3
end
u0 = [:S1=>330.0,:I1=>3.0,:R1=>0.0,
      :S2=>330.0,:I2=>3.0,:R2=>0.0,
      :S3=>331.0,:I3=>3.0,:R3=>0.0]
p = [:N=>1000.0,:=>0.05,:c1=>10.0,:c2=>10.0,:c3=>8.0,
     :m12=>0.5,:m21=>0.5,:m13=>0.3,:m31=>0.4,
     :m23=>0.6,:m32=>0.5,:=>0.25]
tspan = (0.0, 2.0)
ode = solve(ODEProblem(three_group,u0,tspan,p), Tsit5(); saveat=0.1)
Random.seed!(20260713)
jump = solve(JumpProblem(three_group,
    [pair[1]=>round(Int,pair[2]) for pair in u0], tspan,
    p); saveat=0.1)
function total_population(sol)
    sum(sol[name] for name in (:S1,:I1,:R1,:S2,:I2,:R2,:S3,:I3,:R3))
end
ode_error = maximum(abs.(total_population(ode) .- 1000.0))
jump_error = maximum(abs.(total_population(jump) .- 1000))
@assert ode_error < 1e-7
@assert jump_error == 0
(ode_error=ode_error, jump_error=jump_error,
 aggregate_incidence=1000.0-ode[:S1][end]-ode[:S2][end]-ode[:S3][end])

```

```
(ode_error = 2.2737367544323206e-13, jump_error = 0, aggregate_incidence = 14.62930914938840)
```

The aggregate curve cannot generally identify all six cross-group multipliers, especially if only total incidence is observed. Group-specific observations or informative contact data are needed to distinguish the contact rates and mixing factors.

K.7.3.2 Solution to Exercise 2

Run 100 seeded jump realizations, summarize each time point by a central interval, and compare the ensemble mean with the deterministic ODE.

```

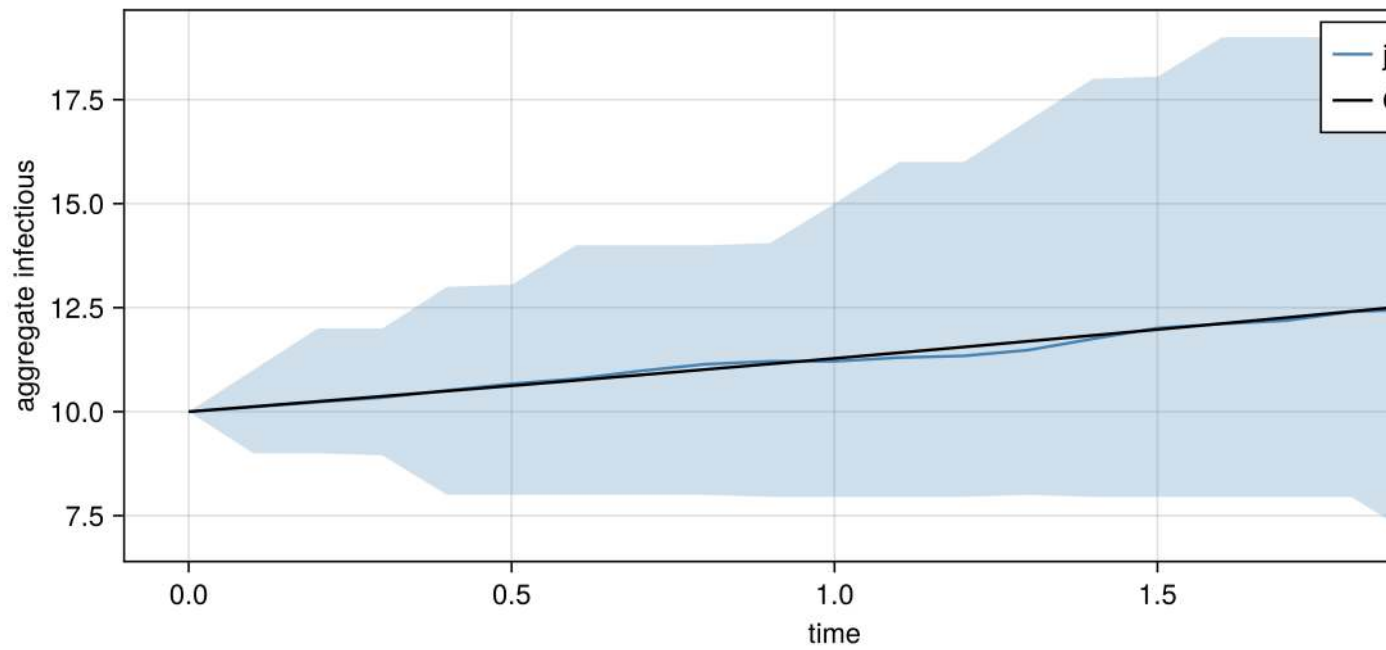
using Catalyst, JumpProcesses, OrdinaryDiffEq, Random, Statistics

two_group = @reaction_network begin
    @parameters N
    (*c1)/N, S1 + I1 --> 2I1
    (*c1*m12)/N, S1 + I2 --> I1 + I2
    (*c2*m21)/N, S2 + I1 --> I2 + I1
    (*c2)/N, S2 + I2 --> 2I2
    , I1 --> R1
    , I2 --> R2
end
u0 = [:S1=>495,:I1=>5,:R1=>0,:S2=>495,:I2=>5,:R2=>0]
p = [:N=>1000.0,:=>0.05,:c1=>10.0,:c2=>10.0,
      :m12=>0.5,:m21=>0.5,:=>0.25]
times = collect(0.0:0.1:2.0)
ode = solve(ODEProblem(two_group,u0,(0.0,2.0),p), Tsit5(); saveat=times)

trajectories = Matrix{Float64}(undef, length(times), 100)
for replicate in axes(trajectories, 2)
    Random.seed!(20260700 + replicate)
    sol = solve(JumpProblem(two_group,u0,(0.0,2.0),p); saveat=times)(times)
    trajectories[:, replicate] = sol[:I1] .+ sol[:I2]
end
mean_infected = vec(mean(trajectories; dims=2))
lower = [quantile(trajectories[i, :], 0.05) for i in axes(trajectories, 1)]
upper = [quantile(trajectories[i, :], 0.95) for i in axes(trajectories, 1)]

fig = Figure(size=(800,350))
ax = Axis(fig[1,1], xlabel="time", ylabel="aggregate infectious")
band!(ax, times, lower, upper; color=:steelblue, 0.25)
lines!(ax, times, mean_infected; label="jump mean", color=:steelblue)
lines!(ax, times, ode[:I1] .+ ode[:I2]; label="ODE", color=:black)
axislegend(ax)
fig

```



The band is a pointwise 90% interval, not a simultaneous confidence band. Increasing N makes the diffusion approximation relatively more plausible, while small counts make discreteness and extinction important. The SDE is a Gaussian approximation and need not remain inside the jump interval or remain nonnegative without additional modeling choices.

References

- Anderson, Roy M., and Robert M. May. 1991. *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press.
- Bosch, Nathanael. 2024. “ProbNumDiffEq.jl: Probabilistic Numerical Solvers for Ordinary Differential Equations in Julia.” *Journal of Open Source Software* 9 (101): 7048. <https://doi.org/10.21105/joss.07048>.
- Britton, Tom, and Lasse Leskelä. 2022. “Optimal Intervention Strategies for Minimising the Epidemic Final Size.” *arXiv*. <https://arxiv.org/abs/2202.07780>.
- Datseris, George, Ali R. Vahdati, and Timothy C. DuBois. 2022. “Agents.jl: A Performant and Feature-Full Agent-Based Modeling Software of Minimal Code Complexity.” *Simulation* 98 (5): 415–30. <https://doi.org/10.1177/00375497211068820>.
- Dowson, Oscar, and Lea Kapelevich. 2021. “SDDP.jl: A Julia Package for Stochastic Dual Dynamic Programming.” *INFORMS Journal on Computing* 33 (1): 27–33. <https://doi.org/10.1287/ijoc.2020.0987>.
- Gelman, Andrew, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis*. 3rd ed. Chapman; Hall/CRC.
- Gillespie, Daniel T. 1977. “Exact Stochastic Simulation of Coupled Chemical Reactions.” *The Journal of Physical Chemistry* 81 (25): 2340–61. <https://doi.org/10.1021/j100540a008>.
- Jacquez, John A., Carl P. Simon, James Koopman, Lisa Sattenspiel, and Theodore Perry. 1988. “Modeling and Analyzing HIV Transmission: The Effect of Contact Patterns.” *Mathematical Biosciences* 92 (2): 119–99. [https://doi.org/10.1016/0025-5564\(88\)90031-4](https://doi.org/10.1016/0025-5564(88)90031-4).
- Kermack, William Ogilvy, and Anderson G. McKendrick. 1927. “A Contribution to the Mathematical Theory of Epidemics.” *Proceedings of the Royal Society of London. Series A* 115 (772): 700–721. <https://doi.org/10.1098/rspa.1927.0118>.
- Munsky, Brian, and Mustafa Khammash. 2006. “The Finite State Projection Algorithm for the Solution of the Chemical Master Equation.” *The Journal of Chemical Physics* 124: 044104. <https://doi.org/10.1063/1.2145882>.

- Rackauckas, Christopher et al. 2020. “Universal Differential Equations for Scientific Machine Learning.” *arXiv*. <https://arxiv.org/abs/2001.04385>.
- Raggett, G. F. 1982. “A Stochastic Model of the Eyam Plague.” *Journal of Applied Statistics* 9 (2): 212–26. <https://doi.org/10.1080/02664768200000021>.
- Venzon, D. J., and S. H. Moolgavkar. 1988. “A Method for Computing Profile-Likelihood-Based Confidence Intervals.” *Applied Statistics* 37 (1): 87–94. <https://doi.org/10.2307/2347496>.
- Whittles, Lilith K., and Xavier Didelot. 2016. “Epidemiological Analysis of the Eyam Plague Outbreak of 1665–1666.” *Proceedings of the Royal Society B* 283 (1830): 20160618. <https://doi.org/10.1098/rspb.2016.0618>.

Julia essentials for modelers

Why this unit matters

- ▶ The course reuses one object throughout: the SIR model of Kermack & McKendrick (1927).
- ▶ Before solving it we need a precise slice of Julia:
 - ▶ **in-place kernels** $f(du, u, p, t)$ — the SciML calling convention;
 - ▶ **broadcasting** and comprehensions to evaluate models on a grid;
 - ▶ **structs / named tuples** and **multiple dispatch** to organise parameters;
 - ▶ **reproducible randomness** with StableRNGs.
- ▶ Goal: the vocabulary, not a language tour.

The running model

Deterministic, continuous-time, population SIR with force of infection $\lambda = \beta c I / N$:

$$\frac{dS}{dt} = -\lambda S, \quad \frac{dI}{dt} = \lambda S - \gamma I, \quad \frac{dR}{dt} = \gamma I. \quad (1)$$

The three right-hand sides sum to zero, so $S + I + R = N$ is conserved. Rate $\rightarrow$ per-step probability for later discretisation: $p(r, \delta t) = 1 - e^{-r \delta t}$.

Julia API: an in-place kernel

$u = [S, I, R]$ maps onto Equation 1; p carries (β, c, γ, N) .

```
function sir_ode!(du, u, p, t)
    S, I, R = u
    , c, , N = p
    = * c * I / N           # force of infection
    du[1] = - * S           # dS/dt
    du[2] = * S - * I       # dI/dt
    du[3] = * I              # dR/dt
    return nothing
end

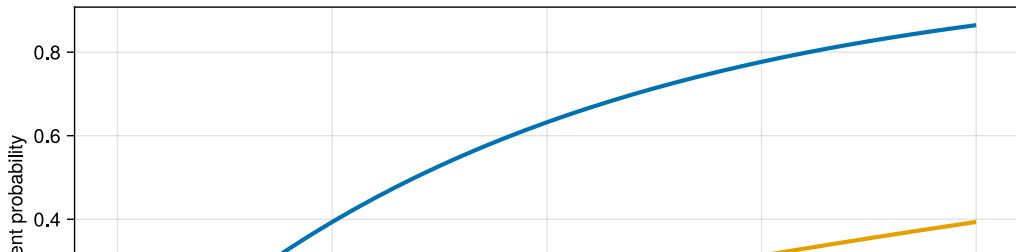
u0 = [990.0, 10.0, 0.0]
p = ( = 0.05, c = 10.0, = 0.25, N = 1000.0)
du = zeros(3); sir_ode!(du, u0, values(p), 0.0); du
```

3-element Vector{Float64}:

-4.95

Broadcasting & a figure from code

```
using CairoMakie
rate_to_proportion(r, t) = -expm1(-r * t)  # stable 1 - exp(-r * t)
ts = 0.0:0.05:2.0
fig = Figure(size = (760, 360))
ax = Axis(fig[1, 1]; xlabel = "t", ylabel = "event probability")
lines!(ax, ts, rate_to_proportion.(1.0, ts); linewidth = 3, label = "r = 1.0")
lines!(ax, ts, rate_to_proportion.(0.25, ts); linewidth = 3, label = "r = 0.25")
axislegend(ax; position = :rb); fig
```



Dispatch & reproducible randomness

```
using StableRNGs
struct SIRParameters; ::Float64; c::Float64; ::Float64; N::Float64; end
foi(I, p::NamedTuple)      = p. * p.c * I / p.N
foi(I, p::SIRParameters) = p. * p.c * I / p.N

params = SIRParameters(0.05, 10.0, 0.25, 1000.0)
rng = StableRNG(2024)                # local, version-stable stream
(foi(10.0, p), foi(10.0, params), rand(rng, 3))
```

(0.005, 0.005, [0.8177021060277301, 0.3804323567786363, 0.5425813266814807])

One function name, two methods — the mechanism SciML relies on everywhere.

Validation

The derivatives must sum to zero (conservation) and `rate_to_proportion` must stay a valid probability and match its Taylor limit $p \approx r \delta t$:

```
sir_ode!(du, u0, values(p), 0.0)
conserved = isapprox(sum(du), 0.0; atol = 1e-12)
taylor     = isapprox(rate_to_proportion(0.25, 1e-6), 0.25e-6; rtol = 1e-3)
(conserved, taylor)
```

(true, true)

Both true: the kernel conserves population and the conversion is well behaved.

Limitations & takeaways

- ▶ **Scope:** this is not a full Julia course — no package management beyond using, no profiling, metaprogramming, or parametric types.
- ▶ **Takeaways:**
 - ▶ write mutating kernels with the `f(du, u, p, t)` signature;
 - ▶ broadcast with `.` and fuse operations; use `expm1` for stability;
 - ▶ group parameters in typed containers and dispatch on them;
 - ▶ seed a *local* StableRNG for bit-reproducible figures.
- ▶ Next: we actually *solve* the model in **Foundations**.

Learn more

- ▶ Book chapter: Julia essentials for modelers
- ▶ Pluto notebook: 01-julia-essentials.jl
- ▶ Julia manual · CairoMakie

Probability and stochastic processes

Why this unit matters

- ▶ Epidemics in finite populations are **random**: whether a contact transmits, or an individual recovers today, are chance events.
- ▶ We build the probability the course needs — Bernoulli/Binomial, mean and variance, the **Markov property** — and apply it to a stochastic SIR chain.
- ▶ Deterministic ODEs describe the *mean*; here we capture the *fluctuations*.

Equations

A sum of n independent Bernoulli(p) trials is Binomial(n, p):

$$\Pr(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad \mathbb{E}[X] = np, \quad \text{Var}[X] = np(1 - p). \quad (1)$$

Per-step probabilities from the SIR rates, then a Markov update:

$$p_I = 1 - e^{-\beta c I_t \delta t / N}, \quad p_R = 1 - e^{-\gamma \delta t}, \quad (2)$$

$$Y_t \sim \text{Bin}(S_t, p_I), \quad Z_t \sim \text{Bin}(I_t, p_R), \quad S_{t+1} = S_t - Y_t, \quad I_{t+1} = I_t + Y_t - Z_t, \quad R_{t+1} = R_t + Z_t \quad (3)$$

Only current counts enter — the **Markov property** — and N is conserved.

Julia API: the Markov kernel

```
using Distributions, StableRNGs

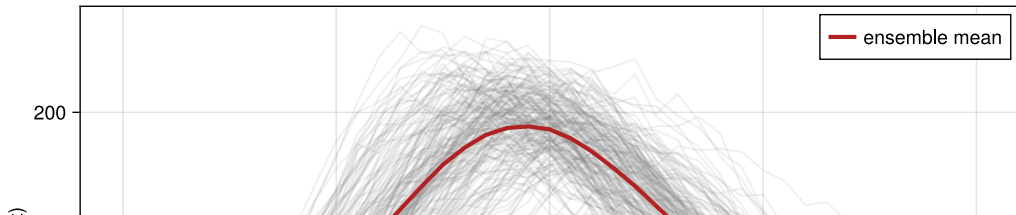
rate_to_proportion(r, t) = -expm1(-r * t)

function chain_step(state, p, t, rng)
    S, I, R = state; , c, , N = p
    Y = rand(rng, Binomial(S, rate_to_proportion( * c * I / N, t))) # in
    Z = rand(rng, Binomial(I, rate_to_proportion( , t))) # rec
    return (S - Y, I + Y - Z, R + Z)
end

function simulate(u0, p, t, nsteps, rng)
    traj = Matrix{Int}(undef, 3, nsteps + 1); traj[:, 1] .= u0
    s = Tuple(u0)
    for k in 1:nsteps; s = chain_step(s, p, t, rng); traj[:, k+1] .= s; end
    traj
end
```

Generated result: an ensemble

```
using CairoMakie
u0 = [990, 10, 0]; p = (0.05, 10.0, 0.25, 1000); t = 1.0; nsteps = 40; nsims = 100
ens = [simulate(u0, p, t, nsteps, StableRNG(seed)) for seed in 1:nsims]
Ipaths = reduce(hcat, (t[2, :] for t in ens)); meanI = vec(sum(Ipaths; dims=2))
ts = 0:nsteps
fig = Figure(size = (760, 380)); ax = Axis(fig[1,1]; xlabel = "day", ylabel = "I")
for j in 1:nsims; lines!(ax, ts, Ipaths[:, j]; color = (:grey, 0.12)); end
lines!(ax, ts, meanI; color = :firebrick, linewidth = 3, label = "ensemble mean")
axislegend(ax; position = :rt); fig
```



Validation

Sample mean of a Binomial draw $\rightarrow np$; every realisation conserves N :

```
rng = StableRNG(7); n, prob = 990, 0.02
samples = [rand(rng, Binomial(n, prob)) for _ in 1:100_000]
mean_ok = isapprox(sum(samples) / length(samples), n * prob; rtol = 0.02)
cons_ok = all(t -> all(k -> sum(t[:, k]) == 1000, 1:nsteps+1), ens)
(mean_ok, cons_ok)
```

(true, true)

Both true: the empirical mean matches np and all 200×41 totals equal N .

Limitations & takeaways

- ▶ The binomial chain captures **demographic stochasticity** the ODE ignores.
- ▶ It is exact only as $\delta t \rightarrow 0$: infections and recoveries within a step are treated in separate binomials and assumed independent.
- ▶ Event-by-event exactness $\rightarrow$ the **Gillespie** algorithm (jump-process chapter).
- ▶ **Takeaways:** Binomial = sum of Bernoullis; convert rates to per-step probabilities; seed local RNGs; average an ensemble to recover the mean.

Learn more

- ▶ Book chapter: Probability and stochastic processes
- ▶ Pluto notebook: `02-probability-and-stochastic-processes.jl`
- ▶ `Distributions.jl`

Calculus and numerical methods

Why this unit matters

- ▶ Differential equations change instant-to-instant, but a computer takes **finite steps**.
- ▶ We review the derivative as a rate, build the **forward Euler** integrator, and analyse its error — local vs global.
- ▶ These ideas justify the adaptive solvers used from the ODE chapter onward.

Equations

A first-order Taylor expansion,

$$u(t + \delta t) = u(t) + \delta t u'(t) + \frac{1}{2} \delta t^2 u''(\xi), \quad (1)$$

dropping the remainder gives the **forward Euler** update

$$u_{n+1} = u_n + \delta t f(u_n). \quad (2)$$

Local truncation error is $O(\delta t^2)$; over $T/\delta t$ steps the **global** error is one order lower, $O(\delta t)$ — slope 1 on log-log axes.

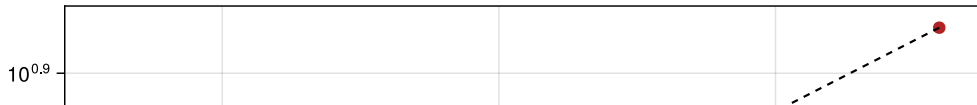
Julia API: the integrator

```
function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p
    = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; return nothing
end

function euler(f, u0, p, t, tmax)
    ts = 0.0:t:tmax
    us = Matrix{Float64}(undef, length(u0), length(ts)); us[:, 1] .= u0
    du = similar(u0)
    for n in 1:length(ts)-1
        f(du, @view(us[:, n]), p, ts[n])
        us[:, n+1] .= us[:, n] .+ t .* du # u_{n+1} = u_n + t f(u_n)
    end
    collect(ts), us
end
```

Generated result: convergence

```
using CairoMakie
t_ref = tmax / 2^16; _, us_ref = euler(sir_ode!, u0, p, t_ref, tmax); u_r
ts = [tmax / 2^k for k in 6:11]
errs = map(ts) do t
    _, u = euler(sir_ode!, u0, p, t, tmax); maximum(abs.(u[:, end] .- u_re
end
fig = Figure(size = (720, 400))
ax = Axis(fig[1,1]; xscale = log10, yscale = log10,
    xlabel = "t", ylabel = "global error at t = 40")
scatter!(ax, ts, errs; markersize = 12, color = :firebrick, label = "measu
lines!(ax, ts, errs[1] .* (ts ./ ts[1]); linestyle = :dash, color = :black
axislegend(ax; position = :rb); fig
```



Validation

Empirical order (successive halvings) should be 1, and Euler preserves the conservation law at any step:

```
orders = [log2(errs[k] / errs[k+1]) for k in 1:length(errs)-1]
mean_order = sum(orders) / length(orders)
_, uc = euler(sir_ode!, u0, p, 0.5, tmax)
cons_ok = all(t -> isapprox(t, 1000.0; atol = 1e-9), vec(sum(uc; dims = 1)))
(round(mean_order; digits = 3), isapprox(mean_order, 1.0; atol = 0.1), cons_ok)
```

(1.019, true, true)

Limitations & takeaways

- ▶ Accuracy improves only **linearly** — halving the error doubles the cost.
- ▶ Forward Euler is only **conditionally stable**: too-large steps oscillate or diverge on stiff problems.
- ▶ Adaptive higher-order methods (Tsit5, ...) get far more accuracy per unit work.
- ▶ **Takeaway**: understand order and stability, then let SciML choose the step.

Learn more

- ▶ Book chapter: Calculus and numerical methods
- ▶ Pluto notebook: `03-calculus-and-numerical-methods.jl`
- ▶ OrdinaryDiffEq solvers

Foundations: SIR theory and the Julia workflow

Why this unit matters

- ▶ The conceptual anchor of the course: derive the SIR model of Kermack & McKendrick (1927) from first principles.
- ▶ Read off the two quantities that govern every epidemic: the reproduction number R_0 and the epidemic threshold.
- ▶ Solve it with the modern **SciML** stack and verify conservation, the threshold, and the analytic **final-size** relation.

Assumptions & equations

Closed, well-mixed population; mass-action transmission; constant recovery; permanent immunity. Force of infection $\lambda = \beta c I / N$:

$$\frac{dS}{dt} = -\lambda S, \quad \frac{dI}{dt} = \lambda S - \gamma I, \quad \frac{dR}{dt} = \gamma I. \quad (1)$$

$$R_0 = \frac{\beta c}{\gamma}, \quad \frac{dI}{dt} > 0 \iff \frac{S}{N} > \frac{1}{R_0}, \quad (2)$$

$$s_\infty = s_0 \exp(-R_0(1 - s_\infty)), \quad s_0 = S(0)/N. \quad (3)$$

Julia API: solve & analyse

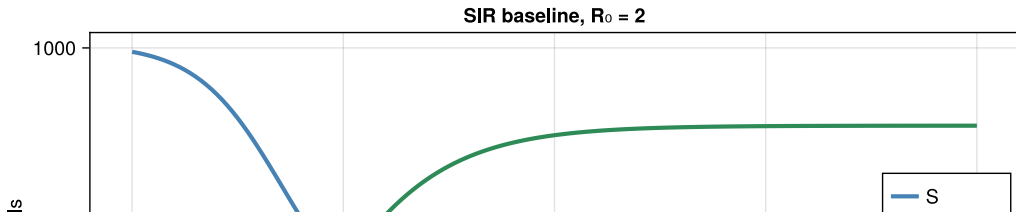
```
using OrdinaryDiffEq, NonlinearSolve
function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p
    = * c * I / N
    du[1] = - * S; du[2] = * S - * I; du[3] = * I; return nothing
end

u0 = [990.0, 10.0, 0.0]; p = (0.05, 10.0, 0.25, 1000.0); tspan = (0.0, 80.0)
sol = solve(ODEProblem(sir_ode!, u0, tspan, p), Tsit5(); saveat = 0.5)
, c, , N = p; R0 = * c / ; s_inf = u0[1] / N
s_inf = solve(NonlinearProblem((s, R0) -> s - s * exp(-R0 * (1 - s)), 0.2,
    NewtonRaphson()).u
(R, round(s_inf; digits = 3), round(1 - s_inf; digits = 3)) # R0, s_inf, at

(2.0, 0.2, 0.8)
```

Generated result

```
using CairoMakie
t = sol.t; S = sol[1, :]; I = sol[2, :]; R = sol[3, :]
fig = Figure(size = (760, 400))
ax = Axis(fig[1,1]; xlabel = "day", ylabel = "individuals", title = "SIR b
lines!(ax, t, S; color = :steelblue, linewidth = 3, label = "S")
lines!(ax, t, I; color = :firebrick, linewidth = 3, label = "I")
lines!(ax, t, R; color = :seagreen, linewidth = 3, label = "R")
hlines!(ax, [N / R]; color = :black, linestyle = :dash, label = "S = N/R ")
axislegend(ax; position = :rc); fig
```



Validation

Population conserved; infectious peak sits on the threshold; long-run S/N matches the analytic s_∞ :

```
cons = all(u -> isapprox(sum(u), N; atol = 1e-6), sol.u)
peak = isapprox(S[argmax(I)] / N, 1 / R; atol = 0.01)
sol_long = solve(remake(ODEProblem(sir_ode!, u0, (0.0, 400.0), p)), Tsit5(
    reltol = 1e-10, abstol = 1e-10, save_everystep = false)
fsize = isapprox(sol_long.u[end][1] / N, s_inf; atol = 1e-3)
(cons, peak, fsize)
```

(true, true, true)

All true: conservation, threshold, and final size are confirmed.

Limitations & roadmap

- ▶ The deterministic ODE is the population **mean** — no extinction, super-spreading, or small-outbreak noise.
- ▶ Homogeneous mixing ignores age, space, contact structure; exponential infectious periods are a convenience.
- ▶ The book relaxes each assumption in turn: discrete-time & stochastic → jump processes → SDEs → agent-based → observation, inference, control.

Learn more

- ▶ Book chapter: Foundations
- ▶ Pluto notebook: `01-foundations.jl`
- ▶ OrdinaryDiffEq · NonlinearSolve

Discrete-time models

Why this unit matters

- ▶ Epidemic data arrive in discrete intervals; many simulations advance in fixed steps.
- ▶ Two flavours of discrete-time SIR:
 - ▶ a **deterministic function map** (continuous state);
 - ▶ a **stochastic Markov chain** (integer counts).
- ▶ As $\delta t \rightarrow 0$ the map converges to the continuous ODE — tying discrete and continuous worlds together.

Equations

Per-step probabilities (Equation 1) from the SIR rates; the map moves expectations, the chain draws binomials:

$$p_I = 1 - e^{-\beta c I_t \delta t / N}, \quad p_R = 1 - e^{-\gamma \delta t}. \quad (1)$$

$$\text{map: } S_{t+1} = S_t - p_I S_t, \quad I_{t+1} = I_t + p_I S_t - p_R I_t, \quad R_{t+1} = R_t + p_R I_t. \quad (2)$$

$$\text{chain: } Y_t \sim \text{Bin}(S_t, p_I), \quad Z_t \sim \text{Bin}(I_t, p_R). \quad (3)$$

The map is the mean-field expectation of the chain.

Julia API: two kernels side by side

```
using Distributions, StableRNGs

rate_to_proportion(r, t) = -expm1(-r * t)

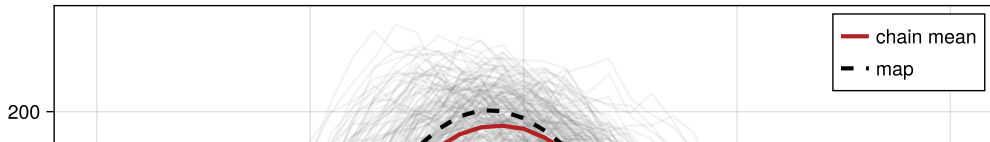
function map_step(state, p, t)                                # deterministic
    S, I, R = state; , c, , N = p
    inf = rate_to_proportion( * c * I / N, t) * S
    rec = rate_to_proportion( , t) * I
    (S - inf, I + inf - rec, R + rec)
end

function chain_step(state, p, t, rng)                        # stochastic
    S, I, R = state; , c, , N = p
    Y = rand(rng, Binomial(round{Int}, S), rate_to_proportion( * c * I / N, t))
    Z = rand(rng, Binomial(round{Int}, I), rate_to_proportion( , t))
    (S - Y, I + Y - Z, R + Z)
end

function iterate_model(step, u0, p, t, n; extra = ())
    traj = Matrix{Float64}(undef, 3, n + 1); traj[:, 1] .= u0; s = Tuple{f
```

Generated result: map vs chain

```
using CairoMakie
u0 = [990, 10, 0]; p = (0.05, 10.0, 0.25, 1000.0); t = 1.0; n = 40; ts = 0:n-1
mapt = iterate_model((s, p, dt) -> map_step(s, p, dt), u0, p, t, n)
nsims = 200
ens = [iterate_model(chain_step, u0, p, t, n; extra = (StableRNG(seed),))
      for seed in 1:nsims]; chainI = reduce(hcat, (t[2, :] for t in ens)); meanI = vec(sum(chainI; dims=2))
fig = Figure(size = (760, 380)); ax = Axis(fig[1,1]; xlabel = "day", ylabel = "population")
for j in 1:nsims; lines!(ax, ts, chainI[:, j]; color = (:grey, 0.1)); end
lines!(ax, ts, meanI; color = :firebrick, linewidth = 3, label = "chain mean")
lines!(ax, ts, mapt[2, :]; color = :black, linestyle = :dash, linewidth = 3, label = "map")
axislegend(ax; position = :rt); fig
```



Validation: convergence to the ODE

```
using OrdinaryDiffEq
function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p;  =  * c * I / N
    du[1] = - * S; du[2] =  * S -  * I; du[3] =  * I; return nothing
end
ref = solve(ODEProblem(sir_ode!, float.(u0), (0.0, 40.0), p), Tsit5();
            reltol = 1e-12, abstol = 1e-12, save_everystep = false).u[end]
ts = [40.0 / 2^k for k in 5:12]
errs = map(ts) do dt
    m = iterate_model((s, p, d) -> map_step(s, p, d), u0, p, dt, round{Int}
    maximum(abs.(m[:, end] .- ref))
end
order = sum(log2(errs[k] / errs[k+1]) for k in 1:length(errs)-1) / (length
cons = all(k -> isapprox(sum(mapt[:, k]), 1000.0; atol = 1e-8), 1:n+1)
(round(order; digits = 3), isapprox(order, 1.0; atol = 0.1), cons)
```

(0.96, true, true)

Limitations & takeaways

- ▶ The **chain** is most faithful: integer counts, chance extinction, run-to-run variability shrinking with population size.
- ▶ The **map** is its cheap, smooth mean-field shadow — blind to fluctuations.
- ▶ Both are exact only as $\delta t \rightarrow 0$; a within-step artefact (no infect *and* recover in one step) vanishes as the step shrinks.
- ▶ **Takeaway:** discrete models bridge data cadence and continuous theory.

Learn more

- ▶ Book chapter: Discrete-time models
- ▶ Pluto notebook: `02-discrete-time-models.jl`
- ▶ `Distributions.jl`

Ordinary differential equations

Why this unit matters

- ▶ Foundations solved SIR to establish theory; this chapter is about the **tools** — the OrdinaryDiffEq workflow.
- ▶ We cover: building an ODEProblem, the Tsit5 solver, dense interpolation `sol(t)`, and **callbacks** for events.
- ▶ Two events: a time-limited **lockdown** and automatic **peak detection**.

Equations

Base SIR with a time-dependent, piecewise-constant $\beta(t)$:

$$\lambda(t) = \frac{\beta(t) c I}{N}, \quad \beta(t) = \begin{cases} \beta_{\text{lock}}, & t_1 \leq t < t_2, \\ \beta, & \text{else.} \end{cases} \quad (1)$$

The infectious peak is the root of the growth rate

$$\frac{dI}{dt} = \lambda(t)S - \gamma I = 0, \quad (2)$$

which a `ContinuousCallback` locates to solver tolerance.

Julia API: problem, solver, interpolation

```
using OrdinaryDiffEq
function sir_ode!(du, u, p, t)
    S, I, R = u; , c, , N = p;  =  * c * I / N
    du[1] = - * S; du[2] =  * S -  * I; du[3] =  * I; return nothing
end
u0 = [990.0, 10.0, 0.0]; p = (0.05, 10.0, 0.25, 1000.0); tspan = (0.0, 80.0)
sol = solve(ODEProblem(sir_ode!, u0, tspan, p), Tsit5(); reltol = 1e-8, abstol = 1e-6)
(length(sol.t), round(sol(12.5)[2]; digits = 3)) # adaptive steps; dense
```

(133, 120.79)

Julia API: a lockdown & peak detection

```
using DiffEqCallbacks
t1, t2 = 10.0, 30.0; _lock = 0.025
lockdown!(integ) = (integ.p[1] = integ.t < t2 ? _lock : 0.05)
cb = PresetTimeCallback([t1, t2], lockdown!)
sol_lock = solve(ODEProblem(sir_ode!, u0, tspan, [0.05, 10.0, 0.25, 1000.0]),
                  Tsit5(); callback = cb, reltol = 1e-8, abstol = 1e-8)

peak_t = Float64[]
dIdt(u, p) = p[1] * p[2] * u[2] / p[4] * u[1] - p[3] * u[2]      # eq-s-ch0
peak_cb = ContinuousCallback((u, t, i) -> dIdt(u, i.p), i -> push!(peak_t,
solve(ODEProblem(sir_ode!, u0, tspan, collect(p)), Tsit5();
      callback = peak_cb, reltol = 1e-8, abstol = 1e-8)
(round.(peak_t; digits = 2), round(maximum(sol[2, :]); digits = 1),
 round(maximum(sol_lock[2, :]); digits = 1))
```

([17.52], 158.3, 86.0)

Generated result

```
using CairoMakie
tt = 0.0:0.25:80.0
fig = Figure(size = (760, 400))
ax = Axis(fig[1,1]; xlabel = "day", ylabel = "I(t)", title = "SIR with a lockdown")
vspan!(ax, t1, t2; color = (:grey, 0.15))
lines!(ax, tt, [sol(t)[2] for t in tt]; color = :firebrick, linewidth = 2)
lines!(ax, tt, [sol_lock(t)[2] for t in tt]; color = :steelblue, linewidth = 2)
vlines!(ax, peak_t; color = :black, linestyle = :dot, label = "detected peak")
axislegend(ax; position = :rt); fig
```



Validation

Conservation, interpolation self-consistency, and event-detected peak vs a brute-force scan:

```
cons = all(u -> isapprox(sum(u), 1000.0; atol = 1e-6), sol.u)
interp = isapprox(sol(sol.t[10]), sol.u[10]; atol = 1e-8)
dense_t = 0.0:0.01:80.0
brute = dense_t[argmax([sol(t)[2] for t in dense_t])]
peak_ok = isapprox(peak_t[1], brute; atol = 0.05)
(cons, interp, peak_ok)
```

(true, true, true)

All true: the solver conserves population, its interpolant is faithful, and the callback peak matches the maximum.

Limitations & takeaways

- ▶ The workflow separates **what** the model is from **how** it is solved and **what happens during** the solve — the same `ODEProblem` scales to stiff solvers, sensitivity, and fitting.
- ▶ Callbacks turn an integrator into an event-driven simulator.
- ▶ Limits are those of deterministic SIR; the lockdown here is a perfect switch.
- ▶ **Takeaway:** master `ODEProblem` + `Tsit5` + callbacks once, reuse everywhere.

Learn more

- ▶ Book chapter: Ordinary differential equations
- ▶ Pluto notebook: `03-ordinary-differential-equations.jl`
- ▶ OrdinaryDiffEq · Callbacks

Continuous-time Markov chains and jump processes

Why events matter

- ▶ A CTMC waits for the next infection or recovery.
- ▶ The state is integer-valued: $X = (S, I, R)$.
- ▶ Exact event timing exposes fade-out and peak variability.

For the baseline population,

$$a_{\text{inf}} = \frac{\beta c S I}{N}, \quad a_{\text{rec}} = \gamma I.$$

Gillespie direct algorithm

At state x , let $a_0 = a_{\text{inf}} + a_{\text{rec}}$.

1. Draw $\tau \sim \text{Exponential}(a_0)$.
2. Choose infection with probability a_{inf}/a_0 .
3. Apply $(-1, +1, 0)$ or $(0, -1, +1)$.

This is a stochastic, continuous-time, discrete-state, population-level model. Every event conserves $S + I + R = N$.

A visible Julia kernel

```
gillespie_sir (generic function with 1 method)
```

Current-code result

```
(events = 1426, peak_I = 138, mass_error = 0)
```

The returned result is generated at render time. `mass_error == 0` is a structural validation; `peak_I` and the event count are realization-specific.

From model to package API

The same reaction structure can be represented with current SciML packages:

```
using Catalyst, JumpProcesses
sir_network = @reaction_network begin
    , S + I --> 2I
    , I --> R
end
jump_problem = JumpProblem(
    sir_network, [:S=>990, :I=>10, :R=>0], (0.0, 40.0),
    [:=>0.05*10/1000, :=>0.25];
    aggregator=Direct(), rng=StableRNG(20250711))
jump_solution = solve(jump_problem, SSAS stepper(); saveat=0.2)
```

The network records species, propensities, and stoichiometry explicitly.

Takeaways and links

- ▶ Propensities determine both waiting times and event probabilities.
- ▶ Exact jumps preserve integer states and population conservation.
- ▶ Ensemble summaries are needed for comparison with an ODE.

Further reading:

- ▶ [JumpProcesses direct stochastic tutorial](#)
- ▶ [Catalyst documentation](#)
- ▶ [Book chapter](#) · [Pluto notebook](#)

Stochastic differential equations

Why a diffusion approximation?

- ▶ A jump model resolves every reaction.
- ▶ An SDE replaces Poisson reaction counts by Gaussian increments.
- ▶ The state becomes continuous-valued, so speed is gained at the boundary of exact integer bookkeeping.

For stoichiometry V and propensity vector $a(X)$,

$$dX = V a(X) dt + V \operatorname{diag}(\sqrt{a(X)}) dW.$$

Reaction-level noise

For infection and recovery,

$$V = \begin{pmatrix} -1 & 0 \\ 1 & -1 \\ 0 & 1 \end{pmatrix}, \quad a = (\kappa SI, \gamma I)^\top.$$

Thus the diffusion columns are

$$G_{.1} = (-\sqrt{a_1}, \sqrt{a_1}, 0)^\top, \quad G_{.2} = (0, -\sqrt{a_2}, \sqrt{a_2})^\top.$$

Each column sums to zero: the reaction-level noise preserves total population.

Current Julia API

retcode: Success

Interpolation: 1st order linear

t: 301-element Vector{Float64}:

0.0

0.1

0.2

0.3

0.4

0.5

0.6

0.7

0.8

0.9

29.2

29.3

29.4

Current-code result

`(2.7284841053187847e-12, -0.06882763332898889)`

The displayed values are generated by the solver. Population conservation is checked independently of the random realization; `minimum_state` diagnoses possible boundary crossing.

Validation and limitations

- ▶ Refine dt and record conservation error and minimum state.
- ▶ Compare an ensemble of SDE paths with the ODE drift.
- ▶ Use the CTMC when integer states, extinction, or small populations matter.

The SDE is not an exact replacement for the jump process. It is a controlled approximation whose assumptions should be reported.

Takeaways and links

- ▶ Stoichiometry determines both drift and covariance.
- ▶ Non-diagonal noise is essential for reaction-level SIR fluctuations.
- ▶ Conservation can hold while nonnegativity fails.

Further reading:

- ▶ StochasticDiffEq SDE tutorial
- ▶ Chemical Langevin construction
- ▶ Book chapter · Pluto notebook

Agent-based models

Why model individuals?

- ▶ An ABM stores who is susceptible, infectious, or recovered.
- ▶ Contact rules can be changed without deriving a new aggregate equation.
- ▶ Aggregation is useful for comparison, but hides contact identities.

This unit uses a stochastic, discrete-time, discrete-state, individual-level model with $\Delta t = 0.1$, $c = 10$, $\beta = 0.05$, and $\gamma = 0.25$.

Individual transition probabilities

For $K_i \sim \text{Poisson}(c\Delta t)$ contacts,

$$\Pr(S \rightarrow I \mid I_t) = 1 - \exp[-c\Delta t \beta I_t/N].$$

Recovery uses

$$\Pr(I \rightarrow R) = 1 - \exp(-\gamma\Delta t).$$

For small Δt , the aggregate limit is

$$\dot{S} = -\beta c S I/N, \quad \dot{I} = \beta c S I/N - \gamma I, \quad \dot{R} = \gamma I.$$

Agents.jl 7 model logic

`agent_step!` (generic function with 1 method)

The second field makes the update synchronous.

Current-code result

```
(peak_I = 49, population_error = 0)
```

The result is generated from the current run! API. Conservation should be exact even though the infectious curve is random.

Extensions and checks

- ▶ Use `GridSpace` and `nearby_agents` for local contacts.
- ▶ Run seeded replicates and compare the mean with the ODE.
- ▶ Check timestep sensitivity before interpreting a policy experiment.

ABMs can represent clustering, households, behavior, and networks, but those mechanisms introduce parameters that may be difficult to observe or identify.

Takeaways and links

- ▶ Agent state makes contact and scheduling assumptions explicit.
- ▶ Synchronous updates protect the intended discrete-time semantics.
- ▶ Aggregate validation must be paired with checks of the individual rule.

Further reading:

- ▶ Agents.jl v7 agent definitions
- ▶ Agents.jl v7 data collection
- ▶ Book chapter · Pluto notebook

Observation models and simulated data

Why this unit matters

- ▶ Every earlier chapter built a **process model**: rules that generate the latent epidemic trajectory.
- ▶ Surveillance data are never the latent process — they are **noisy, partial counts**.
- ▶ An **observation model** links the two, and is the raw material for the inference chapters that follow.

Process and observation, side by side

Baseline SIR with a cumulative-incidence state C

$(S_0 = 990, I_0 = 10, R_0 = 0, N = 1000, c = 10, \beta = 0.05, \gamma = 0.25)$:

$$\dot{S} = -\beta c \frac{SI}{N}, \quad \dot{I} = \beta c \frac{SI}{N} - \gamma I, \quad \dot{R} = \gamma I, \quad \dot{C} = \beta c \frac{SI}{N}.$$

	Process	Observation
Determinism	deterministic	stochastic
Time	continuous	discrete (daily)
State	densities S, I, R, C	counts $Y_t \in \mathbb{Z}_{\geq 0}$

Count models: Poisson vs negative-binomial

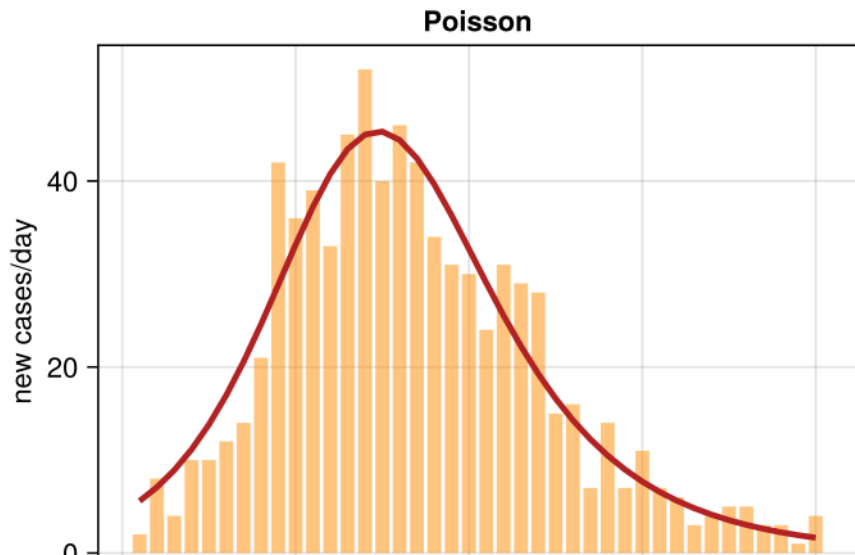
Latent incidence $x_t = C(t) - C(t - 1)$; reporting fraction ρ :

$$Y_t \mid x_t \sim \text{Poisson}(\rho x_t), \quad \mathbb{E}[Y_t] = \text{Var}[Y_t] = \rho x_t,$$

$$Y_t \mid x_t \sim \text{NegBin}\left(r, \frac{r}{r + \rho x_t}\right), \quad \text{Var}[Y_t] = \rho x_t + \frac{(\rho x_t)^2}{r}.$$

Poisson is **equidispersed**; the negative-binomial adds overdispersion and recovers Poisson as $r \rightarrow \infty$.

Simulating case reports



Validation and dispersion

(conservation_error = 5.5e-13, poisson_var_over_mean = 0.98, negbin_var_ov

- ▶ **Conservation:** $C(t) = S_0 - S(t)$ to solver tolerance.
- ▶ At the peak, **replicate** Poisson counts are equidispersed (var/mean ≈ 1); negative-binomial counts are overdispersed.

Julia API

```
using EpiModelingCourse    # sir_cumulative_ode!, synthetic_incidence
df = synthetic_incidence(; seed = 1, tspan = (0.0, 40.0),
                        observation_noise = :poisson, reporting_rate = 1.0)

using Distributions         # Poisson, NegativeBinomial
using StableRNGs           # StableRNG(seed) for reproducible streams
```

- ▶ OrdinaryDiffEq integrates the cumulative-incidence ODE.
- ▶ Distributions supplies the count models; StableRNG fixes the seed.

Takeaways and limitations

- ▶ Separate the **latent process** from the **observation process**.
- ▶ Poisson understates uncertainty when reporting is clustered; use the negative-binomial.
- ▶ ρ and β both scale the mean $\Rightarrow$ **not identifiable** from one incidence series (Chapter 10).
- ▶ We assumed instantaneous, conditionally independent reporting — real systems add delays and weekday effects.

Read the book chapter; see `Distributions.jl` and `StableRNGs.jl`.

Frequentist parameter estimation

Why this unit matters

- ▶ Chapter 7 produced case reports from a **known** process.
- ▶ Now reverse the arrow: given only the reports, what are the parameters?
- ▶ The **frequentist** view: parameters are fixed unknowns; uncertainty is the sampling behaviour of estimators.

Model and likelihood

Deterministic SIR ODE + Poisson observations, estimating $\theta = (i_0, \beta)$ with $c = 10, \gamma = 0.25$ fixed. Model incidence $x_t(\theta)$ gives the Poisson log-likelihood

$$\ell(\theta) = \sum_{t=1}^n [y_t \log x_t(\theta) - x_t(\theta) - \log(y_t!)].$$

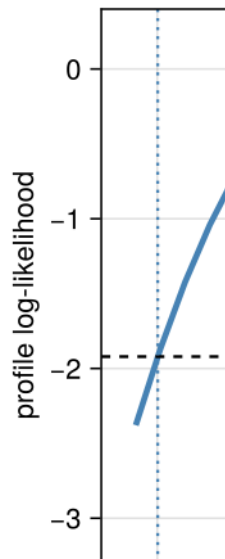
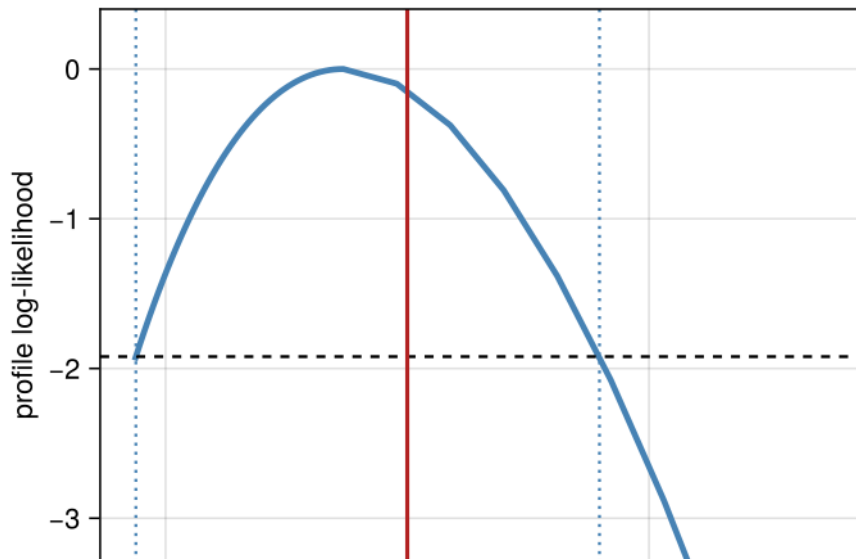
Profile out nuisance parameters and use Wilks' theorem for intervals:

$$\ell_p(\beta) = \max_{i_0} \ell(i_0, \beta), \quad 95\% \text{ CI : } \ell_p(\beta) \geq \ell(\hat{\theta}) - 1.92.$$

Fitting: MLE and profile intervals

	parameter	truth	mle	ci_lower	ci_upper
	String	Float64	Float64	Float64	Float64
1	i	0.01	0.0093	0.0072	0.012
2		0.05	0.05	0.048	0.0521

Profile log-likelihoods



Takeaways and limitations

- ▶ MLE via `Optimization.jl`; profile CIs via `ProfileLikelihood.jl`.
- ▶ Nelder–Mead is **local** — use multistart before trusting one fit.
- ▶ The χ^2 calibration is **asymptotic**; small samples may mis-cover.
- ▶ β and c enter only as $\beta c \Rightarrow$ not separately identifiable (Chapter 10).
- ▶ Intervals summarise sampling variability, not prior knowledge — Chapter 9 is the Bayesian counterpart.

Read the book chapter; see `Optimization.jl` and `ProfileLikelihood.jl`.

Bayesian inference

Why this unit matters

- ▶ Chapter 8: parameters are fixed unknowns, uncertainty is sampling variability.
- ▶ **Bayesian view:** parameters are random variables; a **prior** is updated into a **posterior** by the data.
- ▶ Same SIR model, same reports — different treatment of uncertainty, with Turing.jl.

Bayes' theorem for the SIR model

$$p(\theta \mid y_{1:n}) = \frac{p(y_{1:n} \mid \theta) p(\theta)}{p(y_{1:n})},$$

same Poisson likelihood as Chapter 8, with weakly informative priors on the natural $[0, 1]$ ranges:

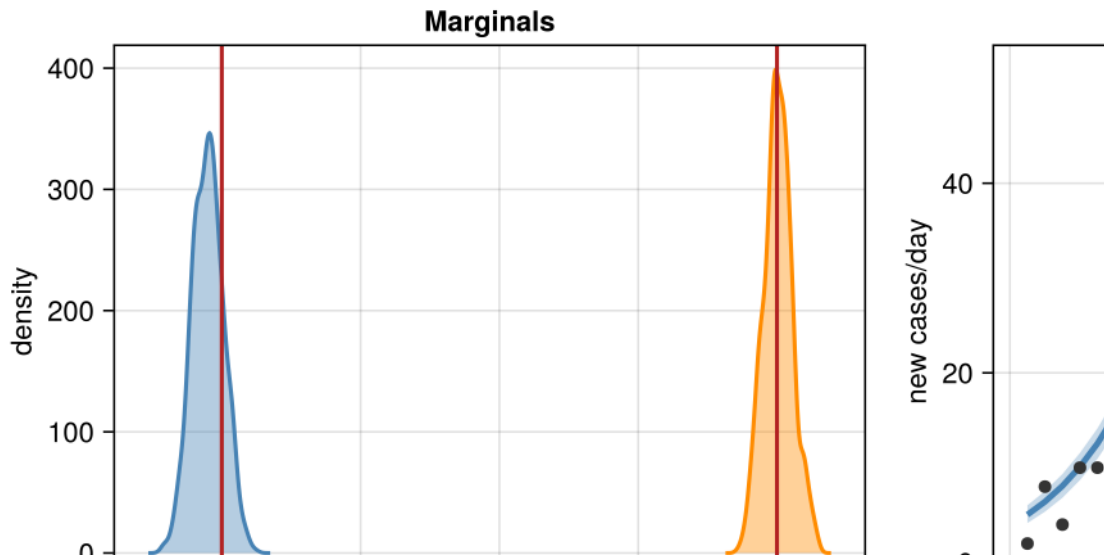
$$i_0 \sim \text{Beta}(1, 20), \quad \beta \sim \text{Beta}(1, 10).$$

The normalising constant is intractable; **NUTS** draws samples whose histogram approximates the posterior.

The probabilistic model and sampler

```
(max_rhat = 1.018, posterior_mean = (i = 0.009,   = 0.0501), truth = (i =
```

Posterior and predictive check



Takeaways and limitations

- ▶ `@model` + NUTS turns Bayes' theorem into executable sampling.
- ▶ Check convergence with $\hat{R} \approx 1$ and effective sample size.
- ▶ Priors are modelling choices; guard the ODE solve and floor the rate.
- ▶ NUTS re-solves + differentiates the ODE each step — costlier than Chapter 8.
- ▶ β - c non-identifiability persists; Poisson still assumes equidispersion.

Read the book chapter; see `Turing.jl` and `MCMCChains.jl`.

Sensitivity and identifiability

Why this unit matters

- ▶ **Sensitivity:** which inputs move a chosen output?
- ▶ **Identifiability:** which parameters can the observation process distinguish?
- ▶ A narrow optimizer interval is not evidence that every biological parameter is learned.

Baseline SIR model

Deterministic, continuous-time, continuous-state, population model:

$$\dot{S} = -\beta c SI/N, \quad \dot{I} = \beta c SI/N - \gamma I, \quad \dot{R} = \gamma I.$$

At the course baseline, $N = 1000$, $(S, I, R) = (990, 10, 0)$, $(\beta, c, \gamma) = (0.05, 10, 0.25)$, and $\mathcal{R}_0 = \beta c / \gamma = 2$.

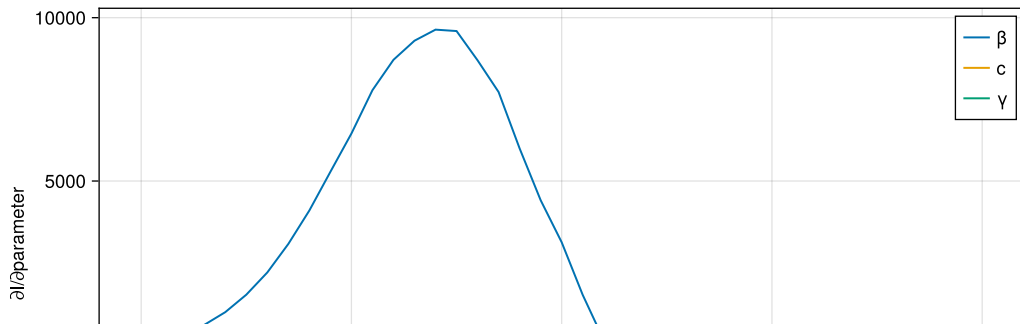
The state sensitivity matrix is $J_x(t) = \partial x(t) / \partial (\beta, c, \gamma)$.

Local sensitivity: a derivative, not an uncertainty interval

For a small perturbation,

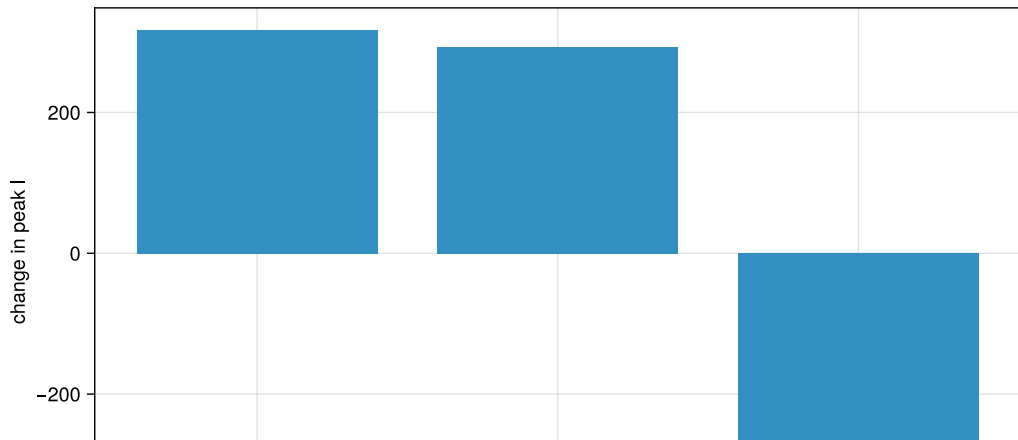
$$\Delta z \approx \nabla_{\theta} z \Delta \theta.$$

The following small solve uses `OrdinaryDiffEq` and centered perturbations to plot the infectious-state derivative at the baseline.



Global variation and the observation map

Global analysis varies a plausible parameter box and summarizes an output such as peak infectious prevalence. `GlobalSensitivity.gsa` can replace the small grid below with a Sobol design; the interpretation still depends on the bounds.



API and validation

```
local_problem = ODEForwardSensitivityProblem(sir_kernel!, u0, (0.0, 40.0),
local_solution = solve(local_problem, Tsit5(); saveat = tgrid)
states, jacobians = extract_local_sensitivities(local_solution, true)

identifiability = assess_identifiability(
    identifiability_system;
    measured_quantities = [y ~ b * c * S * I],
    funcs_to_check = [b, c, g, b * c],
)
```

- ▶ SciMLSensitivity: forward sensitivities or adjoints.
- ▶ GlobalSensitivity: Sobol first-order and total indices.
- ▶ StructuralIdentifiability: structural, noiseless identifiability.
- ▶ Check $S + I + R = N$ and compare one derivative with a finite difference.

Takeaways and limitations

- ▶ Local: neighborhood behavior around one parameter vector.
- ▶ Global: ranking conditional on ranges and a design.
- ▶ Structural: information available in an idealized observation map.
- ▶ Practical identifiability also depends on noise, sampling, initial states, and misspecification.

Read the book chapter for derivations and exercises; see the `SciMLSensitivity`, `GlobalSensitivity`, and `StructuralIdentifiability` documentation for APIs.

Interventions and optimization

Scenario first, optimization second

An intervention analysis has two auditable layers:

1. specify a policy and simulate it;
2. compare admissible policies with an explicit objective and constraints.

The baseline is the course SIR model. A 50% transmission reduction lasts $d = 10$ time units; only its start $t_1 \in [0, 25]$ is optimized.

Controlled SIR equations

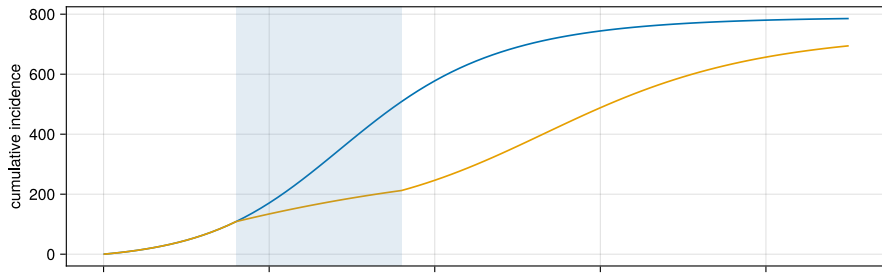
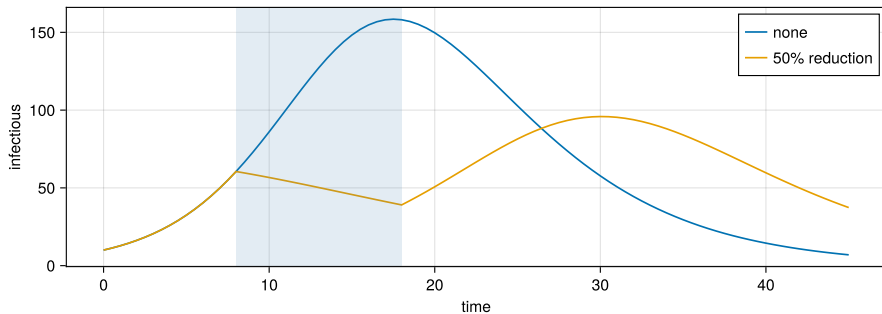
Add cumulative incidence C and a rectangular control:

$$\begin{aligned}\dot{S} &= -\beta c(1 - v(t))SI/N, \\ \dot{I} &= \beta c(1 - v(t))SI/N - \gamma I, \\ \dot{R} &= \gamma I, \\ \dot{C} &= \beta c(1 - v(t))SI/N,\end{aligned}$$

$$v(t; t_1) = \begin{cases} v_{\max}, & t_1 \leq t < t_1 + d, \\ 0, & \text{otherwise.} \end{cases}$$

The teaching objective is $J(t_1) = C(T; t_1)$ at $T = 45$.

A callback defines one policy



Bounded timing optimization

```
function final_incidence(x, _)  
    Array(scenario11(x[1]))[4, end]  
end  
  
objective = OptimizationFunction(final_incidence, AutoFiniteDiff())  
problem = OptimizationProblem(objective, [8.0], nothing;  
    lb = [0.0], ub = [25.0])  
solution = solve(problem, Fminbox(LBFGS())); maxiters = 30)
```

The API makes the decision variable, objective, and box explicit. A grid is a diagnostic for the optimizer, not a substitute for stating the problem.



Validation, interpretation, and limits

For every solved policy check

$$\max_t |S(t) + I(t) + R(t) - N| < 10^{-6}, \quad \Delta C(t) \geq 0.$$

- ▶ The “optimal” result is conditional on reduction, duration, horizon, and cost.
- ▶ Instantaneous switching, perfect adherence, homogeneous mixing, and a closed population are idealizations.
- ▶ Add intervention cost, capacity, equity, uncertainty, or a peak-infection penalty when those are decision requirements.

See the book chapter, `DiffEqCallbacks.jl`, and `Optimization.jl`.

Data-to-decision capstone

From committed data to a conditional decision

The capstone makes each transition inspectable:

provenance → model choice → fit → residuals → parameter envelope → scenario

- ▶ Eight committed Eyam observations, 18 June–20 October 1666.
- ▶ $N = 261$ is the modeled population at the second-wave start.
- ▶ No render-time data download; the notebook has a deterministic fallback.

Model and data caveats

We fit an effective transmission rate $\lambda = \beta c$ and removal rate γ :

$$\dot{S} = -\lambda SI/N, \quad \dot{I} = \lambda SI/N - \gamma I, \quad \dot{R} = \gamma I.$$

The state columns are reconstructed from historical records and assumptions, not direct measurements. A homogeneous closed SIR model omits rodent-to-human transmission, household structure, movement, and reporting changes.

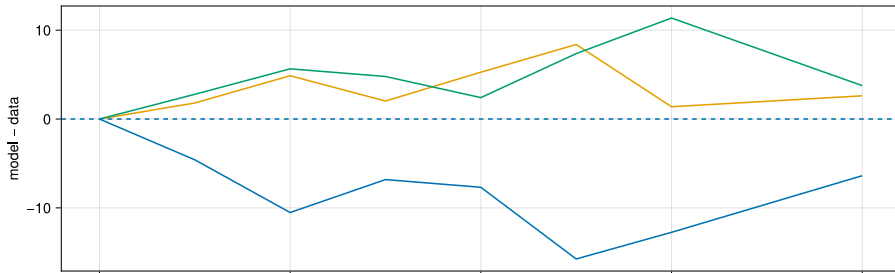
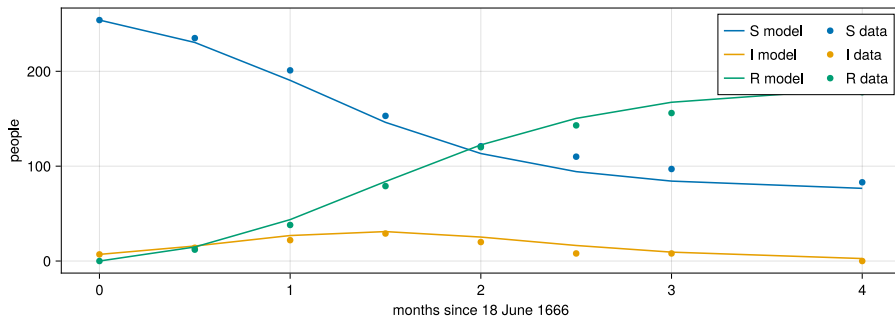
Verify locally, then fit

```
metadata = TOML.parsefile("data/eyam_plague_1666.toml")
data = CSV.read("data/eyam_plague_1666.csv", DataFrame)
@assert bytes2hex(sha256(read("data/eyam_plague_1666.csv")))) ==
    metadata["csv_sha256"]
@assert all(data.S .+ data.I .+ data.R .== metadata["population"])

loss(, _) = sum(((predict() .- observed_states) /
    metadata["population"]) .^ 2)
fit = solve(OptimizationProblem(
    OptimizationFunction(loss, AutoFiniteDiff()), [1.0, 0.5], nothing;
    lb = [1e-4, 1e-4], ub = [8.0, 8.0]), Fminbox(LBFGS()))
```

The normalized all-state least-squares objective is transparent, but it is not a count likelihood.

Fit diagnostics and deterministic envelope



Decision comparison

Add C with

$$\dot{C} = (1 - v(t))\lambda SI/N.$$

Compare no policy with a one-month 50% reduction beginning at month 1, and propagate the accepted parameter envelope through both scenarios.

- ▶ Validate $S + I + R = N$, finite trajectories, and lower modeled final incidence under the policy.
- ▶ Interpret the result as a model-based counterfactual under the chosen assumptions.

Takeaways and next steps

- ▶ A checksum verifies bytes, not historical truth.
- ▶ λ is an effective rate; it does not identify β and c .
- ▶ Residuals and sensitivity determine what conclusions are defensible.
- ▶ Better decisions require better observation and mechanistic models.

Read the capstone chapter, data provenance, and the CSV.jl and Optimization.jl documentation.

Delay differential equations

Why this unit matters

- ▶ Every earlier model assumes an **exponentially distributed** infectious period: memoryless recovery at constant hazard γ .
- ▶ A delay differential equation (DDE) replaces that with the opposite extreme: every infectious period lasts *exactly* τ time units.
- ▶ Question: how much does the **shape**, not just the mean, of the infectious-period distribution change epidemic dynamics?

Model assumptions and classification

Same baseline cohort throughout the course: $N = 1000$, $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $c = 10$, $\beta = 0.05$. Only the recovery mechanism changes.

Property	Classification
Determinism	Deterministic
Time / state	Continuous / continuous
Infectious period	Fixed delay $\tau = 1/\gamma = 4$ (variance 0)

Still **deterministic, continuous, population-level** — but *not* Markovian: the recovery flow at t depends on the state at $t - \tau$.

Equations

$$\frac{dS}{dt} = -\lambda(t)S(t), \quad \frac{dI}{dt} = \lambda(t)S(t) - \lambda(t-\tau)S(t-\tau), \quad \frac{dR}{dt} = \lambda(t-\tau)S(t-\tau)$$

with $\lambda(t) = \beta c I(t)/N(t)$ and history $h(t) = (N, 0, 0)$ for $t \leq 0$: recovery at t equals infection τ time units earlier.

Method of steps

- ▶ On $[0, \tau]$ the delayed term is fully known from the history h , so the DDE is an *ordinary* ODE there.
- ▶ That segment's dense solution becomes the exact history for $[\tau, 2\tau]$, and so on — solving a sequence of ODEs.
- ▶ Derivative discontinuities from the $t = 0$ history mismatch propagate to $t = \tau, 2\tau, \dots$, smoothing at each step.
- ▶ Declaring the lag via `constant_lags` tells the solver exactly where to place step boundaries instead of discovering them by trial and error.

Julia API

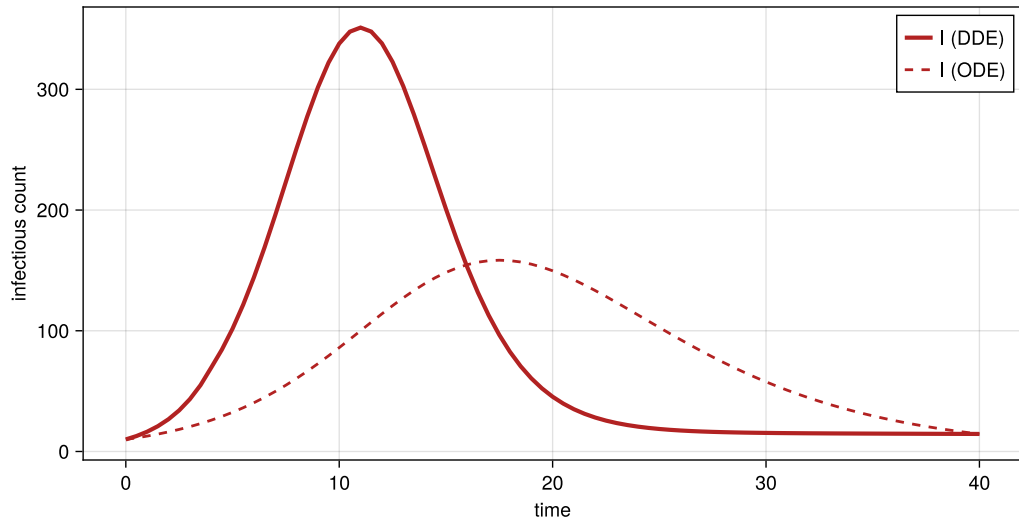
```
using DelayDiffEq, OrdinaryDiffEq

function sir_dde!(du, u, h, p, t)
    S, I, R = u
    , c, = p
    = * c * I / (S + I + R)
    Sd, Id, Rd = h(p, t - )           # delayed state
    d = * c * Id / (Sd + Id + Rd)
    du[1] = - * S
    du[2] = * S - d * Sd
    du[3] = d * Sd
end

prob = DDEProblem(sir_dde!, u0, history, tspan, p; constant_lags = [])
sol = solve(prob, MethodOfSteps(Tsit5())); reltol = 1e-8, abstol = 1e-8)
```

MethodOfSteps wraps any OrdinaryDiffEq algorithm; h(p, t) supplies the delayed

Generated result



Validation

- ▶ **Cross-check:** an independent two-stage (presolved ODE + continued DDE) construction agrees with the single-history run to solver tolerance.
- ▶ **Conservation & nonnegativity:** $S + I + R = 1000$ and all states ≥ 0 throughout $[0, 40]$.
- ▶ **Structural comparison:** at matched mean infectious period, the fixed delay produces a **taller, sharper** peak (≈ 351) than exponential recovery (≈ 158) — the shape of the distribution matters, not just its mean.

Interpretation and limitations

- ▶ A fixed delay has coefficient of variation 0: nobody recovers early, so the initial cohort transmits at full strength for all of $[0, \tau)$.
- ▶ Omitting `constant_lags` still gives a correct answer but forces the adaptive controller to *rediscover* breakpoints at $\tau, 2\tau, \dots$ — slower and less accurate near those points.
- ▶ The “instant seeding” history convention (all $I(0)$ infected exactly at $t = 0$) is convenient, not realistic; the two-stage construction shows how to relax it.
- ▶ Still fully deterministic — no demographic noise. That gap motivates the stochastic delay differential equation of the next unit.

Key takeaways

- ▶ DDEs model *fixed* recovery delays; the method of steps reduces them to a sequence of ODE solves.
- ▶ Declare `constant_lags` for solver efficiency at known breakpoints.
- ▶ Distribution *shape* (not just mean) materially changes epidemic peak height and timing.
- ▶ This model has no stochasticity — see the next two units for the stochastic and individual-level extensions.

Further reading

- ▶ DelayDiffEq.jl documentation
- ▶ SciML DDE problem reference
- ▶ Full derivation, code, and exercises: book chapter

Stochastic delay differential equations

Why this unit matters

- ▶ The DDE of the previous unit is still **deterministic**: same history in, same trajectory out, every time.
- ▶ Real epidemics have finite populations and demographic noise, exactly like the chemical-Langevin SDE.
- ▶ This unit combines both — a diffusion approximation **and** a fixed recovery delay — using the current, maintained DelayDiffEq + StochasticDiffEq route (not the archived StochasticDelayDiffEq.jl).
- ▶ The official docs mark SDDEProblem **experimental** — this deck shows, concretely, what that means.

Model assumptions and classification

Same population, contact process, and fixed delay $\tau = 1/\gamma = 4$ as the DDE unit.
Transmission and recovery now carry chemical-Langevin noise, applied to the *delayed* recovery flow.

Property	Classification
Determinism	Stochastic
Time / state	Continuous / continuous (diffusion approx.)
Noise	Chemical Langevin, non-diagonal, history-dependent

Not guaranteed to stay inside $S, I, R \geq 0$ — the central caveat below.

Equations

$$dS = -\lambda(t)S dt - \sqrt{\lambda(t)S} dW_1,$$

$$dI = [\lambda(t)S - \lambda(t-\tau)S_{t-\tau}]dt + \sqrt{\lambda(t)S} dW_1 - \sqrt{\lambda(t-\tau)S_{t-\tau}} dW_2,$$

$$dR = \lambda(t-\tau)S_{t-\tau} dt + \sqrt{\lambda(t-\tau)S_{t-\tau}} dW_2$$

Noise columns cancel across compartments by construction, so $S + I + R$ stays conserved for *any* realized path — nonnegativity is a separate question.

Julia API

```
using DelayDiffEq, StochasticDiffEq

prob = SDDEProblem(sir_dde!, sir_delayed_noise!, u0, history, tspan, p;
                  noise_rate_prototype = noise_shape())

Random.seed!(4) # NOT the rng= keyword - see caveats
sol = solve(prob, LambdaEM(); dt = 0.1, adaptive = false)
```

DelayDiffEq now recognizes an SDDEProblem directly and wraps LambdaEM in MethodOfSteps, exactly as it wraps Tsit5 for the deterministic DDE.

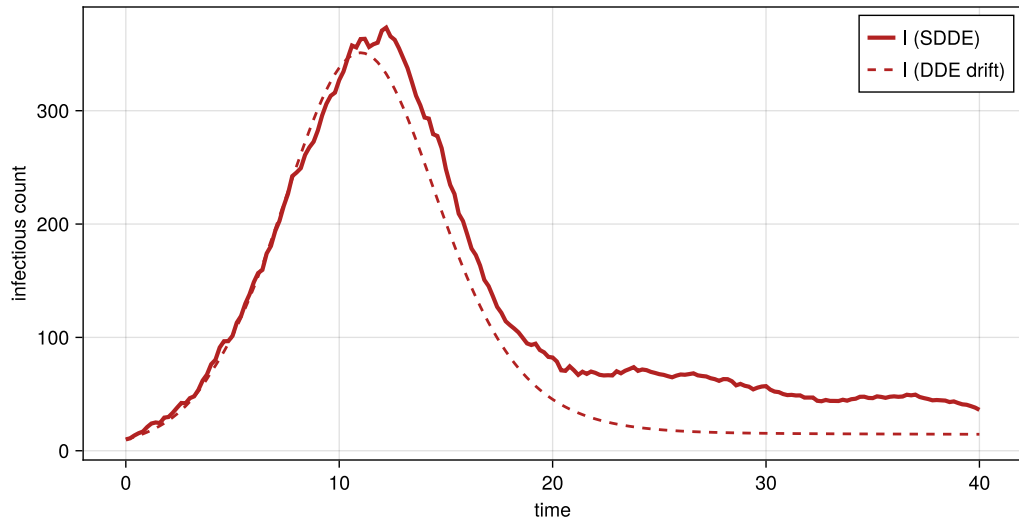
Adaptive stepping breaks down

using DelayDiffEq, OrdinaryDiffEq, StochasticDiffEq, SparseArrays, Random

```
function sir_dde!(du, u, h, p, t)
    S, I, R = u; , c, = p; N = S + I + R
    = * c * I / N
    Sd, Id, Rd = h(p, t - ); Nd = Sd + Id + Rd
    d = * c * Id / Nd
    du[1] = - * S; du[2] = * S - d * Sd; du[3] = d * Sd
    return nothing
end

function sir_delayed_noise!(G, u, h, p, t)
    S, I, R = u; , c, = p; N = S + I + R
    infection = max( * c * I / N * S, 0.0)
    Sd, Id, Rd = h(p, t - ); Nd = Sd + Id + Rd
    recovery = max( * c * Id / Nd * Sd, 0.0)
    G[1,1] = -sqrt(infection); G[2,1] = sqrt(infection)
    G[2,2] = -sqrt(recovery); G[3,2] = sqrt(recovery)
```

Generated result: fixed step



The nonnegativity caveat, quantified

- ▶ Seed 4: this realization happens to stay nonnegative throughout.
- ▶ Seed 1, identical code otherwise: I swings to **roughly** -87 at its worst point, while conservation still holds to solver precision ($< 10^{-8}$).
- ▶ Conservation is *structural* (built into the noise matrix); nonnegativity is *not* — and clamping the current state cannot repair a **delayed** drift/diffusion term that keeps depending on that state for up to τ more time units.

Interpretation and limitations

1. **Adaptive stepping is currently unusable** at the history discontinuity; a fixed step sidesteps it but forfeits automatic error control.
2. **The `rng` keyword is unreliable** for this `SDDEProblem` / `MethodOfSteps` combination — reproducibility currently requires global `Random.seed!` before `solve`, unlike every other stochastic unit in this course.
3. **Nonnegativity is not guaranteed**, and harder to police than for a plain SDE because the delayed term cannot be corrected by clamping the present state alone.

None of this invalidates the chemical Langevin approximation itself — it remains best treated as a teaching device and basis for further testing with the *current* package versions, not a production tool.

Key takeaways

- ▶ `SDDEProblem` unifies delayed drift with chemical-Langevin diffusion, but is explicitly experimental in current SciML docs.
- ▶ Use a **fixed step** (`adaptive = false`) and a **global** `Random.seed!` — both deliberate, documented deviations from the rest of the course.
- ▶ Conservation holds structurally; nonnegativity does not — always check the minimum state, not just the sum.
- ▶ A discrete-event alternative (next unit) avoids the diffusion approximation and its negative-state risk entirely.

Further reading

- ▶ SDDE problem reference (marked experimental)
- ▶ Gillespie (1977) for the chemical Langevin approximation
- ▶ Full derivation, code, and exercises: book chapter

Discrete-event simulation

Why this unit matters

- ▶ Jump-process chapter: aggregate counts, “what happens next, to how many?”
- ▶ Agent-based chapter: individuals, but a fixed time step $\Delta t = 0.1$.
- ▶ Discrete-event simulation (DES): individuals **and** a continuous clock — jump directly from one scheduled event to the next, no discretization.
- ▶ Built two ways here: Agents.jl's EventQueueABM, and the general-purpose, process-oriented ConcurrentSim.jl.

Model assumptions and classification

Usual baseline: $N = 1000$, $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $c = 10$, $\beta = 0.05$, $\gamma = 0.25$. Each individual's next contact and recovery are scheduled as events; the clock jumps to the next event anywhere in the population.

Property	Classification
Determinism	Stochastic
Time	Continuous, event-driven (no fixed Δt)
State	Discrete (individual status)
Representation	Individual-level

Equations

$$T_{\text{contact}} \sim \text{Exponential}(1/c), \quad \Pr(\text{infect} \mid \text{contact w/ infectious}) = \beta$$
$$T_{\text{recovery}} \sim \text{Exponential}(1/\gamma)$$

By memorylessness and superposition of independent Poisson processes, the *population-level* counting process is exactly the jump-process CTMC; the event-queue view additionally retains *who* infected *whom* and *when*.

Julia API: EventQueueABM

```
using Agents

@agent struct Person{NoSpaceAgent}
    status::Symbol
end

transmit_propensity(a, m) = a.status == :S ? m.c : 0.0
recovery_propensity(a, m) = a.status == :I ? m. : 0.0

events = (AgentEvent(action! = transmit!, propensity = transmit_propensity),
          AgentEvent(action! = recover!, propensity = recovery_propensity))
model = EventQueueABM(Person, events; properties, rng)
run!(model, tf; adata = [(:status, infected)])
```

By default an AgentEvent's timing samples an exponential wait parameterized by its propensity — matching the equations above exactly.

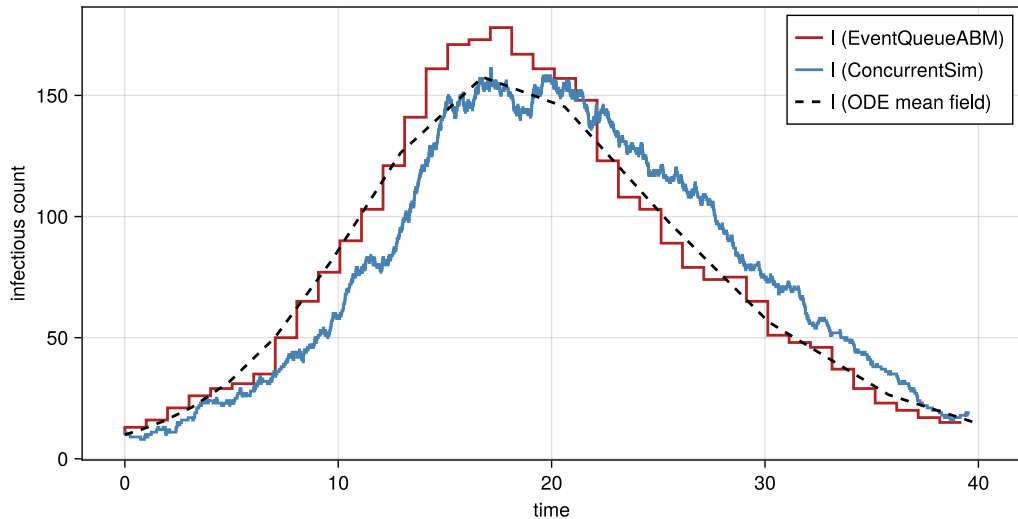
Julia API: ConcurrentSim.jl

```
using ConcurrentSim, ResumableFunctions

@resumable function live(sim, person, m, rng)
    while person.status == :S
        @yield timeout(sim, rand(rng, Exponential(1 / m.c)))
        alter = rand(rng, m.people)
        alter.status == :I && rand(rng) < m. && (person.status = :I)
    end
    if person.status == :I
        @yield timeout(sim, rand(rng, Exponential(1 / m.)))
        person.status = :R
    end
end
end
```

Not agent-centric: coroutine @resumable processes plus Resource/Store primitives for queueing and contention — more bookkeeping, more generality.

Generated result



Validation

- ▶ **Conservation:** every recorded row of both simulations sums to N .
- ▶ **Monotonicity:** the one-way $S \rightarrow I \rightarrow R$ transition keeps S non-increasing and R non-decreasing in both implementations.
- ▶ **Ensemble check:** a 20-run EventQueueABM ensemble's mean final size is within 10% of the deterministic ODE prediction.

Interpretation and limitations

- ▶ `EventQueueABM` automates scheduling but its default `adata` samples once per unit time — coarser than the true event resolution.
- ▶ `ConcurrentSim` needs hand-written recording, but every event is captured exactly when it happens, and its `Resource/Store` primitives extend to models with no natural agent-centric analogue (e.g. limited hospital beds).
- ▶ Both scale worse than the aggregated jump-process CTMC: scheduling $O(N)$ individual events costs more than resampling one aggregate draw.
- ▶ Neither requires exponential timing: `AgentEvent`'s timing and `ConcurrentSim`'s timeout both accept arbitrary — including fixed — waiting times, so a DES can encode the fixed delay of the DDE unit directly, with no history function or method of steps.

Key takeaways

- ▶ DES resolves the same CTMC as the jump-process chapter, per individual, on a continuous event-driven clock.
- ▶ `EventQueueABM` (`Agents.jl` 7) and `ConcurrentSim.jl` are two current, complementary ways to build one: agent/event-centric vs. general process/resource-centric.
- ▶ A DES can express fixed delays and resource contention that are awkward or impossible in the population-level or fixed- Δt models earlier in the course.

Further reading

- ▶ `Agents.jl` EventQueueABM documentation
- ▶ `ConcurrentSim.jl` repository
- ▶ Full derivation, code, and exercises: book chapter

Universal differential equations

Appendix A04 · learning a transmission closure

Why this unit matters

- ▶ Mechanistic SIR models fix the **functional form** of transmission; data may not agree with that choice.
- ▶ A *universal differential equation* (UDE) keeps the trustworthy bookkeeping and replaces one uncertain term with a trainable **universal approximator** [rackauckas2020].
- ▶ We differentiate **through** the ODE solver to fit the hybrid model, then ask the sharpest possible question: does the network recover the *known* closure?

The grey-box model

Keep the SIR flow, learn the force of infection λ_θ :

$$\frac{dS}{dt} = -\lambda_\theta(I) S, \quad \frac{dI}{dt} = \lambda_\theta(I) S - \gamma I, \quad \frac{dC}{dt} = \lambda_\theta(I) S.$$

- ▶ **Deterministic** dynamics, **continuous** time and state, **population** level.
- ▶ Data are generated from the true mass-action closure $\lambda^*(I) = \beta c I = 0.5 I$ (course baseline in proportions).
- ▶ $\mathcal{N}_\theta$ supplies *only* the transmission term; γI and the flow structure are imposed by hand.

Fit by differentiating through the solver

- Loss: Poisson negative log-likelihood of daily incidence
 $\mu_k(\theta) = N[C(t_{k+1}) - C(t_k)]_\theta$ against noisy counts y_k .
- Gradients: reverse-mode adjoint (InterpolatingAdjoint + ReverseDiffVJP) from SciMLSensitivity.jl.
- Optimiser: a short Adam run to make fast progress, then BFGS to polish.
- Network: `Lux.Chain(Dense(1,5,tanh), Dense(5,1,softplus))` — **16 parameters**; softplus output guarantees $\lambda_\theta \geq 0$.

```
using OrdinaryDiffEq, Distributions, StableRNGs, Random
using Lux, DiffEqFlux, SciMLSensitivity
using Optimization, OptimizationOptimisers, OptimizationOptimJL
using CairoMakie

N = 1000.0; c = 0.5;  = 0.25
u0 = [0.99, 0.01, 0.0]; tspan = (0.0, 40.0); t = 1.0; train_time = 30.0

function sir_true!(du, u, p, t)
```

Training the hybrid model

```
function sir_ude!(du, u, , t)
    S, I, C = u;  = force_of_infection(I, )
    du[1] = - *S; du[2] = *S - *I; du[3] = *S
end
prob_ude = ODEProblem(sir_ude!, u0, (0.0, train_time), p_init)
predict() = Array(solve(prob_ude, Tsit5(); p = , saveat = t,
    sensealg = InterpolatingAdjoint(autojacvec = ReverseDiffVJP()))))
loss() = ( = max.(N .* diff(predict())[3, :]), 1e-6); sum( .- noisy .* lo

optf = OptimizationFunction((, _) -> loss(), Optimization.AutoZygote())
res_adam = solve(OptimizationProblem(optf, p_init),
    OptimizationOptimisers.Adam(0.05); maxiters = 120)
res = solve(OptimizationProblem(optf, res_adam.u),
    OptimizationOptimJL.BFGS(); maxiters = 40)
_hat = res.u
round(res.objective; digits = 1)
```

Validation: recovering the known closure

```
Igrid = 0.0:0.005:0.5
_nn = [force_of_infection(I, _hat) for I in Igrid]
Imax = maximum(reduce(hcat, sol_true(ts).u)[2, :])
in_range = Igrid .<= Imax
max_abs_error = maximum(abs.(_nn[in_range] .- c .* Igrid[in_range]))

fig = Figure(size = (900, 400))
ax = Axis(fig[1, 1]; xlabel = "infectious proportion I",
          ylabel = "force of infection ",
          title = "Learned closure (max error  $(round(max_abs_error; sigdigits
lines!(ax, Igrid, c .* Igrid; color = :black, linewidth = 2, label = "true
lines!(ax, Igrid, _nn; color = :firebrick, linestyle = :dash, linewidth =
          label = "neural network")
vlines!(ax, [Imax]; color = :gray, linestyle = :dot)
axislegend(ax; position = :lt)
fig
```

Key takeaways & caveats

- ▶ **Structure buys data efficiency** — 16 parameters suffice *because* recovery and flow are mechanistic; a black box would need far more data.
- ▶ **Identifiability is local** — trust the closure only over the range of I the training data actually visit (left of the dotted line).
- ▶ **Encode priors** — `softplus` enforces $\lambda \geq 0$; monotonicity could be added similarly.
- ▶ **Cost & conditioning** — every loss evaluation is an ODE solve, every gradient an adjoint solve; stiff or larger problems need care with `sensealg`.
- ▶ **APIs move fast** — current `Lux` / `DiffEqFlux` / `Optimization`; legacy `FastChain` / `sciml_train` will not run on the pinned stack.

See the book chapter for derivations, and the `DiffEqFlux`, `Lux`, and `SciMLSensitivity` manuals for the current UDE workflow.

Finite-state projection

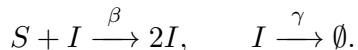
Appendix A05 · solving the master equation directly

Why this unit matters

- ▶ Stochastic SIR is usually studied by **simulating trajectories** (Chapter 4).
- ▶ *Finite-state projection* (FSP) takes the opposite route: solve directly for the **probability distribution** over states by integrating a truncated chemical master equation [munsky2006].
- ▶ One deterministic ODE solve returns the *entire* distribution — including tail probabilities that ensembles estimate only with huge sample sizes.

Model and the projected state space

Two reaction channels on integer counts (S, I) , with $R = N - S - I$:



- ▶ **Stochastic** jump process, **continuous** time, **discrete** state.
- ▶ Reachable states form a **triangle** $\{(s, i) : s + i \leq N\}$: infection maps $(s, i) \mapsto (s-1, i+1)$, recovery maps $(s, i) \mapsto (s, i-1)$.
- ▶ We use $N = 100$, $S(0) = 99$, $I(0) = 1$; $\beta = \beta_{\text{base}} c/N = 0.005$, $\gamma = 0.25$. $N = 100$ is deliberate — the triangle has $(N+1)(N+2)/2$ states.

The chemical master equation as a linear ODE

$$\frac{dp(s, i, t)}{dt} = \beta(s+1)(i-1)p(s+1, i-1) + \gamma(i+1)p(s, i+1) - [\beta si + \gamma i]p(s, i).$$

Stacking probabilities gives $\frac{d\mathbf{p}}{dt} = \mathbf{Q} \mathbf{p}$, with $\mathbf{Q}$ the **sparse CTMC generator**.

***We do not use FiniteStateProjection.jl.** Its resolved release (v0.3.5) omits ModelingToolkit from its dependencies yet calls it internally, so the only way to force it is to mutate a third-party module at runtime. Instead we build $\mathbf{Q}$ **transparently** — the method and results are identical.*

Assembling the sparse generator

using OrdinaryDiffEq, SparseArrays, LinearAlgebra, StableRNGs, CairoMakie

```
function sir_fsp_generator(N, , ; Smax = N, Imax = N)
    index = Dict{Tuple{Int,Int},Int}(); states = Tuple{Int,Int}[]
    for s in 0:Smax, i in 0:Imax
        if s + i <= N
            push!(states, (s, i)); index[(s, i)] = length(states)
        end
    end
    rows = Int[]; cols = Int[]; vals = Float64[]
    add!(r, c, v) = (push!(rows, r); push!(cols, c); push!(vals, v))
    for (col, (s, i)) in enumerate(states)
        a_inf = *s*i; a_rec = *i
        add!(col, col, -(a_inf + a_rec))
        a_inf > 0 && haskey(index, (s-1, i+1)) && add!(index[(s-1, i+1)], col, a_inf)
        a_rec > 0 && haskey(index, (s, i-1)) && add!(index[(s, i-1)], col, a_rec)
    end
end
```

Solve, marginalise, validate

```
p0 = zeros(length(states)); p0[index[(99, 1)]] = 1.0
sol = solve(ODEProblem((dp, p, Q, t) -> mul!(dp, Q, p), p0, (0.0, 40.0), Q,
    Vern7()); abstol = 1e-10, reltol = 1e-8, saveat = 1.0)

marginalI(p) = (m = zeros(Ncap + 1); for (k, (s, i)) in enumerate(states);
    PI20 = marginalI(sol.u[21]); counts = 0:Ncap
    meanI = sum(counts .* PI20)
    leak = 1 - minimum(sum(u) for u in sol.u)

fig = Figure(size = (900, 380))
ax = Axis(fig[1, 1]; xlabel = "infectious count I at t = 20", ylabel = "prob")
    title = "FSP mean I(20)  $(round(meanI; digits = 2)), leakage  $(round(leak; digits = 2))"
barplot!(ax, counts, PI20; color = (:firebrick, 0.7))
vlines!(ax, [meanI]; color = :black, linestyle = :dash)
xlims!(ax, -1, 40)
fig
```

Cost: the defining limitation

```
using Printf
for n in (50, 100, 250, 1000)
    tri = ((n + 1) * (n + 2)) ÷ 2
    @printf("N = %5d → triangular CME dimension = %8d (rectangular %d)\n",
           n, tri, (n + 1)^2)
end
```

N =	50	→	triangular CME dimension =	1326	(rectangular 2601)
N =	100	→	triangular CME dimension =	5151	(rectangular 10201)
N =	250	→	triangular CME dimension =	31626	(rectangular 63001)
N =	1000	→	triangular CME dimension =	501501	(rectangular 1002001)

- ▶ Doubling N roughly **quadruples** the equations; a third species makes it cubic — why we used $N = 100$, not the baseline $N = 1000$ ($\sim 5 \times 10^5$ states).
- ▶ **Truncation is an approximation**; the conservation check is the diagnostic.
- ▶ **Best for small populations / few species**: early-outbreak, extinction, and tail questions. For large N , use ensembles (Ch. 4) or diffusion approximations (Ch. 5).

See the book chapter and the Catalog of Modeling Tools; it does

Probabilistic numerical integration

Appendix A06 · error bars on the ODE solve

Why this unit matters

- ▶ Every numerical ODE solve is an approximation, yet a classical solver returns a single trajectory with **no statement of its own error**.
- ▶ *Probabilistic numerics* reframes integration as Bayesian inference: the solver returns a **distribution** over trajectories whose spread reflects the discretisation uncertainty [bosch2021].
- ▶ Here we make that uncertainty **concrete and executable** by measuring the true error of a coarse solve against a near-exact reference.

Reproducibility gate: ProbNumDiffEq.jl

*The pinned ProbNumDiffEq.jl v0.17 does **not** precompile against the resolved OrdinaryDiffEq stack — OrdinaryDiffEqDifferentiation no longer exposes the standardtag symbol its solvers reference:*

UndefVarError: `standardtag` not defined in `OrdinaryDiffEqDifferentiat

- ▶ We therefore **gate** the probabilistic solve (eval: false) and run an **executable error-quantification core** with the standard stack.
- ▶ The gated block reflects the *current* v0.17 API, verified against the installed source, so it runs as written once the packages are realigned.

Model and the quantity of interest

Baseline SIR (unchanged): $S(0) = 990$, $I(0) = 10$, $R(0) = 0$, $N = 1000$, $c = 10$, $\beta = 0.05$, $\gamma = 0.25$ — **deterministic**, continuous in time and state.

$$\frac{dS}{dt} = -\beta c \frac{SI}{N}, \quad \frac{dI}{dt} = \beta c \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I.$$

- ▶ Numerical error $e(t) = \hat{u}(t) - u(t)$ cannot be evaluated exactly, but we approximate $u(t)$ to machine precision with a tight, high-order solve.
- ▶ A probabilistic solver models $e(t)$ **online** as a Gauss–Markov process; its posterior standard deviation $\sigma(t)$ is a calibrated proxy for $|e(t)|$.

Measuring the error directly

```
using EpiModelingCourse, OrdinaryDiffEq, CairoMakie

p = BASELINE_PARAMETERS; s0 = BASELINE_INITIAL_STATE
u0 = [s0.S, s0.I, s0.R]; tspan = (0.0, 40.0); t = 0.5
prob = ODEProblem(sir_ode!, u0, tspan, p)

ref      = solve(prob, Vern9(); abstol = 1e-12, reltol = 1e-12, saveat = t)
coarse    = solve(prob, Tsit5(); abstol = 1e-1, reltol = 1e-1, saveat = t)
err = abs.(reduce(hcat, coarse.u) .- reduce(hcat, ref.u))

fig = Figure(size = (820, 400))
ax = Axis(fig[1, 1]; xlabel = "time (days)", ylabel = "absolute error",
          yscale = log10, title = "Numerical error vs. reference")
for (i, (lab, col)) in enumerate(zip(("S", "I", "R"),
                                     (:steelblue, :firebrick, :seagreen)))
    lines!(ax, ref.t, max.(err[i, :], 1e-10); color = col, linewidth = 2,
end
```

Calibration check & the gated API

```
using Printf
refm = reduce(hcat, ref.u)
for tol in (1e-1, 1e-3, 1e-5, 1e-7)
    s = solve(prob, Tsit5(); abstol = tol, reltol = tol, saveat = t)
    e = maximum(abs.(reduce(hcat, s.u)[2, :] .- refm[2, :]))
    @printf("tol = %.0e → max |I error| = %.3e\n", tol, e)
end
```

```
tol = 1e-01 → max |I error| = 8.627e-01
tol = 1e-03 → max |I error| = 7.748e-03
tol = 1e-05 → max |I error| = 3.187e-04
tol = 1e-07 → max |I error| = 3.619e-06
```

Tightening tolerance monotonically shrinks the error — the deterministic analogue of a probabilistic solver reporting smaller uncertainty at smaller steps. The **gated** current API (not run here):

```
using ProbNumDiffEq, Statistics
sol = solve(prob, FK1(order = 3, prior = TWP(3))
```

Key takeaways & caveats

- ▶ **Uncertainty is largest where dynamics are fastest** — around the epidemic peak.
- ▶ **Calibration is the whole point** — the reported σ must bracket the true error; the refinement check is that question, deterministically.
- ▶ **Cost** — filters carry a covariance alongside the state; the payoff is an error bar, rarely worth it for routine forward simulation.
- ▶ **Where it matters** — when solver error could be mistaken for model error, e.g. likelihood-based inference with coarse solves.
- ▶ **Environment caveat** — the probabilistic solve is gated by a package-compatibility gap, *not* by the method.

See the book chapter and the `ProbNumDiffEq.jl` and `SciML ODE` documentation.

Surrogate and emulator models

Why emulate a simulator?

- ▶ An ODE solve can be accurate but too expensive inside calibration, uncertainty analysis, or design.
- ▶ A surrogate approximates a declared simulator response on a declared domain.
- ▶ The mechanistic model remains the arbiter for decisions.

A small, explicit target

Classification: deterministic, continuous-time, continuous-state, population-level.

$$\dot{S} = -\beta c SI/N, \quad \dot{I} = \beta c SI/N - \gamma I, \quad \dot{R} = \gamma I.$$

For $\beta \in [0.02, 0.08]$, emulate

$$q(\beta) = R(40; \beta)/N, \quad e_j = \hat{q}(\beta_j) - q(\beta_j).$$

Baseline: $(S, I, R) = (990, 10, 0)$, $N = 1000$, $c = 10$, $\gamma = 0.25$.

Design, simulate, fit

```
(::Surrogates.RadialBasis{Surrogates.var"#cubicRadial##0#cubicRadial##1", 1
```

The diagnostic is out of sample

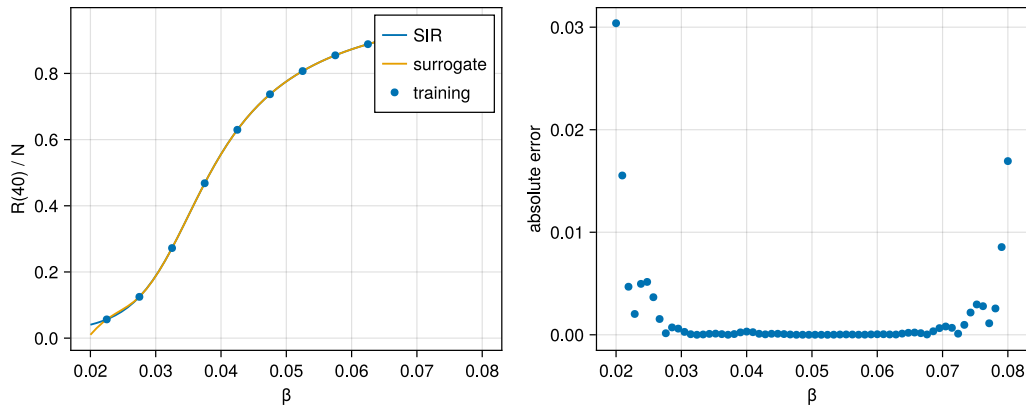


Figure 1: Mechanistic SIR output, radial-basis surrogate, training design, and held-out approximation error.

Validate, then use

- ▶ Check the emulator on held-out inputs, not only training points.
- ▶ Refine locally with `Surrogates.update!` when error identifies a gap.
- ▶ For two or more inputs, scale the design and increase training points.
- ▶ Re-simulate every policy or scientific conclusion with the original model.

(rmse = 0.005079064991333957, max_abs_error = 0.030386676263557774)

Limits and links

- ▶ Extrapolation outside the design interval is not validated.
- ▶ A peak or threshold target can need many more points than a smooth final size.
- ▶ Solver failures and observation noise must be represented in the training pipeline rather than hidden in one scalar response.

See the book appendix, the Surrogates radial-basis docs, and the surrogate interface.

Advanced optimal control

From a timing decision to a control function

- ▶ Chapter 11 chooses one intervention start time.
- ▶ Here $u(t)$ is a bounded, function-valued intervention.
- ▶ Classification: deterministic, continuous-time, continuous-state, population-level, ODE-constrained.

Use normalized states S, I, C with $S(0) = 0.99$, $I(0) = 0.01$ and $C(0) = 0$. The horizon is $T = 20$, $0 \leq u \leq 0.5$, and $\int_0^T u(t) dt \leq 2$.

Controlled SIR and objective

$$\dot{S} = -(1-u)\beta cSI, \quad \dot{I} = (1-u)\beta cSI - \gamma I, \quad \dot{C} = (1-u)\beta cSI.$$

$$\min_u C(T) + 10^{-3} \int_0^T u(t)^2 dt \quad \text{s.t.} \quad 0 \leq u(t) \leq u_{\max}, \quad \int_0^T u(t) dt \leq B.$$

The identity $C(t) = S(0) - S(t)$ is a structural validation.

Current InfiniteOpt formulation

This program contains Ipopt, a library for large-scale nonlinear optimization.

Ipopt is released as open source code under the Eclipse Public License (EPL).

For more information visit <https://github.com/coin-or/Ipopt>

Read the transcription and plot it

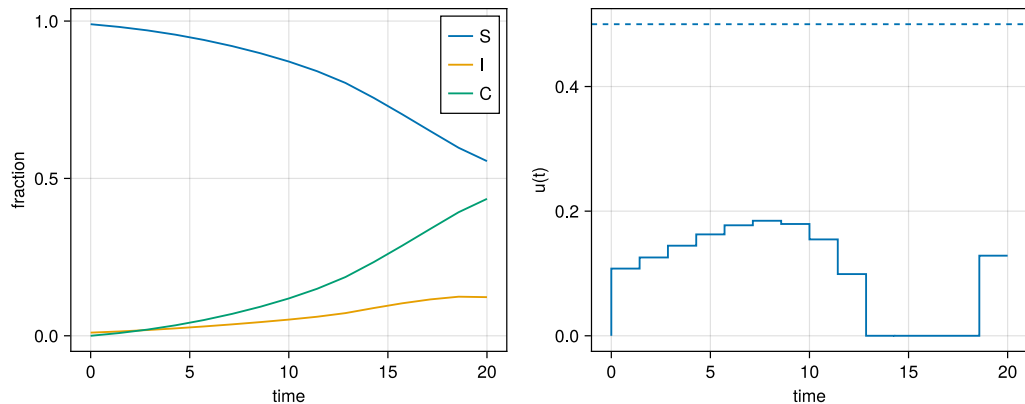


Figure 1: Collocated SIR trajectory and the optimized bounded intervention over time.

Check bounds, nonnegativity, and the incidence identity before interpretation. Refining supports is the relevant numerical convergence experiment

What SDDP can—and cannot—do here

The historical nonlinear SIR script is **not** a current SDDP model: `LinearPolicyGraph` requires affine constraints and objectives. Products such as uSI and exponential infection updates violate that restriction.

- ▶ Defer nonlinear stochastic SIR control.
- ▶ A defensible SDDP exercise first builds an explicit linear inventory-like approximation and reports what epidemiological nonlinearities were discarded.
- ▶ Do not present that proxy as an exact solution of the controlled SIR problem.

See the `InfiniteOpt` quick start, `InfiniteOpt` docs, and SDDP restrictions.

Model composition and structured populations

Why compose models?

- ▶ Named components expose biological assumptions and connections.
- ▶ One Catalyst reaction network can generate deterministic and stochastic problem types.
- ▶ Structure matters: homogeneous mixing hides group-specific contact patterns.

Today: current ModelingToolkit 11 and Catalyst 16 APIs.

Symbolic components

For a deterministic continuous-time continuous-state SIR:

$$\dot{S} = -\beta SI/N, \quad \dot{I} = \beta SI/N - \gamma I, \quad \dot{R} = \gamma I.$$

Keep the flows in separate `System` components, connect shared states and parameters, then call `compose` and `mtkcompile`.

Structured mixing as reactions

For two groups, $m_{12} = m_{21} = 0.5$ modifies cross-group infection:

$$\lambda_1 = \frac{\beta c_1}{N}(S_1 I_1 + m_{12} S_1 I_2), \quad \lambda_2 = \frac{\beta c_2}{N}(m_{21} S_2 I_1 + S_2 I_2).$$

One network, multiple interpretations

The same `ReactionSystem` can be passed to an `SDEProblem` (chemical-Langevin approximation) or `JumpProblem` (integer-count event process). Their state spaces and error claims are different.

Generated comparison

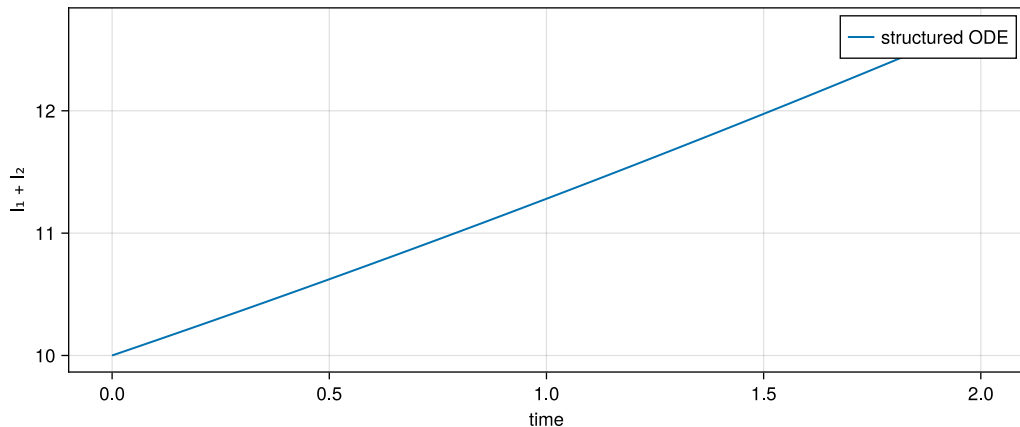


Figure 1: Aggregate infectious prevalence generated from the structured reaction-network model.

Validation: the ODE and jump stoichiometry conserve $S_1 + I_1 + R_1 + S_2 + I_2 + R_2$. A

Takeaways and links

- ▶ Use `System`, `compose`, and `mtkcompile` for inspectable symbolic assembly.
- ▶ Use Catalyst reactions to preserve stoichiometry and reuse a model.
- ▶ Group structure introduces parameters that aggregate incidence may not identify.

Read the book appendix, ModelingToolkit composition, and Catalyst system generation.